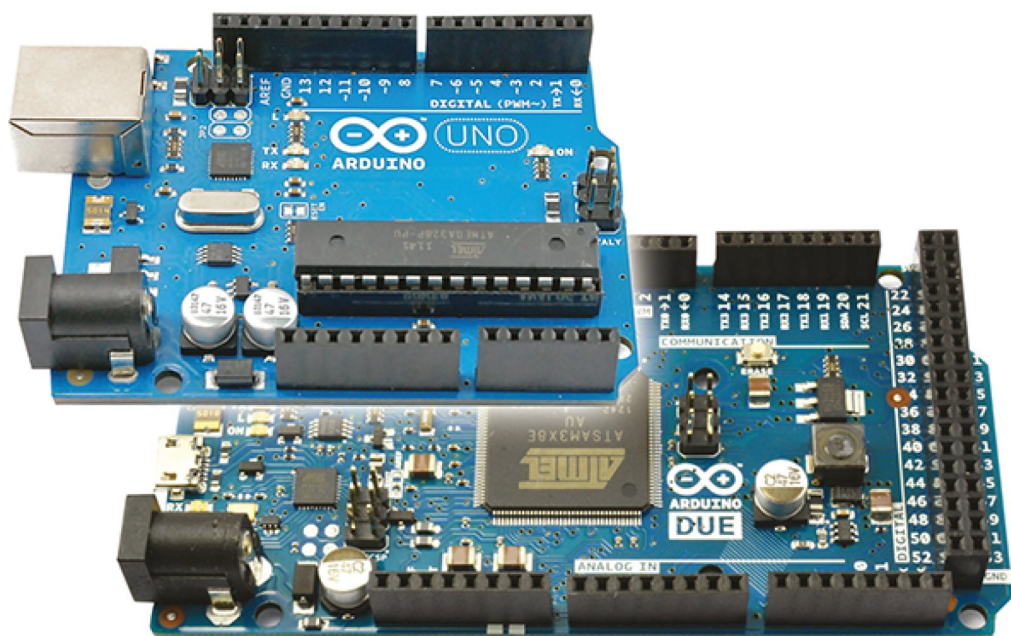


Arduino™ dla początkujących

Kolejny krok

Poznaj tajniki Arduino!



Simon Monk

Helion

Tytuł oryginału: Programming Arduino™ Next Steps: Going Further with Sketches

Tłumaczenie: Konrad Matuk

ISBN: 978-83-283-0016-3

Original edition copyright © 2014 by McGraw-Hill Education.

All rights reserved.

Polish edition copyright © 2015 by HELION S.A.

All rights reserved.

All rights reserved. No part of this book may be reproduced or transmitted in any form or by any means, electronic or mechanical, including photocopying, recording or by any information storage retrieval system, without permission from the Publisher.

Wszelkie prawa zastrzeżone. Nieautoryzowane rozpowszechnianie całości lub fragmentu niniejszej publikacji w jakiejkolwiek postaci jest zabronione. Wykonywanie kopii metodą kserograficzną, fotograficzną, a także kopiowanie książki na nośniku filmowym, magnetycznym lub innym powoduje naruszenie praw autorskich niniejszej publikacji.

Wszystkie znaki występujące w tekście są zastrzeżonymi znakami firmowymi bądź towarowymi ich właścicieli.

Autor oraz Wydawnictwo HELION dołożyli wszelkich starań, by zawarte w tej książce informacje były kompletne i rzetelne. Nie biorą jednak żadnej odpowiedzialności ani za ich wykorzystanie, ani za związane z tym ewentualne naruszenie praw patentowych lub autorskich. Autor oraz Wydawnictwo HELION nie ponoszą również żadnej odpowiedzialności za ewentualne szkody wynikłe z wykorzystania informacji zawartych w książce.

Wydawnictwo HELION

ul. Kościuszki 1c, 44-100 GLIWICE

tel. 32 231 22 19, 32 230 98 63

e-mail: helion@helion.pl

WWW: <http://helion.pl> (księgarnia internetowa, katalog książek)

Drogi Czytelniku!

Jeżeli chcesz ocenić tę książkę, zajrzyj pod adres

http://helion.pl/user/opinie/arpokk_ebook

Możesz tam wpisać swoje uwagi, spostrzeżenia, recenzję.

Pliki z przykładami omawianymi w książce można znaleźć pod adresem:

<ftp://ftp.helion.pl/przyklady/arpokk.zip>

- [Poleć książkę na Facebook.com](#)
- [Kup w wersji papierowej](#)
- [Oceń książkę](#)

- [Księgarnia internetowa](#)
- [Lubię to! » Nasza społeczność](#)

Spis treści

O autorze	9
Podziękowania	10
Wstęp	11
Pobieranie przykładów	12
Czego będę potrzebował?	12
Korzystanie z tej książki	12
Rozdział 1. Programowanie Arduino	15
Czym jest Arduino	15
Instalacja i środowisko programistyczne	17
<i>Instalacja środowiska programistycznego</i>	18
<i>Blink</i>	18
Wycieczka po płytce Arduino	21
<i>Zasilanie</i>	21
<i>Złącza zasilania</i>	22
<i>Wejścia analogowe</i>	22
<i>Złącza cyfrowe</i>	22
Płytki Arduino	23
<i>Uno i pochodne</i>	23
<i>Duże płytki Arduino</i>	24
<i>Małe płytki Arduino</i>	25
<i>Płytki LilyPad i LilyPad USB</i>	26
<i>Nieoficjalne płytki Arduino</i>	27
Język programowania	27
Modyfikacja szkicu Blink	27
Zmienne	29
If	30
Pętle	31
Funkcje	32
Wejścia cyfrowe	33
Wyjścia cyfrowe	35

Monitor portu szeregowego	35
Tablice i macierze	37
Wejścia analogowe	38
Wyjścia analogowe	40
Korzystanie z bibliotek	42
Typy danych obsługiwane przez Arduino	43
Polecenia Arduino	44
Podsumowanie	46
 Rozdział 2. Pod maską	47
Krótka historia Arduino	47
Anatomia Arduino	47
Procesory AVR	48
ATmega328	48
ATmega32u4	50
ATmega2560	50
AT91SAM3X8E	51
Arduino i Wiring	51
Od szkicu do Arduino	55
AVR Studio	56
Instalacja programu rozruchowego	58
Instalacja programu rozruchowego za pomocą aplikacji AVR Studio i programatora	59
Instalacja programu rozruchowego za pomocą zintegrowanego środowiska programistycznego Arduino i drugiej płytki Arduino	60
Podsumowanie	62
 Rozdział 3. Przerwania i zegary	63
Przerwania sprzętowe	63
Piny przerwań	66
Tryby przerwań	67
Aktywacja wbudowanego rezystora podciągającego	67
Procedury obsługi przerwań	67
Zmienne ulotne	68
Podsumowanie wiadomości na temat procedur obsługi przerwań	69
Włączanie i wyłączanie obsługi przerwań	69
Zegary i przerwania	70
Podsumowanie	73
 Rozdział 4. Przyspieszanie Arduino	75
Jak szybko działa Twoje Arduino?	75
Porównanie płytek Arduino	76
Przyspieszanie wykonywania operacji arytmetycznych	77
Czy naprawdę musisz stosować wartości typu float?	77
Przeglądanie kontra obliczanie	78
Szybkie wejścia-wyjścia	80
Podstawowa optymalizacja kodu	80
Bajty i bity	82

<i>Porty układu ATmega328</i>	82
<i>Bardzo szybkie działanie wyjść cyfrowych</i>	84
<i>Szybkie wejścia cyfrowe</i>	84
Przyspieszanie wejść analogowych	86
Podsumowanie	88
 Rozdział 5. Arduino i mały pobór prądu	89
Płytki Arduino i pobór prądu	89
Prąd i akumulatory	91
Zmniejszenie częstotliwości taktowania	92
Wyłączanie komponentów	94
Usypianie	95
<i>Biblioteka Narcoleptic</i>	95
<i>Budzenie za pomocą zewnętrznych przerwań</i>	97
Ograniczanie pobieranego prądu za pomocą wyjść cyfrowych	99
Podsumowanie	101
 Rozdział 6. Pamięć	103
Pamięć Arduino	103
Korzystanie z minimalnej ilości pamięci RAM	104
<i>Korzystanie z właściwych struktur danych</i>	105
<i>Zachowaj ostrożność, korzystając z rekurencji</i>	105
<i>Przechowywanie w pamięci flash stałych będących łańcuchami</i>	107
<i>Rozpowszechnione błędne przekonania</i>	108
<i>Pomiar wolnej pamięci</i>	108
Korzystanie z minimalnej ilości pamięci flash	108
<i>Korzystaj ze stałych</i>	109
<i>Usuń zbędne elementy szkicu</i>	109
<i>Pomiń program rozruchowy</i>	109
Statyczna i dynamiczna alokacja pamięci	109
Łańcuchy	111
<i>Tablice elementów typu char</i>	111
<i>Biblioteka Arduino StringObject</i>	114
Korzystanie z pamięci EEPROM	115
<i>Przykład korzystania z pamięci EEPROM</i>	116
<i>Korzystanie z biblioteki avr/eeprom.h</i>	118
<i>Ograniczenia pamięci EEPROM</i>	120
Korzystanie z pamięci Flash	120
Zapisywanie danych na kartach SD	121
Podsumowanie	122
 Rozdział 7. Korzystanie z magistrali I2C	123
Warstwa sprzętowa	125
Protokół magistrali I2C	126
Biblioteka Wire	126
<i>Inicjacja magistrali I2C</i>	127
<i>Wysyłanie danych przez urządzenie nadrzędne</i>	127
<i>Odbieranie danych przez urządzenie nadrzędne</i>	127

Przykład działania magistrali I2C	128
<i>Radio FM TEA5767</i>	128
<i>Przesyłanie danych pomiędzy dwoma płytkami Arduino</i>	130
<i>Płytki z diodami LED</i>	133
<i>Zegar czasu rzeczywistego DS1307</i>	134
Podsumowanie	135
Rozdział 8. Praca z urządzeniami wyposażonymi w interfejs 1-Wire	137
Sprzęt obsługujący interfejs 1-Wire	137
Protokół 1-Wire	138
Biblioteka OneWire	139
<i>Inicjalizowanie biblioteki OneWire</i>	139
<i>Skanowanie magistrali</i>	139
Korzystanie z układu DS18B20	141
Podsumowanie	143
Rozdział 9. Praca z urządzeniami wyposażonymi w interfejs SPI	145
Operowanie bitami	145
<i>Wartości binarne i szesnastkowe</i>	146
<i>Maskowanie bitów</i>	146
<i>Przesuwanie bitów</i>	148
Sprzęt obsługujący magistralę SPI	150
Protokół SPI	151
Biblioteka SPI	151
Przykład komunikacji za pomocą interfejsu SPI	153
Podsumowanie	157
Rozdział 10. Szeregowa transmisja danych za pośrednictwem układu UART	159
Sprzęt służący do szeregowej transmisji danych	159
Protokół obsługujący szeregową transmisję danych	162
Polecenia służące do obsługi szeregowej transmisji danych	162
Biblioteka SoftwareSerial	164
Przykłady szeregowej transmisji danych	165
<i>Komunikacja pomiędzy komputerem a Arduino za pośrednictwem interfejsu USB</i>	165
<i>Komunikacja pomiędzy dwoma płytkami Arduino</i>	167
<i>Moduł GPS</i>	169
Podsumowanie	172
Rozdział 11. Obsługa interfejsu USB	173
Emulacja klawiatury i myszy	173
<i>Emulacja klawiatury</i>	174
<i>Przykład emulacji klawiatury</i>	175
<i>Emulacja myszy</i>	175
<i>Przykład emulacji myszy</i>	176
Programowanie hosta USB	176
<i>Płytki USB Host i obsługująca ją biblioteka</i>	177
<i>Host USB płytki Arduino Due</i>	180
Podsumowanie	182

Rozdział 12. Obsługa sieci	183
Sprzęt sieciowy	183
<i>Płytkę rozszerzeń wyposażoną w kontroler sieci Ethernet</i>	183
<i>Arduino Ethernet i Arduino EtherTen</i>	184
<i>Arduino i Wi-Fi</i>	185
Biblioteka Ethernet	185
<i>Nawiązywanie połączenia</i>	185
<i>Stawianie serwera sieci Web</i>	188
<i>Tworzenie żądań</i>	189
Przykład szkicu korzystającego z biblioteki Ethernet	189
<i>Sprzętowy serwer sieci Web</i>	190
<i>Pobieranie danych w formacie JSON</i>	194
Biblioteka WiFi	195
<i>Nawiązywanie połączenia</i>	195
<i>Funkcje zdefiniowane w bibliotece WiFi</i>	196
Przykładowy szkic korzystający z sieci Wi-Fi	196
Podsumowanie	197
 Rozdział 13. Cyfrowe przetwarzanie sygnałów	199
Wprowadzenie do cyfrowego przetwarzania sygnałów	199
Uśrednianie odczytów	200
Wstęp do filtrowania	202
Prosty filtr dolnoprzepustowy	203
Cyfrowe przetwarzanie sygnałów przez Arduino Uno	204
Cyfrowe przetwarzanie sygnałów przez Arduino Due	205
Generowanie kodu filtrującego	208
Transformacja Fouriera	210
<i>Analizator spektrum</i>	212
<i>Pomiar częstotliwości</i>	214
Podsumowanie	214
 Rozdział 14. Praca z użyciem tylko jednego procesu	215
Zmiana skali	215
Dlaczego wątki są zbędne	216
Funkcje setup i loop	216
<i>Najpierw wykrywaj, a dopiero później reaguj</i>	216
<i>Pauza, która nie blokuje mikrokontrolera</i>	217
Biblioteka Timer	218
Podsumowanie	220
 Rozdział 15. Tworzenie bibliotek	221
Kiedy należy tworzyć biblioteki?	221
Stosowanie klas i metod	222
Przykładowa biblioteka TEA5767 Radio	222
<i>Określ interfejs programistyczny</i>	223
<i>Utwórz plik nagłówkowy</i>	224
<i>Utwórz plik implementacji</i>	225

<i>Utwórz plik ze słowami kluczowymi</i>	226
<i>Utwórz folder z przykładami</i>	226
Testowanie biblioteki	227
Publikacja biblioteki	227
Podsumowanie	228
 Dodatek A. Podzespoły	 229
Płytki Arduino	229
Płytki rozszerzeń	229
Moduły	229
Dystrybutorzy	230
 Skorowidz	 233

O autorze

Dr Simon Monk (Preston, Wielka Brytania) jest inżynierem cybernetykiem i informatykiem. Posiada doktorat z zakresu inżynierii oprogramowania. Po kilku latach pracy na uczelni został współzałożycielem firmy Momote Ltd., zajmującej się produkcją aplikacji mobilnych. Autor już w dzieciństwie zaczął interesować się elektroniką. Obecnie Simon Monk zajmuje się głównie pisaniem książek dotyczących elektroniki i otwartego sprzętu takiego jak Raspberry Pi i Arduino. Jest również autorem wielu książek poruszających zagadnienia ogólnie związane z elektroniką, a wspólnie z Paulem Scherzem napisał także książkę *Practical Electronics for Inventors*.

Simona możesz śledzić na Twitterze, gdzie występuje jako @simonmonk2.

Podziękowania

Bardzo dziękuję wszystkim pracownikom wydawnictwa McGraw-Hill Education za pracę umożliwiającą wydanie tej książki. Szczególne podziękowania kieruję do redaktora Rogera Stewarta, a także Vastavikty Sharmy, Jody McKenzie, LeeAnn Pickrell i Claire Splan.

Dziękuję również firmom Adafruit, SparkFun i CPC za dostarczenie wielu modułów i komponentów, z których korzystałem podczas pracy nad książką.

Na koniec chciałbym ponownie podziękować Lindzie za cierpliwość i hojne obdarowanie mnie przestrzenią, która umożliwiła mi napisanie tej książki.

Wstęp

Arduino stało się standardowym mikrokontrolerem stosowanym przez twórców, artystów i naukowców. A to dlatego, że mikrokontroler ten jest łatwy w użyciu, tani i posiada wiele dostępnych płytek interfejsowych (tzw. shieldów). Płytki te mogą być podłączane bezpośrednio do Arduino. Umożliwiają one dostęp do internetu, sterowanie robotami i obsługę domowej automatyki.

Proste projekty oparte na Arduino są łatwe do wykonania. Gdy jednak tylko dojdiesz do zagadnień nieopisanych w tekstach wprowadzających użytkownika w świat Arduino, okaże się, że wszystko staje się nagle tak skomplikowane, że aż frustrujące. Komplikacje są wrogiem wszystkich programistów. Zacznijmy z nimi walczyć.

Niniejsza książka stanowi uzupełnienie i dalszy ciąg pozycji *Arduino dla początkujących. Podstawy i szkice*. Przypomnę Ci podstawowe wiadomości dotyczące programowania Arduino, skupię się jednak na bardziej zaawansowanych zagadnieniach. Książka, którą trzymasz w rękach, pomoże Ci:

- pracować efektywnie, korzystając z minimalnej ilości pamięci;
- wykonywać jednocześnie więcej niż jedną operację bez konieczności korzystania z wielowątkowości;
- przekształcać kod na biblioteki, z których mogliby korzystać inni;
- stosować przerwania sprzętowe i zegarowe;
- maksymalizować wydajność;
- minimalizować pobór prądu;
- korzystać z różnych magistrali służących do przesyłu danych (I2C, 1-Wire, SPI i standardowej szeregowej transmisji danych);
- korzystać z interfejsu USB;
- korzystać z sieci;
- przetwarzać sygnały cyfrowe.

Pobieranie przykładów

W książce znajdziesz około 75 przykładowych szkiców będących wolnym oprogramowaniem. Można je pobrać ze strony internetowej książki: <http://www.helion.pl/ksiazki/arpokk.htm>. Pod tym adresem znajdziesz wszystkie szkice, a także ewentualną erratę.

Czego będę potrzebował?

Niniejsza książka dotyczy głównie oprogramowania, a więc działanie większości przykładów będziesz mógł sprawdzić za pomocą Arduino i diod LED lub multimetru. Jeżeli posiadasz płytki zwiększające możliwości Arduino, to mogą Ci się one również przydać. W rozdziale 12. omówiono programy korzystające z płytek obsługujących sieci Ethernet i Wi-Fi. Ponadto w książce zastosowano kilka różnych modułów w celu zademonstrowania działania różnych interfejsów.

Książka dotyczy głównie płytki Arduino Uno (jest to najczęściej stosowana płytka Arduino), ale poruszono w niej również zagadnienia dotyczące specjalnych funkcji płytek takich jak Leonardo i Arduino Due — programowania interfejsu USB i cyfrowego przetwarzania sygnałów.

W dodatku znajdującym się na końcu książki umieszczono listę przykładowych dostawców komponentów.

Korzystanie z tej książki

Każdy z rozdziałów dotyczy jakiegoś zagadnienia związanego z programowaniem Arduino. Wyjątkiem jest rozdział 1., w którym przypominam podstawowe wiadomości dotyczące Arduino. Pozostałe rozdziały możesz czytać w dowolnej kolejności.

Jeżeli jesteś programistą posiadającym doświadczenie w pracy z innymi językami, to lekturę książki możesz rozpocząć od rozdziału 14. Pozwoli Ci to na umieszczenie programowania Arduino w pewnym kontekście.

Poniżej znajduje się lista rozdziałów wraz ze skrótowym opisem ich zawartości.

1. „Programowanie Arduino”. W tym rozdziale zamieszczono streszczenie podstawowych wiadomości dotyczących programowania Arduino. Od tego rozdziału lekturę książki powinny rozpocząć osoby chcące szybko nadrobić braki w podstawach Arduino.
2. „Pod maską”. W tym rozdziale przyjrzymy się działaniu oprogramowania Arduino oraz omówimy jego genezę.
3. „Przerwania i zegary”. Początkujący programiści boją się korzystać z przerwań. W praktyce są one bardzo przydatne w niektórych sytuacjach i ich zaprogramowanie wcale nie jest takie trudne. Trzeba jednak uważać na pewne pułapki, które również zostaną przedstawione w tym rozdziale.

4. „Przyspieszanie Arduino”. Procesor Arduino jest wolny i pobiera bardzo mało prądu, czasem jednak trzeba wycisnąć z niego wyższą wydajność. Na przykład wbudowana funkcja `digitalWrite` jest bezpieczna i łatwa w użyciu, nie jest jednak zbyt wydajna, zwłaszcza gdy stosuje się ją do jednoczesnej obsługi wielu wyjść. W tym rozdziale przyjrzymy się sposobom zwiększania wydajności pracy Arduino i przedstawimy techniki tworzenia bardziej wydajnych szkiców.
5. „Arduino i mały pobór prądu”. Zasilając Arduino z baterii lub ogniw fotowoltaicznych, musisz zadbać o ograniczenie poboru prądu przez mikrokontroler. Poza optymalizacją sprzętu możesz również zmodyfikować kod szkicu pod kątem redukcji prądu pobieranego przez Arduino.
6. „Pamięć”. W tym rozdziale zajmiemy się optymalizacją użycia pamięci, a także omówimy korzyści i zagrożenia związane z dynamicznym użyciem pamięci przez szkice.
7. „Korzystanie z magistrali I2C”. Interfejs I2C znacznie ułatwia komunikację z modułami i komponentami, zmniejszając przy tym liczbę pinów niezbędnych do komunikacji. W tym rozdziale znajdziesz informacje na temat tego, jak działa interfejs I2C, oraz tego, jak należy z niego korzystać.
8. „Praca z urządzeniami wyposażonymi w interfejs 1-Wire”. W tym rozdziale znajdziesz informacje o urządzeniach korzystających z magistrali 1-Wire. Przykładami takich urządzeń są czujniki temperatury firmy Dallas Semiconductor — są one bardzo często stosowane w połączeniu z mikrokontrolerem Arduino. Dowiesz się, jak działa ten protokół komunikacyjny. Ponadto nauczysz się z niego korzystać.
9. „Praca z urządzeniami wyposażonymi w interfejs SPI”. Kolejnym opisanym standardem interfejsu jest SPI. Dowiesz się, jak działa ten interfejs, oraz nauczysz się nim posługiwać.
10. „Szeregowa transmisja danych za pośrednictwem układu UART”. Szeregowa transmisja danych poprzez port USB lub piny Rx i Tx pozwala na łatwą komunikację między Arduino a urządzeniami peryferyjnymi, a także między kilkoma mikrokontrolerami Arduino. W tym rozdziale znajdziesz informacje o tym, jak należy korzystać z szeregowej transmisji danych.
11. „Obsługa interfejsu USB”. W tym rozdziale opisano różne zagadnienia związane z użytkowaniem Arduino wyposażonego w złącze USB. Dowiesz się, jak można emulować mysz lub klawiaturę za pomocą płytki Arduino Leonardo. Znajdziesz również informacje na temat tego, jak można podłączyć klawiaturę USB lub mysz do odpowiednio przygotowanego Arduino.
12. „Obsługa sieci”. Arduino jest mikrokontrolerem często podłączanym do sieci. W tym rozdziale znajdziesz informacje dotyczące tego, jak zaprogramować Arduino, aby działało w sieci Internet. Omówiono tutaj zagadnienia związane z płytkami umożliwiającymi pracę w sieci Wi-Fi i Ethernet. Dowiesz się również tego, jak należy korzystać z usług sieciowych, a także jak zamienić swoje Arduino w miniaturowy serwer sieci Web.

13. „Cyfrowe przetwarzanie sygnałów”. Arduino potrafi przetwarzać sygnały w dość podstawowym zakresie. W tym rozdziale zamieszczono informacje na temat różnych technik, takich jak filtrowanie sygnałów analogowych za pomocą oprogramowania, bez potrzeby stosowania zewnętrznej elektroniki do określania siły sygnałów o różnych częstotliwościach. W tym celu będziemy posługiwać się szybkim przekształceniem Fouriera.
14. „Praca z użyciem tylko jednego procesu”. Programiści, którzy wcześniej zajmowali się programowaniem bardziej złożonych systemów, często zwracają uwagę na to, że Arduino nie obsługuje wielowątkowości. Postarałem się wyjaśnić i zademonstrować techniki związane z programowaniem jednowątkowych układów wbudowanych.
15. „Tworzenie bibliotek”. Prędzej czy później stworzysz coś na tyle dobrego, że będziesz uważać to za przydatne również dla innych. Swój kod możesz umieścić w bibliotece, którą możesz udostępnić całemu światu. W tym rozdziale dowiesz się, jak to zrobić.

Rozdział 1.

Programowanie Arduino

W rozdziale tym znajdziesz podsumowanie podstawowych wiadomości dotyczących Arduino. Jeżeli Arduino jest dla Ciebie czymś zupełnie nowym, to warto, abyś zapoznał się z książką *Arduino dla początkujących. Podstawy i szkice* (Helion, 2014).

Czym jest Arduino

Termin ten jest używany w odniesieniu zarówno do płytki mikrokontrolera Arduino (najpopularniejszą płytką jest Arduino Uno), jak i do całości systemu Arduino. W skład systemu wchodzi również oprogramowanie, które należy uruchomić na komputerze w celu zaprogramowania płytki, a także peryferyjne płytki, które można podłączać do mikrokontrolera Arduino.

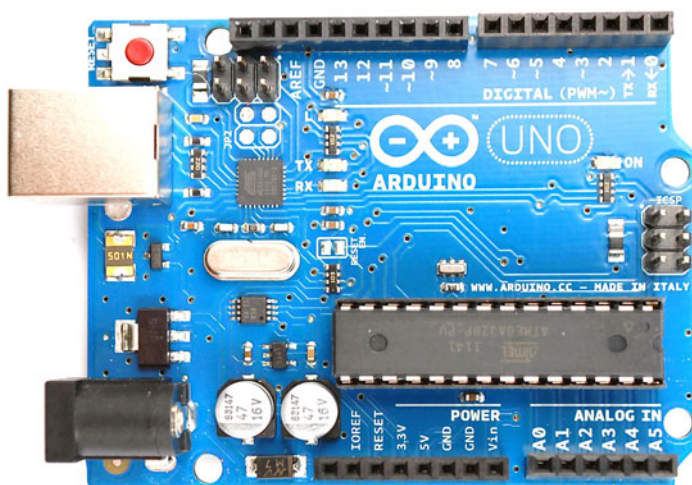
Do korzystania z Arduino będziesz również potrzebował „odpowiedniego” komputera. Powinien być to komputer działający pod kontrolą systemu OS X, Windows lub Linux. Może być to nawet coś tak skromnego, jak Raspberry Pi. Komputer jest Ci potrzebny głównie do ładowania programów do pamięci mikrokontrolera. Po zainstalowaniu będą one uruchamiane samodzielnie na Arduino.

Na rysunku 1.1 znajduje się płytka Arduino Uno.

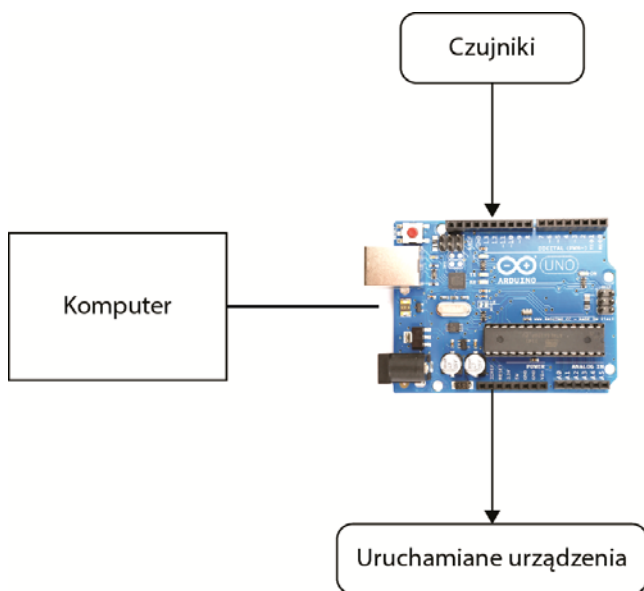
Arduino może komunikować się z komputerem za pośrednictwem interfejsu USB. Komunikaty między komputerem a Arduino mogą być przesyłane w obu kierunkach. Na rysunku 1.2 przedstawiono zależności między komputerem a Arduino.

Arduino, w przeciwieństwie do większości komputerów, posiada bardzo mało pamięci i nie jest wyposażone w żaden system operacyjny. Nie ma klawiatury, myszy ani ekranu. Arduino ma sterować pracą różnych komponentów na podstawie sygnałów generowanych przez czujniki. Na przykład do mikrokontrolera możesz podłączyć czujnik mierzący temperaturę, a także przekaźnik sterujący pracą grzejnika.

Na rysunku 1.3 wymieniono niektóre z elementów, jakie możesz podłączyć do płytki Arduino. Oczywiście do tego mikrokontrolera możesz podłączyć również wiele innych urządzeń.



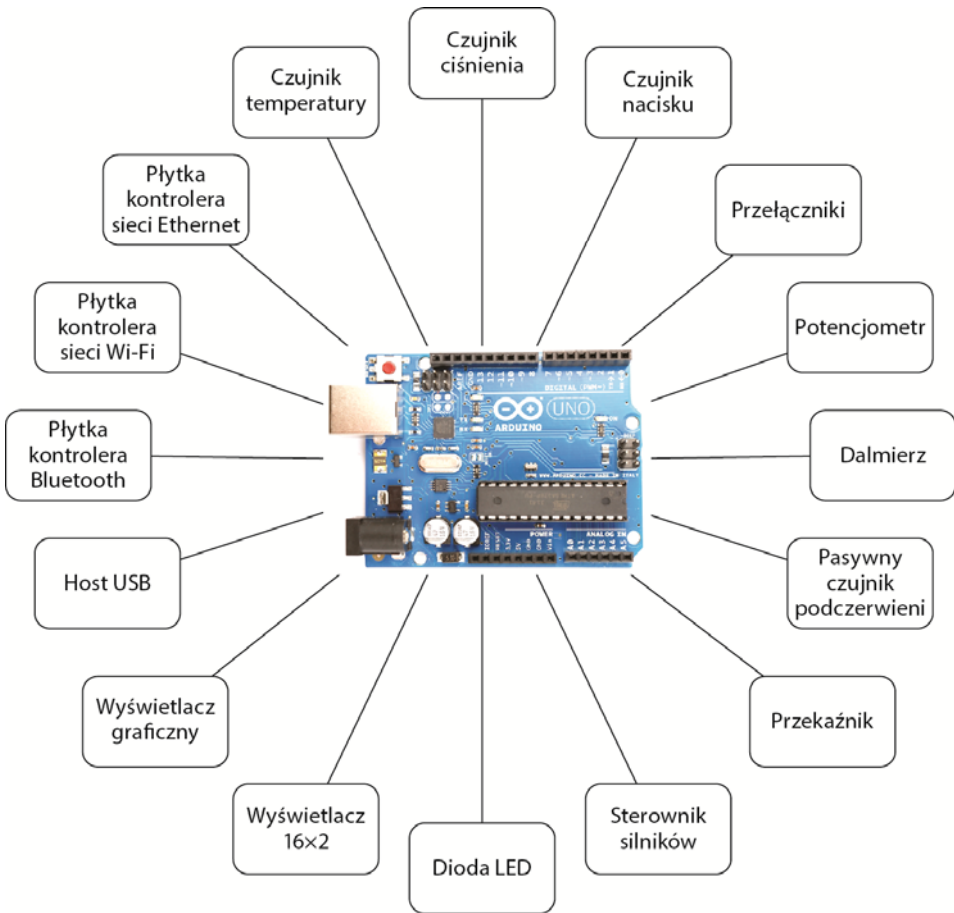
Rysunek 1.1. Arduino Uno



Rysunek 1.2. Arduino i Twój komputer

Na poniższej liście umieściłem tylko kilka wybranych, ciekawych projektów zbudowanych na bazie Arduino:

- Bublino — maszyna generująca bańki mydlane, gdy ktoś otaguje ją na Twitterze,
- trójwymiarowe kwadraty zbudowane z diod LED,
- liczniki Geigera,
- instrumenty muzyczne,



Rysunek 1.3. Komponenty, które można podłączyć do Arduino

- zdalne czujniki,
- roboty.

Instalacja i środowisko programistyczne

Aplikacja stosowana do programowania mikrokontrolera Arduino nosi nazwę **zintegrowanego środowiska programistycznego Arduino**. Jeżeli jesteś programistą przyzwyczajonym do pracy w rozbudowanych środowiskach programistycznych, takich jak Eclipse lub Visual Studio, to środowisko Arduino wyda Ci się bardzo proste. Może brakować Ci funkcji takich jak zintegrowane repozytorium czy funkcja automatycznego uzupełniania poleceń. Jeżeli jesteś dopiero początkującym programistą, to pokochasz prostotę pracy w tym środowisku.

Instalacja środowiska programistycznego

Zacznij od pobrania z oficjalnej strony Arduino oprogramowania w wersji właściwej dla Twojego systemu operacyjnego: <http://arduino.cc/en/Main/Software>.

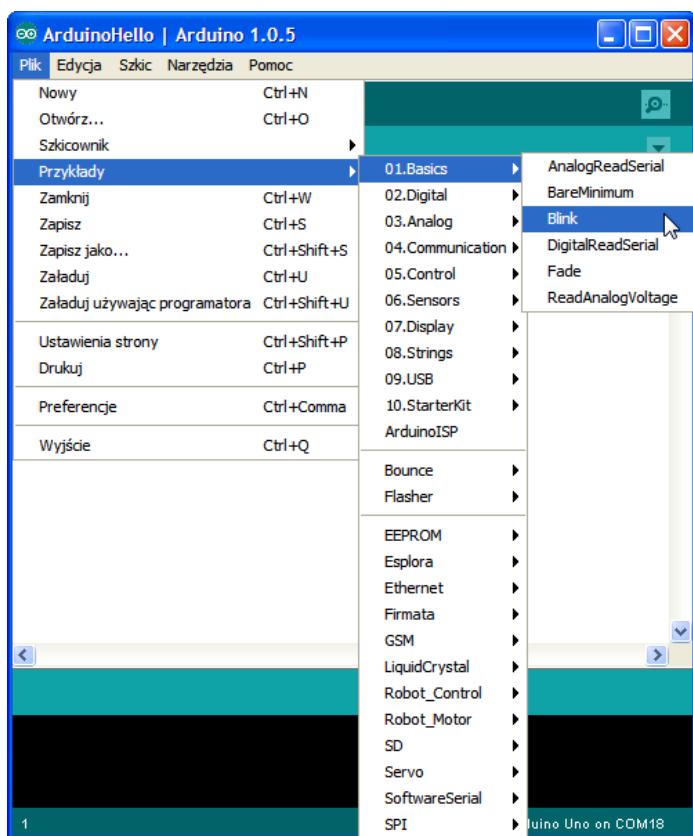
Po pobraniu oprogramowania możesz zapoznać się z dokładnymi (anglojęzycznymi) instrukcjami jego instalacji na różnych platformach: <http://arduino.cc/en/Guide/HomePage>.

Jedną z zalet Arduino jest to, że do rozpoczęcia pracy będziesz potrzebował tylko Arduino, komputera i przewodu USB łączącego te dwa urządzenia. Arduino może być nawet zasilane prądem dostarczany przez komputer za pośrednictwem gniazda USB.

Blink

Aby sprawdzić, czy Arduino działa, zaprogramujemy je tak, aby mikrokontroler migał za pośrednictwem znajdującej się na nim diody LED, oznaczonej literą L. Dioda ta jest często określana jako „dioda L”.

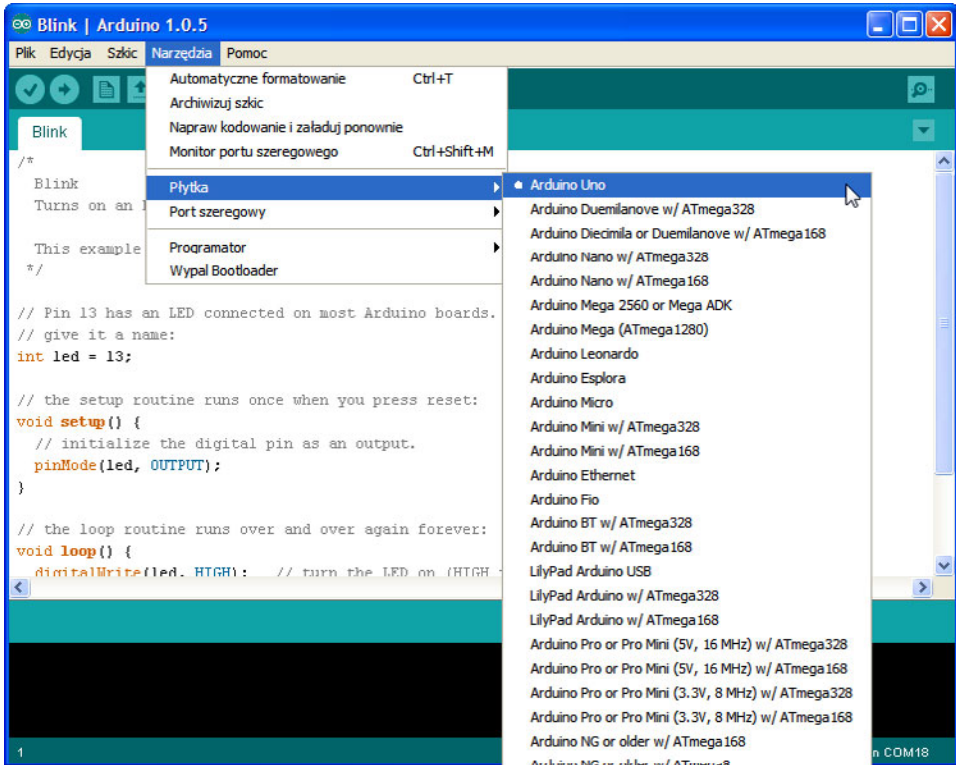
Uruchom środowisko programistyczne Arduino na swoim komputerze, a następnie z menu *Plik* wybierz *Przykłady/01.Basics/Blink* (zobacz rysunek 1.4).



Rysunek 1.4. Wybór szkicu Blink w środowisku programistycznym Arduino

Aby programowanie Arduino wydawało się mniej zniechęcające dla osób niebędących programistami, programy Arduino określa się mianem **szkiców**.

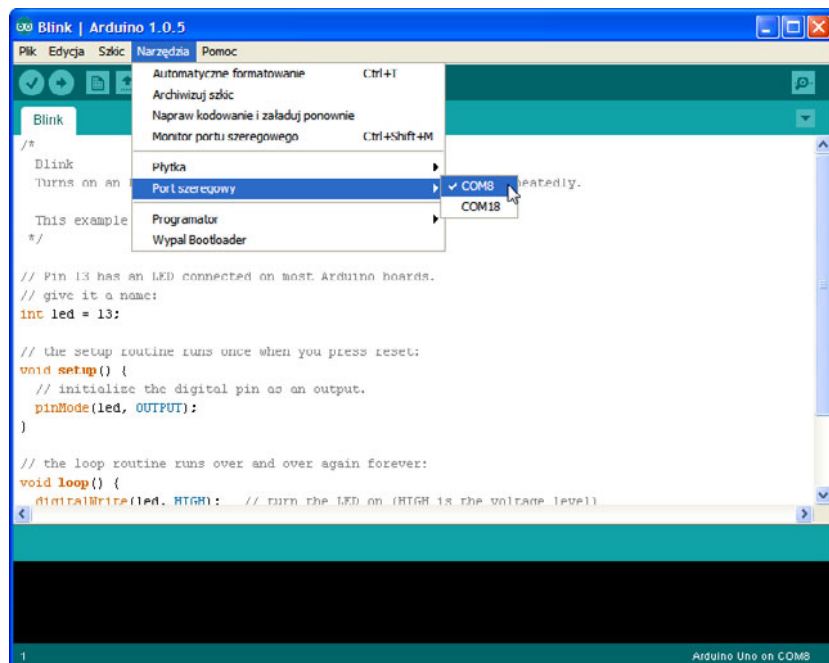
Zanim prześlesz szkic *Blink* do swojego Arduino, musisz określić typ płytki, z jakiej korzystasz. Najczęściej spotykaną płytką jest Arduino Uno. W tym rozdziale zakładam, że korzystasz właśnie z niej. Z menu *Narzędzia* wybierz podmenu *Płytki*, a z niego wybierz opcję *Arduino Uno* (zobacz rysunek 1.5).



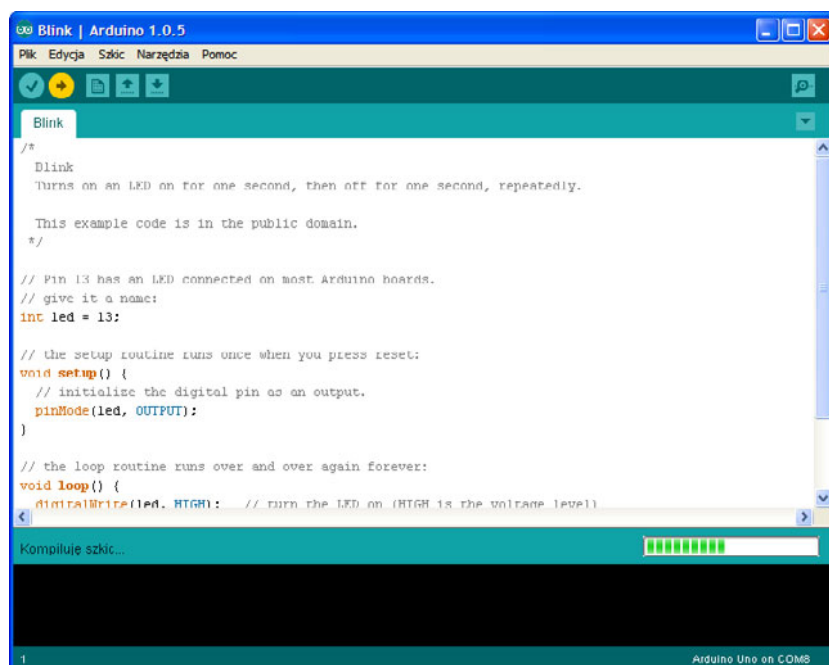
Rysunek 1.5. Wybór płytki

Po wybraniu płytki musisz określić port, do którego podłączone jest Arduino. W systemie Windows jest to dość proste, ponieważ płytka komunikuje się z komputerem zwykle za pośrednictwem portu COM4 i najczęściej jest to jedyny dostępny port na liście (zobacz rysunek 1.6). W systemach OS X i Linux lista urządzeń podłączonych do portów szeregowych będzie zwykle dłuższa. Środowisko programistyczne Arduino umieszcza ostatnio podłączone urządzenia w górnej części listy, a więc Twoja płytka Arduino powinna znajdować się właśnie tam.

Aby załadować szkic do pamięci Arduino, należy kliknąć na znajdujący się w pasku narzędzi przycisk *Załaduj*. Jest to drugi przycisk we wspomnianym pasku — podświetlony na rysunku 1.7.



Rysunek 1.6. Wybór portu szeregowego



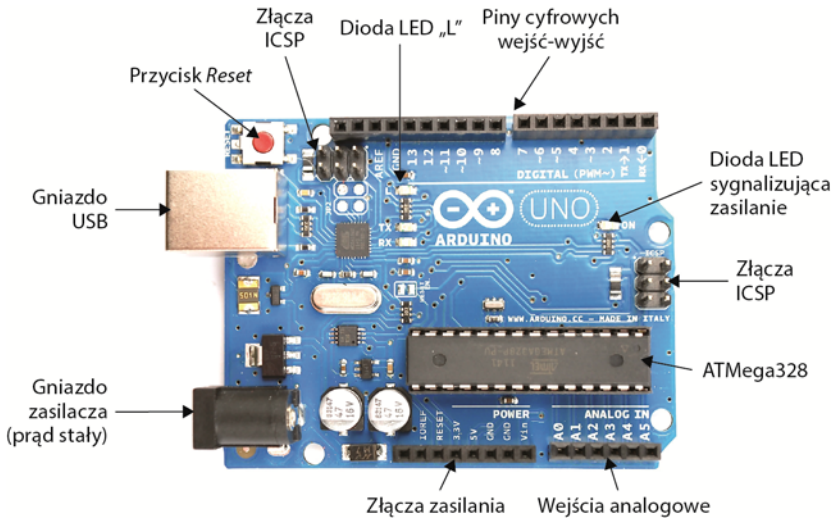
Rysunek 1.7. Ładowanie szkicu Blink

Po tym, gdy zostanie kliknięty przycisk *Załaduj*, środowisko programistyczne wykona kilka rzeczy. W oknie wyświetli się pasek informujący o postępie operacji kompilowania szkicu (szkic jest przetwarzany na formę pozwalającą na zapisanie go w pamięci Arduino). Następnie diody znajdujące się na płytce Arduino, oznaczone etykietami Rx i Tx, powinny migać przez chwilę. Na koniec powinna zacząć migać dioda oznaczona literą L. W oknie środowiska programistycznego zostanie wyświetlony komunikat o treści podobnej do: „Wielkość binarna szkicu: 1084 bajty (maksymalnie 32 256 bajtów)”. Informuje on o tym, że szkic zajmuje około 1 kB z 32 kB pamięci flash dostępnej dla programów.

Zanim zaczniesz programować, przyjrzyj się sprzętowi, na którym uruchamiane będą Twoje programy lub szkice.

Wycieczka po płytce Arduino

Na rysunku 1.8 przedstawiono płytkę Arduino. W lewym górnym rogu płytki, obok gniazda USB, znajduje się przycisk *Reset*. W wyniku jego wciśnięcia nastąpi wysłanie impulsu logicznego do pinu *Reset* znajdującego się na mikrokontrolerze. Impuls ten spowoduje wyczyszczenie pamięci mikrokontrolera i ponowne uruchomienie programu. Programy są przechowywane w **nieulotnej** pamięci flash — zawartość tej pamięci nie ulega skasowaniu na skutek odłączenia prądu od urządzenia.



Rysunek 1.8. Anatomia płytki Arduino

Zasilanie

Arduino może być zasilane za pośrednictwem złącza USB lub znajdującego się pod nim gniazda zasilacza. Płytkę może być zasilana za pomocą zasilacza sieciowego lub baterii.

Prąd stały dostarczany przez gniazdo zasilacza może mieć dowolne napięcie znajdujące się w przedziale od 7,5 V do 12 V. Sama płytki Arduino pobiera prąd o natężeniu około 50 mA, a więc mała bateria PP3 9 V (200 mAh) będzie ją zasilac przez około 40 godzin.

Podłączenie Arduino do prądu jest sygnalizowane przez pracę diody LED znajdującej się w prawej części płytki Uno. W przypadku płytki Leonardo dioda ta znajduje się po lewej stronie.

Złącza zasilania

Teraz przyjrzyjmy się złączom znajdującym się w dolnej części rysunku 1.8. Wszystkie (poza pierwszym) zostały oznaczone etykietami.

Pierwsze, nieoznakowane złącze nie posiada jeszcze żadnej funkcji (jest ono zarezerwowane). Kolejny pin — *IOREF* — określa napięcie, przy którym pracuje Arduino. Płytki Uno i Leonardo pracują pod napięciem 5 V, a więc na tym pinie zawsze będzie znajdował się potencjał 5 V. Nie będziemy jednak korzystać ze wspomnianego pinu, pracując nad projektami opisanymi w niniejszej książce. Pin ten pozwala płytkom rozszerzeń podłączonym do płytek Arduino zasilanych prądem o napięciu 3 V (takich jak Arduino Due) na określenie napięcia prądu, pod jakim pracuje Arduino.

Kolejne złącze oznaczono etykietą *RESET*. Podanie odpowiedniego sygnału na to złącze da taki sam efekt jak wciśnięcie przycisku *Reset* znajdującego się na płytce Arduino. Jego działanie można porównać do ponownego uruchomienia komputera — mikrokontroler zostaje zresetowany, a wykonywanie programu rozpoczyna się od początku. Aby uruchomić ponownie Arduino, wspomniane złącze należy na chwilę zewrzeć z masą. Prawdopodobnie nie będziesz korzystać z tego pinu, ale warto znać jego funkcję.

Pozostałe piny służą do zasilania prądem o różnych potencjałach, zgodnie z etykietami: 3,3 V, 5 V, 0 V (GND) i 9 V. Złącze oznaczone etykietą GND jest złączem masy, czyli jest to potencjał zerowy. To napięcie odniesienia dla wszystkich pozostałych napięć na płytce Arduino.

Oba złącza oznaczone etykietą GND są identyczne. Warto dysponować więcej niż jednym złączem masy. Tak naprawdę w górnej części płytki znajduje się jeszcze jedno gniazdo masy.

Wejścia analogowe

Kolejna sekcja złączy jest oznaczona etykietą *ANALOG IN* i numerami od 0 do 5. Te sześć pinów może być używanych do pomiaru napięcia podawanego na nie prądu. Wartość uzyskana na drodze pomiaru może być później użyta w szkicu. Pomimo tego, że piny te są wejściami analogowymi, można je również stosować jako wejścia lub wyjścia cyfrowe. Po prostu standardowo stosuje się je jako wejścia analogowe.

Złącza cyfrowe

Teraz zajmijmy się złączami znajdującymi się u góry płytki. Zaczniemy od jej prawej strony (zobacz rysunek 1.8). Znajdują się tam piny oznaczone etykietą *DIGITAL* i numerami od 0 do 13. Piny te mogą działać jako cyfrowe wejścia i cyfrowe wyjścia. Gdy działają jako wyjścia,

wtedy ich pracą można sterować za pośrednictwem szkicu. Jeżeli zostaną włączone, to pojawi się na nich napięcie 5 V, a jeżeli zostaną wyłączone — 0 V. Podobnie jak w przypadku złączy zasilających musisz uważać, aby ich nie przeciążyć.

Przez te złącza może płynąć prąd o natężeniu 40 mA przy napięciu 5 V. Taki prąd może z powodzeniem zasilac standardową diodę LED, ale jest to zbyt niskie natężenie, aby zasilac bezpośrednio silnik elektryczny.

Płytki Arduino

Arduino Uno (zobacz rysunek 1.1) jest obecnym wcieleniem oryginalnej płytki Arduino. Jest to najpopularniejsza płytka Arduino i zasadniczo to właśnie o niej myśli większość osób, gdy mówi, że korzysta z Arduino.

Wszystkie inne płytki Arduino powstały w celu spełnienia pewnych wymagań, takich jak potrzeba posiadania większej liczby złączy wejścia-wyjścia, konieczność szybszego działania mikrokontrolera, zapotrzebowanie na płytkę tak małą, że można ją przyczepić do ubrania, podłączyć do telefonu działającego pod kontrolą systemu Android lub zintegrować z urządzeniem bezprzewodowym.

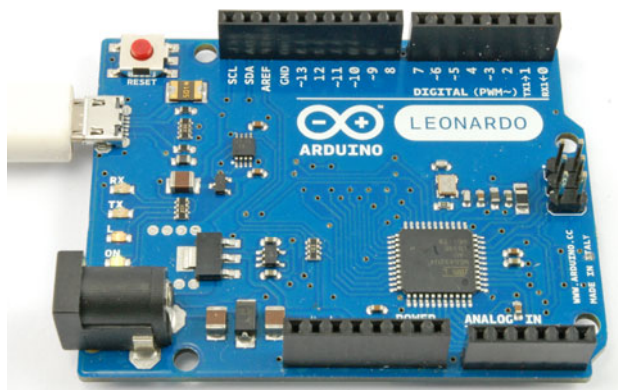
Niezależnie od różnic sprzętowych każdą z płytek można zaprogramować za pomocą środowiska programistycznego Arduino. Istnieją tylko niewielkie różnice między funkcjami oprogramowania, które możesz stosować w swoich szkicach. Jeżeli nauczysz się korzystać z jednej płytki Arduino, to będziesz w stanie obsłużyć wszystkie inne płytki.

Przyjrzyjmy się dostępnemu asortymentowi oficjalnych płytek Arduino. Istnieją również płytki, które nie zostały omówione w tym rozdziale, nie są jednak tak popularne jak te, które Ci przedstawiam. Pełną listę płytek znajdziesz w oficjalnej witrynie internetowej Arduino (www.arduino.cc).

Uno i pochodne

Uno R3 należy do najnowszej serii „standardowych” płytek — w jej skład wchodzi: standardowe Uno, Duemilanove, Diecimila i NG. Na płytkach tych znajduje się mikroprocesor ATmega168 lub ATmega328. Procesory te nie różnią się praktycznie niczym poza ilością dostępnej pamięci.

Inną płytką Arduino, o takich samych wymiarach i takiej samej konfiguracji złączy jak Uno R3, jest Arduino Leonardo (zobacz rysunek 1.9). Jak widzisz, na płytce tej znajduje się mniej komponentów. Wynika to z faktu, że na płytce Leonardo zastosowano inny procesor — ATmega32u4, który jest podobny do procesora ATmega328, ale ma wbudowany kontroler obsługujący interfejs USB. Tak więc na płytce tej nie trzeba było instalować dodatkowych komponentów, które musiały znaleźć się na płytce Uno. Ponadto płytka Leonardo posiada znacznie większą ilość pamięci, więcej wejść analogowych i kilka innych ulepszeń. Jest tańsza od Uno. Pod wieloma względami płytka Leonardo jest lepsza od Uno.



Rysunek 1.9. Arduino Leonardo

Możesz się zastanawiać, dlaczego Leonardo nie jest tak popularną płytką jak Arduino Uno. Dzieje się tak, ponieważ płytka Leonardo została ulepszona kosztem mniejszej kompatybilności z Uno i poprzednikami tej płytki. Niektóre płytki rozszerzeń (zwłaszcza starsze) nie będą współpracować z Leonardo. Z czasem będzie to coraz mniejszy problem, a wtedy Leonardo i jego kolejne wersje mogą stać się popularniejsze.

Czymś względnie nowym w rodzinie Arduino jest płytka Arduino Ethernet. Jest to standardowa płytka Uno wyposażona dodatkowo w interfejs sieci Ethernet, który pozwala na podłączenie jej do sieci bez zewnętrznej płytki kontrolera Ethernet.

Duże płytki Arduino

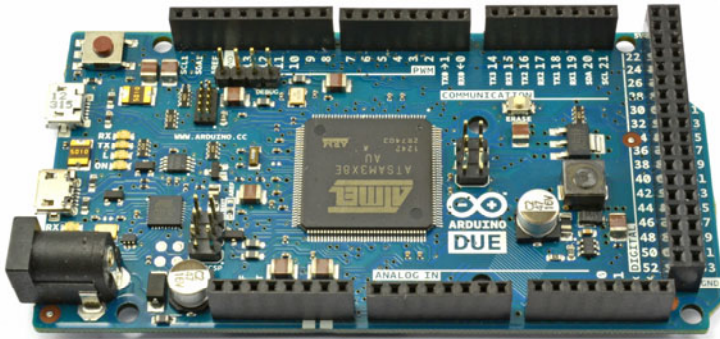
Czasami płytki Uno lub Leonardo mogą po prostu ograniczać nas — możemy znaleźć się w sytuacji, która wymaga zastosowania większej liczby pinów wejścia-wyjścia. Możemy wtedy rozszerzyć sprzętowo płytkę Uno lub zastosować zupełnie inną, większą płytkę.

WSKAZÓWKA

Jeżeli dopiero zaczynasz swoją przygodę z Arduino, to nie kupuj żadnej z tych większych płytek. Ich zakup może wydawać się czymś kuszącym, ponieważ są one większe i szybsze, ale niestety charakteryzują je problemy z kompatybilnością z płytkami rozszerzeń; tak więc znacznie lepiej byłoby, gdybyś kupił „standardową” płytkę Arduino Uno.

Duże płytki Arduino mają ten sam zestaw złączy co płytka Uno, ale posiadają dodatkowe rzędy pinów wejścia-wyjścia umieszczone wzdłuż krawędzi płytki (zobacz rysunek 1.10).

Tradycyjnie, mówiąc o „większej” płytce, ma się na myśli Arduino Mega 2560. Większe płytki Arduino wyposażone są w więcej dostępnej pamięci (wszystkich typów). Procesory płytek Mega 2560 i Mega ADK posiadają moc obliczeniową podobną do procesora Arduino Uno. Arduino Due charakteryzuje się jednak znacznie wyższą mocą obliczeniową. Procesor tej płytki jest taktowany zegarem 84 MHz (dla porównania procesor Arduino Uno był taktowany



Rysunek 1.10. Arduino Due

zegarem o częstotliwości 16 MHz). Niestety powoduje to problemy z kompatybilnością. Największa z płytek — Due działa pod napięciem 3,3 V, a większość wcześniejszych płytek Arduino działała pod napięciem 5 V. Chyba nikogo nie dziwi to, że wiele płytek rozszerzeń Arduino nie jest z nią kompatybilnych.

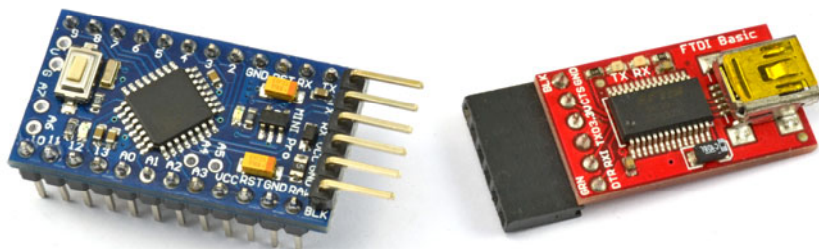
Płytką ta posiada jednakże wiele rzeczy przydatnych podczas pracy z wymagającymi projektami:

- dużą ilość pamięci przeznaczonej dla programu i danych,
- możliwość sprzętowego generowania dźwięku (sprzętowy przetwornik cyfrowo-analogowy),
- cztery porty szeregowy,
- dwa porty USB,
- host USB OTG,
- emulację klawiatury i myszy USB.

Małe płytki Arduino

Płytką Uno czasami może być zbyt mała, ale z kolei w niektórych projektach może okazać się zbyt duża. Płytki Arduino są co prawda dość tanie, ale pozostawianie płytki jako części każdego projektu może stać się dość kosztowne. Istnieje wiele małych płytek Arduino i płytek oznaczonych przedrostkiem „pro”. Są one zaprojektowane tak, aby były mniejsze od standardowej płytki Uno, lub tak, aby były od niej tańsze — usunięto z nich pewne funkcje, które nie są przydatne podczas pracy nad większością projektów.

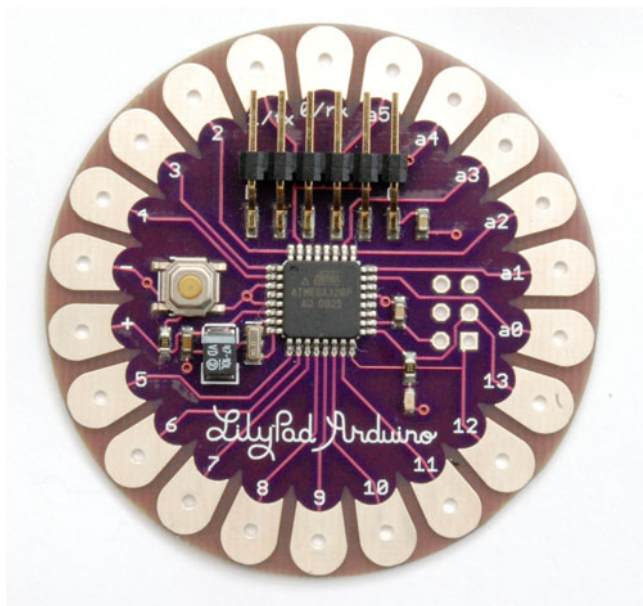
Na rysunku 1.11 przedstawiono płytki Arduino Mini. Nie posiadają one interfejsu USB — żeby je zaprogramować, niezbędny jest specjalny zewnętrzny moduł. Poza płytkami Mini są również Nano i Micro. Płytki te mogą mieć wbudowany kontroler USB, ale są wtedy droższe.



Rysunek 1.11. *Płytki Arduino Mini i programator*

Płytki LilyPad i LilyPad USB

Jedną z najciekawszych stylistycznie płytek Arduino jest płytka LilyPad (zobacz rysunek 1.12), a także jej nowsza wersja — LilyPad USB. Płytki te są zaprojektowane tak, aby można było przyszyć je do ubrania za pomocą przewodzących nici łączących je z modułami zawierającymi diody LED, przełączniki, przyspieszeniomierze itd. Starsze płytki LilyPad wymagały korzystania z zewnętrznego interfejsu USB (takiego samego, jak ten wymagany przez płytkę Arduino Mini). Płytki te są jednak stopniowo wypierane przez płytki Arduino LilyPad USB, które posiadają wbudowane złącze USB.



Rysunek 1.12. *Płytki Arduino LilyPad*

Nieoficjalne płytki Arduino

Poza opisanymi oficjalnymi płytkami istnieje wiele nieoficjalnych kopii i modyfikacji płytek Arduino. Dzieje się tak, gdyż Arduino jest sprzętem otwartym. W serwisach aukcyjnych typu eBay i Allegro można znaleźć wiele tanich kopii oficjalnych płytek Arduino. Są one interesujące tylko z powodu swojej ceny. Istnieją również inne płytki Arduino, które są interesujące z powodu tego, że oferują pewne ciekawe, dodatkowe funkcje.

Warto przyjrzeć się dwóm przykładom płytek tego typu:

- **EtherTen** — płytka Arduino wyposażona w interfejs Ethernet (<http://www.freetronics.com/products/etherten>)
- **Leostick** — odchudzona wersja płytki Leonardo wyposażona w gniazdo USB (<http://www.freetronics.com/collections/arduino/products/leostick>)

Teraz, gdy wiesz już trochę więcej na temat mikrokontrolera Arduino, nadszedł czas, żebyś zaczął go programować.

Język programowania

Często powtarzana, błędna informacja jest taka, że Arduino posiada swój własny język programowania. Tak naprawdę jest ono programowane w języku C, jednym z najstarszych języków programowania. Arduino można zaprogramować za pomocą łatwego w użyciu zestawu poleceń stosowanych w języku C.

Puryści powiedzieliby, że Arduino jest programowane w C++ (obiektywnej wersji języka C). Jest to prawda, ale dysponując tylko 1 kB lub 2 kB dostępnej pamięci, nie warto korzystać z technik obiektowych, chociaż techniki te przydadzą się w pewnych wyspecjalizowanych obszarach. Poza nimi będziemy efektywnie programować w języku C.

Zacznijmy od zmodyfikowania szkicu *Blink*.

Modyfikacja szkicu Blink

Twoje Arduino może błyskać diodą LED od razu po podłączeniu do prądu. Dzieje się tak, ponieważ Arduino jest często fabrycznie programowane za pomocą szkicu *Blink*.

Jeżeli napotkałeś taki egzemplarz, a chcesz zrobić coś ze swoim Arduino, to możesz spróbować zmienić częstotliwość błysków. Przyjrzyjmy się szkicowi *Blink* i spróbujmy zmodyfikować go tak, aby dioda LED migała szybciej.

Pierwsza część szkicu jest komentarzem informującym czytelnika o tym, co robi ten szkic. Komentarz nie jest tak naprawdę kodem programu. Gdy kompilator przygotowuje program do załadowania do pamięci Arduino, usuwa z niego wszystkie komentarze. Wszystko to, co jest umieszczone pomiędzy znakami `/*` i `*/`, jest ignorowane przez komputer, ale powinno

być zrozumiałe dla programisty czytającego kod. Gdyby szkic *Blink* został stworzony przez osobę posługującą się językiem polskim, to jego początek wyglądałby następująco¹:

```
/*
  Blink
  Dioda LED jest wyłączana na jedną sekundę, a następnie włączana na kolejną sekundę. Czynności te są powtarzane.

  Ten przykładowy kod jest oprogramowaniem otwartym.
*/
```

W dalszej części szkicu znajdują się dwa komentarze mieszczące się w jednej linii kodu. Komentarze tego typu są podobne do komentarzy blokowych, ale rozpoczynają się od znaków `//`. Komentarze te informują Cię o tym, co się dzieje. W tym przypadku komentarz podpowiada Ci, że będziemy migać diodą LED podłączoną do pinu numer 13. Wybrano ten pin, dlatego że właśnie do niego podłączona jest dioda LED oznaczona literą *L* na płytce Arduino Uno.

```
// na większości płytek Arduino do pinu numer 13 podłączona jest dioda LED
// numer pinu jest przypisywany zmiennej
int led = 13;
```

Kolejną częścią szkicu jest funkcja `setup`. Każdy szkic Arduino musi mieć tę funkcję. Jest ona uruchamiana za każdym razem, gdy Arduino zostaje uruchomione, czy to na skutek podłączenia do prądu, czy też wciśnięcia przycisku *Reset*.

```
// funkcja setup jest uruchamiana raz po każdym wciśnięciu przycisku Reset
void setup() {
  // cyfrowe złącze będzie działać jako wyjście
  pinMode(led, OUTPUT);
}
```

Jeżeli zaczynasz swoją przygodę z programowaniem, to struktura tego kodu może wydawać Ci się dziwna. **Funkcja** jest częścią kodu, której nadano nazwę (w tym przypadku nazwą tą jest `setup`). Na razie traktuj przytoczony fragment kodu jako arkusz. Wystarczy, żebyś zrozumiał, że Twój szkic musi się rozpoczynać od linii `void setup() {`, po której należy wprowadzić polecenia mające zostać uruchomione. Każda linia poleceń musi kończyć się średnikiem (;). Koniec funkcji zaznaczamy symbolem `}`.

W przytoczonym fragmencie kodu Arduino uruchomi tylko polecenie `pinMode(led, OUTPUT)`, które sprawia, że pin będzie działał jako wyjście.

Teraz dochodzimy do głównej części szkicu — funkcji `loop`.

Każdy szkic Arduino musi zawierać funkcję `setup` i funkcję `loop`. Funkcja `setup` jest uruchamiana tylko raz po włączeniu Arduino, a funkcja `loop` jest uruchamiana w kółko. To znaczy, gdy mikrokontroler skończy wykonywać znajdujące się w niej polecenia, wtedy zostanie ona uruchomiona ponownie.

W funkcji `loop` włączasz diodę LED za pomocą polecenia `digitalWrite(led, HIGH)`. Następnie program odczeka sekundę — uruchamiane jest polecenie `delay(1000)`. Wartość podana w nawiasie określa ilość czasu w milisekundach (1000 milisekund to 1 sekunda). Na-

¹ Dodane przeze mnie — *przyp. tłum.*

stępnie dioda LED jest wyłączana, a program odczekuje kolejną sekundę przed ponownym uruchomieniem całego procesu i zapaleniem diody LED.

```
// funkcja setup jest wykonywana ciągle, aż do wyłączenia Arduino
void loop() {
    digitalWrite(led, HIGH); // włącza diodę LED (argument HIGH sprawia, że do diody zostaje
    // podany prąd o wysokim napięciu)
    delay(1000); // mikrokontroler odczekuje sekundę
    digitalWrite(led, LOW); // dioda LED gaśnie (argument LOW sprawia, że napięcie
    // zasilającego ją prądu spada)
    delay(1000); // mikrokontroler odczekuje sekundę
}
```

Aby dioda LED migała szybciej, zmień występującą w dwóch miejscach liczbę 1000 na 200. Obie zmiany przeprowadzane są w funkcji loop, a więc funkcja ta powinna wyglądać następująco:

```
// funkcja setup jest wykonywana ciągle, aż do wyłączenia Arduino
void loop() {
    digitalWrite(led, HIGH); // włącza diodę LED (argument HIGH sprawia, że do diody zostaje
    // podany prąd o wysokim napięciu)
    delay(200); // mikrokontroler odczekuje sekundę
    digitalWrite(led, LOW); // dioda LED gaśnie (argument LOW sprawia, że napięcie
    // zasilającego ją prądu spada)
    delay(200); // mikrokontroler odczekuje sekundę
}
```

Jeżeli spróbujesz zapisać szkic przed jego załadowaniem, oprogramowanie Arduino przypomni Ci, że jest to jedynie przykładowy szkic przeznaczony tylko do odczytu. Zaofertuje Ci możliwość zapisania kopii tego programu, którą następnie możesz dowolnie modyfikować.

Nie musisz tego robić. Możesz załadować szkic bez jego wcześniejszego zapisywania. Jeżeli zdecydujesz się na zapisanie tego (lub każdego innego) szkicu, to następnie będziesz mógł go znaleźć w menu *Plik* dostępnym w podmenu *Szkieownik* zintegrowanego środowiska programistycznego Arduino.

Tak czy inaczej, kliknij przycisk *Załaduj*. Gdy proces ładowania zostanie zakończony, Arduino samoczynnie uruchomi się ponownie, a dioda LED powinna migać znacznie szybciej.

Zmienne

Zmienne przypisują liczbom nazwy. Tak naprawdę można je stosować w wielu innych celach, ale na razie ograniczmy się do tego jednego zastosowania.

Definiując zmienną w języku C, musisz określić jej typ. Na przykład liczby całkowite mogą być przechowywane przez zmienne typu `int` (skrót od *integer*). Zmienną o nazwie `delayPeriod` i wartości 200 zadeklarujesz za pomocą następującego polecenia:

```
int delayPeriod = 200;
```

Zwróć uwagę na to, że fraza `delayPeriod` jest nazwą, a więc pomiędzy słowami ją tworzącymi nie mogą znajdować się spacje. Istnieje konwencja mówiąca o tym, że nazwy zmiennych

zaczynają się od małej litery, a każde kolejne słowo znajdujące się w nazwie rozpoczyna się od wielkiej litery. Programiści nazywają tę notację *camel case* lub *bumpy case*.

Spróbujmy zmodyfikować szkic *Blink* tak, aby zamiast wpisywania na stałe wartości 200 czas opóźnienia był definiowany przez zmienną:

```
int led = 13;
int delayPeriod = 200;

void setup()
{
    pinMode(led, OUTPUT);
}

void loop()
{
    digitalWrite(led, HIGH);
    delay(delayPeriod);
    digitalWrite(led, LOW);
    delay(delayPeriod);
}
```

Wszędzie tam, gdzie wcześniej odnosiliśmy się do wartości 200, teraz odnosimy się do zmiennej `delayPeriod`. Jeżeli chcesz, aby dioda LED migała szybciej, wystarczy, że zmienisz w jednym miejscu wartość przypisaną zmiennej `delayPeriod`.

If

Normalnie linie kodu programu są wykonywane po kolei, bez żadnych wyjątków. Ale co, jeżeli nie chcesz tego? Co, jeżeli chcesz wykonać tylko pewien fragment programu, w sytuacji gdy spełnione są pewne warunki?

Na przykład chcesz, aby program wykonał pewną operację po wciśnięciu przez użytkownika przycisku podłączonego do Arduino. Kod takiego programu mógłby wyglądać następująco:

```
void setup()
{
    pinMode(5, INPUT_PULLUP);
    pinMode(9, OUTPUT);
}
void loop()
{
    if (digitalRead(5) == LOW)
    {
        digitalWrite(9, HIGH);
    }
}
```

W powyższym przykładzie warunkiem wykonania operacji (podanym po `if`) jest odczytanie z pinu numer 5 wartości `LOW` (pol. *niski*). Podwójny symbol równości `==` jest operatorem porównywania dwóch wartości. Łatwo pomylić go z operatorem przypisującym

zmiennej wartość. Instrukcja `if` mówi, że w sytuacji gdy warunek jest spełniony, zostaną wykonane polecenia znajdujące się wewnątrz nawiasów klamrowych. W niniejszym przypadku poleceniem tym jest podanie sygnału wysokiego (HIGH) na wyjście cyfrowe o numerze 9.

Jeżeli warunek ten nie zostanie spełniony, to Arduino będzie kontynuowało wykonywanie poleceń. W tym wypadku będzie to ponowne uruchomienie funkcji `loop`.

Pętle

Poza wykonywaniem pewnych poleceń po zaistnieniu określonych warunków często w programach niezbędne jest wielokrotne powtarzanie tych samych operacji. Operacje takie należy zapisać w **pętli**. Przykładem pętli jest funkcja `loop`, którą widziałeś w przytoczonym wcześniej szkicu *Blink*.

Czasami potrzebujesz, aby dana operacja została wykonana określoną ilość razy. W tym celu można zastosować polecenie `for`, które pozwala na użycie zmiennej będącej licznikiem. Spróbujmy napisać przykładowy szkic, który spowoduje dziesięciokrotne mignięcie diody LED. Później dowiesz się, dlaczego taka technika nie zawsze się sprawdza — na razie możemy ją zastosować bez problemów.

```
// 01_01_blink_10
int ledPin = 13;
int delayPeriod = 200;
void setup()
{
    pinMode(ledPin, OUTPUT);
}
void loop()
{
    for (int i = 0; i < 10; i++)
    {
        digitalWrite(ledPin, HIGH);
        delay(delayPeriod);
        digitalWrite(ledPin, LOW);
        delay(delayPeriod);
    }
}
```

UWAGA

To jest pierwszy pełny szkic przedstawiony w tej książce. Nazwa pliku zawierającego ten kod jest podana w komentarzu znajdującym się na samym początku kodu. Wszystkie szkice rozpoczynające się od takich komentarzy można pobrać ze strony <http://www.helion.pl/ksiazki/arpokk.htm>.

Aby zainstalować szkice w środowisku Arduino, musisz rozpakować pobrany plik do folderu Arduino znajdującego się w folderze Moje dokumenty. Wspomniany folder jest automatycznie tworzony podczas pierwszego uruchomienia środowiska programistycznego Arduino.

Polecenie `for` definiuje zmienną `i`, a następnie nadaje jej początkową wartość 0. Po średniku pojawia się zapis `i < 10`. Jest to warunek działania pętli. Pętla będzie wykonywana tak długo, aż zmienna `i` będzie miała wartość mniejszą od 10.

Ostatnią częścią polecenia `for` jest zapis `i++`. W języku C jest to skrótowy zapis instrukcji „`i = i + 1`”, czyli polecenia dodającego 1 do wartości zmiennej `i`. Wartość 1 jest dodawana do zmiennej `i` podczas każdego obiegu pętli. Taki zabieg sprawia, że działanie pętli kiedyś się skończy. Jeżeli do zmiennej `i` będziemy każdorazowo dodawać 1, to w pewnym momencie wartość przypisana do tej zmiennej będzie większa od 10.

Funkcje

Funkcje pozwalają zgrupować zbiór poleceń w użyteczny blok. Dzięki funkcjom możliwe jest podzielenie szkicu na bloki, którymi można łatwo zarządzać.

Napiszmy szkic, który sprawi, że dioda znajdująca się na płytce Arduino będzie po uruchomieniu mikrokontrolera migąла szybko dziesięć razy, a następnie będzie zapalana co sekundę.

Przyjrzyj się poniższemu listingowi. Znajdziesz tu wyjaśnienie działania tego programu.

```
// 01_02_blink_szybko_wolno
int ledPin = 13;

void setup()
{
    pinMode(ledPin, OUTPUT);
    flash(10, 100);
}

void loop()
{
    flash(1, 500);
}

void flash(int n, int delayPeriod)
{
    for (int i = 0; i < n; i++)
    {
        digitalWrite(ledPin, HIGH);
        delay(delayPeriod);
        digitalWrite(ledPin, LOW);
        delay(delayPeriod);
    }
}
```

W funkcji `setup` znalazła się linia `flash(10, 100)`. Zapis ten oznacza, że dioda będzie migąла 10 razy, a zmiennej `delayPeriod` zostanie przypisana wartość 100 — odstęp między błyskami będzie wynosił 100 ms. Funkcja `flash` nie jest poleceniem wbudowanym Arduino. Tę przydatną funkcję musisz napisać samodzielnie.

Definicja funkcji znajduje się pod koniec szkicu. Pierwszą linią definicji funkcji jest:

```
void flash(int n, int delayPeriod)
```

Linia informuje Arduino o tym, że definiujesz własną funkcję o nazwie *flash*, przyjmującą dwa parametry. Oba parametry są liczbami typu `int` (integer). Pierwszym parametrem jest zmienna `n`, która określa, ile razy dioda LED ma błyskać, a drugim parametrem jest zmienna `delayPeriod`, która określa czas, jaki ma upłynąć pomiędzy włączeniem i wyłączeniem diody LED.

Te dwie zmienne mogą być użyte tylko wewnątrz tej funkcji. Parametr `n` jest stosowany w poleceniu `for` w celu określenia liczby obiegów pętli, a parametr `delayPeriod` jest stosowany w poleceniach `delay`.

Znajdująca się w szkicu funkcja `loop` również korzysta z funkcji `flash`, w tym przypadku jednak stosowana jest wyższa wartość funkcji `delayPeriod`, a dioda LED jest zapalana tylko jednokrotnie. Ponieważ funkcja `flash` uruchamiana jest wewnątrz funkcji `loop`, dioda LED i tak będzie migać w nieskończoność.

Wejścia cyfrowe

Aby w pełni skorzystać z niniejszego fragmentu książki, będziesz potrzebował krótkiego przewodu. W ostateczności możesz zastosować drut uzyskany w wyniku wyprostowania spinacza do papieru.

Załaduj przedstawiony szkic do pamięci Arduino, a następnie uruchom go.

```
// 01_03_spinacz
int ledPin = 13;
int switchPin = 7;

void setup()
{
  pinMode(ledPin, OUTPUT);
  pinMode(switchPin, INPUT_PULLUP);
}

void loop()
{
  if (digitalRead(switchPin) == LOW)
  {
    flash(100);
  }
  else
  {
    flash(500);
  }
}

void flash(int delayPeriod)
{
  digitalWrite(ledPin, HIGH);
  delay(delayPeriod);
  digitalWrite(ledPin, LOW);
  delay(delayPeriod);
}
```

Za pomocą spinacza do papieru lub przy użyciu przewodu zewrzyj ze sobą pin GND i cyfrowe złącze numer 7 (zobacz rysunek 1.13). Możesz to zrobić, gdy Arduino jest podłączone do prądu, ale dopiero po załadowaniu przedstawionego szkicu do jego pamięci. Należy zwrócić na to szczególną uwagę, ponieważ gdyby w pamięci Arduino znajdował się jakiś wcześniejszy program, w którym pin numer 7 pracowałby w charakterze wyjścia, to zwarcie go z pinem GND doprowadziłoby do uszkodzenia tegoż programu. Piny te można zwierać po załadowaniu powyższego szkicu, ponieważ pin numer 7 jest skonfigurowany tak, aby działał w charakterze wejścia.

Szkic powinien działać w następujący sposób: dioda LED miga szybko, gdy wspomniane piny są zwierane przez spinacz, a gdy piny nie są ze sobą zwarte, dioda miga powoli.

Przeanalizujmy ten szkic i zobaczmy, jak działa.

Mamy do czynienia z nową zmienną o nazwie `switchPin`. Zmiennej tej przypisano pin numer 7 — spinacz może wtedy działać jak przełącznik. W funkcji `setup` za pomocą polecenia `pinMode` określiliśmy, że wspomniany pin będzie działał w charakterze wejścia. Drugim argumentem funkcji `pinMode` nie jest `INPUT`, a `INPUT_PULLUP`. Argument ten powoduje, że standardowo na wejściu będzie odbierany sygnał wysoki (`HIGH`), chyba że zostanie on ściągnięty do sygnału niskiego (`LOW`) przez zwarcie wspomnianego pinu z masą (`GND`) za pomocą spinacza.



Rysunek 1.13. Korzystanie z wejścia cyfrowego

W funkcji `loop` stan wejścia sprawdzamy za pomocą polecenia `digitalRead`. Jeżeli polecenie to zwróci wartość `LOW` (piny są zwarte za pomocą spinacza), to zostanie wywołana funkcja `flash`, której zostanie przekazany parametr `delayPeriod` o wartości 100. Dioda LED będzie migała szybko.

Jeżeli na wejściu odbierany jest sygnał wysoki (HIGH), to uruchamiane są polecenia zawarte w części else instrukcji if. Wywoływana jest wtedy funkcja `flash` ze znacznie wyższą wartością parametru `delayPeriod`, a więc dioda LED miga wolniej.

Funkcja `flash` jest uproszczoną wersją funkcji `flash` zastosowanej w poprzednim szkicu. Zapala ona diodę LED jednokrotnie na określony czas.

Czasami do Arduino będziesz podłączał wyjścia zewnętrznych modułów generujących sygnał, który może przyjmować dwa stany: wysoki (HIGH) i niski (LOW). W takim przypadku będziesz stosował parametr `INPUT` (a nie `INPUT_PULLUP`) funkcji `pinMode`.

Wyjścia cyfrowe

Z punktu widzenia programisty nie mam zbyt wielu nowych rzeczy do powiedzenia na temat wyjść cyfrowych. Korzystałeś już z nich, sterując pracą wbudowanej diody LED podłączonej do pinu o numerze 13.

Najważniejszą rzeczą, o której trzeba pamiętać podczas pracy z wyjściami cyfrowymi, jest to, że należy je zdefiniować w funkcji `setup` tak, aby działały jako wyjścia — za pomocą polecenia:

```
pinMode(outputPin, OUTPUT);
```

Jeżeli chcesz określić sygnał generowany na wyjściu cyfrowym (może on przyjmować dwa stany: HIGH i LOW), możesz to zrobić za pomocą polecenia `digitalWrite`:

```
digitalWrite(outputPin, HIGH);
```

Monitor portu szeregowego

Arduino łączy się z komputerem przez interfejs USB. Pomiędzy tymi urządzeniami możliwe jest przesyłanie komunikatów za pomocą **monitora portu szeregowego**, będącego elementem zintegrowanego środowiska programistycznego Arduino.

Aby sprawdzić jego działanie, zmodyfikujmy poprzedni szkic tak, aby Arduino — zamiast modyfikacji częstotliwości błysków diody LED — wysyłało komunikat sygnalizujący podanie niskiego sygnału na cyfrowe wejście numer 7.

Łaładuj poniższy szkic:

```
// 01_04_port_szeregowy
int switchPin = 7;

void setup()
{
    pinMode(switchPin, INPUT_PULLUP);
    Serial.begin(9600);
}

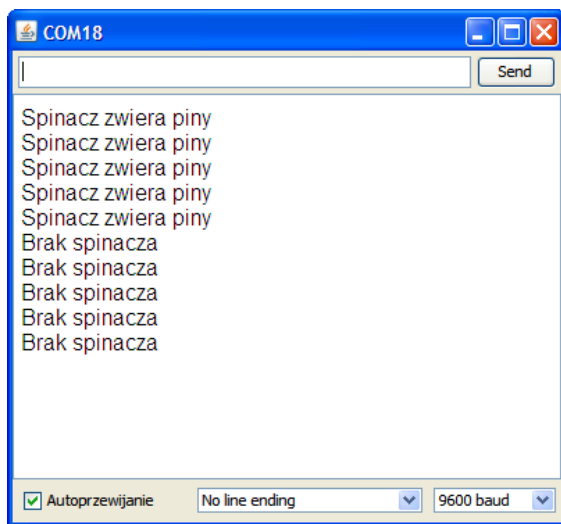
void loop()
{
```

```

if (digitalRead(switchPin) == LOW)
{
    Serial.println("Spinacz zwiera piny");
}
else
{
    Serial.println("Brak spinacza");
}
delay(1000);
}

```

Teraz w środowisku programistycznym Arduino kliknij ikonę znajdującą się w pasku narzędzi, wyglądającą jak szkło powiększające. Otworzy się monitor portu szeregowego. W otwartym oknie, co sekundę, powinny wyświetlać się komunikaty (zobacz rysunek 1.14).



Rysunek 1.14. Monitor portu szeregowego

Odłącz spinacz od któregoś z pinów — zobaczysz, że zmienia się treść wyświetlanego komunikatu.

Nie korzystamy z wbudowanej diody LED, a więc nie potrzebujemy zmiennej `ledPin`. Zamiast tej zmiennej korzystamy z polecenia `Serial.begin` — uruchamia ono szeregową transmisję danych. Niezbędne jest podanie parametru określającego prędkość transmisji sygnału. Więcej informacji na temat szeregowego przesyłania danych znajdziesz w rozdziale 13.

Aby wyświetlić komunikat w oknie monitora portu szeregowego, wystarczy uruchomić polecenie `Serial.println`.

W zaprezentowanym przykładowym szkicu Arduino wysyła komunikaty do monitora portu szeregowego.

Tablice i macierze

Tablice przechowują listy zmiennych. Dotychczas spotykałeś zmienne przechowujące tylko jedną wartość (zwykle była to wartość typu `int`). Tablica zawiera listę wartości. Możesz uzyskać dostęp do dowolnej z przechowywanych wartości, znając jej pozycję na liście.

W języku C, tak jak w większości języków programowania, indeksy list są numerowane od 0, a nie od 1. Oznacza to, że pierwszy element na liście jest tak naprawdę elementem zerowym.

W poprzedniej sekcji poznałeś działanie monitora portu szeregowego i miałeś już do czynienia z jedną tablicą. Komunikaty takie jak „Brak spinacza” nazywa się **tablicami znaków**, ponieważ są tak naprawdę zbiorem znaków.

Na przykład spróbujmy sprawić, aby nasze Arduino wyświetlało jakiś bełkot w oknie monitora portu szeregowego.

W poniższym szkicu stworzymy tablicę zawierającą tablice znaków. Program wybierze losowo jeden z komunikatów, a następnie wyświetli go po określonym losowo odstępie czasu. W szkicu tym przedstawiono sposób generowania liczb losowych w programach uruchamianych na Arduino.

```
// 01_05_belkot

char* messages[] = {
  "To ja Arduino",
  "Czytaj publikacje Simona Monka",
  "Wykonaj ciekawy projekt z Arduino",
  "Raspberry Pi jest owocowe"};

void setup()
{
  Serial.begin(9600);
}

void loop()
{
  int delayPeriod = random(2000, 8000);
  delay(delayPeriod);
  int messageIndex = random(4);
  Serial.println(messages[messageIndex]);
}
```

Każdy z komunikatów-**łańcuchów** (tym terminem określane są zwykle zbiory znaków) jest danymi typu `char*`. Znak `*` informuje o tym, że jest to wskaźnik. Zaawansowane zagadnienia związane ze wskaźnikami zostaną poruszone w rozdziale 6. Znaki `[]` umieszczone na końcu deklaracji zmiennej świadczą o tym, że zmienna jest tablicą elementów typu `char*`, a nie pojedynczym elementem typu `char*`.

Wewnątrz funkcji `loop` zmiennej `delayPeriod` jest przypisywana wartość losowana z zakresu od 2000 do 7999. Drugi z argumentów funkcji `random` jest wartością wykluczoną z zakresu losowanych liczb. Pauza o określonej długości jest następnie generowana przez funkcję `delay`.

Zmiennej `messageIndex` jest również przypisywana wartość losowa. Tym razem jednakże poleceniu `random` przekazany jest tylko jeden parametr określający, że losowana będzie liczba z zakresu od 0 do 3. W ten sposób zostaje określony indeks komunikatu, który ma zostać wyświetlony.

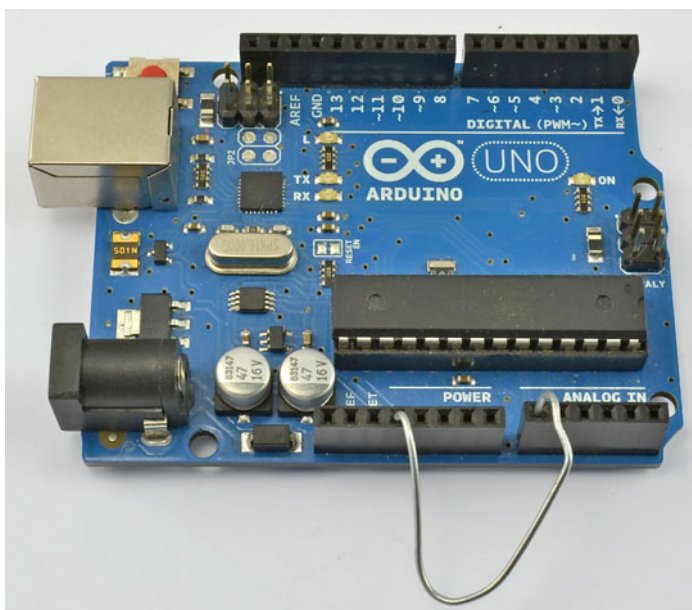
Na koniec komunikat o właściwym indeksie jest wysyłany do monitora portu szeregowego. Wypróbuj działanie zaprezentowanego szkicu. Pamiętaj o konieczności otwarcia okna monitora portu szeregowego.

Wejścia analogowe

Piny Arduino oznaczone etykietami od A0 do A5 mogą mierzyć przyłożone do nich napięcie. Napięcie to musi się znajdować w zakresie od 0 V do 5 V. Czynność tę wykonuje wbudowana funkcja `analogRead`, która zwraca wartość z zakresu od 0 do 1023 — 0 jest zwracane przy napięciu 0 V, a 1023 przy 5 V. Aby przedstawić wartość z zakresu od 0 do 5 za pomocą liczb znajdujących się w przedziale od 0 do 1023, niezbędne jest wykonanie dzielenia $1023:5 = 204,6$.

Liczyby typu `int` nie są idealnym typem danych do przechowywania zmierzonego napięcia. Warto byłoby widzieć ułamkową część napięcia wyrażonego w woltach. Do tego niezbędne będzie zastosowanie danych typu `float`.

załaduj poniższy szkic do pamięci Arduino, a następnie za pomocą spinacza zewrzyj pin A0 z pinem zasilającym o napięciu 3,3 V (zobacz rysunek 1.15).



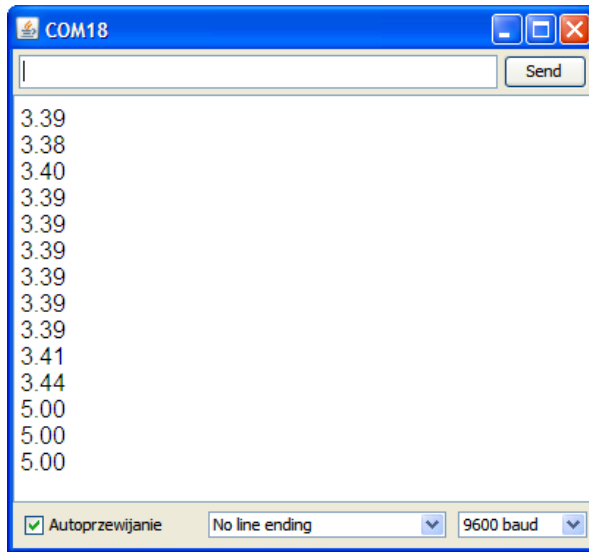
Rysunek 1.15. Podłączanie napięcia 3,3 V do pinu A0

```
// 01_06_wejście_analogowe
int analogPin = A0;

void setup()
{
    Serial.begin(9600);
}

void loop()
{
    int rawReading = analogRead(analogPin);
    float volts = rawReading / 204.6;
    Serial.println(volts);
    delay(1000);
}
```

Otwórz monitor portu szeregowego — powinien zostać na nim wyświetlony ciąg liczb podobny do tego, jaki widać na rysunku 1.16. Wyświetlane wartości powinny być bliskie liczbie 3,3.



Rysunek 1.16. Odczytywane napięcie

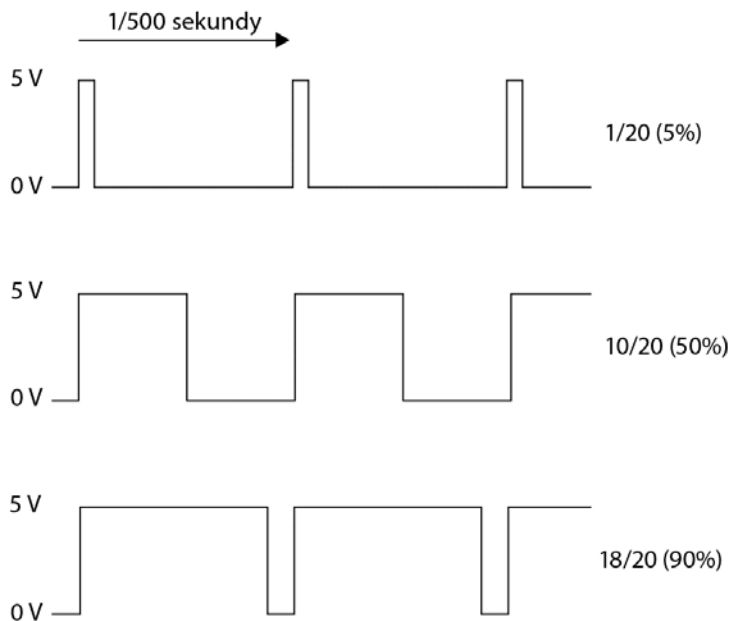
UWAGA

Nie łącz ze sobą pinów zasilających (5V, 3.3V i GND). Doprowadzenie do takiego zwarcia prawdopodobnie uszkodziłoby Twoją płytkę Arduino, a być może nawet i komputer.

Jeżeli jeden z końców spinacza przytrzymasz na pinie A0, a drugi podłączysz do gniazda dostarczającego prąd o napięciu 5 V, to program będzie wyświetlał napięcie 5 V. Dotknięcie tym samym końcem spinacza do złącza masy spowoduje wyświetlenie wartości 0 V.

Wyjścia analogowe

Arduino Uno nie ma wyjść generujących prawdziwy sygnał analogowy (takie wyjścia znajdują się na płytce Arduino Due), ale płytkę tę wyposażono w kilka wyjść mających układy PWM (modulujące czas trwania impulsu). Takie wyjścia mogą dostarczyć sygnał podobny do analogowego, sterując długością generowanych impulsów (zobacz rysunek 1.17).



Rysunek 1.17. Modulacja szerokości impulsu

Im dłużej impuls jest wysoki, tym wyższe jest średnie napięcie sygnału. W związku z tym, że generowane jest około 600 impulsów na sekundę, a większość komponentów podłączanych do wyjść układów PWM reaguje dość wolno na zmiany napięcia, układami takimi można emulować zmiany napięcia.

Piny znajdujące się na płytce Arduino Uno oznaczone symbolem ~ (piny o numerach 3, 5, 6, 9, 10 i 11) mogą funkcjonować jako wyjścia analogowe.

Jeżeli dysponujesz woltomierzem, to przełącz go w tryb pomiaru prądu stałego w zakresie od 0 V do 20 V. Dodatni próbnik miernika podłącz do cyfrowego pinu numer 6, a ujemny próbnik — do złącza oznaczonego etykietą *GND* (zobacz rysunek 1.18). Następnie załaduj do pamięci Arduino poniższy szkic.

```
// 01_07_pwm
int pwmPin = 6;
```

```
void setup()
{
```



Rysunek 1.18. Pomiar napięcia wyjściowego

```
pinMode(pwmPin, OUTPUT);
Serial.begin(9600);
}

void loop()
{
    if (Serial.available())
    {
        int dutyCycle = Serial.parseInt();
        analogWrite(pwmPin, dutyCycle);
    }
}
```

Otwórz okno monitora portu szeregowego i wpisz liczbę w zakresie od 0 do 255 w polu przeznaczonym do wpisywania tekstu, znajdującym się obok przycisku *Send*. Następnie kliknij ten przycisk. Twój multimetr powinien wskazywać teraz inną wartość mierzonego napięcia. Wpisanie 0 powinno spowodować spadek napięcia do około 0 V, wpisanie wartości 127 powinno spowodować wygenerowanie napięcia o wartości o połowę mniejszej od 5 V (czyli 2,5 V), a wpisanie wartości 255 powinno doprowadzić do podania napięcia około 5 V.

Funkcja `loop` znajdująca się w tym szkicu zaczyna się od polecenia `if`. Warunkiem polecenia `if` jest `Serial.available()`. Oznacza to, że gdy Arduino odbierze wiadomość przesłaną za pośrednictwem monitora portu szeregowego, wtedy zostaną wykonane polecenia znajdujące się w nawiasach klamrowych. W tym przypadku polecenie `Serial.parseInt` przekształci wiadomość przesłaną za pośrednictwem monitora portu szeregowego w dane typu `int`, które zostaną następnie użyte jako argument funkcji `analogWrite` sterującej pracą wyjścia wyposażonego w układ PWM.

Korzystanie z bibliotek

Płytki Arduino dysponują dość ograniczonym zasobem pamięci. W związku z tym warto, aby program zawierał tylko kod, który będzie niezbędny do obsługi komponentów podłączonych do płytki. Można to zrobić, na przykład rozsądnie korzystając z bibliotek. W przypadku Arduino i ogólnie języka C **biblioteka** jest zbiorem przydatnych funkcji.

Środowisko programistyczne Arduino zawiera na przykład bibliotekę obsługującą wyświetlacze ciekłokrystaliczne. Dołączenie tej biblioteki do programu wymaga 1,5 kB pamięci. Nie ma sensu łączyć wspomnianej biblioteki do pamięci Arduino, jeżeli nie korzystamy z niej. Takie biblioteki powinny być dołączane tylko wtedy, gdy są potrzebne.

Biblioteki dołącza się do szkicu na początku, za pomocą dyrektywy `#include`. Pozwala ona korzystać ze wszystkich bibliotek dodanych do środowiska programistycznego Arduino — posługując się podmenu *Importuj bibliotekę* znajdującym się w menu *Szkic*.

Do środowiska programistycznego Arduino dodano wiele oficjalnych bibliotek:

- **EEPROM** — przechowywanie danych w pamięci EEPROM,
- **Ethernet** — programowanie sieciowe,
- **Firmata** — obsługa standardowej komunikacji pomiędzy Arduino a komputerem,
- **LiquidCrystal** — współpraca z alfanumerycznymi wyświetlaczami ciekłokrystalicznymi,
- **SD** — zapis danych na kartach pamięci SD, a także odczyt danych już na nich zapisanych,
- **Servo** — obsługa serwowatorów,
- **SPI** — obsługa magistrali pozwalającej na komunikację Arduino z urządzeniami peryferyjnymi,
- **SoftwareSerial** — komunikacja szeregowa za pomocą pinów standardowo do tego nieprzeznaczonych,
- **Stepper** — sterowanie pracą silników krokowych,
- **WiFi** — dostęp do sieci Wi-Fi,
- **Wire** — komunikacja z urządzeniami peryferyjnymi za pomocą magistrali I2C.

Niektóre biblioteki są przeznaczone do użycia tylko z niektórymi płytkami Arduino:

- **Keyboard** — emulacja klawiatury USB (Leonardo, Due i Micro),
- **Mouse** — emulacja myszy USB (Leonardo, Due i Micro),
- **Audio** — narzędzia przeznaczone do generowania dźwięku (tylko płytka Due),
- **Scheduler** — obsługa wykonywania wielu wątków (tylko płytka Due),
- **USB Host** — obsługa urządzeń peryferyjnych wyposażonych w interfejs USB (tylko płytka Due).

Istnieje również wiele bibliotek napisanych przez użytkowników Arduino. Można je pobrać z internetu. Poniżej znajduje się lista najpopularniejszych bibliotek tego typu:

- **OneWire** — umożliwia odczyt danych z urządzeń cyfrowych korzystających z interfejsu 1-Wire firmy Dallas Semiconductor,
- **Xbee** — bezprzewodowa, szeregową transmisję danych,
- **GFX** — biblioteka graficzna obsługująca różne wyświetlacze firmy Adafruit,
- **Capacitive Sensing** — obsługa czujników odległości,
- **FFT** — biblioteka przeznaczona do analizy częstotliwości.

Stale pojawiają się nowe biblioteki. Można je znaleźć w oficjalnej witrynie Arduino (<http://arduino.cc/en/Reference/Libraries>) lub za pomocą wyszukiwarki internetowej.

Jeżeli chcesz korzystać z bibliotek zaliczanych do ostatniej z wymienionych kategorii, to musisz je zainstalować — pobrać z internetu i zapisać w folderze *Moje dokumenty\Arduino\Libraries*. Jeżeli folder *Libraries* nie istnieje, to musisz go utworzyć, instalując pierwszą dodatkową bibliotekę Arduino.

Aby środowisko programistyczne Arduino wykryło zainstalowaną bibliotekę, musisz je zamknąć i uruchomić ponownie.

Typy danych obsługiwane przez Arduino

W języku C zmienna typu `int` zajmuje 2 bajty pamięci Arduino. Jeżeli szkic nie zajmuje bardzo dużego obszaru pamięci, to zmienne typu `int` są stosowane niemalże wszędzie — nawet do przechowywania wartości logicznych lub małych liczb całkowitych, które można zapisać za pomocą 1 bajta.

W tabeli 1.1 przedstawiono listę wszystkich dostępnych typów danych.

Tabela 1.1. Typy danych w języku C obsługiwane przez Arduino

Nazwa typu	Rozmiar (bajty)	Zakres	Uwagi
<code>boolean</code>	1	prawda lub fałsz (0 lub 1)	Używany do zapisu wartości logicznych.
<code>char</code>	1	−128 do +128	Stosowany do kodowania znaków ASCII. Na przykład literze „A” przypisano kod 65. Zwykle korzysta się tylko z dodatnich wartości.
<code>byte</code>	1	0 do 255	Często używane jako pojedyncze pakiety danych podczas szeregowej transmisji danych. Zobacz rozdział 9.
<code>int</code>	2	−32 768 do +32 767	Są to wartości 16-bitowe ze znakiem.

Tabela 1.1. Typy danych w języku C obsługiwane przez Arduino (ciąg dalszy)

Nazwa typu	Rozmiar (bajty)	Zakres	Uwagi
unsigned int	2	0 do 65 536	Stosowane w celu uzyskania większej precyzji, gdy nie ma konieczności zapisu liczby ujemnej. Korzystaj z nich ostrożnie, ponieważ wykonywanie operacji arytmetycznych na nich i na danych typu int może doprowadzić do błędnych wyników
long	4	–2 147 483 648 do 2 147 483 647	Wymagane tylko do przechowywania dużych liczb.
unsigned long	4	0 do 4 294 967 295	Zobacz unsigned int.
float	4	–3,4028235E+38 do +3,4028235E+38	Stosowane do zapisu liczb zmiennoprzecinkowych.
double	4	taki sam jak w przypadku float	Standardowo ten typ danych jest zapisywany z precyzją 8 lub więcej bitów. W przypadku Arduino jednak dane typu double mają taki sam zakres jak dane typu float.

Polecenia Arduino

Standardowa biblioteka Arduino ma wiele wbudowanych funkcji. Najczęściej stosowane polecenia wraz z przykładami przedstawiono w tabeli 1.2.

Tabela 1.2. Funkcje wbudowane w standardową bibliotekę Arduino

Polecenie	Przykład	Opis
Cyfrowe wejścia-wyjścia		
pinMode	pinMode(8, OUTPUT);	Konfiguruje pin numer 8 tak, aby działał w charakterze wyjścia. Może również działać w charakterze wejścia (INPUT lub INPUT_PULLUP).
digitalWrite	digitalWrite(8, HIGH);	Podaje wysoki sygnał na pin numer 8. Aby podać niski sygnał, zamiast stałej HIGH zastosuj stałą LOW.
digitalRead	int i; i = digitalRead(8);	Przypisuje zmiennej i wartość HIGH lub LOW w zależności od napięcia na określonym złączu (w tym przypadku jest to pin numer 8).
pulseIn	i = pulseIn(8, HIGH);	Zwraca wyrażony w milisekundach czas trwania kolejnego impulsu sygnału o stanie HIGH, kierowanego do pinu numer 8.

Tabela 1.2. Funkcje wbudowane w standardową bibliotekę Arduino (ciąg dalszy)

Polecenie	Przykład	Opis
tone	tone(8, 440, 1000);	Generuje na pinie numer 8 sygnał o częstotliwości 440 Hz przez 1000 ms.
noTone	noTone();	Przerywa generowanie wszystkich tonów.
Analogowe wejścia-wyjścia		
analogRead	int r; r = analogRead(0);	Przypisuje zmiennej r wartość z zakresu od 0 do 1023. Jeżeli do pinu numer 0 dopływa prąd o napięciu 0 V, to zmiennej zostanie przypisana wartość 0. Jeżeli prąd ten będzie miał napięcie 5 V (lub 3,3 V w przypadku płytek zasilanych napięciem 3 V), to zmiennej zostanie przypisana wartość 1023.
analogWrite	analogWrite(9, 127);	Generuje sygnał wyjściowy za pomocą układu PWM. Długość cyklu pracy określa liczba z przedziału od 0 do 255 (255 oznacza 100% mocy). Polecenie to może być użyte tylko w odniesieniu do pinów oznaczonych etykietą PWM na płytce Arduino (3, 5, 6, 9, 10 i 11).
Polecenia związane z czasem		
millis	unsigned long l; l = millis();	Zmienne typu long są zapisywane w pamięci Arduino za pomocą 32 b. Polecenie millis() zwraca czas podany w milisekundach, jaki upłynął od ostatniego resetu. Liczba ta jest naliczana od początku po upływie około 50 dni pracy mikrokontrolera.
micros	long l; l = micros();	Zobacz polecenie millis. Polecenie to różni się jedynie tym, że podaje czas od ostatniego resetu, wyrażony w mikrosekundach. Liczba ta jest naliczana od początku po upływie około 70 min pracy mikrokontrolera.
delay	delay(1000);	Wstrzymuje wykonywanie programu na 1000 ms (1 s).
delayMicroseconds	delayMicroseconds(100000);	Wstrzymuje wykonywanie programu na 1000 ms (1 s). Pamiętaj o tym, że minimalny czas, jaki możesz zastosować w przypadku tego polecenia, wynosi 3 µs, a maksymalny — 16 ms.

Tabela 1.2. Funkcje wbudowane w standardową bibliotekę Arduino (ciąg dalszy)

Polecenie	Przykład	Opis
Przerwania (zobacz rozdział 3.)		
<code>attachInterrupt</code>	<code>attachInterrupt(1, myFunction, RISING);</code>	Kojarzy funkcję o nazwie <code>myFunction</code> ze wzrastającym przejściem przerwania 1 (D3 w przypadku płytki Uno).
<code>detachInterrupt</code>	<code>detachInterrupt(1);</code>	Wyłącza wszystkie przerwania skojarzone z przerwaniem 1.

Pełną listę poleceń Arduino znajdziesz w oficjalnej dokumentacji znajdującej się pod adresem <http://arduino.cc>.

Podsumowanie

Rozdział ten stanowi bardzo skondensowane wprowadzenie do świata Arduino. Jeżeli chcesz poczytać więcej na temat podstaw pracy z Arduino, to poszukaj takich informacji w internecie. Na przykład na stronie <https://learn.adafruit.com/> znajdziesz darmowe anglojęzyczne poradniki.

W kolejnym rozdziale wgryziemy się nieco głębiej w zagadnienia związane z Arduino i „zajrzemy pod jego maskę”, zobaczymy, co dzieje się wewnątrz tego przyjemnego i łatwego w użyciu mikrokontrolera.

Rozdział 2.

Pod maską

Jedną z zalet Arduino jest to, że przez większość czasu, po załadowaniu szkicu, tak naprawdę nie musisz mieć świadomości tego, co się dzieje na płytce tego mikrokontrolera. Pracując jednak dłużej z płytką Arduino, możesz chcieć przesuwać granice tego, co możesz robić, a wtedy musisz wiedzieć, co w rzeczywistości dzieje się wewnątrz mikrokontrolera.

Krótką historia Arduino

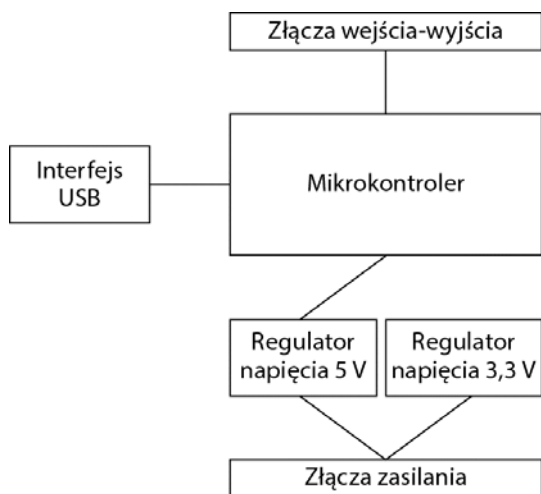
Pierwsza płytka Arduino powstała w 2005 r. we Włoszech, w miejscowości Ivrea nieopodal Turynu, w Interaction Design Institute. Celem projektu było stworzenie taniego, łatwego w użyciu narzędzia pozwalającego studentom na budowę systemów interaktywnych. Programowanie Arduino, które również przyczyniło się do sukcesu tego mikrokontrolera, zostało oparte na otwartej ramie projektowej o nazwie *Wiring*. Rama ta również została stworzona przez studenta wspomnianej uczelni.

Środowisko Arduino jest bardzo podobne do środowiska Wiring. Zintegrowane środowisko programistyczne Arduino zostało napisane w środowisku Processing, będącym bardziej złożoną aplikacją, podobną do Wiring, działającą na komputerach PC i Mac. Jeżeli pracujesz nad projektem, w którym Arduino ma współpracować z komputerem za pośrednictwem interfejsu USB lub Bluetooth, warto, abyś przyjrzał się bliżej środowisku Processing.

Płytki Arduino ewoluowały wraz z upływem czasu, ale płytki Arduino Uno i Leonardo mają taki sam kształt i zestaw złączy jak pierwowzór.

Anatomia Arduino

Na rysunku 2.1 przedstawiono budowę Arduino Uno. Płytką Leonardo jest podobna, z tym że interfejs USB zintegrowano z głównym chipem mikrokontrolera. Płytką Due działa w identyczny sposób, ale jej procesor jest zasilany prądem o napięciu 3,3 V, a nie 5 V.



Rysunek 2.1. Anatomia płytki Arduino Uno

Arduino jest niemalże wyłącznie układem mikrokontrolera wyposażonym w obsługujące go komponenty. Dysponując chipem procesora i kilkoma dodatkowymi podzespołami, możesz zbudować własne Arduino na płytce prototypowej. Możesz również umieścić obwody Arduino na finalnej płytce drukowanej projektu, którego prototyp korzystał z Arduino. Gotowe płytki Arduino ułatwiają pracę, ale tak naprawdę znajdujący się na płytce obwód możesz zmodyfikować tak, aby uzyskać coś, co składa się z chipa mikrokontrolera i kilku niezbędnych podzespołów. Możesz na przykład zaprogramować chip znajdujący się na Arduino, a następnie zamontować go w gnieździe na układy scalone umieszczone na płytce drukowanej innego obwodu niewyposażonego w interfejs USB.

Później dowiesz się, jak zaprogramować Arduino bezpośrednio za pomocą interfejsu ICSP (ang. *In Circuit Serial Programming*).

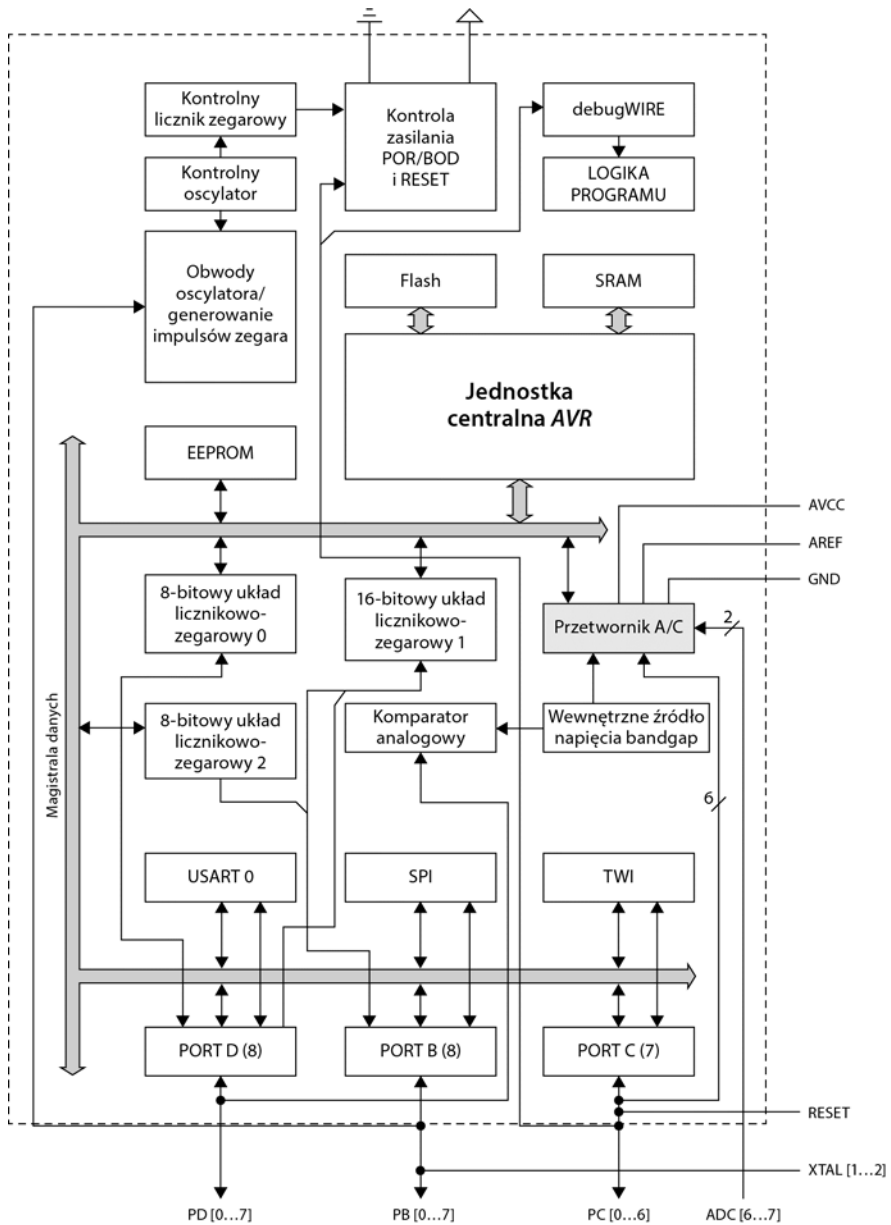
Procesory AVR

Na płytkach z rodziny Arduino stosuje się mikrokontrolery produkowane przez firmę Atmel. Wszystkie te układy charakteryzują się podobną budową. Wyjątek stanowi procesor SAM3X8E ARM Cortex-M3 stosowany na płytce Due.

ATmega328

Procesor ATmega328 jest mikrokontrolerem znajdującym się na płytce Arduino Uno i jej poprzedniczce — Duemilanove. W rzeczywistości procesor ATmega168 użyty w pierwszej płytce Arduino jest procesorem ATmega328 wyposażonym w o połowę mniej pamięci każdego typu.

Na rysunku 2.2 przedstawiono budowę procesora ATmega328. Schemat ten pochodzi z noty katalogowej, którą znajdziesz pod adresem www.atmel.com/Images/doc8161.pdf. Warto zapoznać się z jej treścią, aby przybliżyć sobie szczegóły dotyczące pracy tego komponentu.



Rysunek 2.2. Procesor ATmega328

Jednostka centralna jest miejscem, w którym podejmowane są wszystkie działania. Odczytuje ona instrukcje (skompilowany kod szkicu) z pamięci flash. Jednorazowo odczytywana jest tylko jedna instrukcja. Proces ten przebiega inaczej niż w komputerach, gdzie kod programu, zapisany na dysku, jest ładowany do pamięci o dostępie bezpośrednim (RAM), a dopiero

później uruchamiany. Zmienne używane przez program są przechowywane w oddzielnej pamięci statycznej RAM (SRAM). W przeciwieństwie do pamięci flash, w której znajduje się kod programu, pamięć RAM jest ulotna — jej zawartość jest tracona po odłączeniu od zasilania.

Trzeci rodzaj pamięci — **EEPROM (programowalna pamięć stała wymazywalna elektrycznie)** — jest używany do przechowywania danych, których nie chcemy utracić w wyniku wyłączenia zasilania.

Kolejnymi interesującymi nas elementami schematu są układy kontrolujące licznik zegarowy i zasilanie. Poszerzają one znacznie zakres możliwości mikrokontrolera, umożliwiając wykonywanie czynności ukrytych pod uproszczoną warstwą Arduino (na przykład pozwalają na okresowe usypianie chipa i włączanie go po upływie określonego czasu). Sztuczka ta jest bardzo przydatna, gdy chcemy, aby mikrokontroler pobierał jak najmniej prądu. Więcej informacji na ten temat znajdziesz w rozdziale 5.

Pozostałe elementy widoczne na schemacie blokowym to komponenty zajmujące się konwersją sygnałów analogowych na cyfrowe, porty wejścia-wyjścia i trzy różne interfejsy szeregowo: USART, SPI i TWI (I2C).

ATmega32u4

Procesor ATmega32u4 stosuje się na płytkach Arduino Leonardo, LilyPad USB, a także Micro i Nano. Jest podobny do układu ATmega328, z tym że jest nowocześniejszy od niego — wprowadzono w nim pewne usprawnienia:

- Wbudowano w niego interfejs USB, a więc na płytce nie muszą znajdować się dodatkowe komponenty służące do obsługi złącza USB.
- Więcej pinów wyposażono w układy PWM.
- Procesor posiada dwa porty szeregowo.
- Stworzono piny dedykowane magistrali I2C (na płytce Arduino piny te są współdzielone z interfejsem analogowym).
- Pamięć SRAM została powiększona o 0,5 kB.

Na płytce Leonardo zastosowano procesor w obudowie przeznaczonej do montażu powierzchniowego — układ jest przylutowany bezpośrednio do płytki Arduino, a procesor ATmega328 znajduje się w obudowie podłużnej dwurzędowej, wstawionej w gniazdo układu scalonego znajdujące się na płytce Arduino Uno.

ATmega2560

Procesor ATmega2560 jest montowany na płytkach Arduino Mega 2560 i Arduino Mega ADK. Układ ten nie działa szybciej od pozostałych chipów z rodziny ATmega, ale wyposażono go w o wiele więcej pamięci (256 kB pamięci flash, 8 kB pamięci SRAM i 4 kB pamięci EEPROM), a także w dodatkowe złącza wejścia-wyjścia.

AT91SAM3X8E

Procesor ten to serce Arduino Due. Jest o wiele szybszy od omówionych wcześniej chipów ATmega. Jest on taktowany zegarem o częstotliwości 84 MHz (normalnie układy z rodziny ATmega są taktowane zegarem o częstotliwości 16 MHz). Wyposażono go w 512 kB pamięci flash i 96 kB pamięci SRAM. Mikrokontroler ten nie posiada pamięci EEPROM. Aby zapisywać dane w pamięci trwałej, użytkownik tego chipa musi podłączyć do niego dodatkowy sprzęt, taki jak kontroler kart SD wraz z kartą SD lub układ scalony zawierający pamięć flash lub EEPROM. Procesor AT91SAM3X8E ma wiele zaawansowanych funkcji. Między innymi wyposażono go w dwa analogowe wyjścia, dzięki którym można stosować go do generowania dźwięku.

Arduino i Wiring

Biblioteka Wiring zapewnia łatwe w użyciu funkcje obsługujące złącza sprzętowe. Należy jednak pamiętać o tym, że struktura szkiców Arduino musi być zgodna z językiem C.

Do niedawna w folderze, w którym zainstalowano środowisko programistyczne Arduino, można było znaleźć plik o nazwie *WProgram.h* (program obsługujący złącza). W nowszych wersjach wspomnianego środowiska plik ten zastąpiono podobnym plikiem o nazwie *Arduino.h*. Sugeruje to, że szkielet środowiska Arduino powoli oddala się od oryginalnego projektu Wire.

Jeżeli zajrzysz do folderu z zainstalowanym środowiskiem programistycznym Arduino, znajdziesz tam katalog o nazwie *hardware*, a w nim folder *arduino*, w którym umieszczono podfolder o nazwie *cores*. Jeżeli korzystasz z komputera z systemem OS X, to zawartość tego podfolderu możesz zobaczyć, klikając prawym przyciskiem myszy na aplikację Arduino. Następnie po wybraniu opcji *Pokaż zawartość pakietu* należy skierować się do folderu *Resources/Java/*.

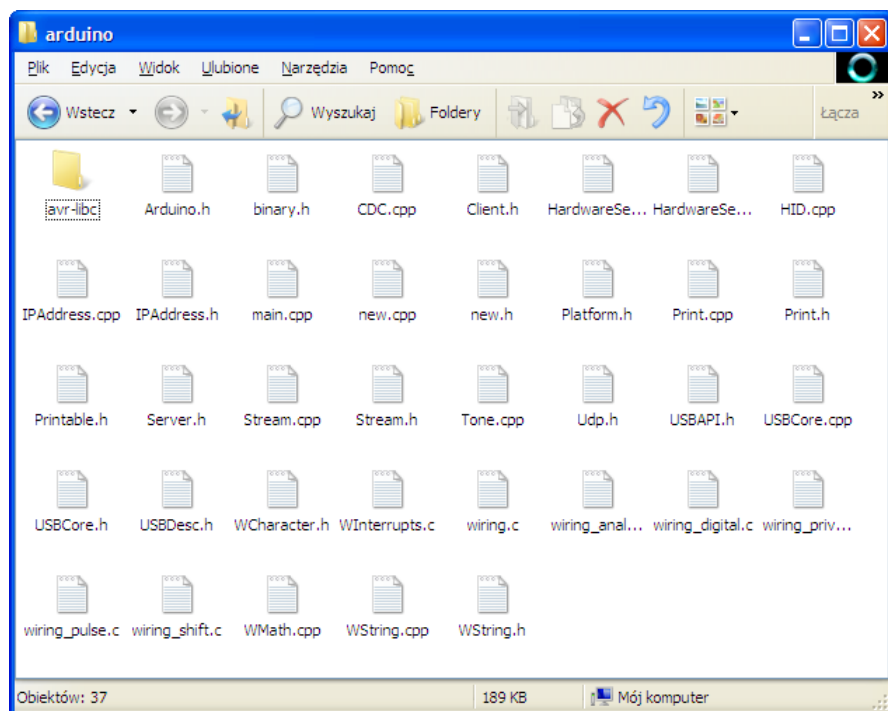
W folderze *cores* umieszczony jest kolejny katalog — o nazwie *arduino*, w którym dostępnych jest wiele plików nagłówkowych języka C — są to pliki z rozszerzeniem *.h*. Znajduje się w nim również wiele plików implementacyjnych C++ — mają one rozszerzenie *.cpp* (zobacz rysunek 2.3).

Po otwarciu pliku *Arduino.h* w edytorze okaże się, że zawiera on wiele linii kodu rozpoczynających się od `#include`. Polecenia te pobierają definicje z innych plików nagłówkowych znajdujących się w folderze *cores\arduino*, dołączanych do szkicu podczas jego kompilacji (procesu przetwarzania kodu programu na taką formę, która może zostać umieszczona w pamięci flash mikrokontrolera).

We wspomnianym pliku znajdziesz również definicje stałych, na przykład:

```
#define HIGH 0cx1
#define LOW 0x0

#define INPUT 0x0
#define OUTPUT 0x1
#define INPUT_PULLUP 0x2
```



Rysunek 2.3. Zawartość folderu arduino

Można je traktować poniekąd jak zmienne — nazwie HIGH przypisano wartość 1. Przypisana wartość to 0x1, a nie po prostu 1, ponieważ określone wartości są zapisane w *systemie szesnastkowym*. Tak naprawdę nie są to definicje zmiennych, lecz tak zwane **przedkompilacyjne dyrektywy języka C** — gdy Twój szkic będzie przetwarzany na postać, którą można zainstalować w pamięci flash mikrokontrolera, wszędzie tam, gdzie występują na przykład słowa HIGH i LOW, będą automatycznie wstawiane odpowiednie wartości liczbowe. Jest to rozwiązanie lepsze od stosowania zmiennych, ponieważ nie wymaga rezerwowania pamięci.

Stałe te są numerami, a więc możesz zastosować je bezpośrednio w swoim szkicu, na przykład w celu skonfigurowania pinu numer 5 tak, aby działał w charakterze wyjścia. Lepiej jednakże, abyś zastosował nazwę OUTPUT — uchroni Cię to przed sytuacją, w której osoby rozwijające Arduino zdecydowały o zmianie wartości stałych. Stosowanie nazw sprawia, że kod szkicu jest czytelniejszy.

```
setMode(5, 1);
setMode(5, OUTPUT);
```

Wewnątrz pliku *Arduino.h* znajdziesz również wiele „sygnatur” funkcji, na przykład:

```
void pinMode(uint8_t, uint8_t);
void digitalWrite(uint8_t, uint8_t);
int digitalRead(uint8_t);
```



```
int analogRead(uint8_t);
void analogReference(uint8_t mode);
void analogWrite(uint8_t, int);
```

Ostrzegają one kompilator o istnieniu funkcji, których właściwe implementacje znajdują się gdzieś indziej. Przyjrzyjmy się na przykład pierwszej linii przytoczonego kodu. Określa ona polecenie `pinMode` jako funkcję przyjmującą dwa argumenty (jak wiesz, są to: numer pinu i tryb pracy), które są określone jako typ `uint8_t`. Zapis `void` oznacza, że funkcja po wywołaniu nie zwraca żadnych wartości.

Być może zastanawia Cię, dlaczego typ parametrów został określony jako `uint8_t`, a nie po prostu `int`. Zwykle, definiując używany pin, określamy go za pomocą zmiennej typu `int`. Tak naprawdę `int` jest uniwersalnym typem danych stosowanym podczas pisania szkiców. Użytkownik nie musi martwić się o sposób, w jaki przetwarzane będą użyte przez niego duże wartości. W przypadku języka C i mikrokontrolera Arduino `int` jest w istocie 16-bitową liczbą ze znakiem — ten typ danych może przedstawić liczby od $-32\,768$ do $32\,767$. W praktyce podczas określania numeru pinu, z którego chcemy korzystać, podawanie liczby ujemnej nie ma sensu, a stworzenie Arduino posiadającego 32 767 pinów również wydaje się mało prawdopodobne.

Zapis `uint_8` jest nieco dokładniejszą konwencją definiowania typu danych. W języku C typ danych `int` może być zapisany za pomocą od 16 do 64 bitów (zależnie od implementacji). Zapis ten należy odczytywać tak: `u` oznacza liczbę bez znaku, `int` — liczbę całkowitą (typu `int`), a po symbolu `_` pojawia się liczba bitów. Zapis `uint_8` oznacza 8-bitową liczbę typu `integer`. Za pomocą tego typu danych można przedstawić liczby z zakresu od 0 do 255.

Tak dokładnie zdefiniowane typy danych mogą być wykorzystywane w szkicach i niektórych programiści rzeczywiście je stosują. Warto pamiętać o tym, że stosowanie takich konwencji sprawia, że program staje się mało czytelny dla osób mniej zaawansowanych w dziedzinie programowania Arduino.

Stosowanie 16-bitowych zmiennych typu `int` zamiast na przykład zmiennych typu `uint_8` jest możliwe, ponieważ kompilator automatycznie przeprowadza konwersję danych. Stosowanie zmiennych typu `int` w rzeczywistości prowadzi do zbędnego zajmowania zbyt dużej przestrzeni pamięci, ale z drugiej strony upraszcza kod i sprawia, że jest on czytelniejszy. Ogólnie rzecz biorąc, w programowaniu lepiej jest napisać bardziej czytelny kod, który zajmuje więcej pamięci, niż minimalizować ilość zajmowanej pamięci za wszelką cenę, chyba że pisziesz bardzo złożony program, który może wypełnić całą dostępną pamięć mikrokontrolera.

Przypomina to przewożenie towarów ciężarówką. Jeżeli masz dość dużo rzeczy do przewiezienia, to warto, abyś układał je tak, żeby zmieścić cały ładunek w dostępnej przestrzeni bagażowej. Jeżeli masz do przewiezienia tylko kilka rzeczy, które z łatwością mieszczą się w ciężarówce, to szkoda tracić czas na ich pieczołowite układanie.

W folderze *arduino* znajdziesz również plik *main.cpp*. Otwórz go. Jego zawartość jest dość interesująca:

```
#include <Arduino.h>

int main(void)
{
    init();
```

```

#if defined(USBCON)
    USBDevice.attach();
#endif

    setup();

    for (;;) {
        loop();
        if (serialEventRun) serialEventRun();
    }

    return 0;
}

```

Jeżeli programowałeś wcześniej w języku C, C++ lub Java, to już wiesz, do czego służy funkcja `main`. Jest ona uruchamiana automatycznie po otwarciu programu. Jest to punkt, od którego zaczyna się wykonywanie całego programu. Funkcja ta istnieje również w przypadku programów pisanych dla Arduino, aczkolwiek jest niewidoczna dla osoby tworzącej szkic, która ma zaimplementować w swoim szkicu dwie funkcje — `setup` i `loop`.

Jeżeli przyjrzyysz się dokładnie zawartości pliku *main.cpp* i pominiesz kilka pierwszych linii, to zauważysz, że przedstawiony kod wywołuje funkcję `setup()`, a następnie — bez żadnych warunków — uruchamia pętlę `for`, w której wywołana zostaje funkcja `loop`.

Polecenie `for(;;)` jest równoważne z zapisem `while(true)`. Zwróć uwagę na to, że poza uruchomioną funkcją `loop` wewnątrz pętli `for` znajduje się polecenie `if` sprawdzające, czy Arduino nie odebrało żadnych danych przekazanych poprzez port szeregowy. Gdyby takie dane zostały odebrane przez mikrokontroler, to polecenie `if` sprawi, że zostaną one obsłużone.

Wróćmy do początkowej części pliku *main.cpp* — pierwsza linia tego pliku to polecenie `include`, które pobiera wszystkie definicje ze wspomnianego wcześniej pliku nagłówkowego *Arduino.h*.

Dalej widzisz początek definicji funkcji `main`, która zaczyna się od wywołania funkcji `init()`. Jeżeli poszukasz, to znajdziesz ją w pliku *wiring.c*; wywołuje ona funkcję `sei` umożliwiającą obsługę przerwań.

Poniższe linie kodu są kolejnym przykładem dyrektywy preprocesora:

```

#if defined(USBCON)
    USBDevice.attach();
#endif

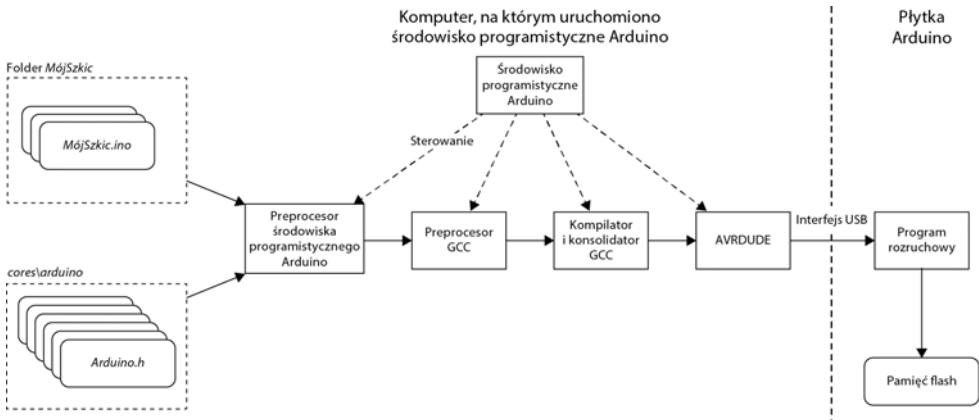
```

Kod ten przypomina nieco polecenie `if` używane w szkicach, ale warunek wykonania kodu nie jest podejmowany podczas pracy Arduino. Warunek wykonania polecenia `#if` jest sprawdzany podczas kompilacji szkicu. Dyrektywa ta pozwala na dołączanie pewnych elementów do kompilowanego programu lub ich wyłączenie, w zależności od rodzaju zastosowanej płytki. W tym przypadku, jeżeli płytka Arduino obsługuje interfejs USB, to odpowiedni kod obsługujący to złącze zostanie dołączony do programu. Jeżeli płytka nie obsługuje interfejsu USB, to kompilacja kodu inicjującego ten interfejs nie ma sensu.

Od szkicu do Arduino

Już wiesz, skąd bierze się cały kod powstający w wyniku napisania prostego szkicu Arduino. Teraz czas przyjrzeć się temu, jak kod jest umieszczany w pamięci flash mikrokontrolera znajdującego się na płytce Arduino (po kliknięciu na przycisk *Zaladuj* w środowisku programistycznym Arduino).

Na rysunku 2.4 przedstawiono to, co dzieje się po kliknięciu na przycisk *Zaladuj*.



Rysunek 2.4. Ładowanie szkicu do pamięci Arduino

Szkice Arduino są zapisywane w plikach tekstowych o rozszerzeniu *.ino* w folderach o takich samych nazwach (oczywiście w nazwie folderu nie umieszcza się rozszerzenia).

Tak naprawdę środowisko programistyczne Arduino steruje pracą wielu programów narzędziowych bezpośrednio wykonujących czynności związane z ładowaniem programu. Na początku część środowiska programistycznego Arduino, której nadałem (moim zdaniem lepszą) nazwę „preprocesor środowiska programistycznego Arduino”, asembluje pliki będące częścią szkicu. Zwróć uwagę na to, że w folderze szkicu znajduje się zwykle tylko jeden plik, w folderze tym jednak możesz umieścić ich więcej, przy czym będziesz musiał utworzyć te pliki za pomocą zewnętrznego edytora.

Jeżeli w folderze znajdują się inne pliki, to zostaną one również uwzględnione w procesie budowy programu. Pliki C i C++ są kompilowane oddzielnie. W górnej części głównego pliku szkicu dodawana jest linia kodu załączająca bibliotekę *arduino.h*.

Istnieje wiele różnych płytek Arduino — znajdują się na nich różne chipy mikrokontrolerów, które posiadają różne nazwy pinów. W związku z tym środowisko programistyczne Arduino musi znać definicje pinów właściwe dla posiadanej płytki. W katalogu *hardware\arduino\variants* znajdziesz foldery odpowiadające różnym płytkom Arduino. W każdym z tych folderów umieszczony jest plik *pins_arduino.h*. Pliki te zawierają nazwy pinów odpowiednie dla danej platformy.

Gdy wszystko zostało już połączone, kolejny krokiem jest wywołanie kompilatora GCC. Program ten jest otwartym kompilatorem języka C++, dołączonym do środowiska

programistycznego Arduino. Na podstawie szkicu, nagłówka i plików implementacyjnych kodu źródłowego języka C tworzy on coś, co może być uruchomione przez płytkę Arduino. Wykonuje szereg operacji:

1. Preprocesor interpretuje wszystkie polecenia `#if` i `#define`, a następnie określa, co będzie budowane.
2. Następnie kod jest kompilowany i konsolidowany do postaci pojedynczego pliku wykonywalnego, odpowiedniego dla procesora zastosowanego na wybranej płytce.
3. Gdy kompilator skończy pracę, kolejny otwarty program, o nazwie *avrdude*, przesyła za pośrednictwem szeregowego interfejsu USB wykonywalny kod binarny, który jest zapisywany w postaci szesnastkowej.

Jesteśmy już w królestwie Arduino. Posiada ono mały program rezydentny zainstalowany na mikrokontrolerze znajdującym się na każdej płytce. Jest to tak zwany **program rozruchowy**. Program ten uruchamia się na chwilę po każdym uruchomieniu Arduino. To właśnie dlatego po rozpoczęciu przesyłania danych sprzęt sterujący transmisją danych, znajdujący się na płytce Arduino Uno, uruchamia ponownie mikrokontroler. W ten sposób program rozruchowy może sprawdzić, czy są do niego przesyłane szkice.

Jeżeli szkic jest przesyłany, to Arduino programuje się samodzielnie, przekształcając kod szesnastkowy z powrotem na binarny. Następnie szkic jest zapisywany w pamięci flash. Podczas kolejnego uruchomienia Arduino program rozruchowy jak zwykle sprawdzi, czy nie podejmujemy próby zainstalowania nowego szkicu, a następnie automatycznie włączy program zapisany w pamięci flash.

Dlaczego komputer nie może zaprogramować mikrokontrolera bezpośrednio, bez tego całego zagmatwanego procesu? Dzieje się tak, ponieważ do zaprogramowania mikrokontrolera niezbędny jest specjalny sprzęt komunikujący się z płytką Arduino w inny sposób. Zastanawiałeś się, do czego służy małe sześciopinowe złącze znajdujące się na płytce Arduino? Za pomocą programu rozruchowego potrafiącego monitorować port szeregowy można zaprogramować Arduino za pośrednictwem interfejsu USB, bez potrzeby posiadania specjalnego, sprzętowego programatora.

Jeżeli posiadasz taki programator (na przykład AVRISPv2, AVRDragon lub USBtinyISP), to możesz zaprogramować Arduino bezpośrednio — pomijając program rozruchowy. Tak naprawdę w roli programatora możesz zastosować drugie Arduino — dowiesz się o tym w dalszej części niniejszego rozdziału.

AVR Studio

Niektórzy cyniczni inżynierowie elektronicy wyśmiewają Arduino. Twierdzą, że nie ma ono żadnej technicznej przewagi nad narzędziami dostarczonymi przez firmę Atmel, służącymi do programowania mikrokontrolerów z serii AVR. Technicznie rzecz biorąc, mają rację. Celem Arduino jednak było odczarowanie pracy z mikrokontrolerami i umożliwienie jej osobom, które nie są ekspertami w dziedzinie elektroniki. Niektóre cechy Arduino mogą sprawiać wrażenie, że jest to platforma przeznaczona dla amatorów, ale co z tego?

AVR Studio jest aplikacją stworzoną przez producenta mikrokontrolerów znajdujących się na płytkach Arduino. Możesz ją stosować do programowania Arduino (zamiast omówionego wcześniej zintegrowanego środowiska programistycznego). Będziesz jednak musiał zgodzić się na pewne warunki:

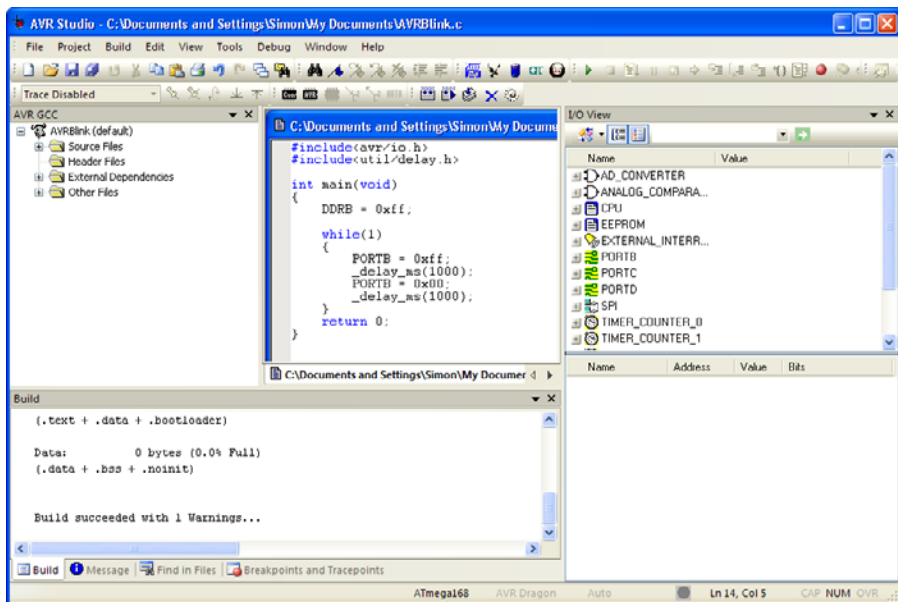
- praca wyłącznie w systemie Windows,
- stosowanie sprzętowego programatora zamiast interfejsu USB,
- obsługa bardziej złożonego środowiska.

Po co więc korzystać z tego programu? Ma on pewne zalety:

- Można pozbyć się programu rozruchowego (w przypadku płytki Uno zajmuje on 500 bajtów pamięci), a więc można rozwiązać problem brakującej pamięci flash lub szybciej uruchamiać Arduino.
- Umożliwia pracę z innymi mikrokontrolerami niż te standardowo montowane na płytkach Arduino; takimi kontrolerami może być na przykład tańsza i mniejsza seria układów ATtiny.
- Możesz po prostu nauczyć się czegoś nowego.

Wszystkie płytki Arduino są wyposażone w sześciostykowe złącze pozwalające na bezpośrednie programowanie Arduino za pomocą programu AVR Studio. Niektóre płytki są wyposażone nawet w dwa sześciostykowe złącza: jedno jest przeznaczone do programowania głównego procesora, a drugie — do programowania interfejsu USB. Podłącz programator do właściwego gniazda.

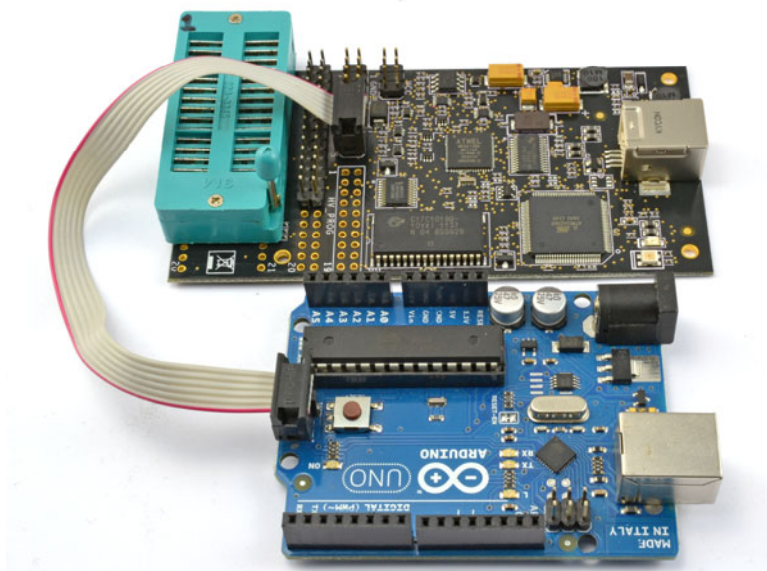
Na rysunku 2.5 pokazano uruchomioną aplikację AVR Studio 4.



Rysunek 2.5. AVR Studio

Praca w programie AVR Studio nie jest tematem tej książki. Na rysunku 2.5 widać, że szkic *Blink* nie stał się dłuższy, ale z pewnością wygląda na bardziej skomplikowany! Ten sam program po skompilowaniu w AVR Studio będzie zajmował o wiele mniej pamięci flash od tej samej aplikacji skompilowanej za pomocą środowiska programistycznego Arduino.

Na rysunku 2.6 pokazano Arduino połączone do programatora AVR Dragon. Urządzenie to charakteryzuje się dużymi możliwościami. Między innymi pozwala na przeprowadzanie diagnostyki programu, którego kod jest przetwarzany przez procesor ATmega.



Rysunek 2.6. Arduino połączone do programatora AVR Dragon

W rozdziale 4. omówię bardziej szczegółowo bezpośrednie manipulowanie portami, które widać na rysunku 2.5. Opiszę sposób na poprawę wydajności wejść i wyjść bez konieczności całkowitej rezygnacji z pracy w zintegrowanym środowisku programistycznym Arduino.

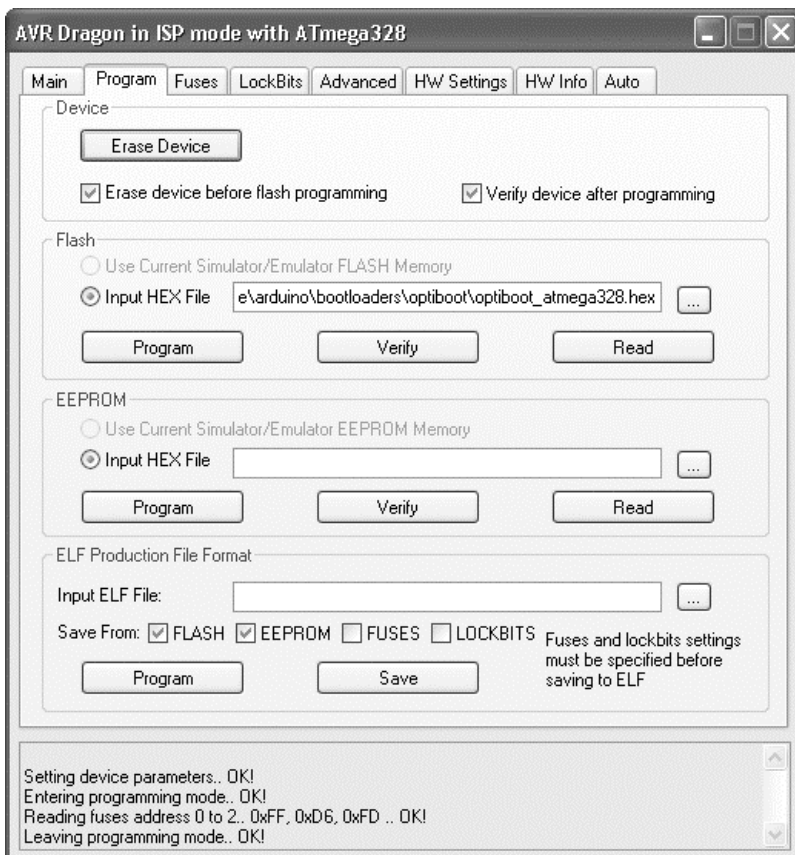
Instalacja programu rozruchowego

Do instalacji programu rozruchowego Arduino możesz być zmuszony w kilku sytuacjach. Zainstalowania programu rozruchowego może wymagać nowy chip ATmega328, który został wstawiony w płytkę Arduino, ponieważ oryginalny układ scalony został uszkodzony. Instalacja wspomnianego programu może być również konieczna podczas przenoszenia układu ATmega328 poza płytkę Arduino — podczas tworzenia własnego projektu obwodu znajdującego się na płytce.

Niezależnie od powodu, dla którego chcesz zainstalować program rozruchowy w pustej pamięci układu ATmega328, możesz w tym celu zastosować jeden z omówionych wcześniej programatorów lub drugą płytkę Arduino.

Instalacja programu rozruchowego za pomocą aplikacji AVR Studio i programatora

W folderze z zainstalowanym oprogramowaniem Arduino znajdują się heksadecymalne pliki z programem rozruchowym, który można umieścić w pamięci układu ATmega328 za pomocą aplikacji AVR Studio. Wspomniane pliki dostępne są w katalogu *hardware\arduino\bootloaders*. Znajdziesz tam pliki przeznaczone do różnych płytek. Aby zainstalować program rozruchowy przeznaczony do płytki Arduino Uno, należy skorzystać z pliku *optiboot_atmega328.hex* umieszczonego w podfolderze *optiboot* (zobacz rysunek 2.7).



Rysunek 2.7. Instalowanie programu rozruchowego płytki Arduino Uno za pomocą aplikacji AVR Studio

Zachowaj ostrożność. Jeżeli wykonasz opisane dalej czynności, to pamiętaj o tym, że możesz „zablokować” swój procesor. Układy scalone tego typu zawierają „bezpieczniki”, które czasami mogą okazać się niemożliwe do zdjęcia. Ich zadaniem jest zablokowanie możliwości przeprogramowania procesora, tak aby nie można go było stosować w celach komercyjnych (ochrona praw autorskich). Przed przystąpieniem do pracy dokładnie sprawdź, czy bezpieczniki znajdujące się na płytce Arduino są odpowiednio skonfigurowane. W przeciwnym wypadku możesz uszkodzić płytkę. Na oficjalnym forum Arduino <http://www.arduino.cc/forum> dostępnych jest wiele wątków dotyczących tego problemu. Opisano tam również sposoby jego rozwiązania.

Aby umieścić program rozruchowy w pamięci procesora za pomocą aplikacji AVR Studio i programatora AVR Dragon, podłącz go do pinów interfejsu ISP znajdujących się na płytce Arduino (przedstawiono to wcześniej na rysunku 2.6). Na płytce Arduino umieszczony jest również drugi, identyczny zestaw styków. Umożliwia on ingerencję w program rozruchowy chipa sterującego pracą interfejsu USB.

Z menu *Tools* wybierz opcję *Program AVR*, a następnie podłącz przewody do procesora ATmega328, znajdującego się na płytce Arduino. W dalszej kolejności w sekcji *Flash* wybierz właściwy plik z rozszerzeniem *.hex* i kliknij na przycisk *Program*.

Instalacja programu rozruchowego za pomocą zintegrowanego środowiska programistycznego Arduino i drugiej płytki Arduino

Zapisywanie programu rozruchowego na płytce Arduino za pomocą drugiej płytki Arduino jest dość łatwe. To zadanie prostsze i stwarzające mniej zagrożeń niż wykonywanie tej samej czynności za pomocą AVR Studio. W środowisku programistycznym Arduino znajduje się specjalna, służąca do tego opcja. Na początek będziesz potrzebował:

- dwóch płytek Arduino Uno,
- sześciu przewodów połączeniowych obustronnie zakończonych wtykami (możesz również zamiast nich korzystać z drutów),
- jednego krótkiego kawałka drutu,
- kondensatora 10 μF 10 V (możesz zastosować również kondensator o pojemności 100 μF).

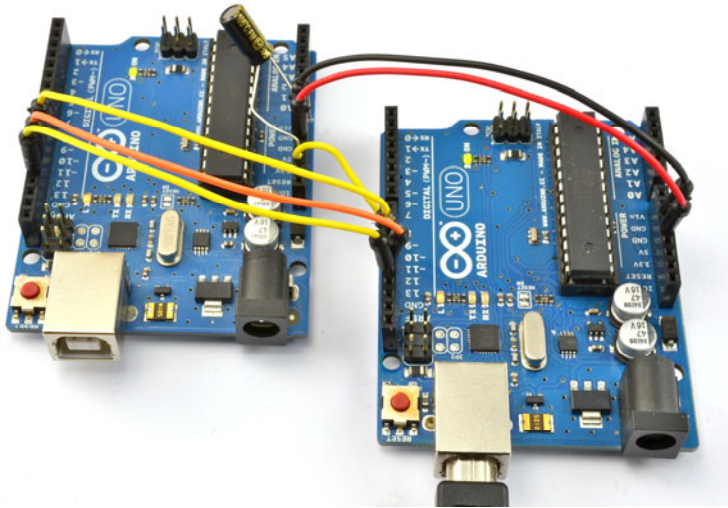
Pracę zacznij od wykonania połączeń przedstawionych w tabeli 2.1.

Tabela 2.1. Połączenia wymagane do zaprogramowania płytki Arduino Uno

Arduino Uno działające jako programator	Programowana płytka Arduino Uno
GND	GND
5V	5V
13	13
12	12
11	11
10	Reset

Złącza *Reset* i *GND* programowanej płytki Arduino należy połączyć za pośrednictwem kondensatora 10 μ F. Dłuższą nóżkę kondensatora (biegun dodatni) należy podłączyć do gniazda *Reset*.

Na rysunku 2.8 przedstawiono połączone ze sobą płytki Arduino. Płytką widoczną po prawej stronie działa w charakterze programatora. Zwróć uwagę na to, jak za pomocą drutu połączono pin numer 10 tej płytki z pinem *Reset* drugiej płytki. Połączenie to wykonano tak, aby wspomniany drut, jak również nóżka kondensatora mogły jednocześnie zmieścić się w gnieździe *Reset*.



Rysunek 2.8. Instalacja programu rozruchowego płytki Arduino za pomocą drugiej płytki Arduino

Zwróć uwagę na to, że płytka działająca jako programator zasilą płytkę programowaną, a więc tylko pierwsza z nich musi być podłączona do gniazda USB komputera.

W pamięci Arduino działającego jako programator musi być zainstalowany odpowiedni szkic. Znajdziesz go w menu *Plik/Przykłady* pod nazwą *ArduinoISP* (umieszczony jest on w górnej części listy zawierającej przykłady).

Jak zwykle, wybierz odpowiedni typ płytki i numer portu, a następnie załaduj szkic *ArduinoISP* do pamięci Arduino działającego jako programator. Teraz w menu *Narzędzia* przejdź do podmenu *Programator* i wybierz opcję *Arduino as ISP*.

Na koniec z menu *Narzędzia* wybierz opcję *Wypal Bootloader*. Proces ten trwa od minuty do dwóch minut. W tym czasie powinny migać diody LED oznaczone etykietami *Rx* i *Tx*, znajdujące się na płycie działającej jako programator. Na programowanej płycie powinna migać dioda LED oznaczona literą *L*.

W wyniku tego procesu w pamięci flash programowanej płytki Arduino zostanie zainstalowany nowy program rozruchowy.

Podsumowanie

W tym rozdziale przyjrzeliśmy się dokładniej, jak działa Arduino. Pokazałem Ci to, co ukrywa się za środowiskiem Arduino.

W kolejnym rozdziale przyjrzymy się temu, jak należy stosować przerwania, oraz temu, jak za pomocą przerwań zmusić Arduino do reagowania na zdarzenia wywoływane przez zewnętrzny zegar.

Rozdział 3.

Przerwania i zegary

Przerwania pozwalają mikrokontrolerom reagować na pewne zdarzenia bez konieczności generowania ciągłych zapytań sprawdzających, czy coś się nie zmieniło. Poza kojarzeniem przerwań z pinami możliwe jest również stosowanie przerwań generowanych przez układy zegarowe.

Przerwania sprzętowe

W poniższym przykładzie użycia przerwań korzystamy z wejść cyfrowych. Kod ten przedstawia najpopularniejszy sposób sprawdzania tego, czy doszło do jakiegoś zdarzenia wejściowego (na przykład możemy sprawdzić, czy przycisk został wciśnięty).

```
void loop
{
    if (digitalRead(inputPin) == LOW)
    {
        // zrób coś
    }
}
```

Kod ten ciągle sprawdza wejście `inputPin`; w chwili, gdy jego stan będzie niski (LOW), zostaną wykonane polecenia znajdujące się w miejscu oznaczonym komentarzem `// zrób coś`. Taki program działa poprawnie, ale co, jeżeli wewnątrz pętli chciałbyś wykonywać również wiele innych operacji? Wykonanie innych operacji zajmowałoby czas, a więc krótkie przyciśnięcie guzika mogłoby zostać pominięte przez procesor zajęty czymś innym. Tak naprawdę przeoczenie przez mikrokontroler przyciśnięcia guzika jest mało prawdopodobne, ponieważ zwykle guzik jest wciskany przez czas, który z punktu widzenia mikrokontrolera jest dość długi.

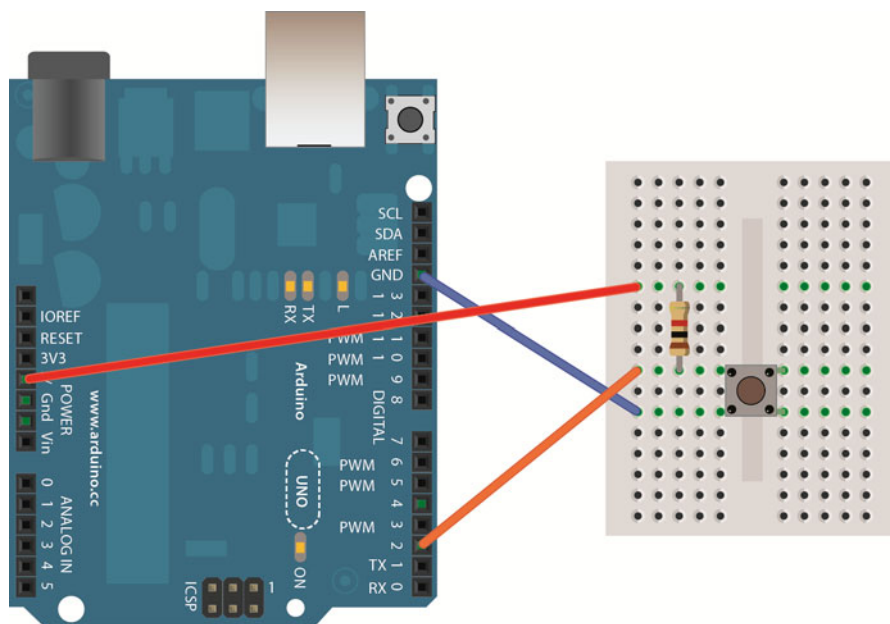
Ale co, jeżeli będziemy mieli do czynienia z generowanymi przez czujnik impulsami, które będą trwać tylko przez kilka milionowych części sekundy? Do obsługi tego typu zdarzeń

lepiej użyć przerwań i napisać funkcję, która będzie uruchamiana, gdy do nich dojdzie, niezależnie od tego, jakie operacje będą wykonywane przez mikrokontroler. W ten sposób właśnie działają tak zwane **przerwania sprzętowe**.

Na płytce Arduino Uno znajdują się tylko dwa złącza pozwalające na korzystanie z przerwań sprzętowych. Jest to jeden z powodów, dla których przerwania są dość rzadko używane. Płytkę Leonardo jest wyposażona w cztery złącza obsługujące przerwania, a większe płytki, takie jak Mega 2560, mają ich więcej — wszystkie złącza znajdujące się na płytce Duo mogą być użyte w celu wygenerowania przerwania.

Teraz czas przyjrzeć się przykładowi ilustrującemu działanie przerwań. Aby go wykonać, będziesz potrzebował płytki prototypowej, przełącznika dotykowego, rezystora 1 k Ω i kilku przewodów połączeniowych.

Obwód, który powinieneś wykonać, przedstawiono na rysunku 3.1. Rezystory podciągają pin D2 (podawany jest na niego sygnał HIGH), dopóki przycisk nie jest wciśnięty. Wciśnięcie przycisku spowoduje połączenie pinu z masą — podany zostanie na niego sygnał LOW.



Rysunek 3.1. Obwód przeznaczony do testowania przerwań

Załaduj poniższy szkic do pamięci Arduino:

```
// 03_01_przerwania
```

```
int ledPin = 13;
```

```
void setup()
```

```
{
  pinMode(ledPin, OUTPUT);
  attachInterrupt(0, stuffHappened, FALLING);
}
```

```
}  
  
void loop()  
{  
}  
  
void stuffHappened()  
{  
  digitalWrite(ledPin, HIGH);  
}
```

Funkcja `setup` konfiguruje pin określany przez zmienną `ledPin` tak, aby działał jako wyjście. Ponadto zawiera wiersz kojarzący funkcję `stuffHappened` z przerwaniem. A więc zawsze, gdy dojdzie do przerwania, spowoduje ono uruchomienie określonej funkcji. Przyjrzyjmy się uważnie tej linii kodu, ponieważ argumenty, jakie przyjmuje polecenie `attachInterrupt`, mogą wydawać się nieco dziwne:

```
attachInterrupt(0, stuffHappened, FALLING);
```

Pierwszym argumentem (0) jest numer przerwania. Zapis ten wydawałby się sensowniejszy, jeżelibyśmy podali tutaj po prostu numer odpowiedniego pinu znajdującego się na płytce Arduino. Jednakże w przypadku płytki Arduino Uno przerwanie o numerze 0 jest wywoływane przez pin *D2*, a przerwanie numer 1 jest wywoływane przez pin *D3*. W praktyce jest to jeszcze bardziej skomplikowane, ponieważ w innych płytkach Arduino numeracja pinów jest zupełnie inna, a korzystając z płytki Arduino Due, należy po prostu podać nazwę pinu. W przypadku tej płytki wszystkie piny mogą być zastosowane w celu wywołania przerwania.

Wróć do tego zagadnienia nieco później. Teraz przyjrzyjmy się drugiemu argumentowi — `stuffHappened`. Jest to nazwa funkcji wywoływanej w wyniku wystąpienia przerwania. Funkcja ta jest zdefiniowana w dalszej części szkicu. Takie funkcje określa się mianem **procedur obsługi przerwania**. Nie mogą one posiadać żadnych parametrów i nie powinny niczego zwracać. To ma sens, ponieważ można je wywołać z innego miejsca szkicu — ale tak naprawdę w szkicu nie ma żadnej linii kodu wywołującego funkcje określające procedury obsługi przerwań, a więc nie ma sposobu przypisania im parametrów, a funkcje takie nie mogą również zwracać wartości.

Ostatni parametr `attachInterrupt` jest stałą, której w tym przypadku przypisano parametr `FALLING`. Sprawia on, że przerwanie uruchamiające procedurę obsługi przerwań zostanie wywołane tylko wtedy, gdy sygnał podawany na złącze *D2* przejdzie ze stanu `HIGH` na `LOW` (innymi słowy „opadnie”), co będzie miało miejsce, gdy zostanie wciśnięty przycisk podłączony do wspomnianego złącza.

Jak widzisz, w funkcji `loop` nie umieszczono żadnego kodu. Zwykle w tym miejscu umieszczany jest kod, który będzie wykonywany aż do przerwania. Procedura obsługująca przerwanie włącza po prostu diodę LED oznaczoną literą *L*.

W czasie eksperymentowania z zaprezentowanym szkicem wspomniana dioda LED powinna zgasnąć po ponownym uruchomieniu Arduino. Dioda ta powinna zostać zapalona na stałe natychmiast po wciśnięciu przycisku.

Teraz zmień ostatni argument funkcji `attachInterrupt` na `RISING` i załaduj zmodyfikowany szkic do pamięci Arduino. Po tym, gdy Arduino zostanie ponownie uruchomione, dioda LED powinna w dalszym ciągu się palić. Dzieje się tak, ponieważ w tej sytuacji do przerwania dochodzi, gdy sygnał „wzrasta”, a stan sygnału odbieranego na złączu nie był jeszcze niski, a więc nie miał szansy wzrosnąć.

Gdy wciśniesz przycisk i przytrzymasz go, dioda LED powinna w dalszym ciągu świecić. Dioda ta zgaśnie dopiero po zwolnieniu przycisku. Czynność ta wywoła przerwanie, ponieważ gdy trzymałeś przycisk, na złącze `D2` był podawany sygnał `LOW`, a zwolnienie przycisku powoduje wzrost stanu sygnału do stanu `HIGH`.

Jeżeli tak się nie dzieje, to przełącznik prawdopodobnie generuje stuki — nie przechodzi idealnie pomiędzy stanami zwarcia i rozwarcia. Zamiast tego jego styki poruszają się, kilkakrotnie zwierając się i rozwierając. Aby zminimalizować to zjawisko, postaraj się wcisnąć przycisk dość mocno.

Działanie programu możesz również sprawdzić, przytrzymując przełącznik podczas wciskania przycisku *Reset*, znajdującego się na płytce Arduino. Gdy będziesz gotowy, zwolnij przycisk zainstalowany na płytce prototypowej, a wtedy dioda LED, oznaczona literą *L*, powinna zostać zapalona.

Piny przerwań

Wróćmy do skomplikowanych zagadnień związanych z nazewnictwem przerwań — w tabeli 3.1 przedstawiono sposób mapowania numerów przerwań do numerów pinów na najpopularniejszych płytkach Arduino.

Tabela 3.1. Piny przerwań różnych płytek Arduino

Numer przerwania							
Płytka	0	1	2	3	4	5	Uwagi
Uno	D2	D3	–	–	–	–	–
Leonardo	D3	D2	D0	D1	–	–	Numery przerwań są przypisane dokładnie odwrotnie niż w przypadku płytki Arduino Uno.
Mega 2560	D2	D3	D21	D20	D19	D18	–
Due	–	–	–	–	–	–	Zamiast numerów przerwań stosuje się numery pinów.

Łatwo jest wpaść w pułapkę — numery pierwszych dwóch przerwań na płytkach Uno i Leonardo są zamienione ze sobą. Praca z płytką Due jest znacznie bardziej logiczna — zamiast numerów przerwań stosuje się numery pinów.

Tryby przerwań

Najbardziej przydatnymi trybami są przedstawione wcześniej RISING i FALLING. Istnieją również inne tryby przerwań, które uwzględniono w tabeli 3.2.

Tabela 3.2. Tryby przerwań

Tryb	Działanie	Uwagi
LOW	Podanie sygnału o stanie LOW wywołuje przerwanie.	W tym trybie procedura obsługi przerwania działa cały czas, dopóki na pin podawany jest niski sygnał.
RISING	Uruchamia przerwanie, gdy sygnał podawany na pin zmieni stan z LOW na HIGH.	–
FALLING	Uruchamia przerwanie, gdy sygnał podawany na pin zmieni stan z HIGH na LOW.	–
CHANGE	Uruchamia przerwanie, gdy sygnał podawany na pin zmieni swój stan.	–
HIGH	Podanie sygnału o stanie HIGH wywołuje przerwanie.	Tryb ten jest obsługiwany tylko przez płytkę Due i podobnie jak z trybu LOW rzadko się z niego korzysta.

Aktywacja wbudowanego rezystora podciągającego

W schemacie poprzedniego przykładu zastosowano rezystor podciągający. Bardzo często przerwanie wywołuje się za pomocą czujnika wyposażonego w wyjście cyfrowe. W takim przypadku nie ma potrzeby używania rezystora podciągającego.

Jeżeli w roli czujnika stosujesz przełącznik podłączony do Arduino tak, jak pokazano na rysunku 3.1, to nie musisz używać w jego obwodzie zewnętrznego rezystora. Możesz skorzystać z wewnętrznego rezystora podciągającego (około 40 kΩ). W tym celu musisz zdefiniować odpowiedni typ pinu przerwania — INPUT_PULLUP. Należy to zrobić w funkcji setup. W poniższym przykładzie interesującą nas linię kodu oznaczono pogrubieniem:

```
void setup()
{
    pinMode(ledPin, OUTPUT);
pinMode(2, INPUT_PULLUP);
    attachInterrupt(0, stuffHappened, RISING);
}
```

Procedury obsługi przerwań

Możliwość przerwania operacji wykonywanych w funkcji loop może wydawać się prostym sposobem na przykład na przechwytywanie wciśnień klawiszy. Istnieją jednak pewne warunki ograniczające to, co można zrobić za pomocą procedury obsługi przerwań.

Zwykle procedura obsługi przerwań powinna być tak krótka, jak to tylko możliwe. Jeżeli podczas wykonywania poleceń znajdujących się w procedurze obsługującej dane przerwanie dojdzie do kolejnego przerwania, to wykonywana procedura nie zostanie przerwana. Sygnał przerwania będzie ignorowany aż do zakończenia wykonywania wspomnianych poleceń. Oznacza to, że stosując procedurę obsługi przerwań, na przykład do pomiaru częstotliwości, możesz uzyskać nieprawidłowy wynik pomiaru.

Ponadto aż do zakończenia wykonywania procedury obsługi przerwań kod znajdujący się w funkcji `loop` nie będzie wykonywany.

Podczas wykonywania poleceń znajdujących się w procedurze obsługi przerwań kolejne przerwanie są automatycznie ignorowane. Zapobiega to przerwaniu procedury obsługującej przerwanie przez inną procedurę obsługującą kolejne przerwanie. Niestety takie rozwiązanie ma również pewne wady. Funkcja `delay` nie będzie działać, ponieważ korzysta z zegarów i przerwań. Funkcja `millis` również nie będzie działała prawidłowo. Funkcja `delay` korzysta z `millis`. Funkcja `millis` podaje czas od uruchomienia mikrokontrolera aż do uruchomienia procedury obsługującej przerwanie, ale podczas wykonywania poleceń zawartych w procedurze nie będzie zmieniała swojej wartości. Możesz jednak stosować funkcję `delayMicroseconds`, ponieważ nie korzysta ona z przerwań.

Szeregowa transmisja danych również korzysta z przerwań, a więc nie odczytuj danych za pomocą polecenia `Serial.print`. Czasem może się okazać, że polecenie to będzie działało prawidłowo, ale nie oczekuj, że będzie tak zawsze.

Zmienne ulotne

Funkcja będąca procedurą obsługującą przerwanie nie może przyjmować parametrów i zwracać wartości, musi istnieć jakiś inny sposób na przepływ danych pomiędzy tą funkcją a resztą programu. Zwykle stosuje się w tym celu zmienne globalne, co zaprezentowano w tym przykładzie:

```
// 03_02_przerwanie_blysk

int ledPin = 13;
volatile boolean flashFast = false;

void setup()
{
    pinMode(ledPin, OUTPUT);
    attachInterrupt(0, stuffHappened, FALLING);
}

void loop()
{
    int period = 1000;
    if (flashFast) period = 100;
    digitalWrite(ledPin, HIGH);
    delay(period);
    digitalWrite(ledPin, LOW);
    delay(period);
}
```



```
void stuffHappened()  
{  
    flashFast = ! flashFast;  
}
```

W tym szkicu zastosowano zmienną globalną o nazwie `flashFast`. W funkcji `loop` służy ona do określania czasu opóźnienia. Procedura obsługi przerwania zmienia wartość tej zmiennej na `true` lub `false`.

Zwróć uwagę na to, że w deklaracji zmiennej `flashFast` zastosowano słowo `volatile`. Szkic może działać bez użycia polecenia `volatile`, ale powinieneś je stosować, ponieważ jeżeli nie zadeklarujesz za pomocą tego polecenia zmiennej ulotnej, to kompilator może wygenerować kod maszynowy, który będzie przechowywał jej stan w pamięci podręcznej w celu poprawy wydajności działania programu. Jeżeli proces zapisywania w pamięci podręcznej zostanie przerwany (może do tego dojść w tym szkicu), to zmienna może zostać błędnie zapisana.

Podsumowanie wiadomości na temat procedur obsługi przerwań

Tworząc procedury obsługi przerwań, pamiętaj o następujących rzeczach:

- Muszą one być szybkie.
- Przekazuj dane pomiędzy procedurami obsługi przerwań a resztą programu za pomocą zmiennych ulotnych.
- Nie stosuj polecenia `delay`, możesz korzystać z polecenia `delayMicroseconds`.
- Nie oczekuj, że zapis lub odczyt danych za pomocą portu szeregowego będzie zawsze przebiegał poprawnie.
- Nie oczekuj, że wartość zwracana przez funkcję `millis` będzie się zmieniać.

Włączanie i wyłączanie obsługi przerwań

Standardowo przerwania są obsługiwane przez szkic, ale ich obsługa jest automatycznie zawieszana na czas potrzebny na wykonanie poleceń znajdujących się w procedurze obsługi przerwań, o czym pisałem już wcześniej. Istnieje jednak sposób na jawne włączenie lub wyłączenie obsługi przerwań za pomocą funkcji `interrupts` i `noInterrupts`. Obydwie funkcje nie przyjmują żadnych parametrów. Pierwsza z funkcji zezwala na obsługę przerwań, a druga wyłącza obsługę przerwań.

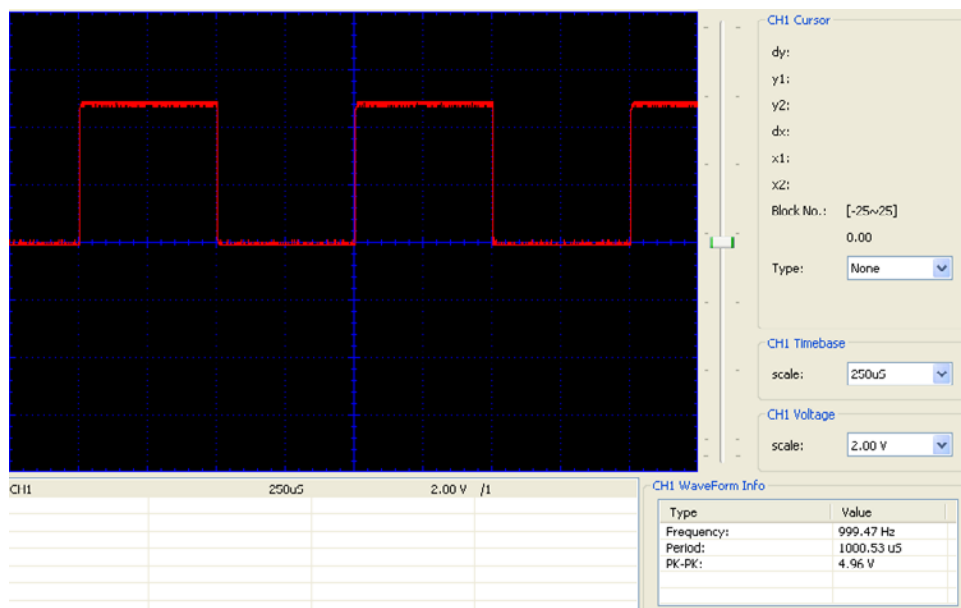
Jawne włączenie i wyłączenie obsługi przerwań przydaje się w okolicy kodu, którego wykonywanie nie powinno zostać przerwane, na przykład wtedy, gdy dochodzi do transferu danych za pomocą interfejsu szeregowego lub generowania impulsów o dokładnie określonej długości za pomocą funkcji `delayMicroseconds`.

Zegary i przerwania

Przerwania są wywoływane przez zdarzenia zewnętrzne, procedurę obsługi przerwania można jednakże uruchomić jako zdarzenie zaplanowane. Taka możliwość jest przydatna, gdy chcesz uzależnić wykonanie pewnych czynności od określonego czasu.

Do wywoływania przerwań po upływie określonego czasu można zastosować bibliotekę `TimerOne`. Można ją znaleźć pod adresem <http://playground.arduino.cc/Code/Timer1>.

Poniższy przykład ilustruje to, jak można zastosować wspomnianą bibliotekę do wygenerowania fali kwadratowej o częstotliwości 1 kHz. Jeżeli posiadasz oscyloskop lub multimetr potrafiący wykonywać pomiar częstotliwości, to podłącz go do pinu numer 12 w celu zaobserwowania sygnału (zobacz rysunek 3.2).



Rysunek 3.2. Fala kwadratowa wygenerowana za pomocą zegara

```
// 03_03_1kHz

#include <TimerOne.h>

int outputPin = 12;
volatile int output = LOW;

void setup()
{
  pinMode(12, OUTPUT);
  Timer1.initialize(500);
  Timer1.attachInterrupt(toggleOutput);
}

void loop()
```

```

{
}

void toggleOutput()
{
    digitalWrite(outputPin, output);
    output = ! output;
}

```

Mogłeś uzyskać taki sam efekt przy użyciu polecenia `delay`, ale stosując przerwanie wywołane za pomocą zegara, możesz wykonywać również inne polecenia znajdujące się w pętli. Ponadto, korzystając z funkcji `delay`, nie sposób jest wygenerować częstotliwości tak dokładnie — czas potrzebny w rzeczywistości na wygenerowanie wysokiego sygnału nie byłby brany pod uwagę przez tę funkcję.

UWAGA

Procedury obsługi przerwań wywołanych za pomocą zegara podlegają takim samym ograniczeniom, jak w przypadku przerwań wywołanych przez czynniki zewnętrzne.

Przerwanie zegarowe można wywołać w celu wykonania dowolnej czynności po upływie od 1 μ s do 8 388 480 μ s (8,4 s). Czas ten (w mikrosekundach) należy podać jako argument funkcji `initialize`.

Biblioteka `TimerOne` pozwala na wygenerowanie na pinach o numerach 9 i 10 sygnału PWM (modulacja szerokości impulsu) za pomocą zegara. Zabieg taki może wydawać się czymś zbędnym, ponieważ identyczny efekt można osiągnąć, stosując funkcję `analogWrite`, ale metoda ta pozwala na uzyskanie większej kontroli nad sygnałem PWM. Cykl pracy można określić za pomocą wartości od 0 do 1023, a podczas korzystania z funkcji `analogWrite` możliwe jest określenie go z mniejszą dokładnością — od 0 do 255. Ponadto częstotliwość sygnału PWM generowanego za pomocą funkcji `analogWrite` jest stała — 500 Hz, a biblioteka `TimerOne` pozwala na jej modyfikację przy użyciu zegara.

Aby wygenerować sygnał PWM za pomocą biblioteki `TimerOne`, należy skorzystać ze znajdującej się w niej funkcji `Timer1.pwm` tak, jak pokazano w poniższym szkicu:

```

// 03_04_pwm

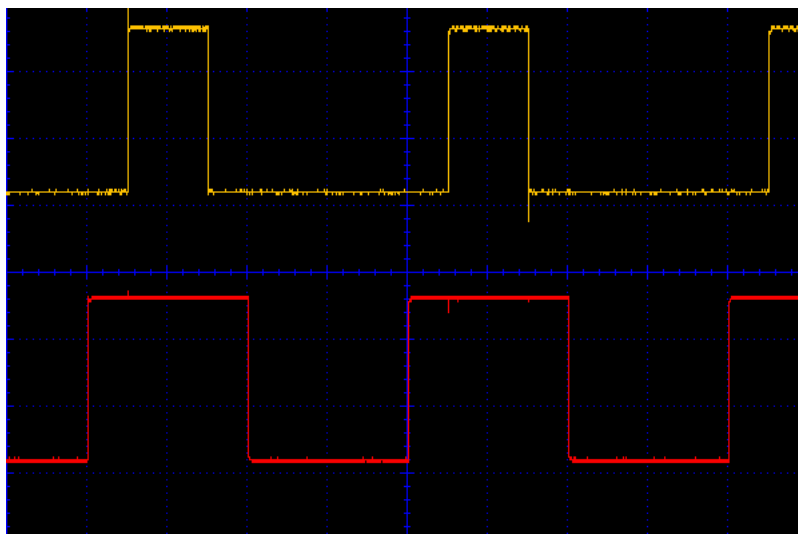
#include <TimerOne.h>

void setup()
{
    pinMode(9, OUTPUT);
    pinMode(10, OUTPUT);
    Timer1.initialize(1000);
    Timer1.pwm(9, 512);
    Timer1.pwm(10, 255);
}

void loop()
{
}

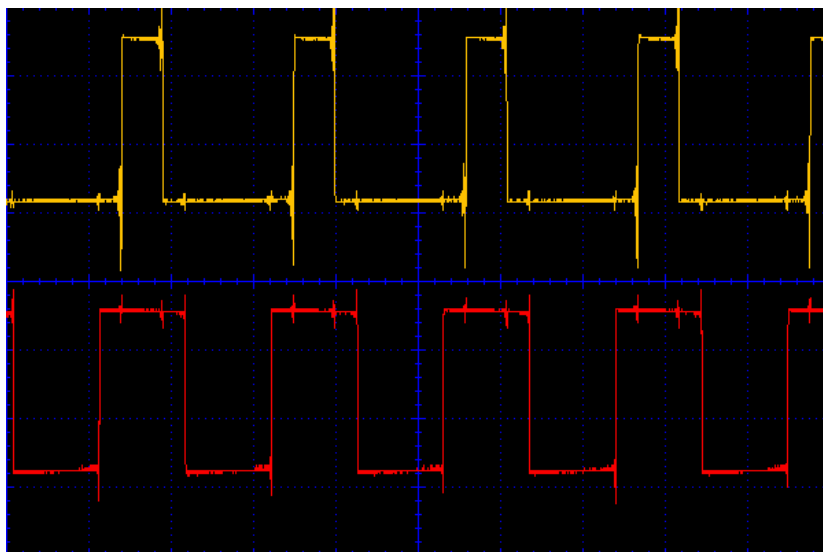
```

W tym przykładzie skonfigurowałem całkowity okres tak, aby trwał 1000 μ s. W ten sposób sygnał PWM jest generowany z częstotliwością 1 kHz. Na rysunku 3.3 przedstawiono sygnały wygenerowane na pinach numer 10 (górny wykres) i 9 (dolny wykres).



Rysunek 3.3. Sygnał PWM o częstotliwości 1 kHz, wygenerowany za pomocą biblioteki TimerOne

Sprawdźmy, jak bardzo możemy modyfikować częstotliwość sygnału PWM. Zmiana okresu na 10 spowoduje wygenerowanie sygnału o częstotliwości 100 kHz. Wykresy wygenerowanych sygnałów pokazano na rysunku 3.4.



Rysunek 3.4. Sygnał PWM o częstotliwości 100 kHz, wygenerowany za pomocą biblioteki TimerOne

Sygnal, jak się pewnie domyślałeś, zawiera sporo zakłóceń, ale jak widzisz, cykl pracy jest wciąż dość dobrze odwzorowany i osiąga wartości zbliżone do, odpowiednio, 25% i 50%.

Podsumowanie

Przerwania mogą czasem wydawać się idealnym rozwiązaniem podczas pracy nad trudnymi projektami, ale ich stosowanie może sprawić, że usuwanie błędów z kodu stanie się o wiele trudniejsze. Nie zawsze są one najlepszym sposobem wykonania pewnych zadań. Zastosowanie przerwań należy dokładnie rozważyć. W rozdziale 14. omówię różne techniki pozwalające rozwiązywać problemy spowodowane tym, że Arduino potrafi jednocześnie wykonywać tylko jedną operację.

W rozdziale 5. wrócimy do zagadnień dotyczących przerwań. Poznasz techniki pozwalające na zastosowanie ich w celu czasowego usypiania Arduino, co pozwala na ograniczenie pobieranej energii. W rozdziale 13. omówię zastosowanie przerwań do dokładnego określania czasu cyfrowego przetwarzania sygnałów.

W kolejnym rozdziale poznasz techniki pozwalające uzyskać przez Arduino maksymalną wydajność.

Rozdział 4.

Przyspieszanie Arduino

W tym rozdziale omówiono zagadnienia związane z tym, jak szybko działa Arduino i jak można je w razie konieczności przyspieszyć.

Jak szybko działa Twoje Arduino?

Zanim przejdziemy do tematyki związanej z przyspieszaniem Arduino, sprawdźmy, jak sprawnie działa ten układ, porównajmy go z innymi komputerami. Zaczniemy od częstotliwości taktowania mikrokontrolera.

Arduino Uno jest taktowane zegarem 16 MHz. Większość poleceń (dodawanie lub sortowanie wartości zmiennej) jest wykonywana w ciągu jednego cyklu zegara, a więc układ ten jest w stanie wykonać 16 milionów operacji w ciągu sekundy. Wydaje się to dość dobrym wynikiem, nieprawdaż? Nie można tego stwierdzić z całą pewnością. Szkic napisany w języku C jest przetwarzany na wiele instrukcji.

Chciałbym porównać teraz wydajność Arduino z wydajnością mojego komputera. Dysponuję laptopem Mac wyposażonym w dwa procesory. Każdy z nich jest taktowany zegarem o częstotliwości 2,5 GHz. Procesor mojego laptopa jest taktowany zegarem ponad 150 razy szybszym od Arduino. Co prawda wykonanie instrukcji przez procesor zajmuje kilka cykli pracy, ale — tak jak mogłeś się spodziewać — jest on o wiele szybszy od Arduino.

Wypróbujmy działanie poniższego programu testującego wydajność. Uruchomię go na Arduino oraz (w nieco zmodyfikowanej wersji) na swoim laptopie.

```
// 04_01_test_wydajności

void setup()
{
  Serial.begin(9600);
  while (!Serial) {};
  Serial.println("Rozpoczynam test");
  long startTime = millis();
```

```

// początek kodu testującego
long i = 0;
long j = 0;
for (i = 0; i < 20000000; i++)
{
    j = i + i * 10;
    if (j > 10) j = 0;
}
// koniec kodu testującego
long endTime = millis();

Serial.println(j); // zapobiega optymalizacji pętli
Serial.println("Koniec testu");
Serial.print("Czas (sekundy): ");
Serial.println((endTime - startTime) / 1000);
}

void loop()
{
}

```

UWAGA

Ten sam program, napisany w języku C, znajdziesz w postaci pliku `test_wzorcowy.c` na stronie <http://www.helion.pl/ksiazki/arpokk.htm>.

Wykonanie tego programu na laptopie MacBook Pro, wyposażonym w procesor taktowany zegarem 2,5 GHz, zajęło 0,068 s, a wykonanie tego samego programu przez Arduino Uno zajęło 28 s. Arduino wykonało to zadanie w czasie około 400 razy dłuższym.

Porównanie płytek Arduino

W tabeli 4.1 podano czas wykonywania przedstawionego wcześniej programu przez różne płytki Arduino.

Tabela 4.1. Wyniki testu wydajności płytek Arduino

Płytką	Czas potrzebny na wykonanie zadania (s)
Uno	28
Leonardo	29
Arduino Pro Mini	28
Mega 2560	28
Due	2

Jak widzisz, większość płytek charakteryzuje się podobną wydajnością, zaskakujący jest jednak czas wykonania zadania przez płytkę Due. Działa ona dziesięć razy szybciej od innych.

Przyspieszanie wykonywania operacji arytmetycznych

Spróbujmy zmienić kod programu testującego tak, aby działania arytmetyczne były wykonywane na liczbach typu `float`, a nie `long`. Oba typy danych zajmują 32 bity, a więc wykonanie operacji powinno zająć podobną ilość czasu. Poniższy kod jest przeznaczony dla Arduino Uno:

```
// 04_02_test_wydajności_float

void setup()
{
    Serial.begin(9600);
    while (! Serial) {};
    Serial.println("Rozpoczynam test");
    long startTime = millis();

    // początek kodu testującego
    long i = 0;
    float j = 0.0;
    for (i = 0; i < 20000000; i++)
    {
        j = i + i * 10.0;
        if (j > 10) j = 0.0;
    }
    // koniec kodu testującego
    long endTime = millis();

    Serial.println(j); // zapobiega optymalizacji pętli
    Serial.println("Koniec testu");
    Serial.print("Czas (sekundy): ");
    Serial.println((endTime - startTime) / 1000);
}

void loop()
{
}
```

Niestety wykonanie tego zadania zajęło o wiele więcej czasu niż wykonanie programu opartego na liczbach typu `float`. Wykonanie tego programu przez Arduino trwało 467 s, a poprzedniego programu — tylko 28 s. Korzystając z wartości typu `float`, mój kod stał się około 16 razy wolniejszy od kodu opartego na wartościach `double`. Spadek wydajności był częściowo wywołany przez konwersję pomiędzy typami `float` i `integer`, co również jest dość czasochłonne.

Czy naprawdę musisz stosować wartości typu `float`?

Często popełnianym błędem jest myślenie, że pomiar czegoś takiego jak temperatura wymaga stosowania zmiennej typu `float`, ponieważ temperatura może być wyrażona liczbą typu 23,5. Co prawda czasem może zająć potrzeba wyświetlenia temperatury za pomocą zmiennej typu `float`, ale możesz przechowywać tę wartość w szkicu w postaci zmiennej innego typu.

Dane odczytywane z wejść analogowych są zmiennymi typu `int`, a tak naprawdę 12-bitowymi zmiennymi typu `int` — są to liczby z zakresu od 0 do 1023. Jeżeli chcesz, to możesz zapisać te 12 bitów za pomocą 32 bitów (zmiennej typu `float`), ale taki zabieg nie zwiększy precyzjności danych.

Czujnik może wskazywać na przykład temperaturę wyrażoną w stopniach Celsjusza ($^{\circ}\text{C}$). Popularny czujnik TMP36 generuje napięcie wyjściowe proporcjonalne do temperatury. Poniższe obliczenia stosuje się często do konwersji danych odczytanych z wejścia analogowego (liczby z zakresu od 0 do 1023) na temperaturę wyrażoną w stopniach Celsjusza.

```
int raw = analogRead(sensePin);
float volts = raw / 205.0;
float tempC = 100.0 * volts - 50;
```

Tak naprawdę musisz zaprezentować tę wartość za pomocą liczby zmiennoprzecinkowej tylko w momencie jej wyświetlania. Inne operacje wykonywane na tej wartości (na przykład porównywanie temperatury lub obliczanie średniej temperatury na podstawie kilku pomiarów) będą przebiegać znacznie szybciej, gdy operacje arytmetyczne będą wykonywane bezpośrednio na zmiennych typu `int`.

Przeglądanie kontra obliczanie

Jak już wiesz, warto jest unikać zmiennych typu `float`. Ale jeżeli chciałbyś wygenerować falę sinusoidalną za pomocą analogowego wyjścia Arduino, to musiałbyś zastosować funkcję matematyczną `sin` w celu „narysowania” fali przez wyjście analogowe. Wygenerowanie sinusoidy wiąże się z obliczeniem wartości kąta w zakresie od 0 do 2π radianów. Następnie wartość tę należy wysłać do wyjścia analogowego. W praktyce jest to nieco bardziej skomplikowane, ponieważ musisz wyśrodkować wygenerowaną falę tak, aby sygnał wyjściowy nie przekraczał połowy maksymalnej wartości.

Poniższy kod generuje sinusoidę w 64 krokach na cykl, na wyjściu DAC0 płytki Arduino Due. Tylko płytka Arduino wyposażona w prawdziwe analogowe wyjście będzie w stanie prawidłowo wykonać ten program.

```
// 04_03_sinus

void setup()
{

}

float angle = 0.0;
float angleStep = PI / 32.0;

void loop()
{
    int x = (int)(sin(angle) * 127) + 127;
    analogWrite(DAC0, x);
    angle += angleStep;
    if (angle > 2 * PI)
```

```

    {
        angle = 0.0;
    }
}

```

W wyniku pomiaru sygnału wyjściowego okazało się, że Arduino generuje ładną sinusoidę o częstotliwości zaledwie 310 Hz. Procesor znajdujący się na płycie Arduino Due jest taktowany zegarem 80 MHz, a więc można było się spodziewać wygenerowania sygnału o wyższej częstotliwości. Problemem jest ciągle wykonywanie tych samych obliczeń. Jeżeli powtarzamy sekwencję tych samych działań, to możemy wykonać obliczenia tylko raz, a następnie używane wyniki umieścić w tablicy.

Poniższy kod również generuje sinusoidę w 64 krokach, ale robi to, przekazując bezpośrednio do przetwornika C/A gotowe wartości odczytane z tablicy przeglądowej:

```

// 04_04_tablica_przegladowa

byte sin64[] = {127, 139, 151, 163, 175, 186, 197, 207, 216, 225, 232, 239, 244, 248,
251, 253, 254, 253, 251, 248, 244, 239, 232, 225, 216, 207, 197, 186, 175, 163, 151,
139, 126, 114, 102, 90, 78, 67, 56, 46, 37, 28, 21, 14, 9, 5, 2, 0, 0, 0, 2, 5, 9, 14,
21, 28, 37, 46, 56, 67, 78, 90, 102, 114, 126};

void setup()
{

}

void loop()
{
    for (byte i = 0; i < 64; i++)
    {
        analogWrite(DAC0, sin64[i]);
    }
}

```

Sinusoida wygenerowana przez ten kod wygląda dokładnie tak, jak ta wygenerowana wcześniej, ale jej częstotliwość to już 4,38 kHz. Sinusoida jest generowana 14 razy szybciej niż poprzednio.

Tablicę wartości funkcji sinus można wygenerować na kilka sposobów. Nie jest to trudniejsze od obliczeń stosowanych w arkuszach kalkulacyjnych. Możesz napisać szkic, który będzie wyświetlał generowane wartości w oknie monitora portu szeregowego, skąd można je skopiować do kolejnego szkicu.

```

// 04_05_sinus_print

float angle = 0.0;
float angleStep = PI / 32.0;

void setup()
{
    Serial.begin(9600);
    Serial.print("byte sin64[] = {");
    while (angle < 2 * PI)
    {

```

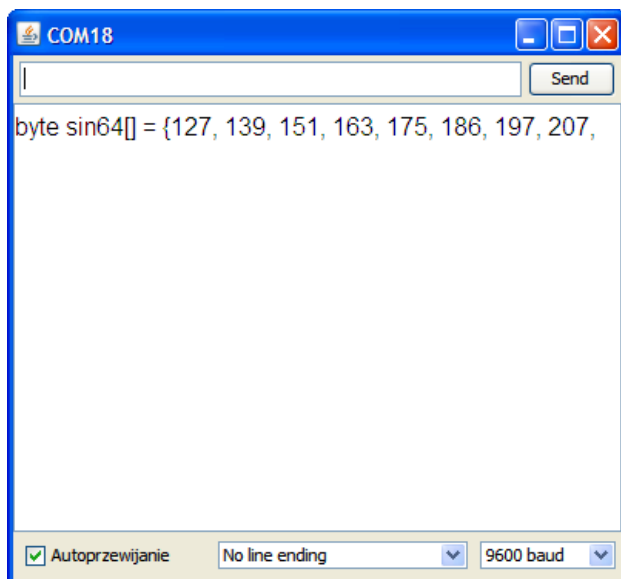
```

    int x = (int)(sin(angle) * 127) + 127;
    Serial.print(x);
    angle += angleStep;
    if (angle < 2 * PI)
    {
        Serial.print(", ");
    }
}
Serial.println("};");
}

void loop()
{
}

```

Otwórz monitor portu szeregowego. Znajdziesz tam wygenerowane wartości (zobacz rysunek 4.1).



Rysunek 4.1. Generowanie kodu za pomocą szkicu

Szybkie wejścia-wyjścia

W tej sekcji omówię zagadnienia związane z przyspieszaniem włączania i wyłączania cyfrowych złączy. Podniesiemy podstawową maksymalną częstotliwość z 73 kHz do prawie 4 MHz.

Podstawowa optymalizacja kodu

Przyjrzyjmy się podstawowemu kodowi włączającemu i wyłączającemu sygnał podawany na cyfrowe wyjście za pomocą funkcji `digitalWrite`.

```
// 04_06_fala_kwadratowa

int outPin = 10;
int state = 0;

void setup()
{
    pinMode(outPin, OUTPUT);
}

void loop()
{
    digitalWrite(outPin, state);
    state = ! state;
}
```

Jeżeli uruchomisz ten program, a do cyfrowego pinu numer 10 podłączysz oscyloskop lub miernik częstotliwości, to okaże się, że częstotliwość generowanego sygnału wynosi około 73 kHz (mój oscyloskop wskazał wartość 73,26 kHz).

Zanim zajmiemy się bezpośrednią obsługą portów za pomocą kodu, możemy dokonać pewnych optymalizacji szkicu. Żadna ze zmiennych nie musi być 16-bitową zmienną typu `int`. Obie zmienne mogą być typu `byte`. Taka zmiana zwiększy częstotliwość do 77,17 kHz. Teraz zmienną określającą numer pinu możemy zamienić na stałą — przed jej nazwą dodajmy słowo klucz `const`. Taka modyfikacja szkicu zwiększy częstotliwość do 77,92 kHz.

W rozdziale 2. dowiedziałeś się, że funkcja `loop` to coś więcej niż zwykła pętla `while` — funkcja ta sprawdza również, czy żadne dane nie są transmitowane do Arduino za pomocą portu szeregowego. Dlatego kolejna optymalizacja wydajności szkicu powinna polegać na przeniesieniu kodu z głównej funkcji `loop` do funkcji `setup`. Poniżej umieszczono kod poddany tym wszystkim modyfikacjom. Szkice poddawane kolejnym modyfikacjom¹ znajdziesz pod adresem <http://www.helion.pl/ksiazki/arpokk.htm>.

```
// 04_09_bez_funkcji_loop

const byte outPin = 10;
byte state = 0;

void setup()
{
    pinMode(outPin, OUTPUT);
    while (true)
    {
        digitalWrite(outPin, state);
        state = ! state;
    }
}
```

¹ Nagły przeskok w numeracji plików jest spowodowany tym, że autor tworzył pliki zawierające stopniowe modyfikacje szkiców, ale w książce zamieścił tylko wersję ostateczną, poddaną w sumie trzem modyfikacjom — *przyp. tłum.*

```
void loop()
{
}
```

Ten zabieg podniósł częstotliwość generowanego sygnału do 86,39 kHz.

W tabeli 4.2 przedstawiono wpływ kolejnych modyfikacji kodu na sprawność działania programu. Kolejnym, ostatnim działaniem, jakie możemy wykonać, jest rezygnacja z użycia funkcji `digitalWrite` i zastosowanie jakiegoś szybszego rozwiązania.

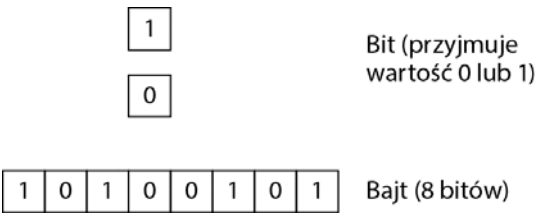
Tabela 4.2. *Optymalizacja kodu*

Modyfikacja kodu	Szkic	Częstotliwość (kHz)
Oryginalny kod	04_06	73,26 kHz
Zastosowanie zmiennych typu <code>byte</code> zamiast zmiennych typu <code>int</code>	04_07	77,17 kHz
Zamiana zmiennej określającej numer złącza na stałą	04_08	77,92 kHz
Przeniesienie kodu z funkcji <code>loop</code> do funkcji <code>setup</code>	04_09	86,39 kHz

Bajty i bity

Zanim zaczniesz bezpośrednio sterować pracą portów wejścia-wyjścia, musisz zrozumieć, czym są bajty, bity, jak również zmienne typu `byte` i `int`.

Na rysunku 4.2 porównano bity i bajty.



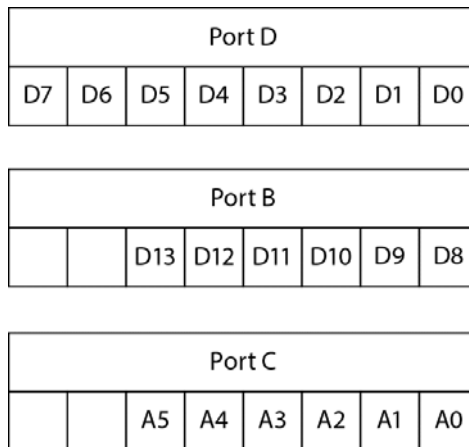
Rysunek 4.2. *Bity i bajty*

Bit (cyfra binarna) może przyjmować jedną z dwóch wartości (0 lub 1). **Bajt** składa się z 8 bitów. Każdy z tych bitów może być zerem lub jedynką, a więc za pomocą 8 bitów możemy utworzyć 256 różnych liczb binarnych. Bajt może być użyty do zapisu dowolnej liczby w zakresie od 0 do 255.

Każdy z tych bitów może być wykorzystany w celu określenia tego, czy coś jest włączone, czy wyłączone. Jeżeli chciałbyś włączyć jakiś pin, to musiałbyś przypisać określonemu bitowi wartość 1, co spowodowałoby podanie wysokiego sygnału (HIGH) na danym wyjściu.

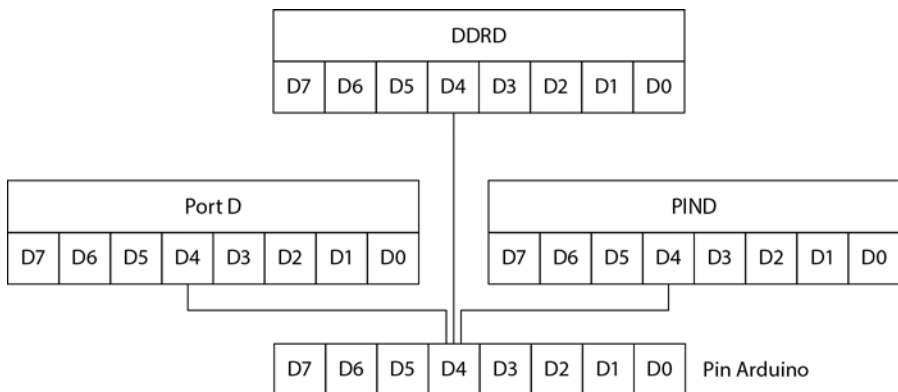
Porty układu ATmega328

Na rysunku 4.3 przedstawiono relacje pomiędzy portami układu ATmega328 a pinami płytki Arduino Uno.



Rysunek 4.3. Porty układu ATmega328

To, że wszystkie porty są 8-bitowe, a porty *B* i *C* zaledwie 6-bitowe, nie jest żadnym błędem. Każdy port jest sterowany za pomocą trzech **rejestrów**. Rejestr można rozumieć jako specjalną zmienną, której można przypisać jakąś wartość lub z której można tę wartość odczytać. Rejestr portu *D* przedstawiono na rysunku 4.4.

Rysunek 4.4. Rejestry portu *D*

Rejestr kierunku danych *D* (DDRD) składa się z 8 bitów. Każdy z nich określa, czy odpowiadający mu pin mikrokontrolera działa jako wejście czy wyjście. Jeżeli dany bit przyjmuje wartość 1, to odpowiadający mu pin działa jako wyjście. Gdy wartość ta jest zerem, wówczas pin działa jako wejście. Funkcja `pinMode` korzysta z tego rejestru.

Rejestr *Port D* jest używany do określania sygnału wyjściowego przez funkcję `digitalWrite`. Zmiana wartości bitu na 1 lub 0 spowoduje wygenerowanie sygnału wysokiego lub niskiego na odpowiednim pinie.

Do omówienia pozostał nam jedynie **rejestr wejścia portu D** (*PIND*). Odczytując wartość tego rejestru, możemy dowiedzieć się, na które piny podawany jest sygnał wysoki, a na które — niski.

Każdy z trzech portów jest obsługiwany przez trzy rejestry. Rejestry portu *B* mają oznaczenia *DDRB*, *PORTB* i *PINB*, a rejestry portu *C* można opisać za pomocą etykiet *DDRC*, *PORTC* i *PINC*.

Bardzo szybkie działanie wyjść cyfrowych

Poniższy kod obsługuje porty bezpośrednio, nie skorzystano w nim z funkcji `pinMode` i `digitalWrite`.

```
// 04_10_fala_kwadratowa_porty

byte state = 0;

void setup()
{
  DDRB = B00000100;
  while (true)
  {
    PORTB = B00000100;
    PORTB = B00000000;
  }
}

void loop()
{
}
```

W podanym przykładzie sterujemy pracą pinu *D10* należącego do portu *B*, a więc ustawiamy cyfrę 1 w miejsce trzeciego bitu od lewej (*D10*). Zwróć uwagę na zastosowanie stałej binarnej *B00000100*. W głównej pętli wystarczy najpierw w miejsce interesującego nas bitu wstawić 1, a następnie 0. W tym celu należy przypisać portowi *PORTB* wartość tak, jakby był on zmienną.

Kod po uruchomieniu generuje sygnał o częstotliwości 3,97 MHz (zobacz rysunek 4.5), czyli prawie 4 miliony impulsów na sekundę. Program działa 46 razy szybciej od wcześniejszej wersji szkicu, w której zastosowano funkcję `digitalWrite`.

Fala nie jest kwadratowa, ale takich zniekształceń można było spodziewać się po fali wygenerowanej z tak wysoką częstotliwością.

Kolejną zaletą bezpośredniego korzystania z rejestrów portów jest możliwość jednoczesnego skonfigurowania sygnału podawanego na osiem pinów, co jest bardzo przydatne podczas przekazywania danych poprzez magistralę szeregową.

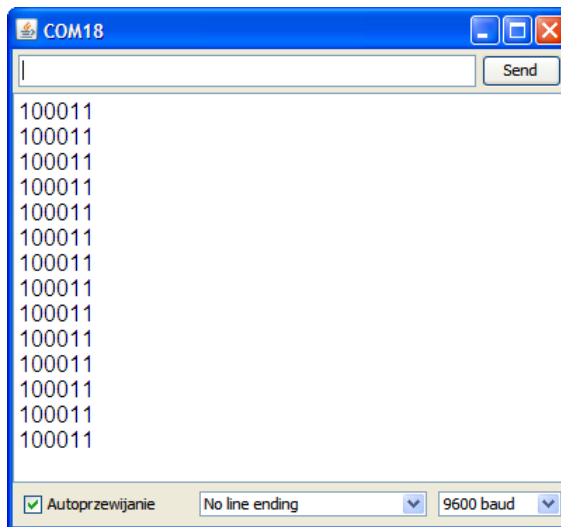
Szybkie wejścia cyfrowe

Metodę bezpośredniego dostępu do rejestrów portów można zastosować również w celu przyspieszenia odczytu danych. Jeżeli jednak chcesz przechwytywać bardzo krótkie impulsy, prawdopodobnie lepiej, żebyś skorzystał z przerw (zobacz rozdział 3.).



Rysunek 4.5. Sygnał o częstotliwości 4 MHz, wygenerowany za pomocą Arduino

Uzyskanie bezpośredniego dostępu do portu przydaje się na przykład do jednoczesnego odczytania kilku bitów. Poniższy szkic odczytuje stany wszystkich wejść portu *B* (od *D8* do *D13*) i wyświetla odczytane dane w postaci liczby binarnej w oknie monitora portu szeregowego (zobacz rysunek 4.6):



Rysunek 4.6. Jednoczesny odczyt danych z sześciu wejść

```
// 04_11_bezpośredni_odczyt

byte state = 0;

void setup()
{
  DDRB = B00000000; // wszystkie wejścia
  Serial.begin(9600);
}

void loop()
{
  Serial.println(PINB, 2);
  delay(1000);
}
```

Wszystkie bity znajdujące się w rejestrze *DDRB* są zerami — wszystkie piny działają w charakterze wejść. W funkcji *loop* użyto funkcji *Serial.println* w celu przesłania danych za pomocą monitora portu szeregowego. Dane w jego oknie zostaną wyświetlone w formie binarnej. Standardowo są one wyświetlane w formacie dziesiętnym, zastosowanie dodatkowego argumentu (2) pozwoliło na pokazanie zapisu binarnego.

Przyspieszanie wejść analogowych

Zmodyfikujmy szkic, za pomocą którego wcześniej testowaliśmy wydajność. Sprawdźmy, jak sprawnie działa funkcja *analogRead*, a następnie spróbujmy przyspieszyć działanie tego programu.

```
// 04_12_test_wejścia_analogowego

void setup()
{
  Serial.begin(9600);
  while (! Serial) {};
  Serial.println("Rozpoczynam test");
  long startTime = millis();

  // początek kodu testującego
  long i = 0;
  for (i = 0; i < 1000000; i++)
  {
    analogRead(A0);
  }
  // koniec kodu testującego
  long endTime = millis();

  Serial.println("Koniec testu");
  Serial.print("Czas (sekundy): ");
  Serial.println((endTime - startTime) / 1000);
}
```

```
void loop()
{

}
```

Wykonanie tego kodu przez Arduino Uno zajmuje 112 s, czyli płytka ta jest w stanie wykonać niemalże 9000 odczytów sygnałów analogowych na sekundę.

Funkcja `analogRead` korzysta z przetwornika analogowo-cyfrowego (A/C) zintegrowanego z mikrokontrolerem Arduino. Przetwornik ten wykonuje pomiar na zasadzie **kompensacji równoległej**. Porównuje sygnał wejściowy z regulowanym napięciem odniesienia. Przetwornik jest sterowany za pomocą układu zegarowego. Podnosząc częstotliwość jego pracy, możemy sprawić, że konwersja będzie odbywała się szybciej.

W prezentowanym dalej kodzie częstotliwość pracy przetwornika A/C zwiększono z 128 kHz do 1 MHz, w wyniku czego program powinien zostać wykonany około ośmiu razy szybciej.

```
// 04_13_test_wejścia_analogowego
const byte PS_128 = (1 << ADPS2) | (1 << ADPS1) | (1 << ADPS0);
const byte PS_16 = (1 << ADPS2);

void setup()
{
  ADCSRA &= ~PS_128; // usuwa przelicznik wstępny 128
  ADCSRA |= PS_16; // dodaje przelicznik wstępny 16 (1 MHz)
  Serial.begin(9600);
  while (! Serial) {};
  Serial.println(PS_128, 2);
  Serial.println(PS_16, 2);
  Serial.println("Rozpoczynam test");
  long startTime = millis();

  // początek kodu testującego
  long i = 0;
  for (i = 0; i < 1000000; i++)
  {
    analogRead(A0);
  }
  // koniec kodu testującego
  long endTime = millis();

  Serial.println("Koniec testu");
  Serial.print("Czas (sekundy): ");
  Serial.println((endTime - startTime) / 1000l);
}

void loop()
{
}
```

Wykonanie programu zajmuje teraz tylko 17 s, czyli szkic działa 6,5 razy szybciej, a mikrokontroler wykonuje około 58 000 odczytów na sekundę. Taka częstotliwość pozwala na próbkowanie sygnału audio, ale nie będziesz w stanie zapisać go. 2 kB pamięci RAM nie pozwolą Ci na zapis zbyt długich próbek!

Gdybyśmy uruchomili szkic *04_13_test_wejścia_analogowego* na płytce Arduino Due, to zostałby on wykonany w 39 s. Zaprezentowanej sztuczki związanej z obsługą rejestrów nie da się zaimplementować na płytce Due, ponieważ charakteryzuje się ona inną architekturą.

Podsumowanie

W tym rozdziale przedstawiłem Ci pewne sposoby na uzyskanie maksymalnej wydajności dość wolnego procesora o częstotliwości taktowania 16 MHz. W kolejnym rozdziale zaprezentuję Ci sposoby umożliwiające zminimalizowanie poboru prądu przez Arduino. Porady te są szczególnie przydatne podczas pracy nad projektami, w których Arduino będzie zasilane za pomocą baterii lub ogniw fotowoltaicznych.

Rozdział 5.

Arduino i mały pobór prądu

Warto zaznaczyć, że Arduino pobiera dość mało prądu bez stosowania jakichkolwiek zabiegów. Zwykle w czasie pracy płytka mikrokontrolera pobiera prąd o natężeniu 40 mA — podczas zasilania za pośrednictwem gniazda USB, przy napięciu 5 V, moc układu wynosi zaledwie 200 mW. Oznacza to, że Arduino może być zasilane z małej baterii 9 V (o pojemności 150 mAh) przez około 4 godzin.

Ograniczenie poboru prądu staje się ważne, gdy Arduino musi być zasilane przez długi czas za pomocą baterii. W projektach związanych ze zdalnym monitorowaniem lub sterowaniem ogniwa fotowoltaiczne i baterie są jedynymi dostępnymi środkami zasilającymi. Na przykład ostatnio wykonałem automatyczne drzwi do kurnika sterowane przez Arduino. W projekcie zastosowałem małe ogniwo słoneczne ładujące akumulator, który musi dostarczyć tyle prądu, aby można było otworzyć i zamknąć drzwi dwa razy dziennie.

Płytki Arduino i pobór prądu

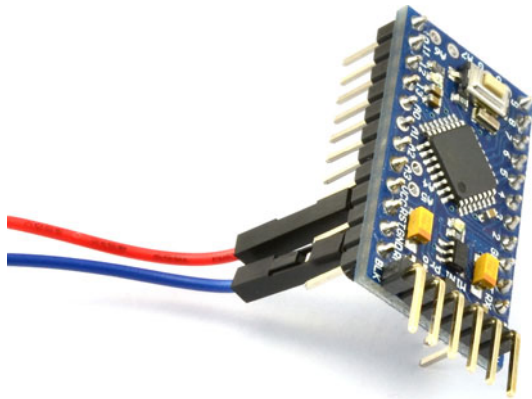
Jak dużo prądu pobierają najpopularniejsze płytki Arduino? W tabeli 5.1 przedstawiono wartości uzyskane w wyniku bezpośredniego pomiaru, za pomocą amperomierza, natężenia prądu pobieranego przez Arduino. Pamiętaj o tym, że realne natężenie prądu pobieranego przez Arduino zmienia się — układy zegarowe i inne podzespoły mikrokontrolera są czasem włączane w celu wykonania pewnych zadań.

Zwróć uwagę na to, że różnica w poborze prądu (przy zasilaniu Arduino prądem o napięciu 5 V) między płytką z procesorem a płytką bez procesora wynosi zaledwie 15 mA. A więc elektronika umieszczona na płycie pobiera prąd o natężeniu 32 mA. Na płycie Arduino znajduje się chip obsługujący interfejs USB, dioda LED sygnalizująca podłączenie urządzenia do zasilania, a także regulator napięcia 3,3 V. Wszystkie te komponenty pobierają prąd nawet wtedy, gdy na płycie nie znajduje się chip mikrokontrolera. Zwróć uwagę również na to, jak mało prądu pobiera Arduino zasilane prądem o napięciu 3,3 V.

Tabela 5.1. Natężenie prądu pobieranego przez różne płytki Arduino

Płytki	Natężenie prądu (mA)
Uno (5 V, USB)	47
Uno (9 V, zasilacz sieciowy)	48
Uno (5 V, wymontowany procesor)	32
Uno (9 V, wymontowany procesor)	40
Leonardo (5 V, USB)	42
Due (5 V, USB)	160
Due (9 V, zasilacz sieciowy)	70
Pro Mini (9 V, zasilacz sieciowy)	42
Pro Mini (5 V, USB)	22
Pro Mini (3,3 V, bezpośrednio)	8

Techniki programistyczne przedstawione w tym rozdziale pozwalają na redukcję prądu pobieranego przez sam procesor, ale nie zmniejszą ilości prądu pobieranego przez pozostałe komponenty znajdujące się na płytce. W przedstawionych przykładach korzystam z płytki Arduino Pro Mini zasilanej prądem o napięciu 3,3 V podłączonym bezpośrednio do złączy oznaczonych etykietami *VCC* i *GND* (zobacz rysunek 5.1). W ten sposób pomijamy regulator napięcia, a więc poza diodą LED, sygnalizującą podłączenie płytki do prądu, zasilamy tylko chip mikrokontrolera.

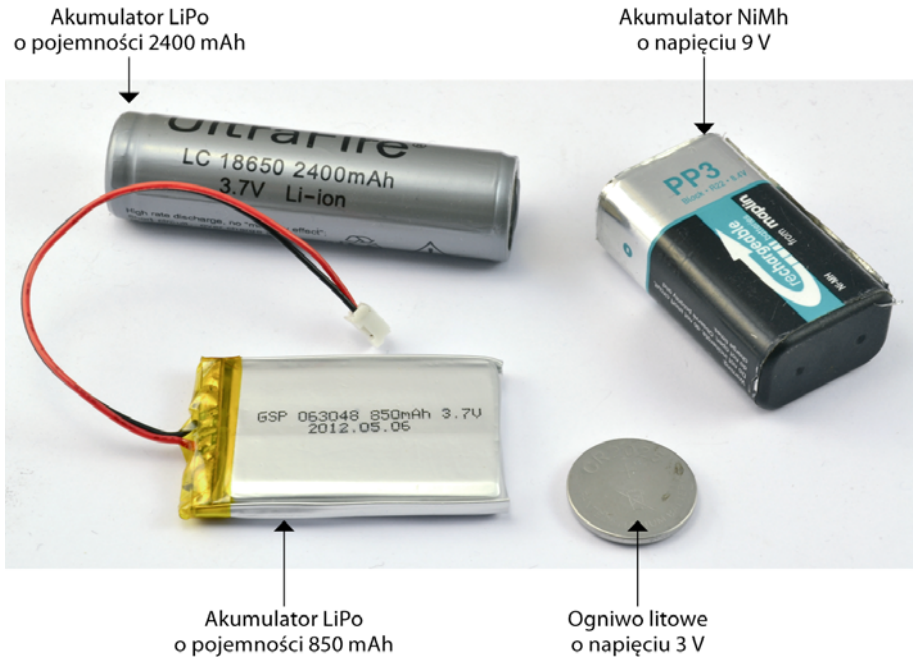


Rysunek 5.1. Płytki Arduino Pro Mini zasilana bezpośrednio prądem o napięciu 3 V

W ten sposób płytkę Arduino można zasiląć za pomocą ogniwa LiPo, które zapewnia prąd o napięciu pomiędzy 4,2 V (gdy jest w pełni naładowane) a 2,7 V (gdy jest rozładowane). Prąd o napięciu w tym przedziale może bezpośrednio zasiląć układ mikrokontrolera ATmega328.

Prąd i akumulatory

Niniejsza książka dotyczy programowania, a więc omówię tylko niezbędne zagadnienia związane z bateriami i ogniwami. Na rysunku 5.2 przedstawiono wybrane ogniwa i baterie, za pomocą których możesz zasilać Arduino.



Rysunek 5.2. Akumulatory i baterie, za pomocą których możesz zasiląć Arduino

W lewym górnym rogu rysunku znajduje się cylindryczne ogniwo LiPo o pojemności 2400 mAh. Poniżej umieszczono płaski akumulator LiPo o pojemności 850 mAh. Ogniwa LiPo są lekkie i mogą być wielokrotnie ładowane. Charakteryzują się dobrym stosunkiem pojemności do objętości i masy. W prawym górnym rogu rysunku znajduje się bateria NiMH o napięciu 9 V i pojemności 200 mAh. Bateria ta jest również ładowalna, ale wykonano ją w starszej technologii. Charakteryzuje się ona napięciem 9 V, a więc można ją stosować do zasilania Arduino tylko za pośrednictwem regulatora napięcia umieszczonego na płytce mikrokontrolera. Możesz kupić klips baterii pozwalający na podłączenie jej do okrągłego złącza zasilania znajdującego się na płytce Arduino. W prawym dolnym rogu rysunku widoczne jest nieladowalne ogniwo litowe o napięciu 3 V (CR2025), charakteryzujące się pojemnością 160 mAh.

Istnieje praktyczny sposób pozwalający określić, na ile roboczogodzin wystarczy dana bateria lub ogniwo. Wystarczy podzielić jej pojemność wyrażoną w miliamperogodzinach (mAh) przez natężenie pobieranego prądu wyrażone w miliamperach (mA):

$$\text{żywytność baterii (godziny)} = \text{pojemność baterii (mAh)} / \text{natężenie prądu (mA)}$$

Gdybyśmy zasilali płytkę Pro Mini prądem o napięciu 3 V za pomocą ogniwa CR2025, to mikrokontroler mógłby pracować przez $160 \text{ mAh} / 8 \text{ mA} = 20 \text{ h}$. Gdybyśmy podłączyli tę samą płytkę do akumulatora LiPo o pojemności 2400 mAh, to mikrokontroler mógłby pracować przez $2400 \text{ mAh} / 8 \text{ mA} = 300 \text{ h}$.

Zmniejszenie częstotliwości taktowania

Procesory znajdujące się na większości płytek Arduino są taktowane zegarem o częstotliwości 16 MHz. Mikrokontroler pobiera znaczącą ilość prądu tylko w momencie przełączania stanów logicznych z HIGH na LOW, a więc częstotliwość taktowania zegara ma znaczny wpływ na pobór prądu. Oczywiście obniżenie tej częstotliwości spowolni pracę mikrokontrolera, co czasami może stać się problemem.

Częstotliwość pracy układu ATmega328 może być zmieniona z poziomu szkicu. Wygodnym sposobem na zmianę szybkości pracy zegara jest zastosowanie biblioteki Prescaler (<http://playground.arduino.cc/Code/Prescaler>).

Biblioteka ta pozwala użytkownikowi na określenie częstotliwości pracy mikrokontrolera, ale zapewnia też funkcje `trueMillis` i `trueDelay`, zastępujące polecenia `millis` i `delay`. Korzystanie z tych funkcji staje się koniecznością, ponieważ obniżenie częstotliwości pracy układu spowoduje proporcjonalne wydłużenie czasu określanego przez funkcje `delay` i `millis`.

Poniższy przykładowy szkic włącza diodę LED, oznaczoną etykietą *L*, na 1 s, a następnie wyłącza ją na kolejne 5 s. W tym czasie można przeprowadzić pomiar prądu pobieranego przez mikrokontroler po zastosowaniu różnych poleceń regulujących szybkość pracy mikrokontrolera za pomocą biblioteki Prescaler.

```
// 05_01_biblioteka_prescaler

#include <Prescaler.h>

void setup()
{
    pinMode(13, OUTPUT);
    setClockPrescaler(CLOCK_PRESCALER_256);
}

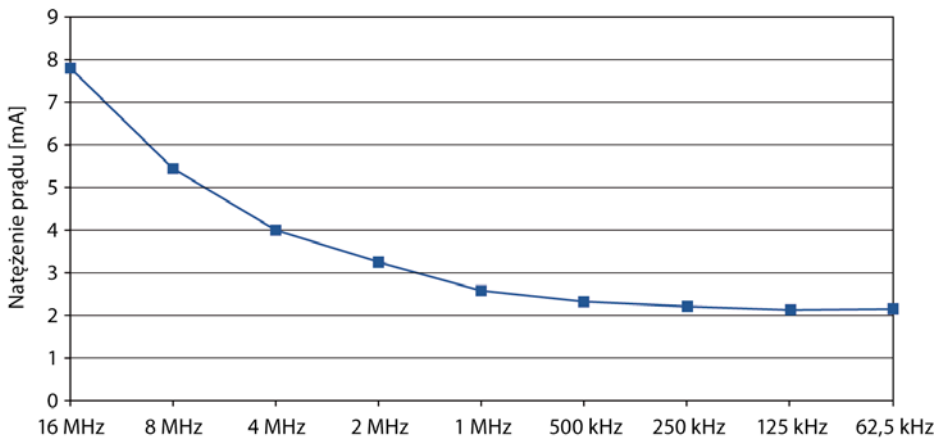
void loop()
{
    digitalWrite(13, HIGH);
    trueDelay(1000);
    digitalWrite(13, LOW);
    trueDelay(5000);
}
```

Biblioteka zapewnia obsługę zbioru stałych, które pozwalają na określenie dzielnika częstotliwości pracy zegara. Wartość `CLOCK_PRESCALER_1` nie zmienia częstotliwości zegara — zegar pracuje ze standardową częstotliwością 16 MHz. Zastosowanie stałej `CLOCK_PRESCALER_256` spowoduje zmniejszenie częstotliwości 256 razy, czyli zostanie ona zmniejszona do 62,5 kHz.

W tabeli 5.2 przedstawiono wartości natężenia prądu pobieranego przez układ pracujący z różnymi częstotliwościami. Na rysunku 5.3 dane te zaprezentowano w formie wykresu. Krzywa widoczna na wykresie staje się od pewnego momentu coraz wyraźniej płaska — częstotliwość 1 MHz wydaje się najlepszym kompromisem pomiędzy częstotliwością taktowania a poborem prądu.

Tabela 5.2. Pobór prądu w zależności od częstotliwości zegara

Stała	Częstotliwość taktowania	Natężenie pobieranego prądu — po wyłączeniu diody LED (mA)
CLOCK_PRESCALER_1	16 MHz	7,8
CLOCK_PRESCALER_2	8 MHz	5,4
CLOCK_PRESCALER_4	4 MHz	4,0
CLOCK_PRESCALER_8	2 MHz	3,2
CLOCK_PRESCALER_16	1 MHz	2,6
CLOCK_PRESCALER_32	500 kHz	2,3
CLOCK_PRESCALER_64	250 kHz	2,2
CLOCK_PRESCALER_128	125 kHz	2,1
CLOCK_PRESCALER_256	62,5 kHz	2,1



Rysunek 5.3. Wykres ilustrujący zależność między natężeniem pobieranego prądu a częstotliwością zegara

Obniżenie częstotliwości pracy zegara ma również inne konsekwencje poza koniecznością korzystania z nowych wersji funkcji `millis` i `delay`. Tak naprawdę żadna czynność zależna od czasu, taka jak generowanie sygnału PWM czy sterowanie serwomotorami, nie będzie wykonywana tak, jak się spodziewałeś.

Większość z 2,1 mA pobierana przez układ pracujący przy najniższej częstotliwości jest najprawdopodobniej pobierana przez diodę LED sygnalizującą podłączenie urządzenia do prądu, a więc jeżeli chcesz, jak najbardziej obniżyć pobór prądu, możesz ją ostrożnie wylutować.

Wyłączanie komponentów

Układy ATmega pozwalają na bardzo złożone zarządzanie energią — nawet na wyłączanie funkcji, które akurat nie są używane, co umożliwia niewielkie zmniejszenie natężenia prądu pobieranego przez chip.

Komponenty możesz wyłączać i włączać z poziomu szkicu. Na przykład istnieje możliwość włączania przetwornika analogowo-cyfrowego (A/C), wtedy gdy wywołana zostaje funkcja `analogRead`. Po zakończeniu działania tej funkcji przetwornik może zostać z powrotem wyłączony.

Zarządzanie energią odbywa się za pomocą biblioteki *avr/power.h*, w której znajdują się pary funkcji włączających i wyłączających poszczególne komponenty mikrokontrolera. W bibliotece tej zdefiniowano na przykład funkcję `power_adc_disable`, która wyłącza przetwornik A/C, a także funkcję `power_adc_enable`, która włącza go z powrotem.

Szczerze mówiąc, nie oszczędzi się w ten sposób zbyt wiele energii. Po przeprowadzeniu testów okazało się, że wyłączenie wszystkich dodatkowych komponentów układu Pro Mini, pracującego z częstotliwością 16 MHz i zasilanego prądem o napięciu 5 V, spowodowało zmniejszenie natężenia pobieranego prądu zaledwie o 1,5 mA. W pełni funkcjonalny układ pobierał prąd o natężeniu 16,4 mA, a po wyłączeniu wszystkich dodatkowych komponentów — 14,9 mA. Pomiary przeprowadziłem, korzystając z następującego szkicu:

```
// 05_02_wyłączanie

#include <avr/power.h>

void setup()
{
    pinMode(13, OUTPUT);
    // power_adc_disable();
    power_spi_disable();
    // power_twi_disable();
    // power_usart0_disable();
    // power_timer0_disable();
    // power_timer1_disable();
    // power_timer2_disable();
    // power_all_disable();
}

void loop()
{
}
```

Dostępne funkcje wymieniono w tabeli 5.3. Każda z funkcji ma swój mający przeciwnie działanie odpowiednik, którego nazwa kończy się na `disable`, a nie `enable`.

Tabela 5.3. Funkcje zarządzające zasilaniem układów ATmega

Funkcja	Opis
<code>power_adc_disable</code>	Wyłącza wejścia analogowe.
<code>power_spi_disable</code>	Wyłącza interfejs SPI.
<code>power_twi_disable</code>	Wyłącza magistralę TWI (I2C).
<code>power_usart0_disable</code>	Wyłącza interfejs szeregowy UART odpowiedzialny za komunikację przez złącze USB.
<code>power_timer0_disable</code>	Wyłącza zegar o numerze 0 (korzystają z niego funkcje <code>millis</code> i <code>delay</code>).
<code>power_timer1_disable</code>	Wyłącza zegar o numerze 1.
<code>power_timer2_disable</code>	Wyłącza zegar o numerze 2.
<code>power_all_disable</code>	Wyłącza wszystkie wymienione wyżej moduły.

Usypianie

Ostatecznym sposobem oszczędzania energii pobieranej przez Arduino jest usypianie mikrokontrolera, gdy nie wykonuje on żadnych czynności.

Biblioteka Narcoleptic

Peter Knight jest autorem łatwej w użyciu biblioteki o nazwie *Narcoleptic*. Możesz ją pobrać z <https://code.google.com/p/narcoleptic/>.

Oczywiście usypianie Arduino nie miałoby sensu, gdyby nie można go było z powrotem obudzić! Mikrokontroler może zostać wzbudzony na dwa sposoby: w wyniku wygenerowania zewnętrznego przerwania lub przez zegar skonfigurowany tak, aby włączył Arduino po określonym czasie. Biblioteka *Narcoleptic* korzysta z tej drugiej metody.

W bibliotece *Narcoleptic* zdefiniowana jest alternatywna funkcja `delay`, która usypia Arduino na określony czas. W związku z tym, że Arduino nie wykonuje żadnych operacji, gdy aktywna jest funkcja `delay`, metoda ta sprawdza się znakomicie.

Przyjrzyjmy się na przykład dobrze nam znanemu szkicowi *Blink*. Włącza on diodę LED na 1 s, a następnie wyłącza ją na czas 10 s. Czynności te powtarzane są w nieskończoność. Przeanalizuj poniższy szkic:

```
// 05_03_standardowy_blink

void setup()
{
  pinMode(13, OUTPUT);
}

void loop()
{
  digitalWrite(13, HIGH);
  delay(1000);
```

```
digitalWrite(13, LOW);
delay(10000);
}
```

A teraz przyjrzyj się temu samemu szkicowi, ale w wersji z wykorzystywaną biblioteką *Narcoleptic*:

```
// 05_04_blink_biblioteka_narcoleptic
#include <Narcoleptic.h>

void setup()
{
  pinMode(13, OUTPUT);
}

void loop()
{
  digitalWrite(13, HIGH);
  Narcoleptic.delay(1000);
  digitalWrite(13, LOW);
  Narcoleptic.delay(10000);
}
```

Jedyna różnica między programami jest taka, że w drugim szkicu zaimportowano bibliotekę *Narcoleptic* i korzystano ze znajdującej się w niej funkcji `delay`.

Po uruchomieniu obu szkiców na płycie Pro Mini zasilanej prądem o napięciu 5 V, przy taktowaniu zegara o częstotliwości 16 MHz, okazało się, że podczas wykonywania pierwszego programu, po wyłączeniu diody LED, płytka pobierała prąd o natężeniu 17,2 mA, a podczas wykonywania drugiego szkicu zużycie prądu po wyłączeniu diody LED spadło do 3,2 mA. Warto mieć na uwadze, że większość z tego prądu jest pobierana przez diodę LED sygnalizującą działanie Arduino, a więc gdybyś ją wymontował, to średni pobór prądu mógłby spaść do wartości mniejszej niż 1 mA.

Mikrokontroler przechodzi w tryb uśpienia bardzo szybko, a więc jeżeli w Twoim projekcie Arduino ma wykonywać jakieś działania po wciśnięciu przycisku przez użytkownika, to nie musisz koniecznie stosować zewnętrznego przerwania w celu wybudzenia procesora. Prawdopodobnie łatwiej będzie Ci napisać kod, który będzie wybudzał mikrokontroler 10 razy na sekundę i sprawdzał, czy na wejście nie podano sygnału wysokiego (HIGH), a następnie w razie wystąpienia takiego sygnału wykonywał odpowiednie operacje. Tego typu rozwiązanie zademonstrowano w kolejnym szkicu:

```
// 05_05_wejście_biblioteka_narcoleptic
#include <Narcoleptic.h>

const int ledPin = 13;
const int inputPin = 2;

void setup()
{
  pinMode(ledPin, OUTPUT);
  pinMode(inputPin, INPUT_PULLUP);
}
```

```

void loop()
{
    if (digitalRead(inputPin) == LOW)
    {
        doSomething();
    }
    Narcoleptic.delay(100);
}

void doSomething()
{
    for (int i = 0; i < 20; i++)
    {
        digitalWrite(ledPin, HIGH);
        Narcoleptic.delay(200);
        digitalWrite(ledPin, LOW);
        Narcoleptic.delay(200);
    }
}

```

Po uruchomieniu tego szkicu na płytce Pro Mini zasilanej prądem o napięciu 5 V, przy taktowaniu zegara o częstotliwości 16 MHz, okazało się, że Arduino, czekając na wciśnięcie przycisku, pobiera prąd o natężeniu 3,25 mA. Podłączenie pinu numer 2 do ujemnego biegunu zasilającego spowoduje wygenerowanie 20 błysków diody LED. Średnie zużycie prądu podczas migania diody będzie się utrzymywać na poziomie 4 – 5 mA, ponieważ korzystamy również ze specjalnej funkcji `delay` zdefiniowanej w bibliotece `Narcoleptic`.

Jeżeli w funkcji `loop` zastosujesz inny argument funkcji `delay` — Arduino będzie wybudzane, powiedzmy, 100 razy na sekundę — spowoduje to odczuwalny wzrost poboru prądu, ponieważ usypianie mikrokontrolera jest czasochłonne. Zmiana argumentu wspomnianej funkcji na 50 (Arduino będzie budzone 20 razy na sekundę) nie powinno jednakże spowodować znacznego wzrostu natężenia pobieranego prądu.

Budzenie za pomocą zewnętrznych przerw

Opisana metoda sprawdza się w większości zastosowań, czasami jednak może zaistnieć potrzeba wygenerowania szybszej odpowiedzi na zewnętrzne zdarzenie. W takim przypadku będziesz musiał skonfigurować mikrokontroler tak, aby budził się w wyniku zaistnienia zewnętrznego przerwania.

Przebudowa poprzedniego szkicu w taki sposób, aby zewnętrzne przerwanie było generowane przez przycisk podłączony do pinu *D2*, wymaga znacznej ingerencji w kod, ale z pewnością Arduino będzie sprawniej reagować na zewnętrzne zdarzenie i nie będzie na okrągło skanować stanu sygnału podawanego na wejście. Zmodyfikowany szkic charakteryzuje się dość rozbudowanym kodem, a więc najpierw Ci go przedstawię, a dopiero później opiszę jego działanie. Jeżeli podczas lektury książki pominąłeś rozdział 3., w którym omawiałem zagadnienia związane z przerwaniami, to przed przystąpieniem do analizy tego szkicu powinieś zapoznać się ze wspomnianym rozdziałem.

```

// 05_06_budzenie_przerwanie
#include <avr/sleep.h>

```

```

const int ledPin = 13;
const int inputPin = 2;

volatile boolean flag;

void setup()
{
    pinMode(ledPin, OUTPUT);
    pinMode(inputPin, INPUT_PULLUP);
    goToSleep();
}

void loop()
{
    if (flag)
    {
        doSomething();
        flag = false;
        goToSleep();
    }
}

void setFlag()
{
    flag = true;
}

void goToSleep()
{
    set_sleep_mode(SLEEP_MODE_PWR_DOWN);
    sleep_enable();
    attachInterrupt(0, setFlag, LOW); // pin D2
    sleep_mode(); // teraz następuje uśpienie
    // uśpienie aż do podania niskiego sygnału, a wtedy...
    sleep_disable();
    detachInterrupt(0);
}

void doSomething()
{
    for (int i = 0; i < 20; i++)
    {
        digitalWrite(ledPin, HIGH);
        delay(200);
        digitalWrite(ledPin, LOW);
        delay(200);
    }
}

```

Zwróć uwagę na to, że w szkicu zastosowano funkcje zdefiniowane w bibliotece *avr/sleep.h*, wcześniej w podobny sposób stosowałem funkcje z biblioteki *avr/power.h*. Biblioteka ta nie jest standardową biblioteką Arduino. Została ona napisana z myślą o mikrokontrolerach należących do serii AVR. W związku z tym zaprezentowany szkic nie będzie działał na płytce Arduino Due, ale powtórzę to jeszcze raz — w pracy nad projektami charakteryzującymi się niskim poborem prądu płytka Due jest najgorszym możliwym wyborem.

Po zdefiniowaniu pinów, z których będę korzystać, zdefiniowałem zmienną `volatile` pozwalającą na komunikację pomiędzy procedurą obsługującą przerwania a resztą szkicu.

Funkcja `setup` konfiguruje piny, a następnie wywołuje funkcję `goToSleep`. Funkcja ta określa tryb uśpienia — w tym przypadku jest to `SLEEP_MODE_PWR_DOWN`. W trybie tym mikrokontroler będzie pobierał najmniej prądu, a więc warto z niego skorzystać.

Następnie należy wywołać funkcję `sleep_enable`. Wywołanie tej funkcji nie usypia mikrokontrolera. Wcześniej należy skojarzyć przerwanie z przerwaniem numer 0 (pin `D2`) — pozwoli to na obudzenie Arduino, gdy będzie to konieczne.

UWAGA

Zwróć uwagę na to, że przerwanie jest typu LOW. Jest to jedyny typ przerwania, jaki można zastosować w zaprezentowanym szkicu. Przerwania typu RISING, FALLING i CHANGE nie będą działały prawidłowo.

Po skojarzeniu przerwania wywołanie funkcji `sleep_mode()` usypia procesor. Gdy mikrokontroler zostanie wybudzony, zacznie wykonywać polecenia znajdujące się w procedurze obsługi przerwań, a następnie dalsze polecenia dostępne w funkcji `goToSleep`. Najpierw wywołana zostanie funkcja `disable_sleep`. Następnie wyłączona zostanie obsługa przerwań — procedura obsługi przerwań nie może zostać ponownie wywołana aż do ponownego uśpienia mikrokontrolera.

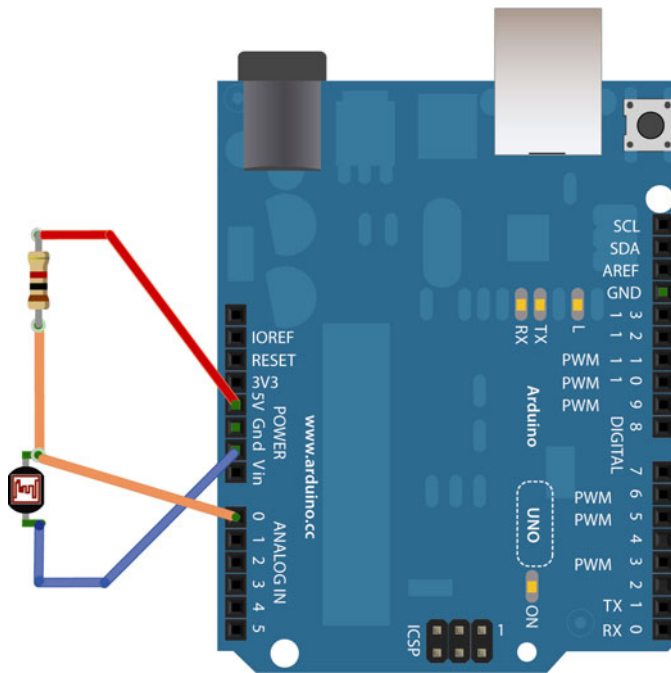
Gdy dochodzi do przerwania za pośrednictwem wejścia `D2`, procedura obsługi przerwań (`setFlag`) generuje znacznik, którego obecność sprawdza funkcja `loop`. Pamiętaj o tym, że w procedurze obsługi przerwań nie możesz stosować funkcji typu `delay`. W związku z tym funkcja `loop` musi sprawdzać, czy znacznik nie został wygenerowany, a następnie wywoływać funkcję `doSomething` — jest to ta sama funkcja, którą wcześniej stosowaliśmy w przykładzie wykorzystującym bibliotekę `Narcoleptic`. Po tym, gdy zadanie zostanie wykonane, znacznik jest wyzerowywany, a Arduino ponownie usypiane.

Mikrokontroler pobierał podobną ilość prądu do pobieranej przez niego podczas wykonywania szkicu, w którym korzystaliśmy z biblioteki `Narcoleptic`. Większa ilość prądu była pobierana tylko w momencie świecenia diodami LED — zastosowałem normalną funkcję `delay`.

Ograniczanie pobieranego prądu za pomocą wyjść cyfrowych

Rozdział dotyczy programowego minimalizowania pobieranego prądu, ale warto wspomnieć również o pewnych zabiegach sprzętowych, które także mogą się przyczynić do zmniejszenia poboru prądu.

Na rysunku 5.4 przedstawiono czujnik światła składający się z fototranzystora (jego rezystancja zmienia się w zależności od ilości padającego na niego światła) oraz rezystora. Oba te komponenty podłączone są do analogowego wejścia Arduino mierzącego natężenie światła.



Rysunek 5.4. Pomiar natężenia światła za pomocą fototranzystora

Problem związany z przedstawionym obwodem polega na tym, że przez fototranzystor i rezystor cały czas płynie prąd o napięciu 5 V. Jeżeli na fototranzystor pada jasne światło, to charakteryzuje się on rezystancją 500 Ω , a więc z prawa Ohma możemy obliczyć natężenie prądu płynącego przez obwód: $I = U/R = 5 \text{ V}/(1000 \Omega + 500 \Omega) = 3,3 \text{ mA}$.

Zamiast korzystać ze złącza stale zasilającego obwód prądem o napięciu 5 V, układ przeznaczony do pomiaru natężenia światła można zasilać za pomocą cyfrowego wyjścia (zobacz rysunek 5.5). W chwili przeprowadzania pomiaru należy na taki pin podać sygnał HIGH, później można zmienić stan sygnału na LOW. W ten sposób prąd o natężeniu 3,3 mA będzie płynął przez obwód tylko przez chwilę, za każdym razem gdy będzie wykonywany pomiar. W ten sposób średnie zużycie prądu zostanie znacząco zmniejszone.

```
// 05_07_pomiar_światłości
```

```
const int inputPin = A0;
const int powerPin = 12;
```

```
void setup()
{
  pinMode(powerPin, OUTPUT);
  Serial.begin(9600);
}
```


Rozdział 6.

Pamięć

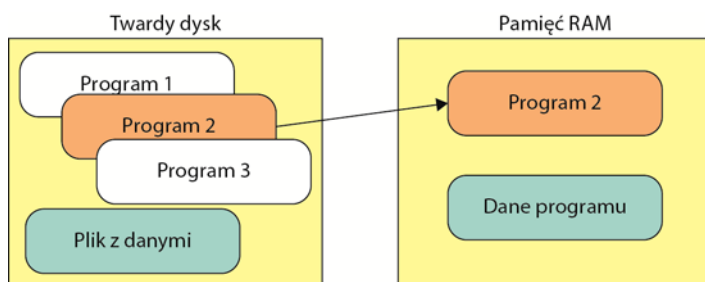
W większości komputerów ilość dostępnej pamięci można wyrazić w gigabajtach. Arduino posiada zaledwie 2 kB pamięci. Jest to ponad milion razy mniej od konwencjonalnego komputera. Dysponując tak małą przestrzenią, musimy zachować pewną ostrożność podczas pisanie kodu. Nie ma tutaj miejsca na „rozdęcie”, czyli zajmowanie przez oprogramowanie nadmiernej ilości pamięci, co trapi większość komputerów.

Napisanie kodu wydajnie korzystającego z pamięci jest czymś ważnym, ale nie należy tego robić kosztem czytelności lub możliwości dalszych modyfikacji programu. Mimo tak ograniczonej przestrzeni pamięci RAM większość szkiców Arduino nawet nie zbliży się do zajęcia całej dostępnej pamięci operacyjnej. Ograniczeniem dostępnej pamięci RAM należy się przejmować tylko podczas pracy nad bardzo złożonym szkicem lub takim szkicem, w którym korzysta się z dużej ilości danych.

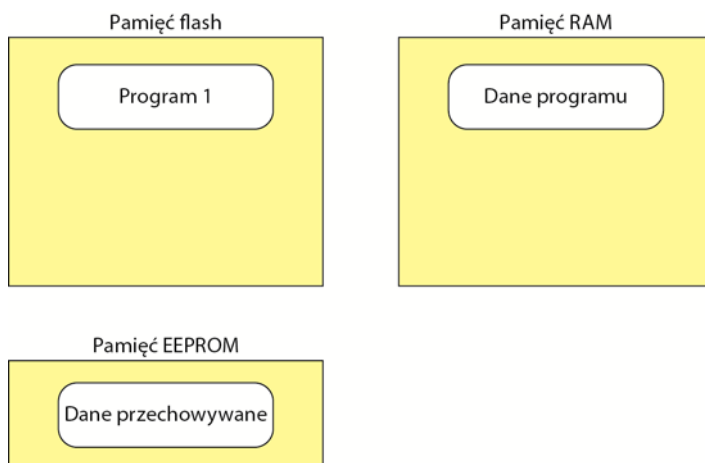
Pamięć Arduino

Porównywanie pamięci operacyjnej typowego komputera z pamięcią płytki Arduino nie jest sprawiedliwe. Urządzenia te korzystają z pamięci RAM w zupełnie inny sposób. Na rysunku 6.1 pokazano schemat ilustrujący sposób zarządzania pamięcią przez komputer PC podczas działania programu.

Gdy program jest uruchamiany na komputerze klasy PC, jest kopiowany w całości z dysku twardego do pamięci RAM. W dalszej kolejności uruchamiana jest kopia programu znajdująca się we wspomnianej pamięci. Zmienne programu następnie zajmują dalszą część pamięci operacyjnej. Na rysunku 6.2 przedstawiono sposób zarządzania pamięcią przez Arduino, na którym uruchomiono program. Sam program jest uruchamiany bezpośrednio z pamięci flash, nie jest kopiowany do pamięci RAM.



Rysunek 6.1. Zarządzanie pamięcią przez komputer PC



Rysunek 6.2. Zarządzanie pamięcią przez Arduino

Arduino umieszcza w pamięci RAM tylko zmienne i inne dane, z których korzysta uruchomiony program. Pamięć RAM nie jest pamięcią stałą. Po odłączeniu jej od zasilania wszystkie znajdujące się w niej dane zostają utracone. Jeżeli program wymaga zapisu pewnych danych na stałe, to musi tego dokonać w pamięci EEPROM. Dane zapisane w tej pamięci będą mogły być odczytane z powrotem po kolejnym uruchomieniu szkicu.

Aby wykorzystać w pełni możliwości Arduino, należy pamiętać zarówno o ilości danych umieszczonych w pamięci RAM, jak również i (w mniejszym stopniu) o tym, ile miejsca zajmuje program zapisany w pamięci flash. Płytki Arduino Uno jest wyposażona w 32 kB pamięci flash — limit ten nie jest zbyt często osiągany.

Korzystanie z minimalnej ilości pamięci RAM

Jak już wiesz, ilość pamięci RAM używanej przez program można zmniejszyć, redukując ilość tej pamięci zajmowanej przez zmienne.

Korzystanie z właściwych struktur danych

Jak dotąd najczęściej stosowaliśmy zmienne typu `int`. Każda taka zmienna zajmuje 2 bajty, ale w większości przypadków nie musisz korzystać z tego typu zmiennej mogącej przechowywać dowolną liczbę z zakresu od $-32\,768$ do $+32\,767$. Zwykle wystarcza zakres od 0 do 255, oferowany przez zmienną typu `byte`. Większość wbudowanych funkcji działających ze zmiennymi typu `int` będzie działała w ten sam sposób ze zmiennymi typu `byte`.

Najłatwiej działanie zmiennych tego typu jest przedstawić za pomocą szkicu, w którym zmienne te przechowują numery złączy. Zwykle numery te zapisuje się za pomocą zmiennych typu `int`, co ilustruje poniższy, przykładowy szkic:

```
// 06_01_int
int ledPins[] = {2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13};

void setup()
{
  for (int i = 0; i < 12; i++)
  {
    pinMode(ledPins[i], OUTPUT);
    digitalWrite(ledPins[i], HIGH);
  }
}

void loop()
{
}
```

Z łatwością tablicę elementów typu `int` można zamienić na tablicę elementów typu `byte`. Program będzie działał tak samo, ale tablica będzie zajmowała o połowę mniej pamięci.

Bardzo dobrym sposobem na zminimalizowanie zajmowanej przez program pamięci jest zadeklarowanie zmiennych, których wartość nie zmienia się podczas działania programu, jako stałe. Kompilator, wiedząc, że dana zmienna będzie przyjmować tylko jedną wartość, zastosuje tę wartość w generowanym kodzie, co ogranicza zajmowaną pamięć. Wcześniej zaprezentowaną tablicę można zadeklarować jako tablicę stałych typu `byte`:

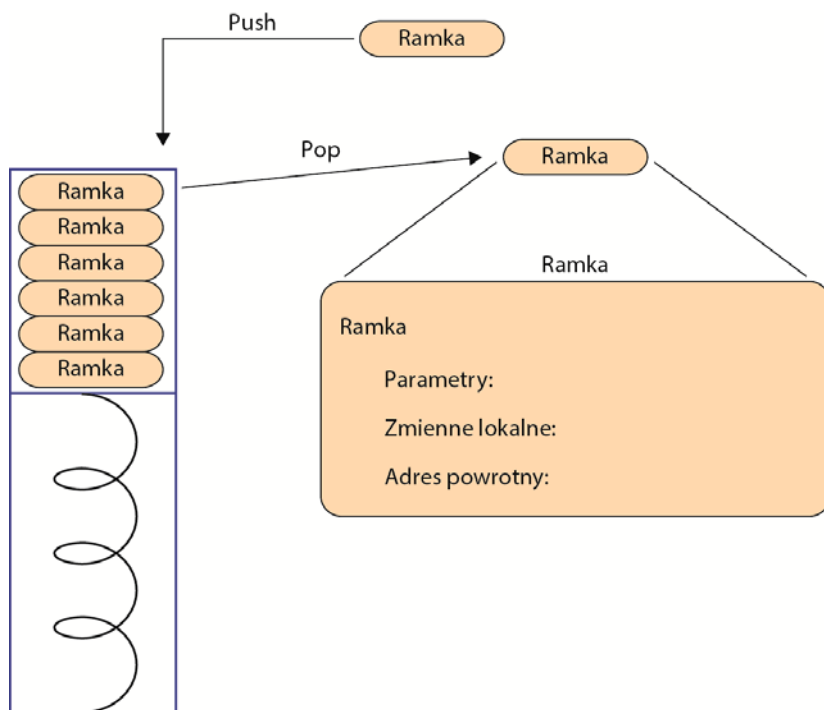
```
const byte ledPins[] = {2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13};
```

Całość zmodyfikowanego szkicu znajdziesz w pliku o nazwie `06_01_byte_const.ino`, który możesz pobrać ze strony <http://www.helion.pl/ksiazki/arpokk.htm>.

Zachowaj ostrożność, korzystając z rekurencji

Rekurencja jest techniką polegającą na tym, że funkcja wywołuje się sama. Jest to narzędzie, za pomocą którego można rozwiązać wiele problemów. Stosuje się je często w językach programowania funkcyjnego takich jak Lisp i Scheme.

Po wywołaniu funkcji używany jest obszar pamięci zwany **stosem**. Wyobraź sobie pojemnik na drażetki wyposażony w sprężynę. Drażetki mogą być do niego dokładane od góry. Słodczyce można wyjmować również tylko od góry (zobacz rysunek 6.3). Operacja `push` odkłada dane na stos, a operacja `pop` zdejmuje je z niego.



Rysunek 6.3. Stos

Za każdym razem, gdy wywołujesz funkcję, tworzona jest ramka stosu. **Ramka stosu** jest małym rekordem pamięci, w którym znajduje się przestrzeń przeznaczona dla parametrów i zmiennych lokalnych danej funkcji. Znajduje się w niej również adres powrotny określający miejsce w programie, od którego należy zacząć go wykonywać po zakończeniu funkcji i zwróceniu przez nią danych.

Na początku stos jest pusty, ale po wywołaniu funkcji (nazwijmy ją dla przykładu „funkcja A”) dochodzi do alokacji pamięci i wepchnięcia ramki na stos. Jeżeli funkcja A wywołuje inną funkcję (funkcję B), to do górnej części stosu dodawany jest drugi rekord — stos składa się w takiej sytuacji z dwóch rekordów. Gdy funkcja B zostanie zakończona, jej ramka jest ściągana ze stosu, a gdy później zakończy się działanie funkcji A, wtedy jej ramka również zostanie ściągnięta ze stosu. Zmienne lokalne funkcji są przechowywane w ramce stosu, a więc nie są one zapisywane w pamięci między kolejnymi wywołaniami funkcji.

Stos zajmuje część naszej cennej pamięci. Przez większość czasu nie znajdują się na nim więcej niż trzy – cztery rekordy. Wyjątek stanowi sytuacja, w której pozwalasz funkcjom na wywoływanie siebie albo wywołujesz funkcje w pętli. W takich przypadkach może dojść do realnego zapełnienia całej pamięci stosu przez program.

Na przykład matematyczna funkcja obliczająca silnię wykonuje działanie mnożenia przez siebie wszystkich kolejnych liczb aż do liczby, której silnię chcemy obliczyć. Na przykład silnia 6 wynosi $6 \cdot 5 \cdot 4 \cdot 3 \cdot 2 \cdot 1 = 720$.

Rekurencyjna definicja silni liczby n jest następująca: jeżeli $n = 0$, to silnia n wynosi 1. W przeciwnym wypadku silnia liczby n wynosi n razy $(n-1)$.

W szkicu Arduino możemy przedstawić tę definicję w następujący sposób:

```
long factorial(long n)
{
    if (n == 0)
    {
        return 1;
    }
    else
    {
        return n * factorial(n - 1);
    }
}
```

Cały kod szkicu wyświetlającego wynik tej operacji znajdziesz w pliku *06_02_silnia.ino*. Ogólnie rzecz biorąc, osoby myślące w sposób matematyczny uważają, że jest to dobry zapis. Gdy przeanalizujesz działanie tego kodu, zauważysz, że wielkość stosu zależy od liczby, której silnię obliczamy. Nierekurencyjna wersja funkcji `factorial` jest również dość prosta:

```
long factorial(long n)
{
    long result = 1;
    while (n > 0)
    {
        result = result * n;
        n--;
    }
    return result;
}
```

Kod ten jest prawdopodobnie czytelniejszy. Ponadto zajmuje mniej pamięci i działa szybciej. Cały kod szkicu wyświetlającego wynik tej operacji znajdziesz w pliku *06_03_silnia_bez_rekurencji.ino*. Warto jest unikać rekurencji albo ograniczyć jej stosowanie do wysoce wydajnych algorytmów rekurencyjnych takich jak algorytm szybkiego sortowania (http://pl.wikipedia.org/wiki/Sortowanie_szybkie), który pozwala na bardzo wydajne uporządkowanie tablicy liczb.

Przechowywanie w pamięci flash stałych będących łańcuchami

Domyślnie łańcuchy będące stałymi, zadeklarowane tak, jak pokazano poniżej, będą przechowywane zarówno w pamięci RAM, jak i flash — raz w ramach kodu programu, a drugi raz, gdy zostaną zapisane w pamięci RAM podczas uruchamiania szkicu.

```
Serial.println("Uruchomiono program");
```

Możesz jednak zmodyfikować kod tak, aby stała będąca łańcuchem była przechowywana tylko w pamięci flash:

```
Serial.println(F("Uruchomiono program"));
```

W znajdującej się w tym rozdziale sekcji „Korzystanie z pamięci flash” poznasz inne sposoby wykorzystywania pamięci flash.

Rozpowszechnione błędne przekonania

Często spotykana błędna teoria jest taka, że zmienne o krótszych nazwach zabierają mniej pamięci. Kompilator zajmuje się rzeczami tego typu — finalne nazwy zmiennych zastosowane w binarnej wersji szkicu są zupełnie inne. Spotkałem się również z osobami twierzącymi, że liczba komentarzy znajdujących się w szkicu ma wpływ na wielkość ostatecznej binarnej wersji programu. Ta teoria jest również nieprawdziwa.

Zwykle może Ci się wydawać, że podział kodu na wiele małych funkcji również doprowadzi do zwiększenia rozmiaru pliku z programem. Najczęściej nie dochodzi do takiej sytuacji, ponieważ kompilator potrafi zastąpić wywołania funkcji kopiami znajdującego się w niej kodu. Ma to miejsce podczas procesu optymalizacji kodu. Dzięki temu możliwe jest pisanie kodu bardziej czytelnego.

Pomiar wolnej pamięci

Biblioteka `MemoryFree` pozwala na określenie w dowolnym momencie tego, ile pamięci RAM zajmuje uruchomiony szkic. Bibliotekę tę możesz pobrać ze strony <http://playground.arduino.cc/Code/AvailableMemory>.

Wspomniana biblioteka jest łatwa w użyciu. Zdefiniowano w niej funkcję `freeMemory`, która zwraca liczbę dostępnych bajtów. Użycie tej funkcji ilustruje poniższy szkic:

```
#include <MemoryFree.h>
void setup()
{
    Serial.begin(115200);
}
void loop()
{
    Serial.print("freeMemory()=");
    Serial.println(freeMemory());
    delay(1000);
}
```

Funkcja ta umożliwia sprawdzenie, czy nieprawidłowe działanie szkicu wynika z braku dostępnej pamięci. Oczywiście korzystanie z omawianej biblioteki wiąże się ze zwiększeniem ilości pamięci zajmowanej przez program.

Korzystanie z minimalnej ilości pamięci flash

Po udanej kompilacji szkicu środowisko programistyczne wyświetla komunikat o treści w rodzaju:

Wielkość binarna szkicu: 1 344 bajty (maksymalnie 32 256 bajtów)¹

¹ Komunikat wyświetlany przez polską wersję Arduino IDE (1.0.5) — *przyp. tłum.*

Komunikat ten informuje Cię dokładnie o tym, ile pamięci flash dostępnej na płycie Arduino będzie zajmował skompilowany szkic (w ten sposób wiesz, czy nie zbliżasz się do limitu 32 kB). Jeżeli Twój szkic zajmuje znacznie mniej pamięci, niż wynosi wartość graniczna, to tak naprawdę nie musisz optymalizować go pod kątem zajmowanej pamięci flash. Jeżeli jednak zbliżasz się do wartości granicznej, możesz podjąć pewne działania.

Korzystaj ze stałych

Zmienne określające numery pinów zwykle są definiowane w następujący sposób:

```
int ledPin = 13;
```

Jeżeli podczas działania szkicu nie planujesz zmiany numeru pinu, do którego jest podłączona dioda LED, to możesz zadeklarować ten numer jako stałą. Wystarczy przed jego deklaracją dodać słowo `const`:

```
const int ledPin = 13;
```

Taka modyfikacja zmniejsza rozmiar programu o 2 bajty oraz o kolejne 2 bajty w każdym miejscu, w którym zastosowano stałą zamiast zmiennej. W przypadku szkiców, w których zmienne występują wielokrotnie, taka technika może spowodować zaoszczędzenie kilkudziesięciu bajtów pamięci.

Usuwanie zbędne elementy szkicu

Przeprowadzając diagnostykę szkiców Arduino, warto korzystać z polecenia `Serial.println` w celu wyświetlenia wartości zmiennych. Pozwala to na wykrycie błędów w programie. Kod obsługujący to polecenie zajmuje jednak dość dużo pamięci flash. Każde użycie polecenia `Serial.println` wiąże się z zajęciem 500 bajtów pamięci przez kod biblioteki dołączany do szkicu. A więc gdy jesteś już pewien, że szkic działa prawidłowo, wtedy oznacz jako komentarze wiersze zawierające wspomniane polecenie.

Pomiń program rozruchowy

W rozdziale 2. dowiedziałeś się, jak można zaprogramować mikrokontroler umieszczony na płycie Arduino — bezpośrednio przy użyciu złącza ISP i programatora. Takie rozwiązanie pozwala zaoszczędzić kilka kilobajtów pamięci, ponieważ możliwe jest wtedy pominięcie instalacji programu rozruchowego.

Statyczna i dynamiczna alokacja pamięci

Jeżeli, podobnie jak ja, posiadasz doświadczenie w programowaniu złożonych systemów w językach takich jak Java czy C#, to jesteś przyzwyczajony do tworzenia obiektów w czasie działania programu, a następnie ich kasowania za pomocą algorytmu odzyskującego pamięć. Tego typu praktyk nie należy stosować w przypadku pisania programu, który ma zostać

uruchomiony na mikroprocesorze wyposażonym zaledwie w 2 kB pamięci. Po pierwsze, nie mamy do dyspozycji algorytmu odzyskującego pamięć. Po drugie, alokowanie i dealokowanie pamięci w trakcie działania programu jest czymś zbędnym w przypadku większości szkiców uruchamianych na Arduino.

W poniższym przykładzie definiujemy tablicę w standardowy, statyczny sposób.

```
// 06_04_statyczna_alokacja_pamięci
```

```
int array[100];

void setup()
{
    array[0] = 1;
    array[50] = 2;
    Serial.begin(9600);
    Serial.println(array[50]);
}

void loop()
{
}
```

W chwili kompilacji szkicu znana jest ilość pamięci zajmowana przez tablicę, a więc kompilator jest w stanie przypisać jej niezbędny obszar pamięci. W kolejnym szkicu stworzymy tablicę o tym samym rozmiarze, ale pamięć dla tej tablicy będzie alokowana w trakcie działania programu z puli dostępnej pamięci. Oprogramowanie Arduino w wersji starszej niż 1.0.4 nie obsługuje funkcji `malloc`.

```
// 06_05_dynamiczna_alokacja_pamięci
```

```
int *array;

void setup()
{
    array = (int *)malloc(sizeof(int) * 100);
    array[0] = 1;
    array[50] = 2;
    Serial.begin(9600);
    Serial.println(array[50]);
}

void loop()
{
}
```

Program rozpoczyna się od deklaracji zmiennej `int *array`. Symbol `*` oznacza, że mamy do czynienia ze wskaźnikiem wartości typu `integer`, a w tym przypadku — tablicy wartości typu `integer`. Nie jest to zwykła wartość. Ilość pamięci, która ma być zarezerwowana na tablicę, jest określana dopiero w momencie wykonania poniższej linii kodu znajdującej się w funkcji `setup`:

```
array = (int *)malloc(sizeof(int) * 100);
```

Polecenie `malloc` (skrót od ang. *memory allocate*) służy do alokowania pamięci z obszaru pamięci RAM, nazywanego **stertą**. Funkcja ta przyjmuje w roli argumentu liczbę bajtów, które mają być alokowane. Tablica ma zawierać 100 zmiennych typu `int`, a więc w celu określenia tego, ile pamięci musisz alokować, potrzebne jest wykonanie pewnych obliczeń. Tak naprawdę w roli parametru funkcji `malloc` mógłbyś zastosować wartość 200, ponieważ każda wartość typu `int` zajmuje 2 bajty pamięci. Zastosowanie funkcji `sizeof` pozwala jednak upewnić się, że zarezerwowana zostanie właściwa liczba bajtów.

Po zaalokowaniu pamięci możesz korzystać z tablicy tak samo, jakby była alokowana statycznie. Jedyną zaletą dynamicznej alokacji pamięci jest to, że możemy przesunąć w czasie podjęcie decyzji dotyczącej jej rozmiaru. Rozmiar tablicy może być określony podczas działania programu.

Zagrożenie związane z korzystaniem z dynamicznej alokacji pamięci jest takie, że gdy alokowana pamięć nie zostanie później zwolniona, może dojść do sytuacji, w której podczas wykonywania szkicu zabraknie pamięci operacyjnej. Brak dostępnej pamięci może spowodować zawieszenie się Arduino. Do takiej sytuacji nie może dojść, gdy pamięć jest alokowana statycznie.

Pomimo tego, że pracowałem nad setkami projektów korzystających z Arduino, bardzo rzadko znajdowałem praktyczne uzasadnienie zastosowania dynamicznej alokacji pamięci, z której korzysta mikrokontroler.

Łańcuchy

Łańcuchy (ciągi znaków alfanumerycznych) stosowane są znacznie rzadziej w przypadku Arduino, niż ma to miejsce podczas konwencjonalnego programowania. W większości programów łańcuchy są najczęściej stosowanym typem danych, ponieważ zwykle programowanie sprowadza się do tworzenia interfejsów użytkownika lub baz danych, a to z kolei wiąże się z przetwarzaniem jakiegoś tekstu.

W większości programów Arduino nie zachodzi potrzeba przetwarzania tekstu. W praktyce najczęściej stosuje się polecenia `Serial.println` podczas usuwania błędów znajdujących się w programie.

Istnieją dwie metody obsługi łańcuchów w szkicach Arduino: stary sposób (tablice elementów typu `char` w języku C) i nowy sposób (biblioteka `StringObject`).

Tablice elementów typu `char`

W tej metodzie stała będąca łańcuchem jest definiowana w następujący sposób:

```
char message[] = "Hello World";
```

W powyższej linii kodu zdefiniowano w sposób statyczny tablicę 12 elementów typu `char`. Znajduje się w niej 11 znaków tworzących hasło „Hello World” (pol. „Witaj, świecie”) oraz znak końca łańcucha. Jest to konwencja zapisu łańcuchów stosowana w języku C. Pozwala

ona na zapis znacznie dłuższych łańcuchów. Każdy znak (niezależnie od tego, czy jest to litera, czy liczba) jest zapisywany za pomocą kodu ASCII (zobacz rysunek 6.4).

H	e	l	l	o		W	o	r	l	d	\0
72	101	108	108	111	32	87	111	114	108	100	0

Kod ASCII (wartości dziesiętne)

Rysunek 6.4. Łańcuch zakończony znakiem wartości zerowej

Stałą będącą łańcuchem można również zapisać za pomocą innej konwencji:

```
char *message = "Hello World";
```

Składnia ta działa w podobny sposób, ale tym razem deklarujemy wskaźnik message określający to, gdzie znajduje się pierwszy znak tablicy.

Formatowanie łańcuchów za pomocą wielokrotnego wywoływania polecenia `print`

Zwykle łańcuchy są używane do wyświetlania komunikatów. Stosuje się je na przykład w roli parametru funkcji `Serial.println`, która może wyświetlić treść komunikatów na wyświetlaczu ciekłokrystalicznym. Możesz mieć wrażenie, że podczas wyświetlania tekstu niezbędne jest łączenie łańcuchów i przeprowadzanie konwersji liczb na łańcuchy. Przeanalizujmy na przykład to, jak można wyświetlić komunikat „Temperatura: 32°C”. Może Ci się wydawać, że do łańcucha „Temperatura:” należy dołączyć liczbę 32, a następnie na końcu tak zmodyfikowanego łańcucha dodać kolejny „°C”. Jeżeli programujesz w języku Java, to prawdopodobnie spodziewasz się, że w języku C trzeba wykonać następującą operację:

```
String text = "Temperatura: " + tempC + "°C";
```

Niestety język C nie działa w ten sposób. Aby wyświetlić wspomniany komunikat, należy użyć wielu poleceń `print`, co pokazano w poniższej linii kodu:

```
lcd.print("Temperatura: "); lcd.print(tempC); lcd.print("°C");
```

Taki sposób wyświetlania łańcuchów zwalnia programistę z obowiązku kopiowania danych podczas łączenia łańcuchów.

Za pomocą tej samej metody możliwe jest wyświetlanie danych w oknie monitora portu szeregowego. W tym celu należy posługiwać się poleceniami `Serial.print`. Zwykle ostatnie polecenie `println` służy do wygenerowania na wyjściu znaku nowego wiersza.

Formatowanie łańcuchów za pomocą polecenia `sprint`

Standardowa biblioteka obsługująca łańcuchy w języku C (nie należy jej mylić z biblioteką Arduino `StringObject`, która zostanie omówiona w dalszej części rozdziału) posiada bardzo przydatną funkcję formatującą tablice znaków. Funkcja ta nosi nazwę `sprintf`. Poniżej znajduje się przykład ilustrujący jej działanie:

```
char line1[17];
int tempC = 30;
sprintf(line1, "Temperatura: %d°C", tempC);
```

Tablica znaków `line1` jest buforem mającym formę łańcucha. Zawiera on sformatowany tekst. Może się w nim zmieścić 17 znaków (wraz ze znakiem symbolizującym koniec łańcucha). Tablicę tę nazwałem `line1`, aby pokazać to, że można za jej pomocą wyświetlić tekst w górnym wierszu dwuwierszowego ekranu LCD. W każdym wierszu takiego wyświetlacza można umieścić 16 znaków.

Pierwszym parametrem polecenia `sprintf` jest nazwa tablicy znaków, w której zostanie zapisany efekt działania funkcji. Kolejnym argumentem jest łańcuch znaków zawierający tekst, taki jak np. `Temperatura:` , oraz formatujące polecenia, takie jak np. `%d`. Zapis `%d` oznacza liczby dziesiętne ze znakiem. Pozostałe elementy będą umieszczone w formatowanym łańcuchu w takiej kolejności, w jakiej umieszczone są jako parametry polecenia formatującego.

Gdybyś chciał wyświetlić w drugim wierszu swojego wyświetlacza LCD dokładny czas, to mógłbyś użyć w tym celu odpowiednio sformatowanego łańcucha zbudowanego na bazie niezależnych zmiennych określających godziny, minuty i sekundy:

```
char line2[17];
int h = 12;
int m = 30;
int s = 5;
sprintf(line2, "Czas: %2d:%02d:%02d", h, m, s);
```

Po wyświetleniu w oknie monitora portu szeregowego lub na wyświetlaczu LCD tablica znaków `line2` wyglądałaby następująco:

```
Czas: 12:30:05
```

Liczby zostały ustawione we właściwych miejscach, a przed cyfrą 5 zostało dodane zero. W zaprezentowanym szkicu między dwukropkami znalazły się polecenia formatujące każdą z trzech składowych czasu. Godziny sformatowano za pomocą parametru `%2d`. Wyświetla on dwucyfrową liczbę dziesiętną. Minuty i sekundy sformatowano za pomocą nieco innego polecenia (`%02d`). Polecenie to służy do wyświetlania liczb dwucyfrowych, ale dodatkowo poprzedza zerem liczbę jednocyfrową.

Zachowaj pewną ostrożność. Zaprezentowana technika działa z liczbami typu `int`, ale osoby odpowiedzialne za środowisko programistyczne Arduino nie zaimplementowały standardowej biblioteki formatującej języka C dla innych typów zmiennych, takich jak np. zmienne typu `float`.

Ustalanie długości łańcucha

Łańcuch znajdujący się w tablicy elementów typu `char` może być krótszy od tej tablicy. Funkcja `strlen` służy do określania ilości znaków umieszczonych w łańcuchu przed znakiem kończącym.

Wspomniana funkcja zwraca rozmiar łańcucha (bez znaku `\0`), przyjmując jako jedyny argument tablicę znaków. Poniżej znajduje się przykład jej użycia:

```
strlen("abc")
```

Funkcja zwróci wartość 3.

Biblioteka Arduino StringObject

Środowisko programistyczne Arduino, począwszy od wersji 019 wydanej kilka lat temu, jest wyposażone w bibliotekę Arduino StringObject. Jest ona intuicyjna w obsłudze dla osób mających doświadczenie w pracy z językami takimi jak Java, Ruby czy Python. W takich językach normą jest scalanie łańcuchów za pomocą znaku „+”. Biblioteka ta zawiera również wiele przydatnych funkcji pozwalających na przeszukiwanie łańcuchów lub przeprowadzanie na nich innych operacji.

Oczywiście korzystanie z tej biblioteki pociąga za sobą konieczność przeznaczenia na nią kilku kilobajtów pamięci, w której będzie zapisany szkic. Omawiana biblioteka korzysta z dynamicznej alokacji pamięci, z czym wiążą się pewne problemy, takie jak zagrożenie zapelnieniem całej pamięci operacyjnej. Warto więc przemyśleć korzystanie z niej. Wielu użytkowników Arduino pozostaje przy możliwościach manipulowania łańcuchami oferowanymi standardowo przez język C.

Biblioteka Arduino StringObject jest bardzo łatwa w użyciu, zwłaszcza gdy stosowałeś wcześniej łańcuchy w języku Java.

Tworzenie łańcuchów

Łańcuch może być utworzony za pomocą tablicy elementów typu char, int lub float, co ilustruje poniższy przykład:

```
String message = "Temperatura: ";
String temp = String(123);
```

Łączenie łańcuchów

Łańcuchy mogą być łączone z sobą lub z danymi innego typu za pomocą operatora +. Wypróbuj działanie poniższego kodu. Umieść go w funkcji setup pustego szkicu.

```
Serial.begin(9600);
String message = "Temperatura: ";
String temp = String(123);
Serial.println(message + temp + "°C");
```

Zwróć uwagę na to, że do łańcucha znaków dołączana jest liczba, która tak naprawdę jest tablicą znaków. Jeżeli pierwszy z elementów łączonych za pomocą operatora + będzie łańcuchem, to pozostałe elementy będą przed dołączeniem automatycznie konwertowane na łańcuchy.

Inne funkcje zdefiniowane w bibliotece StringObject

W tabeli 6.1 wymieniono najczęściej stosowane funkcje znajdujące się w omawianej bibliotece. Więcej informacji na temat dostępnych funkcji umieszczono pod adresem <http://arduino.cc/en/Reference/StringObject>.

Tabela 6.1. Niektóre przydatne funkcje biblioteki *StringObject*

Funkcja	Przykład	Opis
<code>[]</code>	<code>char ch = String("abc")[0]</code>	ch przypisywana jest wartość a.
<code>trim</code>	<code>String s = " abc "; s.trim();</code>	Usuwa znaki spacji znajdujące się po obu stronach łańcucha abc. s przypisywana jest wartość abc.
<code>toInt</code>	<code>String s = "123"; int x = s.toInt();</code>	Przeprowadza konwersję liczby znajdującej się w łańcuchu na typ <code>int</code> lub <code>long</code> .
<code>substring</code>	<code>s = "abcdefg"; String s2 = s.substring(1, 3);</code>	Zwraca część oryginalnego łańcucha. s2 — w tym przypadku będą to znaki bc. Parametry są umieszczone w indeksie i przed nim.
<code>replace</code>	<code>String s = "abcdefg"; s.replace("de", "DE");</code>	We wszystkie miejsca łańcucha, w których występuje kombinacja znaków de, wstawia kombinację znaków DE. W wyniku operacji łańcuch s będzie miał wartość abcDEfg.

Korzystanie z pamięci EEPROM

Dane przechowywane przez każdą zmienną zastosowaną w szkicu Arduino zostaną utracone w wyniku odłączenia mikrokontrolera od zasilania lub jego ponownego uruchomienia. Jeżeli chcesz zachować pewne dane, to musisz je zapisać w pamięci EEPROM. Jednocześnie można w niej zapisywać tylko 1 bajt danych. Płytką Arduino Uno została wyposażona w 1 kB pamięci EEPROM.

UWAGA

Płytką Arduino Due nie posiada pamięci EEPROM. Płytką ta pozwala na zapis danych na karcie micro-SD.

Korzystanie z pamięci EEPROM wymaga użycia biblioteki dołączonej do środowiska programistycznego Arduino. W poniższym przykładzie pokazano, jak za pomocą kodu umieszczonego w funkcji `setup` można zapisać 1 bajt danych w pamięci EEPROM.

```
#include <EEPROM.h>
void setup()
{
    byte valueToSave = 123;
    EEPROM.write(0, valueToSave);
}
```

Pierwszym argumentem funkcji `write` jest adres komórki pamięci EEPROM, pod którym zostanie zapisany bajt danych. Drugim argumentem tej funkcji jest wartość, która zostanie zapisana pod tym adresem.

Aby odczytać dane z pamięci EEPROM, należy skorzystać z polecenia `read`. W celu odczytania pojedynczego bajta danych wystarczy skorzystać z poniższego kodu:

```
EEPROM.read(0);
```

W miejsce cyfry 0 należy podać adres komórki pamięci EEPROM.

Przykład korzystania z pamięci EEPROM

W kolejnym przykładzie przedstawiono typowy program, który w trakcie działania zapisuje pewną wartość w pamięci. Zostanie ona odczytana podczas kolejnego uruchomienia programu. Aplikacja ta jest częścią projektu zamykającego i otwierającego drzwi przy użyciu hasła wprowadzanego za pomocą monitora portu szeregowego. Hasło to jest przechowywane w pamięci EEPROM. Dzięki takiemu rozwiązaniu można je zmienić. Gdyby kod otwierający drzwi musiał być zmieniany podczas każdego uruchomienia Arduino, to umożliwianie użytkownikowi jego edycji nie miałoby sensu.

W dalszej części niniejszego rozdziału zostaną omówione pewne elementy szkicu, który znajdziesz w pliku o nazwie `06_06_przykład_EEPROM.ino`. Plik ten dostępny jest w internecie na stronie <http://www.helion.pl/ksiazki/arpokk.htm>. Zrozumienie budowy tego szkicu może okazać się łatwiejsze, gdy zobaczysz jego działanie w praktyce. Uruchomienie programu nie wymaga łączenia Arduino z żadnymi urządzeniami zewnętrznymi.

Funkcja `setup` wywołuje funkcję `initializeCode`:

```
void initializeCode()
{
    byte codeSetMarker = EEPROM.read(0);
    if (codeSetMarker == codeSetMarkerValue)
    {
        code = readSecretCodeFromEEPROM();
    }
    else
    {
        code = defaultCode;
    }
}
```

Funkcja ta ma za zadanie określić wartość zmiennej `code` (hasło). Wspomniana wartość jest zwykle odczytywana z pamięci EEPROM, ale wiążą się z tym pewne trudności.

Pamięć EEPROM nie jest czyszczona podczas ładowania nowego szkicu. Zawartość tej pamięci może być zmieniona tylko w wyniku nadpisanie przez nowe wartości. W związku z tym podczas pierwszego uruchomienia szkicu może dojść do wczytania nieokreślonej wartości z pamięci EEPROM. Wartość ta mogła zostać zapisana przez jakiś szkic uruchomiony w przeszłości. Może to doprowadzić do sytuacji, w której użytkownik nie będzie w stanie otworzyć drzwi, ponieważ nie będzie znał hasła wprowadzonego do programu.

Problem ten można rozwiązać za pomocą innego szkicu zapisującego w pamięci EEPROM domyślne hasło. Szkic ten musi zostać zainstalowany na Arduino przed głównym szkicem.

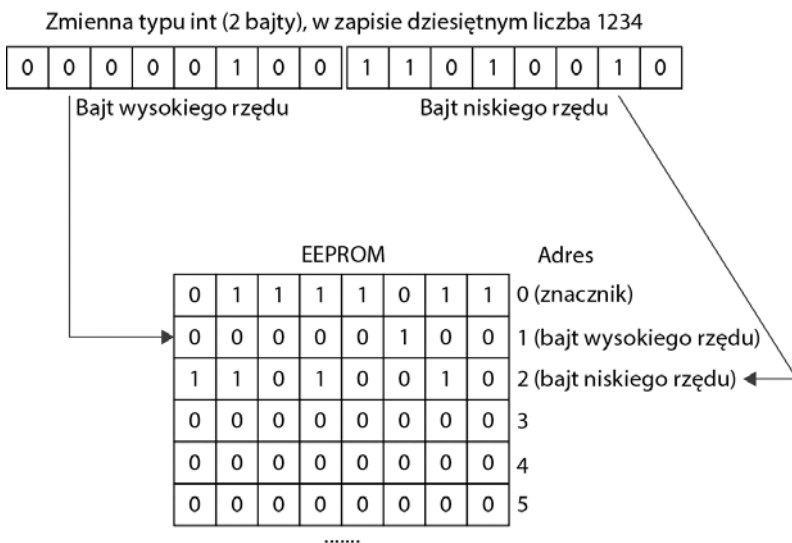
Innym, mniej pewnym, aczkolwiek bardziej wygodnym sposobem rozwiązania tego problemu jest zapisanie w pamięci EEPROM znacznika (określonej wartości). Znacznik ma

informować program o tym, czy w pamięci EEPROM zapisano hasło. Wadą tego rozwiązania jest istnienie małego prawdopodobieństwa, iż w danej komórce pamięci EEPROM zapisano wcześniej identyczny znacznik. Rozwiązanie takie jest nieakceptowane w przypadku pracy nad produktem komercyjnym, ale pracując nad własnym projektem, możesz zaakceptować takie ryzyko.

Funkcja `initializeCode` odczytuje pierwszy bajt pamięci EEPROM. Jeżeli odczytana wartość jest identyczna z zawartością zmiennej `codeMarkerValue`, której przypisano wartość 123, to program zakłada, że w pamięci EEPROM zapisano hasło, i wywołuje funkcję `readSecretCodeFromEEPROM`.

```
int readSecretCodeFromEEPROM()
{
    byte high = EEPROM.read(1);
    byte low = EEPROM.read(2);
    return (high << 8) + low;
}
```

Funkcja ta odczytuje dwubajtowy kod będący wartością typu `int`, zapisany w bajtach numer 1 i 2 pamięci EEPROM (zobacz rysunek 6.5).



Rysunek 6.5. Zapis zmiennej typu `int` w pamięci EEPROM

Aby przeprowadzić konwersję dwóch oddzielnych bajtów na pojedynczą zmienną typu `int`, należy przesunąć wysokie bajty na odpowiednie pozycje (`high << 8`), a następnie dodać do nich niskie bajty.

Hasło zapisane w pamięci jest odczytywane tylko po uruchomieniu Arduino. Szkic powinien je jednak zapisywać w pamięci EEPROM za każdym razem, gdy jest zmieniane, tak aby Arduino po odłączeniu od prądu lub po ponownym uruchomieniu mogło wczytać to hasło z powrotem.

Odpowiada za to funkcja `saveSecretCodeToEEPROM`:

```
void saveSecretCodeToEEPROM()
{
    EEPROM.write(0, codeSetMarkerValue);
    EEPROM.write(1, highByte(code));
    EEPROM.write(2, lowByte(code));
}
```

Znacznik jest zapisywany w pamięci EEPROM w komórce o numerze 0. Informuje on program, że w pamięci zapisano właściwe hasło. Następnie wspomniana funkcja zapisuje hasło w dwóch kolejnych komórkach pamięci EEPROM. Zmienna typu `int` jest zapisywana za pomocą dwóch oddzielnych funkcji narzędziowych Arduino — `highByte` i `lowByte`.

Korzystanie z biblioteki `avr/eeprom.h`

Biblioteka Arduino EEPROM umożliwia wykonywanie również innych operacji niż tylko jednorazowy zapis lub odczyt 1 bajta danych. W poprzednim przykładzie zobaczyłeś, jak można zapisać dwubajtową zmienną typu `int` w pamięci EEPROM, rozdzielając ją na dwie części. Korzystać z tej pamięci możesz również za pomocą specjalnej biblioteki stworzonej przez firmę AVR. Poszerza ona znacznie możliwości programów, pozwalając między innymi na zapis i odczyt słów (16 bitów), jak również bloków pamięci o dowolnym rozmiarze.

W poniższym szkicu zastosowano wspomnianą bibliotekę do bezpośredniego zapisu i odczytu zmiennej typu `int`, poddawanej inkrementacji podczas każdego uruchomienia Arduino:

```
// 06_07_avr_eeprom_int

#include <avr/eeprom.h>

void setup()
{
    int i = eeprom_read_word((uint16_t*)10);
    i++;
    eeprom_write_word((uint16_t*)10, i);
    Serial.begin(9600);
    Serial.println(i);
}

void loop()
{
}
```

Argument (10) funkcji `eeprom_read_word` i pierwszy argument funkcji `eeprom_write_word` określają początek słowa. Zwróć uwagę na to, że struktura ta zajmuje 2 bajty; aby zapisać kolejną zmienną typu `int`, należy podać adres 12, a nie 11. Zapis `(uint16_t*)` przed liczbą 10 jest konieczny, aby utworzyć odpowiedni (oczekiwany przez funkcję zdefiniowaną w bibliotece) typ indeksu.

Inne dwie przydatne funkcje zdefiniowane w bibliotece to `EEPROM_read_block` i `EEPROM_write_block`. Funkcje te umożliwiają zapis i odczyt struktur danych o dowolnej długości (pozwalając na występowanie odstępów).

Przyjrzyjmy się szkicowi, w którym tablica łańcuchów zostanie zapisana w pamięci EEPROM, począwszy od indeksu o numerze 100:

```
// 06_08_avr_eeprom_string

#include <avr/eeprom.h>

void setup()
{
    char message[] = "I am written in EEPROM";
    eeprom_write_block(message, (void *)100,
        strlen(message) + 1);
}

void loop()
{
}
```

Pierwszym argumentem przekazany funkcji `EEPROM_write_block` jest wskaźnik tablicy znaków, którą chcemy zapisać. Drugim argumentem tej funkcji jest początkowy adres w pamięci EEPROM (100). Ostatnim argumentem jest liczba bajtów, które chcemy zapisać. Jest ona o jeden większa od liczby znaków, z których składa się łańcuch, ponieważ na końcu łańcucha musi zostać umieszczony znak `\0`.

Poniższy szkic wczytuje zapisany wcześniej łańcuch i wyświetla go, wraz z informacją o jego długości, w oknie monitora portu szeregowego:

```
// 06_09_avr_eeprom_string_odczyt

#include <avr/eeprom.h>

void setup()
{
    char message[50]; // wystarczający rozmiar
    eeprom_read_block(&message, (void *)100, 50);
    Serial.begin(9600);
    Serial.println(message);
    Serial.println(strlen(message));
}

void loop()
{
}
```

W celu odczytania łańcucha tworzona jest tablica, w której może zmieścić się 50 elementów. Następnie do tablicy `message` funkcja `EEPROM_read_block` wczytuje 50 znaków. Znak `&` przed nazwą `message` dostarcza funkcji adres tej tablicy w pamięci RAM.

Na końcu łańcucha umieszczony jest znak `\0`. W oknie monitora portu szeregowego wyświetli się tylko ciąg znaków znajdujący się przed nim, nie zostanie wyświetlone pełne 50 znaków.

Ograniczenia pamięci EEPROM

Operacje zapisu i odczytu danych przeprowadzane w pamięci EEPROM przebiegają dość wolno (około 3 ms). Producenci gwarantują możliwość przeprowadzenia 100 000 cykli zapisu, zanim pamięć zacznie gubić zapisane w niej dane. W związku z tym powinieneś uważać, aby nie nadpisywać zawartości pamięci za pomocą instrukcji umieszczonej w pętli.

Korzystanie z pamięci Flash

Najbardziej pojemną pamięcią znajdującą się na płycie Arduino jest pamięć flash. Arduino Uno posiada aż 32 kB pamięci flash, a tylko 2 kB pamięci RAM. Sprawia to, że perspektywa zapisu danych w tej pamięci jest kusząca, ponieważ dane te nie są tracone wraz z odłączeniem mikrokontrolera od prądu.

Istnieją jednak pewne problemy związane z zapisem danych w pamięci flash:

- Pamięć flash znajdująca się na płycie Arduino może być zapisana około 10 000 razy, później może stać się nieużyteczna.
- W pamięci flash zapisany jest kod programu, a więc jeżeli pomylisz się i nadpiszesz go, to może dojść do bardzo dziwnych rzeczy.
- W pamięci flash zapisano również program rozruchowy. Nadpisanie tego programu uniemożliwi korzystanie z Arduino do momentu odblokowania go za pomocą specjalnego programatora (zobacz rozdział 2.).
- W pamięci flash można jednorazowo zapisywać jeden blok danych (64 bajty).

Biorąc to pod uwagę, można stwierdzić, że w pamięci flash warto przechowywać dane, które nie są modyfikowane podczas działania szkicu.

Istnieje nieoficjalna biblioteka umożliwiająca odczyt i zapis danych w pamięci flash płytki Arduino Due, co pozwala na rozwiązanie pewnych problemów wynikających z braku pamięci EEPROM. Więcej informacji na temat tego projektu znajdziesz na przykład na stronie <http://sebastiannilsson.com/en/blogg/ersattare-for-eprom-pa-arduino-due/>.

Stałe będące łańcuchami najłatwiej jest zapisać w pamięci flash za pomocą funkcji `F`, którą opisałem już wcześniej w tym rozdziale. Przypomnę jednak składnię tej funkcji:

```
Serial.println(F("Uruchomiono program"));
```

Funkcja ta działa tylko wtedy, gdy używasz komunikatu będącego stałą (łańcuchem). Nie możesz na przykład przypisać efektu działania tej funkcji wskaźnikowi elementu typu `char`.

Bardziej złożoną techniką jest korzystanie z dyrektywy `PROGMEM`. Pozwala ona na pracę z dowolną strukturą danych. Dane te jednak muszą być stałymi — nie mogą być modyfikowane w trakcie działania programu.

Poniższy przykładowy szkic ilustruje to, w jaki sposób możesz utworzyć tablicę elementów typu `int`, a następnie zapisać ją w pamięci flash:

```
// 06_10_PROGMEM_tablica
```

```
#include <avr/pgmspace.h>
```

```

PROGMEM int value[] = {10, 20, 25, 25, 20, 10};

void setup()
{
  Serial.begin(9600);
  for (int i = 0; i < 6; i++)
  {
    int x = pgm_read_word(&value[i]);
    Serial.println(x);
  }
}

void loop()
{
}

```

Umieszczając dyrektywę `PROGMEM` przed deklaracją tablicy, sprawimy, że zostanie ona zapisana w pamięci flash. Aby odczytać zawarte w niej dane, musisz posłużyć się funkcją `pgm_read_word`, zdefiniowaną w bibliotece `avr/pgmspace.h`:

```
int x = pgm_read_word(&value[i]);
```

W parametrze tej funkcji znajduje się symbol `&`, umieszczony przed nazwą tablicy. Wskazuje on, że wymagany jest adres tablicy znajdującej się w pamięci flash, a nie sama wartość.

Funkcja `pgm_read_word` odczytuje słowo (2 bajty) z pamięci flash. Za pomocą funkcji `pgm_read_byte` możliwy jest odczyt 1 bajta, a funkcja `pgm_read_dword` umożliwia odczyt 4 bajtów.

Zapisywanie danych na kartach SD

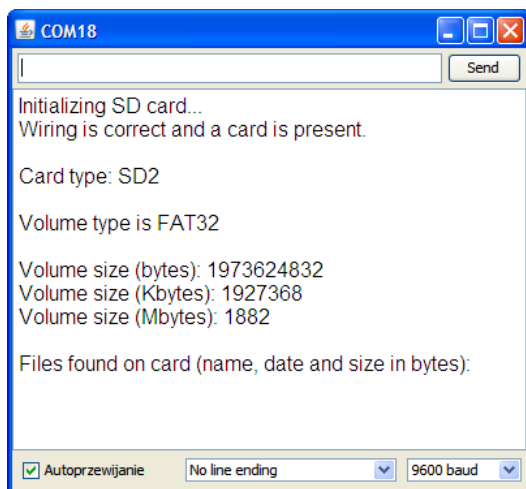
Płytki Arduino nie są wyposażone w slot kart SD, istnieje jednakże kilka różnych płytek mających gniazda kart SD lub micro-SD — takich jak płytka obsługująca sieć Ethernet czy widoczna na rysunku 6.6 płytka pozwalająca na odtwarzanie plików MP3.



Rysunek 6.6. Płytkę rozszerzeń umożliwiającą odtwarzanie plików MP3, wyposażoną w gniazdo kart micro-SD.

Karty SD korzystają z interfejsu SPI (omówionego w rozdziale 9.). Na szczęście w celu obsługi kart SD przez Arduino nie będziesz musiał zajmować się niskopoziomowym programowaniem interfejsu SPI, ponieważ istnieje gotowa biblioteka obsługująca karty SD. Biblioteka ta nosi nazwę „SD” i wchodzi w skład środowiska programistycznego Arduino.

Do wspomnianej biblioteki dołączony jest zestaw przykładowych szkiców wykonujących różne operacje na kartach SD. Jednym z nich jest program wyświetlający informacje o karcie SD w oknie monitora portu szeregowego (zobacz rysunek 6.7).



Rysunek 6.7. Efekt działania przykładowego szkicu *Cardinfo*

Zapis danych na karcie SD jest bardzo łatwy, o czym świadczy poniższy fragment kodu:

```
File dataFile = SD.open("dataLog.txt", FILE_WRITE);
// jeżeli plik jest dostępny, to zapisz w nim:
if (dataFile) {
    dataFile.println(dataString);
    dataFile.close();
    // prześlij dane również do portu szeregowego
    Serial.println(dataString);
}
```

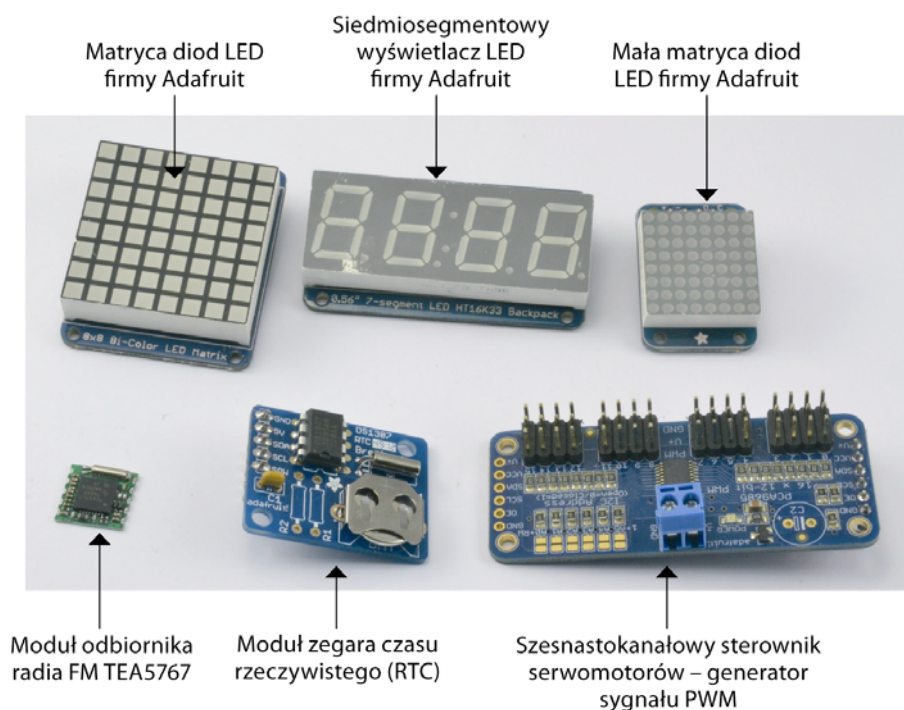
Podsumowanie

Podczas lektury tego rozdziału poznałeś różne zagadnienia związane z pamięcią oraz zapisem danych przez Arduino. W kolejnych rozdziałach poruszę zagadnienia dotyczące tworzenia kodu korzystającego z różnych interfejsów szeregowych — zacznę od magistrali I2C.

Rozdział 7.

Korzystanie z magistrali I2C

Magistrala I2C (a właściwie I²C) jest interfejsem pozwalającym na komunikację pomiędzy mikrokontrolerami a urządzeniami peryferyjnymi. Jest ona czasem nazywana *interfejsem dwuprzewodowym* (ang. *Two Wire Interface* — TWI). Wszystkie płytki Arduino wyposażone są przynajmniej w jeden interfejs I2C, do którego można podłączyć różne urządzenia peryferyjne. Przykłady takich urządzeń pokazano na rysunku 7.1.

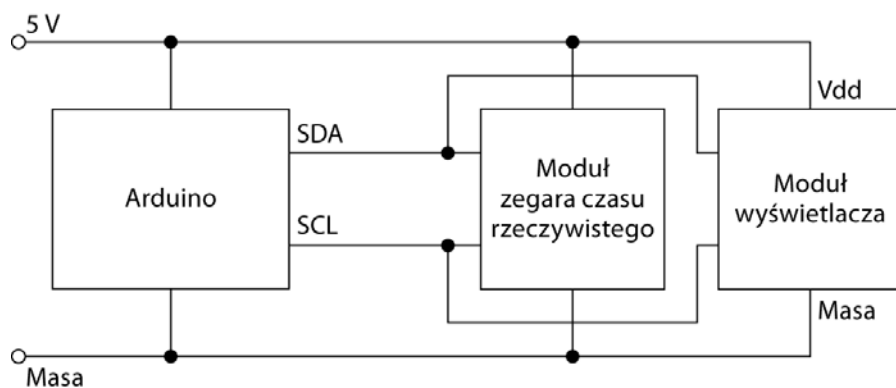


Rysunek 7.1. Urządzenia korzystające z magistrali I2C

Trzy urządzenia widoczne u góry rysunku 7.1 są wyświetlaczami wyprodukowanymi przez firmę Adafruit. W dolnym rzędzie, po lewej stronie, znajduje się moduł odbiornika radia FM TEA5767. Moduł ten można kupić już za kilkanaście złotych za pośrednictwem serwisu Allegro.pl. Pozwala on na odbiór sygnału radiowego nadawanego w paśmie FM. Moduł ten jest dostrajany do wybranej częstotliwości za pomocą poleceń wysyłanych poprzez magistralę I2C. U dołu rysunku, na środku, widoczny jest moduł zegara czasu rzeczywistego (RTC). Na tej płytce znajduje się układ umożliwiający komunikację za pośrednictwem magistrali I2C, a także rezonator kwarcowy, który pozwala na dokładne określenie czasu i daty. Po podaniu aktualnej daty i godziny za pośrednictwem magistrali I2C zegar może w dowolnej chwili przesłać z powrotem do Arduino dane dotyczące aktualnej daty i czasu. Na płycie tego modułu zainstalowano również ogniwo litowe o długiej żywotności, które umożliwia działanie zegara nawet wtedy, gdy moduł jest odłączony od zewnętrznego źródła zasilania. W prawym dolnym rogu rysunku znajduje się szesnastokanałowy sterownik serwowmotorów — generator sygnału PWM. Za jego pomocą możemy sterować przy użyciu Arduino pracą dodatkowych 16 wyjść analogowych.

Standard I2C określa się mianem magistrali, ponieważ nie jest on ograniczony do bezpośredniego łączenia ze sobą dwóch komponentów. Wyobraź sobie, że do mikrokontrolera podłączono wyświetlacz. Za pomocą tych samych złączy magistrali I2C możesz podłączyć do urządzenia „nadrzędnego” cały zestaw urządzeń „podrzędnych”. Arduino działa jako urządzenie „nadrzędne”, a każdemu urządzeniu „podrzędnemu” przypisywany jest unikalny adres identyfikujący je po podłączeniu do magistrali.

Na rysunku 7.2 przedstawiono układ dwóch urządzeń korzystających z magistrali I2C, podłączonych do Arduino — jest to zegar czasu rzeczywistego i moduł wyświetlacza.



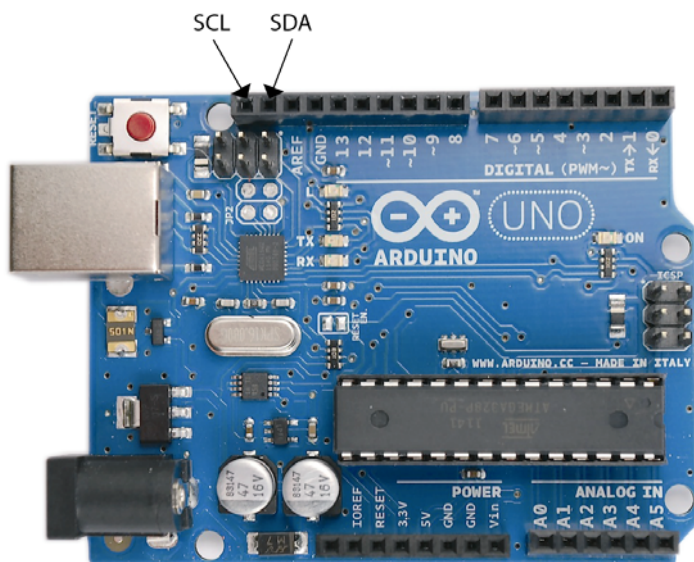
Rysunek 7.2. Arduino sterujące pracą dwóch urządzeń połączonych za pomocą magistrali I2C

Za pomocą magistrali I2C możliwe jest połączenie ze sobą dwóch płytek Arduino i prowadzenie wymiany danych pomiędzy nimi. W takim układzie jedna z płytek Arduino musi zostać skonfigurowana tak, aby działała jako urządzenie „nadrzędne”, a druga — jako urządzenie „podrzędne”.

Warstwa sprzętowa

Interfejs I2C tworzą elektryczne linie łączące mikrokontroler z urządzeniami peryferyjnymi, mogące działać zarówno jako cyfrowe wyjście, jak i cyfrowe wejście (są one określane mianem **trójstanowych**). Sygnał przesyłany w trybie trójstanowym nie przybiera stanu wysokiego lub niskiego, jego wartość zmienia się w inny sposób. Wyjścia są tak naprawdę **otwartym kolektorem**, co oznacza, że wymagają rezystorów podciągających. Rezystory te powinny charakteryzować się oporem $4,7\text{ k}\Omega$. Cała magistrala wymaga zastosowania pary takich rezystorów podciągających napięcie do $3,3\text{ V}$ lub 5 V — w zależności od tego, przy jakim napięciu ma działać magistrala. Jeżeli niektóre z urządzeń podłączonych do magistrali są zasilane prądem o innym napięciu, to musisz zastosować układ zmieniający napięcie. Firma Adafruit ma w swojej ofercie dwukierunkowy regulator napięcia DSS138, który może współpracować z magistralą I2C: <http://www.adafruit.com/products/757>.

Różne modele płytek Arduino przypisują różnym pinom możliwość obsługi magistrali I2C. Na przykład na płytce Uno funkcje złączy SDA i SCL przypisano pinom A4 i A5, a płytka Leonardo przypisuje te same funkcje pinom D2 i D3. Więcej informacji na temat złączy SDA i SCL znajdziesz w kolejnej sekcji. Na obu płytkach złącza SDA i SCL umiejscowiono obok gniazda AREF (zobacz rysunek 7.3).



Rysunek 7.3. Złącza magistrali I2C na płytce Arduino Uno

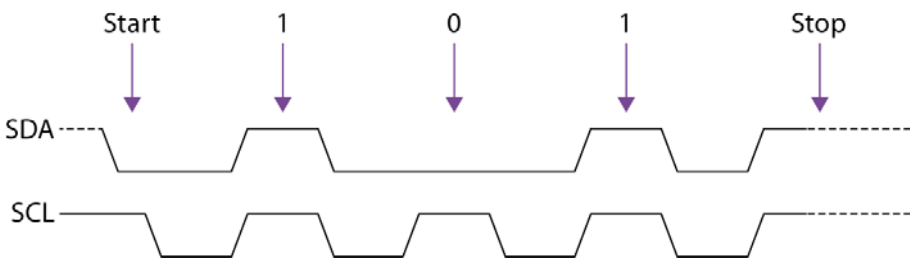
W tabeli 7.1 opisano umiejscowienie złączy magistrali I2C na popularnych płytkach Arduino.

Tabela 7.1. Umieszczenie złączy magistrali I2C na płytkach Arduino

Płytką	Piny	Uwagi
Uno	A4 (SDA) i A5 (SCL)	Piny oznaczone etykietami SCL i SDA pełnią również funkcje złączy A4 i A5. Znajdują się obok gniazda AREF.
Leonardo	D2 (SDA) i D3 (SCL)	Piny oznaczone etykietami SCL i SDA pełnią również funkcje złączy D2 i D3. Znajdują się obok gniazda AREF.
Mega 2560	D20 (SDA) i D21 (SCL)	–
Due	D20 (SDA) i D21 (SCL)	Na płytce Due umieszczona jest również druga para złączy magistrali I2C. Styki te oznaczono etykietami SDA1 i SCL1.

Protokół magistrali I2C

Magistrala I2C w celu transmisji i odbioru danych korzysta z dwóch przewodów (dlatego czasem określa się ją mianem interfejsu dwuprzewodowego). Przewody te to linia zegara (SCL — ang. *Serial Clock Line*) i linia danych (SDA — ang. *Serial Data Line*). Na rysunku 7.4 przedstawiono wykres logiki sygnału.



Rysunek 7.4. Wykres logiki sygnału transmitowanego za pośrednictwem magistrali I2C

Urządzenie nadrzędne generuje sygnał zegara. Gdy zaistnieje potrzeba przesłania danych, urządzenie nadawcze (nadrzędne lub podrzędne) przełącza linię danych z trybu trójstanowego wejścia cyfrowego na tryb przesyłu danych. Dane są transmitowane jako impulsy logiczne (sygnał wysoki lub niski). Czas trwania tych impulsów zależy od sygnału zegara. Gdy dane zostaną przesłane, zegar może być zatrzymany, a złącze SDA przełączone z powrotem w tryb trójstanowy.

Biblioteka Wire

Oczywiście impulsy te możesz wygenerować ręcznie za pomocą techniki **bit banging**, polegającej na włączaniu i wyłączaniu cyfrowych wyjść przy użyciu kodu programu. Jednakże, w celu uproszczenia komunikacji, do oprogramowania Arduino dołączono bibliotekę Wire.

Obsługuje ona czynności związane z czasowaniem sygnału, co znacznie ułatwia przesyłanie danych.

Aby korzystać z biblioteki Wire, należy najpierw dołączyć się do szkicu za pomocą polecenia:

```
#include <Wire.h>
```

Inicjacja magistrali I2C

Zwykle funkcję urządzenia nadrzędnego magistrali I2C będzie pełniło Arduino. W tym celu w funkcji setup należy umieścić polecenie begin:

```
void setup()
{
  Wire.begin();
}
```

Zwróć uwagę na to, że gdy Arduino pełni funkcję urządzenia nadrzędnego, wtedy nie ma potrzeby określania jego adresu. Gdyby Arduino było inicjowane jako urządzenie podrzędne, to musiałbyś określić jego adres identyfikacyjny w magistrali I2C (jako parametr funkcji należy podać liczbę z zakresu od 0 do 127).

Wysyłanie danych przez urządzenie nadrzędne

Aby przesłać dane do innego urządzenia korzystającego z magistrali I2C, należy skorzystać z funkcji beginTransmission. Należy określić przy tym adres urządzenia podłączonego do magistrali I2C, do którego chcemy przesłać dane:

```
Wire.beginTransmission(4);
```

Do urządzeń podłączonych do magistrali I2C możliwe jest jednoczesne przekazywanie 1 bajta danych lub tablicy elementów typu char, co ilustruje poniższy kod:

```
Wire.send(123)    // przesyła bajt 123
Wire.send("ABC"); // przesyła łańcuch znaków „ABC”
```

Po przesłaniu danych należy uruchomić funkcję endTransmission:

```
Wire.endTransmission();
```

Odbieranie danych przez urządzenie nadrzędne

Urządzenie nadrzędne, do którego chcemy przesłać dane z urządzenia podrzędnego, musi określić liczbę bajtów danych, które mają być przesłane za pomocą funkcji requestFrom:

```
Wire.requestFrom(4, 6); // żądamy przesłania 6 bajtów przez urządzenie znajdujące się pod adresem 4
```

Pierwszym argumentem tej funkcji jest adres urządzenia podrzędnego, od którego chcemy, aby urządzenie nadrzędne otrzymało dane. Drugim argumentem jest liczba bajtów, które mają być przesłane. Może zaistnieć sytuacja, w której urządzenie podrzędne zwróci mniejszą ilość danych. Funkcja available określa liczbę przetransmitowanych bajtów — to, czy dane zostały przesłane. Poniższy szkic (jeden z przykładowych szkiców dołączonych do biblioteki

Wire) ilustruje proces odbioru danych przez urządzenie nadrzędne. Dane te są następnie wyświetlane w oknie monitora portu szeregowego.

```
#include <Wire.h>
void setup()
{
    Wire.begin(); // inicjuje obsługę magistrali I2C (w przypadku urządzenia nadrzędnego podawanie
                  // adresu nie jest konieczne)
    Serial.begin(9600); // uruchamia szeregową transmisję danych
}
void loop()
{
    Wire.requestFrom(4, 6); // żąda przesłania 6 bajtów przez urządzenie znajdujące się pod adresem 4
    while(Wire.available()) // urządzenie podrzędne może przesłać mniejszą ilość danych
    {
        char c = Wire.receive(); // bajt danych jest odbierany jako zmienna typu char
        Serial.print(c); // wyświetla odebrane znaki
    }
    delay(500);
}
```

Biblioteka Wire będzie buforować dane odbierane za pośrednictwem magistrali I2C.

Przykład działania magistrali I2C

Do każdego urządzenia korzystającego z magistrali I2C powinna być dołączona dokumentacja, w której określone są komunikaty obsługiwane przez dany moduł. Dokumentacja ta może być przydatna podczas formułowania własnych komunikatów i wysyłania ich do zewnętrznego modułu oraz podczas tworzenia instrukcji interpretujących komunikaty kierowane przez urządzenie do Arduino. W praktyce nierzadko okazuje się, że jeżeli jakieś urządzenie korzystające z magistrali I2C jest często podłączane do Arduino, oznacza to, że ktoś napisał gotową bibliotekę obsługującą przesyłane komunikaty za pomocą prostych w użyciu funkcji. Jeżeli biblioteka nie jest dostępna, a samodzielnie doszedłeś to tego, jak należy komunikować się z jakimś zewnętrznym modulem, to dla dobra ogółu warto byłoby, abyś napisał własną bibliotekę. Karma może wrócić do Ciebie.

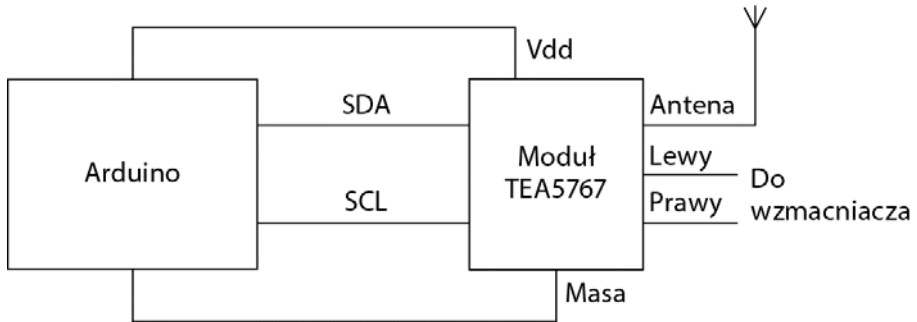
Gdy pełnoprawna biblioteka nie istnieje, najprawdopodobniej możesz znaleźć w internecie fragmenty szkiców korzystających z danego urządzenia.

Radio FM TEA5767

W pierwszym przykładzie nie będziemy korzystać z biblioteki. Będziemy komunikować się bezpośrednio z modulem TEA5767 za pomocą Arduino. W internecie znajdziesz dystrybutorów sprzedających ten moduł w bardzo przystępnych cenach. TEA5767 można w prosty sposób podłączyć do Arduino. Tak otrzymasz radioodbiornik sterowany za pomocą Arduino.

Problem związany z tym modulem bierze się stąd, że znajdujące się na nim złącza są tak usytuowane, że aby podłączyć go do Arduino za pośrednictwem płytki prototypowej lub przewodów połączeniowych, niezbędny jest zakup jakichś przejściówek.

Na rysunku 7.5 przedstawiono sposób, w jaki należy połączyć moduł TEA5767 z płytką Arduino.



Rysunek 7.5. Moduł TEA5767 podłączony do płytki Arduino Uno za pośrednictwem magistrali I2C

Pełną notę katalogową modułu TEA5767 znajdziesz pod adresem <http://www.sparkfun.com/datasheets/Wireless/General/TEA5767.pdf>. Dokumentacja ta zawiera wiele danych technicznych dotyczących chipa, ale można w niej znaleźć również sekcję, w której wymieniono komunikaty obsługiwane przez moduł. Układ TEA5767 oczekuje, że będą do niego przesyłane komunikaty pięciobajtowe. Przedstawiony tutaj przykładowy szkic umożliwi dostrojenie częstotliwości pracy modułu po uruchomieniu płytki Arduino. W praktyce do ustawiania odbieranej częstotliwości będziesz potrzebował innych dodatkowych, zewnętrznych komponentów, takich jak przyciski i wyświetlacz LCD.

```
// 07_01_I2C_TEA5767

#include <Wire.h>

void setup()
{
    Wire.begin();
    setFrequency(93.0); // MHz
}

void loop()
{
}

void setFrequency(float frequency)
{
    unsigned int frequencyB = 4 * (frequency * 1000000 + 225000) / 32768;
    byte frequencyH = frequencyB >> 8;
    byte frequencyL = frequencyB & 0xFF;

    Wire.beginTransaction(0x60);
    Wire.write(frequencyH);
    Wire.write(frequencyL);
    Wire.write(0xB0);
}
```

```
Wire.write(0x10);
Wire.write(0x00);
Wire.endTransmission();
delay(100);
}
```

Interesujący nas kod znajduje się w całości w funkcji `setFrequency`. Wymieniona funkcja w roli parametru przyjmuje zmienną typu `float`. Wartość ta określa częstotliwość wyrażoną w megahercach. Gdybyś chciał w praktyce zastosować ten szkic, to warto, żebyś dowiedział się, na jakiej częstotliwości w Twojej okolicy nadaje jakaś dobra radiostacja (jej sygnał powinien być dość mocny). Ustaloną wartość należy podać podczas wywoływania funkcji `setFrequency` w funkcji `setup`.

Aby przekształcić częstotliwość (MHz), wyrażoną przez liczbę zmiennoprzecinkową, w dwubajtową wartość, która może zostać przesłana jako część pięciobajowego komunikatu, należy dokonać pewnych obliczeń. Obliczenia te są wykonywane przez poniższe linie kodu:

```
unsigned int frequencyB = 4 * (frequency * 1000000 + 225000) / 32768;
byte frequencyH = frequencyB >> 8;
byte frequencyL = frequencyB & 0xFF;
```

Polecenie `>>` przesuwa bity w prawo, a więc `>>8` przesuwa 8 najbardziej znaczących bitów na pozycję 8 najmniej znaczących bitów. Operator `&` sprawia, że 8 górnych bitów jest maskowanych, a więc widoczne są tylko pozostałe bity. Więcej informacji na temat tej operacji znajdziesz w rozdziale 9.

Dalszy ciąg kodu znajdującego się w funkcji `setFrequency` rozpoczyna transmisję komunikatów poprzez magistralę I2C do urządzenia podrzędnego o adresie `0x60`. Adres ten jest przypisany do chipa TEA5767. Szkic przesyła 5 bajtów danych, które rozpoczynają się od 2 bajtów określających częstotliwość.

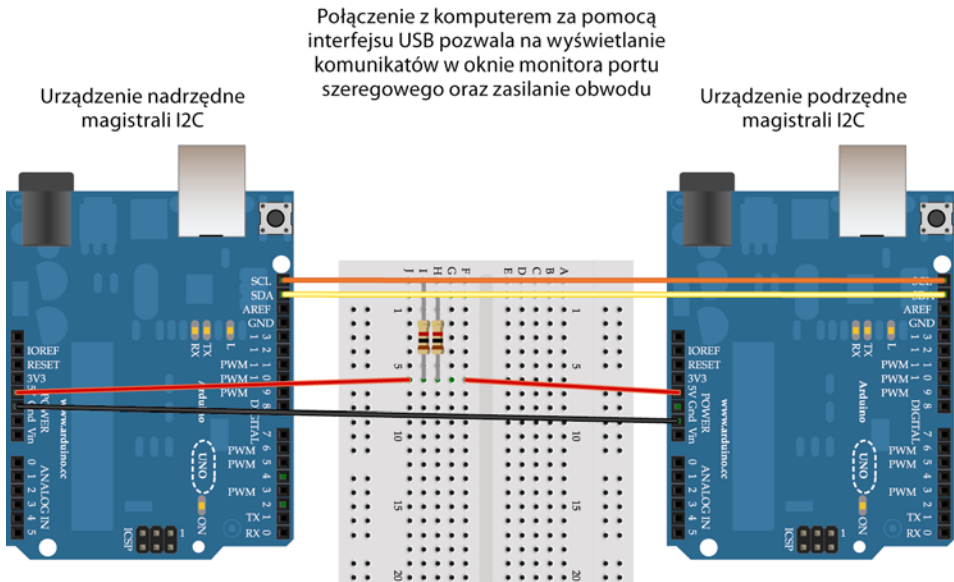
Czytając dokumentację modułu, dowiesz się, jakie czynności wykona on po wysłaniu innych komunikatów. Radioodbiornik może między innymi skanować pasmo, wyciszać któryś z kanałów, pracować w trybie mono i stereo.

W rozdziale 15. przyjrzymy się jeszcze raz przedstawionemu przykładowi i stworzymy bibliotekę ułatwiającą korzystanie z modułu TEA5767.

Przesyłanie danych pomiędzy dwoma płytkami Arduino

W kolejnym przykładzie będziemy korzystać z dwóch płytek Arduino. Pierwsza z nich będzie działała jako urządzenie nadrzędne magistrali I2C, a druga — jako urządzenie podrzędne. Urządzenie nadrzędne będzie wysyłało komunikaty do urządzenia podrzędnego, które będzie wyświetlało je w oknie monitora portu szeregowego (w ten sposób będzie można sprawdzić, czy dane są przesyłane pomiędzy dwoma płytkami).

Urządzenia należy połączyć ze sobą tak, jak pokazano na rysunku 7.6. W używany wcześniej moduł TEA5767 wbudowano rezystor podciągający sygnał magistrali I2C. Takiego rezystora nie wbudowano w płytkę Arduino, a więc łącząc dwie płytki Arduino, powinieneś umieścić w obwodzie rezystor $4,7\text{ k}\Omega$ (zobacz rysunek 7.6).



Rysunek 7.6. Dwie płytki Arduino połączone za pośrednictwem magistrali I2C

Każda z dwóch płytek Arduino musi zostać zaprogramowana za pomocą innego szkicu. Oba szkice dołączono jako przykłady użycia biblioteki Wire. Płytkę nadrzędną zaprogramuj przy użyciu szkicu *master_writer* (znajdziesz go w menu *Plik/Przykłady/Wire*), natomiast Arduino podrzędne zaprogramuj za pomocą szkicu *slave_receiver* (znajdziesz go w menu *Plik/Przykłady/Wire*).

Gdy zaprogramujesz już obie płytki Arduino, podłącz do komputera płytkę pracującą w charakterze urządzenia podrzędnego. Będzie ona wysyłała dane do okna monitora portu szeregowego, a także zasilala Arduino pracujące w charakterze urządzenia nadrzędnego.

Przyjrzyjmy się programowi uruchomionemu na nadrzędnej płytce Arduino:

```
#include <Wire.h>

void setup()
{
    Wire.begin(); // inicjuje magistralę I2C (w przypadku urządzenia nadrzędnego podawanie adresu
                  // jest zbędne)
}

byte x = 0;

void loop()
{
    Wire.beginTransmission(4); // transmituje do urządzenia numer 4
    Wire.write("x is ");      // wysyła 5 bajtów
    Wire.write(x);             // wysyła 1 bajt
    Wire.endTransmission();    // koniec transmisji
    x++;
    delay(500);
}
```

Kod ten generuje komunikat w rodzaju „x is 1” (pol. *x wynosi 1*), gdzie 1 jest liczbą inkrementowaną co pół sekundy. Komunikat ten jest następnie wysyłany do urządzenia podrzędnego podłączonego do magistrali I2C. W wywołaniu funkcji `beginTransmission` podano numer identyfikacyjny tego urządzenia (4).

Szkic zainstalowany w urządzeniu podrzędnym ma za zadanie odbierać komunikaty wysyłane przez urządzenie nadrzędne, a następnie przekazywać je dalej do okna monitora portu szeregowego:

```
#include <Wire.h>

void setup()
{
    Wire.begin(4);           // dołącza do magistrali I2C jako urządzenie numer 4
    Wire.onReceive(receiveEvent); // rejestruje zdarzenie
    Serial.begin(9600);      // rozpoczyna szeregową transmisję danych
}

void loop()
{
    delay(100);
}

// funkcja przetwarzająca wszystkie dane przesłane przez urządzenie nadrzędne
// funkcja ta jest zarejestrowana jako zdarzenie, zobacz setup()
void receiveEvent(int howMany)
{
    while(1 < Wire.available()) // pętla przetwarzająca wszystkie elementy poza ostatnim
    {
        char c = Wire.read(); // odbiera znak przesłany jako zmienna typu byte
        Serial.print(c);      // wyświetla znak
    }
    int x = Wire.read();      // odbiera zmienną typu integer przesłaną jako zmienna typu byte
    Serial.println(x);        // wyświetla zmienną typu integer
}
```

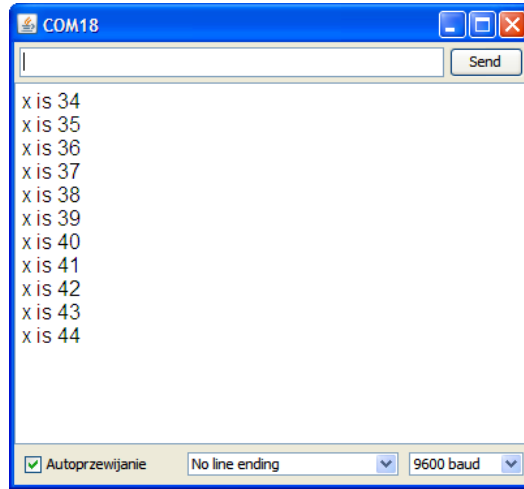
Zauważ, że funkcji `Wire.begin` przekazano parametr 4. Wartość ta określa adres, jaki przypisujemy urządzeniu podrzelnemu podłączonemu do magistrali I2C. Wartość ta musi być zgodna z adresem, na jaki urządzenie nadrzędne wysyła komunikaty.

WSKAZÓWKA

Do magistrali I2C możesz podłączyć wiele płytek Arduino pracujących w charakterze urządzeń podrzędnych, pod warunkiem że każdej płytce przypiszesz inny adres.

Szkic urządzenia podrzędnego jest skonstruowany inaczej niż szkic urządzenia nadrzędnego. Odpowiedzi na wiadomości przesyłane przez urządzenie nadrzędne są generowane za pomocą przerwania. Służy do tego funkcja `onReceive`. Jest ona wywoływana w charakterze procedury obsługi przerwania (zobacz rozdział 3.). Wspomnianą funkcję należy umieścić w funkcji `setup`, wtedy nadejście komunikatu będzie wywoływało napisaną przez użytkownika funkcję `receiveEvent`.

Funkcja `receiveEvent` oczekuje podania jednego parametru wskazującego liczbę bajtów przeznaczonych do odczytu. Tym razem liczba ta jest ignorowana. Pętla `while` najpierw odczytuje wszystkie dostępne znaki, a następnie przekazuje je kolejno dalej. Potem odczytywana jest jednobajtowa liczba znajdująca się na końcu komunikatu. W dalszej kolejności jest ona wyświetlana za pomocą monitora portu szeregowego. Zastosowanie funkcji `println` zamiast `writeln` zapewnia, że zostanie ona wyświetlona jako liczba, a nie ciąg znaków (zobacz rysunek 7.7).



Rysunek 7.7. Dane wyświetlane w monitorze portu szeregowego dzięki komunikowaniu się płytek Arduino za pośrednictwem magistrali I2C

Płytki z diodami LED

Do magistrali I2C dość często podłączane są również wyświetlacze. Przykładem takich urządzeń są matryce diodowe oraz wyświetlacze siedmiosegmentowe produkowane przez firmę Adafruit. Składają się one z diod oraz chipów obsługujących magistralę I2C. Korzystając z takich modułów, możemy sterować wyświetlaczem diodowym za pomocą zaledwie 2 złączy Arduino (pinów SDA i SCL).

Do urządzeń tego typu (zobacz górny rząd na rysunku 7.1) dołączone są specjalne biblioteki, w których zdefiniowano zestaw funkcji pozwalających na wyświetlanie za pomocą diod LED tekstu i elementów graficznych. Więcej informacji na temat tego typu modułów znajdziesz na stronie <http://www.adafruit.com/products/902>.

Po zainstalowaniu bibliotek obsługa magistrali I2C przestaje być kłopotliwa dla użytkownika. Może on posługiwać się poleceniami wysokiego poziomu, co ilustruje poniższy przykładowy szkic dołączony do jednej z bibliotek:

```
#include <Wire.h>
#include "Adafruit_LEDBackpack.h"
#include "Adafruit_GFX.h"
Adafruit_8x8matrix matrix = Adafruit_8x8matrix();
```

```
void setup()
{
  matrix.begin(0x70);
  matrix.clear();
  matrix.drawLine(0,0, 7,7, LED_RED);
  matrix.writeDisplay();
}
```

Zegar czasu rzeczywistego DS1307

Kolejnym popularnym modulem korzystającym z magistrali I2C jest zegar czasu rzeczywistego DS1307. Istnieje specjalna biblioteka obsługująca ten moduł, która ułatwia pracę użytkownikowi i ukrywa przed nim komunikaty przesyłane pomiędzy urządzeniami. Biblioteka ta nosi nazwę *RTClib*. Można ją pobrać ze strony <https://github.com/adafruit/RTClib>.

Chciałbym przedstawić Ci przykładowy szkic dołączony do tej biblioteki:

```
#include <Wire.h>#
#include "RTClib.h"

RTC_DS1307 RTC;

void setup () {
  Serial.begin(9600);
  Wire.begin();
  RTC.begin();

  if (! RTC.isrunning()) {
    Serial.println("RTC is NOT running!");
    // przestawia zegar czasu rzeczywistego na datę i godzinę kompilacji szkicu
    RTC.adjust(DateTime(__DATE__, __TIME__));
  }
}

void loop () {
  DateTime now = RTC.now();
  Serial.print(now.year(), DEC);
  Serial.print('/');
  Serial.print(now.month(), DEC);
  Serial.print('/');
  Serial.print(now.day(), DEC);
  Serial.print(' ');
  Serial.print(now.hour(), DEC);
  Serial.print(':');
  Serial.print(now.minute(), DEC);
  Serial.print(':');
  Serial.print(now.second(), DEC);
  Serial.println();
  delay(1000);
}
```

Jeżeli chcesz poznać sposób, w jaki moduł ten komunikuje się z Arduino za pośrednictwem magistrali I2C, to otwórz plik z biblioteką i przyjrzyj się jej kodowi. W bibliotece *RTClib* znajdują się pliki *RTC.h* i *RTClib.cpp*. Dostępne są w katalogu *libraries\RTClib*.

Przyjrzyj się, na przykład, definicji funkcji *now* znajdującej się w pliku *RTClib.cpp*:

```

DateTime RTC_DS1307::now() {
    Wire.beginTransmission(DS1307_ADDRESS);
    Wire.write(i);
    Wire.endTransmission();

    Wire.requestFrom(DS1307_ADDRESS, 7);
    uint8_t ss = bcd2bin(Wire.read() & 0x7F);
    uint8_t mm = bcd2bin(Wire.read());
    uint8_t hh = bcd2bin(Wire.read());
    Wire.read();
    uint8_t d = bcd2bin(Wire.read());
    uint8_t m = bcd2bin(Wire.read());
    uint16_t y = bcd2bin(Wire.read()) + 2000;

    return DateTime (y, m, d, hh, mm, ss);
}

```

Wartości przekazywane za pośrednictwem magistrali I2C są zakodowane w systemie BCD (wartości dziesiętne zakodowane dwójkowo). Muszą być one przekonwertowane na zmienne typu byte za pomocą funkcji `bcd2bin` zdefiniowanej w bibliotece.

Zmienne typu byte podczas konwersji na format BCD są dzielone na dwa czterobitowe półbajty. Każdy półbajt definiuje jedną cyfrę dwucyfrowej liczby dziesiętnej. Na przykład liczba 37 po zapisaniu w kodzie BCD ma formę 0011 0111. Pierwsze cztery bity w systemie dziesiętnym oznaczają liczbę 3, a pozostałe cztery bity — liczbę 7.

Podsumowanie

W tym rozdziale dowiedziałeś się, czym jest magistrala I2C. Poznałeś techniki pozwalające na komunikację pomiędzy płytką Arduino a urządzeniami peryferyjnymi i innymi płytkami Arduino.

W kolejnym rozdziale omówię inny interfejs szeregowy stosowany do komunikowania się z urządzeniami peryferyjnymi — interfejs 1-Wire. Nie jest on stosowany tak często jak magistrala I2C, ale wyposażono w niego na przykład popularny czujnik temperatury DS18B20.

Rozdział 8.

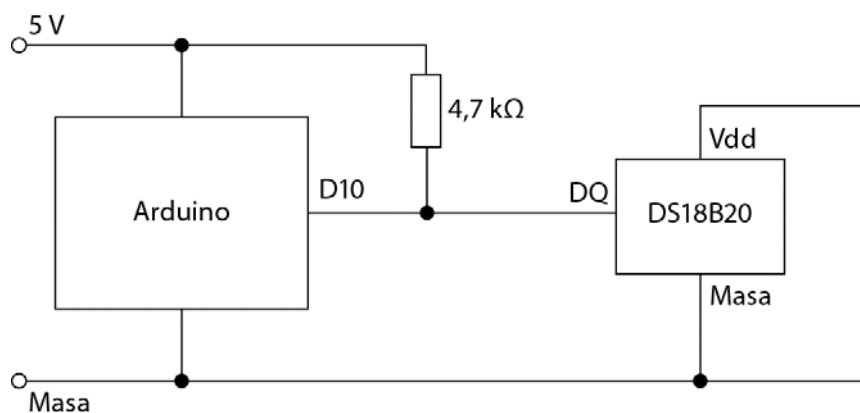
Praca z urządzeniami wyposażonymi w interfejs 1-Wire

Interfejs 1-Wire został zaprojektowany po to, aby służyć, podobnie jak magistrala I2C (zobacz rozdział 7.), do komunikacji pomiędzy mikrokontrolerami a układami scalonymi urządzeń peryferyjnych za pomocą jak najmniejszej linii przesyłu danych. Standard 1-Wire został stworzony przez firmę Dallas Semiconductor. Osiągnął on ekstremum w redukcji linii, po których przesyłane są dane. Sygnał przekazywany jest zaledwie po jednej linii. Interfejs ten jest wolniejszy od magistrali I2C, ale umożliwia **pasożytnicze zasilanie** urządzeń podłączonych do mikrokontrolera. Wystarczy je połączyć za pomocą przewodu masowego i linii, która służy do przesyłu danych oraz zasilania.

O wiele więcej urządzeń jest wyposażanych w magistralę I2C niż w interfejs 1-Wire. Zwykle w interfejs ten wyposażane są moduły produkowane przez firmę Dallas Semiconductor lub Maxim. Wśród nich można znaleźć urządzenia służące do identyfikacji kaset w drukarkach, pamięci flash EEPROM i przetworniki analogowo-cyfrowe. Czujnik temperatury DS18B20 to wyposażone w interfejs 1-Wire urządzenie, które jest najczęściej używane przez elektroników hobbystów.

Sprzęt obsługujący interfejs 1-Wire

Na rysunku 8.1 przedstawiono moduł DS18B20 (zasilany pasożytniczo) podłączony do Arduino za pomocą dwóch przewodów.



Rysunek 8.1. Urządzenie wyposażone w interfejs 1-Wire, podłączone do Arduino

1-Wire jest magistralą. Za pomocą tego interfejsu, w sposób przedstawiony na rysunku 8.1, można połączyć ze sobą aż 255 urządzeń.

Jeżeli chcesz zasilac urządzenie w sposób standardowy, to nie musisz włączać w obwód rezystora 4,7 kΩ, a złącze Vdd modułu DS18B20 możesz połączyć bezpośrednio z pinem zasilającym 5 V, zamiast łączyć je z masą.

Protokół 1-Wire

Do interfejsu 1-Wire, podobnie jak do magistrali I2C, podłączane są urządzenia nadrzędne i podrzędne. Mikrokontroler pełni funkcję urządzenia nadrzędnego, a moduły peryferyjne działają jako urządzenia podrzędne. Każdemu urządzeniu podrzdnemu w fazie produkcji nadawany jest numer identyfikacyjny. Jest to adres identyfikujący dane urządzenie po podłączeniu do jednej magistrali z wieloma innymi urządzeniami podrzdnymi. Adres ten zajmuje 64 bity, a więc może istnieć $1,8 \cdot 10^{19}$ różnych numerów identyfikacyjnych.

Protokół interfejsu 1-Wire przypomina protokół obsługujący magistralę I2C — urządzenie nadrzędne przełącza kierunek przesyłu danych przez przewód. Dzięki takiemu rozwiązaniu możliwa jest komunikacja w obu kierunkach. Interfejs 1-Wire nie korzysta z oddzielnej szyny zegara. Dane (jedyńki i zera) przekazywane są za pomocą jednego przewodu za pośrednictwem długich i krótkich impulsów. Impuls trwający 60 μ s symbolizuje 0, a impuls trwający 15 μ s symbolizuje 1.

Normalnie napięcie prądu płynącego przez szynę danych jest wysokie. Gdy mikrokontroler (urządzenie nadrzędne) ma wysłać polecenie do urządzenia podrzdnego, jest to sygnalizowane za pomocą specjalnego „resetującego” impulsu niskiego napięcia, trwającego 480 μ s. Następnie przesyłany jest ciąg impulsów symbolizujących zera i jedynki.

Biblioteka OneWire

Korzystanie z interfejsu 1-Wire za pomocą biblioteki OneWire jest bardzo proste. Wspomnianą bibliotekę można pobrać ze strony <http://playground.arduino.cc/Learning/OneWire>.

Inicjalizowanie biblioteki OneWire

Jeżeli Arduino ma pracować w charakterze urządzenia nadrzędnego interfejsu 1-Wire, to należy zaimportować bibliotekę OneWire za pomocą polecenia:

```
#include <OneWire.h>
```

Następnie należy utworzyć egzemplarz OneWire i określić, który z pinów Arduino ma pełnić funkcję magistrali do przesyłania danych. Te dwie operacje można wykonać za pomocą jednego polecenia. Komunikacja może przebiegać za pośrednictwem dowolnego pinu, wystarczy, że przekażesz jego numer jako parametr dla wspomnianej funkcji:

```
OneWire bus(10);
```

W omawianym przykładzie, do zainicjowania magistrali, zastosujemy pin numer 10.

Skanowanie magistrali

Każdemu z urządzeń podrzędnych nadaje się w fabryce indywidualny numer identyfikacyjny. Musisz dowiedzieć się, jakie urządzenia połączono ze sobą. Lepiej, żebyś nie stosował tego numeru w szkicu, ponieważ, w przypadku gdybyś musiał wymienić któryś z zewnętrznych modułów, nie mógłbyś korzystać z tego samego szkicu, a to z tego powodu, że nowemu modułowi byłby przypisany zupełnie inny numer identyfikacyjny. Arduino (jako urządzenie nadrzędne) może wygenerować listę urządzeń podłączonych do magistrali. Na podstawie pierwszych 8 bitów adresu każdego z urządzeń można określić typ urządzenia, a więc możemy powiedzieć, czy jest to na przykład moduł DS18B20, czy jakieś zupełnie inne urządzenie.

W tabeli 8.1 przedstawiono kody serii najpopularniejszych urządzeń korzystających z magistrali 1-Wire. Większą listę modułów znajdziesz na stronie <http://owfs.sourceforge.net/family.html>.

Tabela 8.1. Kody serii różnych urządzeń korzystających z interfejsu 1-Wire

Kod serii (szesnastkowy)	Seria urządzeń	Opis
06	iButton 1993	Moduł pamięci pastylkowej
10	DS18S20	Precyzyjny czujnik temperatury (rozdzielczość 9 b)
28	DS18B20	Precyzyjny czujnik temperatury (rozdzielczość 12 b)
1C	DS28E04-100	4 kB pamięci EEPROM

W bibliotece OneWire zdefiniowano funkcję wyszukującą wszystkie urządzenia podrzędne podłączone do magistrali. Przedstawiony szkic wyświetla adresy wszystkich urządzeń w oknie monitora portu szeregowego:

```
// 08_01_OneWire_Lista

#include <OneWire.h>

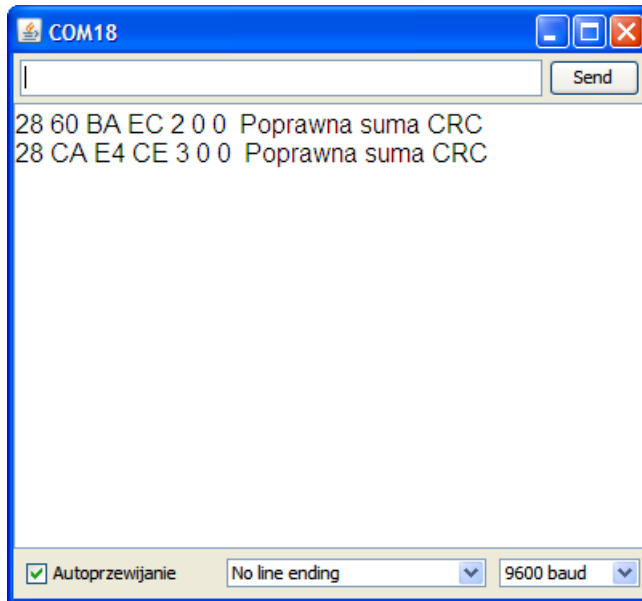
OneWire bus(10);

void setup()
{
  Serial.begin(9600);
  byte address[8]; // 64 bity
  while (bus.search(address))
  {
    for(int i = 0; i < 7; i++)
    {
      Serial.print(address[i], HEX);
      Serial.print(" ");
    }
    // Czy suma kontrolna jest poprawna?
    if (OneWire::crc8(address, 7) == address[7])
    {
      Serial.println("Poprawna suma CRC");
    }
    else
    {
      Serial.println("Bład CRC");
    }
  }
}

void loop()
{
}
```

Na rysunku 8.2 przedstawiono efekt działania szkicu po podłączeniu do interfejsu 1-Wire dwóch czujników temperatury DS18B20. Zauważ, że w przypadku obu tych urządzeń pierwszy bajt danych (liczba 28 wyrażona w systemie szesnastkowym) określa serię, do której należą urządzenia.

Funkcja `search` wymaga utworzenia ośmiobajtowej tablicy, w której zapisany zostanie kolejny znaleziony przez nią adres. Gdy funkcja skończy proces wyszukiwania urządzeń, wtedy zwróci wartość 0. Pozwala to na iterację pętli `while` dopóty, dopóki nie zostaną wyświetlone wszystkie adresy. Ostatni bajt adresu to tak naprawdę bajt danych CRC (cyklicznej kontroli nadmiarowej). Dane te pozwalają sprawdzić poprawność adresu. W bibliotece OneWire dostępna jest również funkcja sprawdzająca poprawność danych.



Rysunek 8.2. Lista urządzeń podrzędnych podłączonych do magistrali 1-Wire

Korzystanie z układu DS18B20

W przedstawionym dalej przykładzie do obsługi chipa DS18B20 zastosowano bibliotekę One-Wire. Na rysunku 8.3 pokazano ten moduł podłączony do Arduino. Zwróć uwagę na to, że chip ten umieszczono w obudowie mającej trzy wyprowadzenia. Przypomina on swym wyglądem tranzystor.

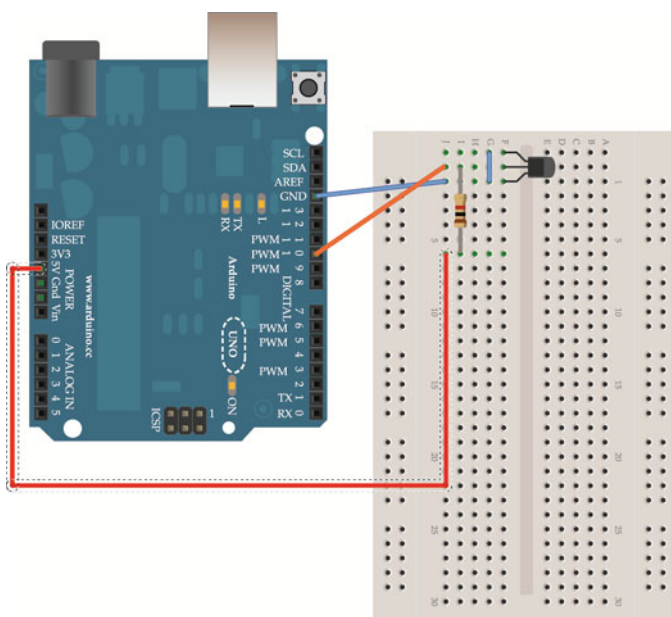
Firma Dallas Semiconductor stworzyła specjalną bibliotekę ułatwiającą wysyłanie żądań pomiaru temperatury i dekodującą dane zwracane przez układ DS18B20. Bibliotekę tę można znaleźć na stronie <https://github.com/milesburton/Arduino-Temperature-Control-Library>.

```
// 08_02_OneWire_DS18B20

#include <OneWire.h>
#include <DallasTemperature.h>

const int busPin = 10;

OneWire bus(busPin);
DallasTemperature sensors(&bus);
DeviceAddress sensor;
```



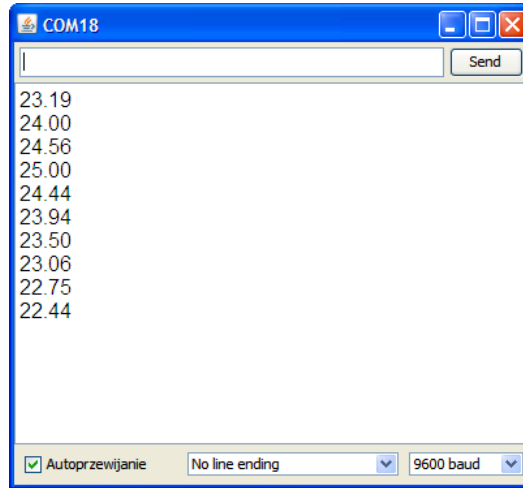
Rysunek 8.3. Układ DS18B20 podłączony do Arduino

```
void setup()
{
    Serial.begin(9600);
    sensors.begin();
    if (!sensors.getAddress(sensor, 0))
    {
        Serial.println("Nie znaleziono chipu DS18B20");
    }
}

void loop()
{
    sensors.requestTemperatures();
    float tempC = sensors.getTempC(sensor);
    Serial.println(tempC);
    delay(1000);
}
```

Przedstawiony szkic w oknie monitora portu szeregowego wyświetla temperaturę wyrażoną w stopniach Celsjusza, mierzoną przez jeden czujnik (zobacz rysunek 8.4).

W podanym przykładzie zastosowano tylko jeden czujnik temperatury, ale z łatwością możesz go zmodyfikować tak, aby korzystać z wielu czujników. Biblioteka DallasTemperature obudowuje proces uzyskiwania adresów urządzeń podłączonych do interfejsu 1-Wire za pomocą funkcji `getAddress`. Drugim parametrem tej funkcji jest indeks czujnika. Jeżeli chcesz korzystać z drugiego czujnika, to musisz utworzyć nową zmienną przechowującą jego adres, a następnie ustalić go za pomocą funkcji `getAddress`. Ze strony <http://www.helion.pl/ksiazki/arpokk.htm> możesz pobrać szkic obsługujący dwa czujniki. Znajdziesz go w pliku o nazwie `08_02_OneWire_DS18B20.ino`.



Rysunek 8.4. Wyświetlanie temperatury mierzonej przez czujnik DS18B20

Podsumowanie

W tym rozdziale zdobyłeś pewne informacje związane z interfejsem 1-Wire. Dowiedziałeś się również, jak komunikować się za jego pośrednictwem z popularnym czujnikiem temperatury DS18B20.

W kolejnym rozdziale przyjrzymy się innemu interfejsowi, o nazwie *SPI*, służącemu do szeregowej transmisji danych.

Rozdział 9.

Praca z urządzeniami wyposażonymi w interfejs SPI

Szeregowy interfejs urządzeń peryferyjnych (SPI) jest kolejną magistralą, za pomocą której możesz łączyć swoje Arduino z urządzeniami peryferyjnymi. Interfejs ten jest dość szybki, ale korzysta z czterech pinów. Omówione wcześniej interfejsy korzystały z jednego pinu (1-Wire) lub dwóch pinów (I2C). Interfejs ISP nie jest tak naprawdę magistralą. Czwarte złącze, z którego korzysta ten interfejs, służy do wyboru układu podrzędnego. Na każde urządzenie peryferyjne podłączone do tej magistrali musi przypadać jeden pin wyboru układu (ang. *Slave Select* — SS). Takie rozwiązanie pozwala na efektywną komunikację z jednym modulem peryferyjnym, podczas gdy wszystkie pozostałe urządzenia zostają uśpione.

Istnieje wiele urządzeń wyposażonych w interfejs SPI. Często są to takie same moduły jak te, opisane wcześniej, mogące współpracować z magistralą I2C. Nierzadko spotyka się urządzenia wyposażone jednocześnie w magistralę I2C i interfejs SPI.

Operowanie bitami

Przesyłanie danych przez interfejs SPI bardzo często wiąże się z koniecznością przeprowadzania wielu operacji na bitach. Zanim zaczniesz analizę pierwszego, przykładowego projektu (zastosowano w nim układ przetwornika A/C MCP3008), warto, abyś zrozumiał operację przestawiania bitów, a także ich maskowania w celu uzyskania zmiennej całkowitoliczbowej, będącej wartością odczytaną przez analogowe złącze. Dlatego, zanim zagłębię się w zagadnienia związane z pracą interfejsu SPI, wyjaśnię Ci proces operowania bitami.

Wartości binarne i szesnastkowe

W rozdziale 4. (zobacz rysunek 4.2) przedstawiłem Ci pojecie bitu i bajta. Operując bitami znajdującymi się w bajcie lub słowie (dwóch bajtach), możesz korzystać z ich wartości dziesiętnych, ale konwersja pomiędzy systemem dwójkowym i dziesiętnym nie jest czymś, co można łatwo zrobić w pamięci. Dlatego w języku C, używanym do programowania Arduino, wartości są często wyrażane jako stałe zapisane w systemie dwójkowym. Do takiej deklaracji należy użyć specjalnej składni, którą zademonstrowano w poniższym przykładzie:

```
byte x = 0b00000011; // 3
unsigned int y = 0b0000000000000011; // 3
```

W pierwszej linii kodu zdefiniowano zmienną typu `byte`. Przypisana jej wartość w systemie dziesiętnym wynosi 3 (2+1). Zera znajdujące się na początku liczby są elementem opcjonalnym, przypominającym o tym, że zmienna może przechowywać 8 bitów.

W drugiej linii kodu zmiennej typu `int` przypisano wartość 16-bitową. Przed nazwą zmiennej umieszczono kwalifikator `unsigned`, co oznacza, że zmienna powinna być używana tylko do przechowywania liczb dodatnich. Kwalifikator ten tak naprawdę ma znaczenie tylko wtedy, gdy korzystasz z operatorów `+`, `-`, `*`. Nie powinienes ich stosować, jeżeli masz zamiar operować bitami. Korzystanie z kwalifikatora `unsigned` jest dobrą praktyką.

Binarna reprezentacja liczby 16-bitowej wygląda na dość długą i nieporęczną. Dlatego programiści często, w celu zapisu dłuższych liczb binarnych, stosują notację **szesnastkową** (zwaną również zapisem **heksadecymalnym**).

Podstawą systemu szesnastkowego jest liczba 16. W związku z tym poza standardowymi cyframi z zakresu od 0 do 9 stosuje się również litery od A do F (w celu przedstawienia wartości z zakresu od 10 do 15). W ten sposób jedna cyfra może posłużyć do zapisu 4 bitów. W tabeli 9.1 porównano zapis liczb z zakresu od 0 do 15 w systemie dziesiętnym, binarnym i szesnastkowym.

Stałe szesnastkowe są zapisywane za pomocą specjalnej notacji, podobnej do tej, którą stosowaliśmy w przypadku liczb binarnych.

```
int x = 0x0003; // 3
int y = 0x010F; // 271 (256 + 15)
```

Z notacją tego typu spotkasz się również poza językiem C. Stosuje się ją w dokumentach w celu podkreślenia tego, że dana liczba jest zapisana w systemie szesnastkowym, a nie dziesiętnym.

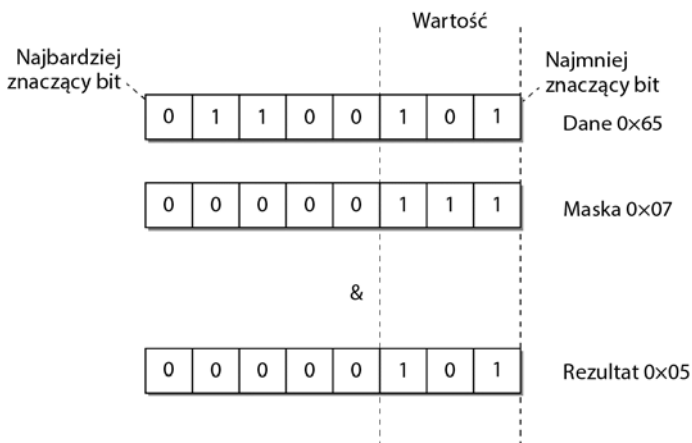
Maskowanie bitów

Pracując z danymi przesłanymi z urządzenia peryferyjnego za pomocą dowolnego interfejsu, często można spotkać się z problemem polegającym na tym, że dane są przesyłane za pomocą bajtów, a nie wszystkie przesłane bajty mogą być nam akurat potrzebne. Osoby projektujące urządzenia peryferyjne często starają się, aby były one w stanie przesłać jak najwięcej informacji za pomocą jak najmniejszej liczby bitów. Przyspiesza to proces wymiany danych, ale sprawia, że programowanie takiego urządzenia staje się trudniejsze.

Tabela 9.1. Liczby w systemie dziesiętnym, binarnym i szesnastkowym

System dziesiętny	System binarny	System szesnastkowy
0	0000	0
1	0001	1
2	0010	2
3	0011	3
4	0100	4
5	0101	5
6	0110	6
7	0111	7
8	1000	8
9	1001	9
10	1010	A
11	1011	B
12	1100	C
13	1101	D
14	1110	E
15	1111	F

Proces „maskowania” bitów pozwala na pominięcie pewnych bajtów znajdujących się w większej strukturze danych. Na rysunku 9.1 pokazano, jak bajt zawierający wiele danych może być częściowo zamaskowany w celu uzyskania liczby składającej się z trzech najmniej znaczących bitów.

**Rysunek 9.1.** Maskowanie bitów

W stosunku do bitów, z których składa się liczba, spotyka się określenia „najbardziej znaczący” i „najmniej znaczący”. Jeżeli spojrzysz na standardowy zapis liczby binarnej, to cyfrę znajdującą się skrajnie po lewej stronie można określić mianem najbardziej znaczącego bitu lub najstarszego bitu, a skrajną cyfrę znajdującą się po prawej stronie można nazwać najmniej znaczącym bitem lub bitem najmłodszym. Tak naprawdę cyfra znajdująca się po prawej stronie może mieć tylko wartość 1 lub 0. Najmniej znaczący bit nazywany jest również **bitem 0**. Kolejny bit można nazwać bitem numer 1.

W przykładzie pokazanym na rysunku 9.1 bajt danych zawiera pewne dane, którymi nie jesteśmy zainteresowani, znajdujące się w okolicy najbardziej znaczącego bitu. Chcemy uzyskać wartość składającą się tylko z trzech bitów znajdujących się na najmniej znaczących pozycjach. Do danych należy w tym celu dodać za pomocą operatora `&` maskę, której trzy interesujące nas bity są jedynkami. Otrzymana wartość będzie rezultatem koniunkcji bitowej kolejnych bitów. W wyniku tej operacji wartość 1 jest otrzymywana tylko wtedy, gdy obu „dodawanych do siebie” bitem przypisano wartość 1.

Poniższy przykład ilustruje użycie operatora `&` w języku C używanym do programowania Arduino. Operatorem **koniunkcji bitowej** jest znak `&`. Operatorem zwykłej koniunkcji logicznej był znak `&&`.

```
byte data = 0b01100101;
byte result = (data & 0b00000111);
```

Na koniec zmiennej `result` zostanie przypisana wartość 5 (w systemie dziesiętnym).

Przesuwanie bitów

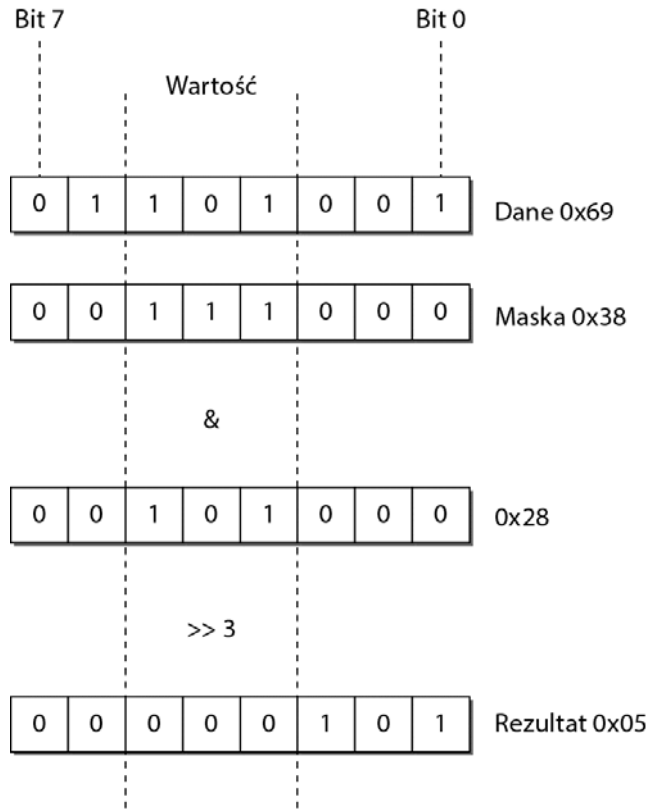
Kolejnym problemem napotykanym podczas pracy z danymi przesyłanymi przez urządzenie zewnętrzne jest to, że potrzebne bity nie zawsze znajdują się w najmniej znaczącym końcu bajta.

Na przykład gdyby interesująca nas wartość, widoczna na rysunku 9.1, była zapisana pomiędzy bitami o numerach 5 i 3 (zobacz rysunek 9.2), to najpierw trzeba by było zamaskować niepotrzebne nam bity (tak jak zrobiliśmy to w poprzednim przykładzie), a następnie przesunąć interesującą nas wartość o trzy bity w prawo.

W języku C operator `>>` służy do przesuwania bitów w prawo. Po tym operatorze należy podać liczbę pozycji, o jaką mają zostać przesunięte bity. W wyniku tej operacji niektóre bity mogą zostać wysunięte poza obszar bajta. Poniższy przykład ilustruje użycie wspomnianego operatora:

```
byte data = 0b01101001;
byte result = (data & 0b00111000) >> 3;
```

Co, jeżeli musiałbyś połączyć dwa 8-bitowe bajty w jedną 16-bitową zmienną typu `int`? Operację taką można wykonać, przesuwając bity najbardziej znaczącego bajta na koniec zmiennej typu `int`, a następnie dodając do niej drugi bajt. Proces ten przedstawiono na rysunku 9.3.



Rysunek 9.2. Maskowanie i przesuwanie bitów



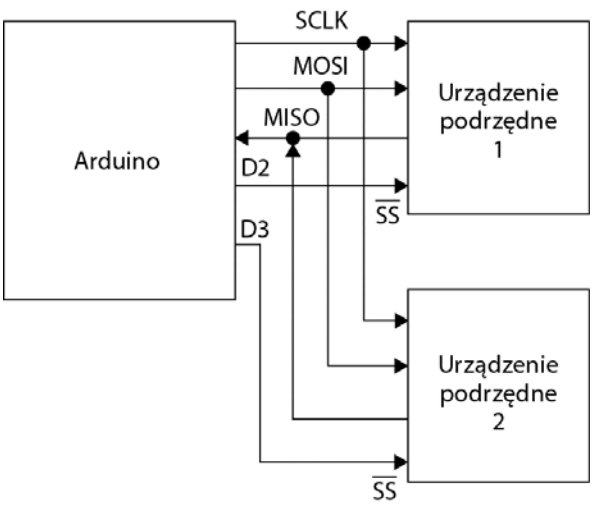
Rysunek 9.3. Tworzenie zmiennej typu int z dwóch zmiennych typu byte

W szkicu Arduino do zmiennej `int` należy najpierw wstawić najbardziej znaczący bajt (`highByte`), przesunąć go w lewo, a następnie dodać najmniej znaczący bajt (`lowByte`):

```
byte highByte = 0x6A;
byte lowByte = 0x0F;
int result = (highByte << 8) + lowByte;
```

Sprzęt obsługujący magistralę SPI

Na rysunku 9.4 pokazano typowy układ składający się z Arduino i dwóch urządzeń podrzędnych.



Rysunek 9.4. Arduino połączone za pomocą interfejsu SPI z dwoma urządzeniami podrzędnymi

Pinom D13, D11 i D12 znajdującym się na płytce Arduino Uno przypisane są funkcje kolejnych złączy interfejsu: SCLK (taktujący sygnał zegarowy), MOSI (dane dla układu peryferyjnego) i MISO (dane z układu peryferyjnego). W tabeli 9.2 podano etykiety pinów popularnych płytek Arduino, którym to pinom przypisano funkcje złączy interfejsu SPI.

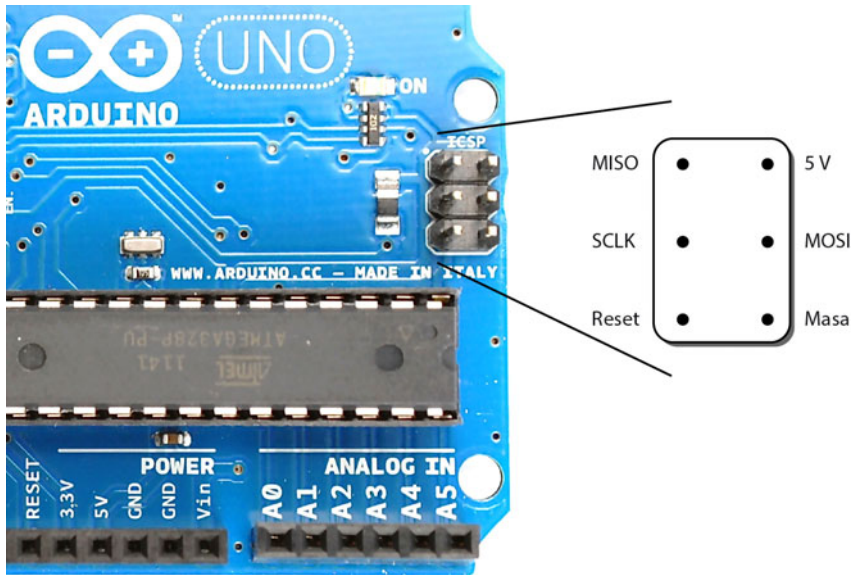
Tabela 9.2. Złącza interfejsu SPI znajdujące się na różnych płytkach Arduino

Płytki	SCLK	MOSI	MISO
Uno	13 (ICSP3)	11 (ICSP4)	12 (ICSP1)
Leonardo	ICSP3	ICSP4	ICSP1
Mega 2560	52 (ICSP3)	51 (ICSP4)	50 (ICSP1)
Due	ICSP3	ICSP4	ICSP1

Każdy z pinów płytki Arduino może generować sygnał aktywujący urządzenie podrzędne. Sygnał taki aktywuje urządzenie podrzędne tuż przed rozpoczęciem transmisji danych i wyłącza je po zakończeniu komunikacji.

Żadna z linii interfejsu nie wymaga stosowania rezystorów podciągających.

Niektóre płytki Arduino (w tym Leonardo) mają złącza interfejsu SPI w formie styków goldpin, a więc niektóre płytki rozszerzeń mają specjalne żeńskie gniazda przeznaczone dla tych styków. Na rysunku 9.5 przedstawiono złącze ICSP. Opisano na rysunku funkcje poszczególnych pinów tego złącza.



Rysunek 9.5. Złącze ICSP znajdujące się na płytce Arduino Uno

Zauważ, że płytka Arduino Uno posiada drugie złącze ICSP, znajdujące się obok przycisku *Reset*. Służy ono do programowania interfejsu USB.

Protokół SPI

Protokół SPI może wydawać się na pierwszy rzut oka dość skomplikowany, ponieważ dane są jednocześnie transmitowane i odbierane zarówno przez urządzenie nadrzędne, jak i wybrane urządzenie podrzędne. Gdy bit danych jest wysyłany z pinu MOSI płytki Arduino do odpowiadającego mu pinu MOSI urządzenia podrzędnego, inny bit danych jest równocześnie wysyłany z pinu MISO urządzenia podrzędnego do pinu MISO znajdującego się na płytce Arduino.

Zwykle Arduino wysyła bajt danych, a następnie przesyła ciąg ośmiu zer. W tym czasie płytka ta odczytuje dane przesyłane do niej z urządzenia peryferyjnego. Urządzenie nadrzędne określa częstotliwość transmisji danych. Upewnij się, że nie jest ona za wysoka dla urządzenia podrzędnego.

Biblioteka SPI

Do środowiska programistycznego Arduino dołączona jest biblioteka obsługująca interfejs SPI, a więc nie ma konieczności jej dodatkowego instalowania. Biblioteka ta potrafi obsłużyć tylko komunikację, w której Arduino pełni funkcję urządzenia nadrzędnego. Obsługuje ona tylko bezpośrednią transmisję całych bajtów. Ograniczenia te nie są kłopotliwe w przypadku pracy

z większością urządzeń peryferyjnych, niektóre z nich jednak mogą oczekiwać przesyłania 12-bitowych komunikatów, co może wiązać się z koniecznością wykonywania skomplikowanych operacji na bitach. Przekonasz się o tym, analizując przykład przedstawiony w kolejnej sekcji tego rozdziału.

Jak zwykle, szkic trzeba rozpocząć od dołączenia biblioteki:

```
#include <SPI.h>
```

Następnie w funkcji `setup` trzeba uruchomić komunikację za pomocą interfejsu SPI. W tym celu należy zastosować polecenie `SPI.begin`.

```
void setup()
{
    SPI.begin();
    pinMode(chipSelectPin, OUTPUT);
    digitalWrite(chipSelectPin, HIGH);
}
```

O ile nie korzystasz z płytki Due, musisz również skonfigurować piny SS obsługujące wszystkie urządzenia podrzędne. W tym celu możesz stosować dowolnie wybrane piny. Ustaw je tak, aby działały jako wyjścia, a następnie natychmiast podaj na nie sygnał wysoki, ponieważ urządzenia podrzędne charakteryzują się odwróconą logiką — podanie sygnału niskiego na ich złącze oznaczone etykietą SS spowoduje aktywację danego modułu.

Płytką Due charakteryzuje się rozszerzoną obsługą biblioteki SPI. W tym przypadku wystarczy zadeklarować pin służący do wyboru urządzenia podrzędnego, a biblioteka automatycznie poda na nim sygnał niski przed rozpoczęciem transmisji danych, natomiast po jej zakończeniu poda sygnał wysoki. Funkcję `SPI.begin` możesz stosować, podając jako jedyny jej argument numer pinu, z którego chcesz korzystać. Wada korzystania z tego rozwiązania jest taka, że Twój szkic przestanie być kompatybilny z innymi płytkami Arduino. W przykładowych szkicach przedstawionych w tym rozdziale piny służące do wyboru urządzenia podrzędnego są sterowane ręcznie. Przykładowe programy mogą zostać uruchomione na dowolnej płycie Arduino.

W bibliotece znajduje się wiele funkcji narzędziowych pozwalających na konfigurację interfejsu SPI. W praktyce zwykle sprawdzają się ustawienia domyślne. Interfejs SPI może wymagać konfiguracji tylko wtedy, gdy wskazują na to wytyczne znajdujące się w nocie aplikacyjnej urządzenia podrzędnego. Funkcje narzędziowe wymieniono w tabeli 9.3.

Jednoczesne wysyłanie i odbiór danych obsługuje funkcja `transfer`. Funkcja ta wysyła bajt danych i zwraca bajt danych odebrany podczas wykonywania operacji transmisji danych.

```
byte sendByte = 0x23;
byte receiveByte = SPI.transfer(sendByte);
```

Komunikacja z urządzeniem peryferyjnym zwykle polega na tym, że urządzenie nadrzędne wysyła żądanie do urządzenia podrzędnego, które w jakiś sposób na nie reaguje. W związku z tym będziesz musiał przeprowadzać dwie transmisje danych: za pierwszym razem trzeba wysłać dane do urządzenia zewnętrznego, a za drugim (wysyłając np. ciąg zer) odebrać dane wygenerowane przez ten moduł. Zobaczysz, jak to działa w praktyce, gdy przeanalizujesz następny przykładowy szkic.

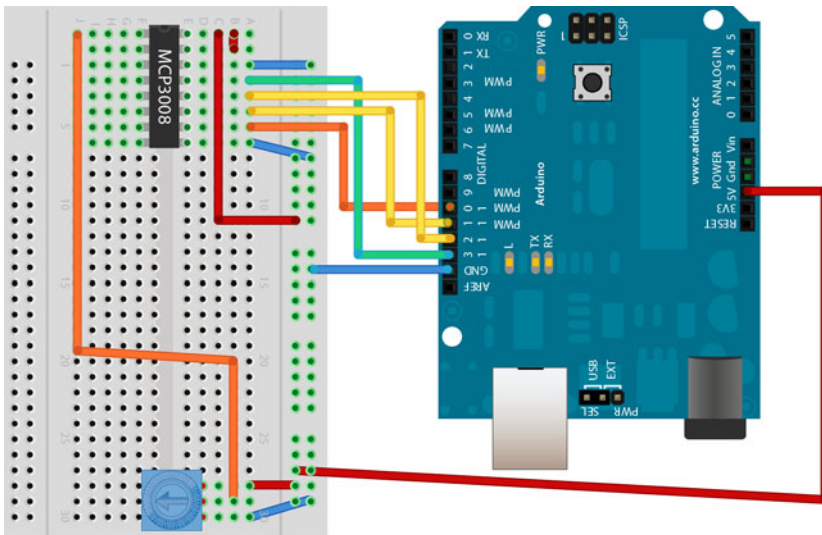
Tabela 9.3. Funkcje konfigurujące interfejs SPI

Funkcja	Opis
<code>SPI.setClockDivider(SPI_CLOCK_DIV64)</code>	Dzieli standardową częstotliwość zegara (4 MHz) przez 2, 4, 8, 16, 32, 64 lub 128.
<code>SPI.setBitOrder(LSBFIRST)</code>	Określa kolejność przesyłanych bitów: parametr <code>LSBFIRST</code> spowoduje rozpoczęcie transmisji danych od najmniej znaczącego bitu, a parametr <code>MSBFIRST</code> spowoduje rozpoczęcie transmisji danych od najbardziej znaczącego bitu. Domyślnym parametrem jest <code>MSBFIRST</code> .
<code>SPI.setDataMode(SPI_MODE0)</code>	Funkcji tej możliwe jest przypisanie argumentów od <code>SPI_MODE0</code> do <code>SPI_MODE3</code> . Funkcja ta określa polaryzację i fazę sygnału zegara. Zwykle nie musisz zmieniać tych ustawień, chyba że nota aplikacyjna danego urządzenia wskazuje na taką konieczność.

Przykład komunikacji za pomocą interfejsu SPI

W kolejnym przykładzie będziemy komunikować się z ośmiokanałowym przetwornikiem A/C MCP3008. Dzięki temu układowi scalonemu możemy wyposażyć Arduino w kolejne dziesięć wejść analogowych. Wspomniany układ jest tani i łatwy do podłączenia.

Na rysunku 9.6 przedstawiono układ podłączony do Arduino za pośrednictwem płytki prototypowej i przewodów połączeniowych. Rezystor nastawny (potencjometr) służy do regulacji napięcia na analogowym wejściu numer 0 w zakresie od 0 V do 5 V.

**Rysunek 9.6.** Obwód obsługiwany przez przykładowy szkic korzystający z interfejsu SPI

To jest szkic obsługujący zaprezentowany obwód:

```
// 09_01_SPI_przetwornik

#include <SPI.h>

const int chipSelectPin = 10;

void setup()
{
    Serial.begin(9600);
    SPI.begin();
    pinMode(chipSelectPin, OUTPUT);
    digitalWrite(chipSelectPin, HIGH);
}

void loop()
{
    int reading = readADC(0);
    Serial.println(reading);
    delay(1000);
}

int readADC(byte channel)
{
    unsigned int configWord = 0b11000 | channel;
    byte configByteA = (configWord >> 1);
    byte configByteB = ((configWord & 1) << 7);
    digitalWrite(chipSelectPin, LOW);
    SPI.transfer(configByteA);
    byte readingH = SPI.transfer(configByteB);
    byte readingL = SPI.transfer(0);
    digitalWrite(chipSelectPin, HIGH);

    // printByte(readingH);
    // printByte(readingL);

    int reading = ((readingH & 0b00011111) << 5) + ((readingL & 0b11111000) >> 3);

    return reading;
}

void printByte(byte b)
{
    for (int i = 7; i >= 0; i--)
    {
        Serial.print(bitRead(b, i));
    }
    Serial.print(" ");
}
```

Funkcja `printByte` była używana w trakcie prac nad programem do wyświetlania danych binarnych. Co prawda funkcja `Serial.print` może wyświetlać wartości binarne, ale bez początkowych zer. Sprawia to, że interpretacja danych jest trudna. Funkcja `printByte` zawsze wyświetla pełne 8 bitów.

Aby zobaczyć dane przesyłane przez układ MCP3008, usuń znaki // znajdujące się na początku dwóch linii wywołujących funkcję `printByte`. Oczekiwane przez Ciebie wartości zostaną wyświetlone.

Interesujące nas operacje wykonywane są przez funkcję `readADC`. Jako parametr przyjmuje ona numer kanału przetwornika A/C (liczbę z zakresu od 0 do 7). Pierwszą rzeczą, jaką należy zrobić, jest przeprowadzenie operacji na bitach — w efekcie zostanie utworzony bajt określający typ konwersji sygnału i kanał, z którego chcemy korzystać.

Chip może pracować w dwóch trybach przetwarzania sygnału analogowego na cyfrowy. W pierwszym trybie układ porównuje sygnały analogowe podawane do dwóch złączy, a w drugim trybie (z którego korzystamy) przetwornik zwraca wartość odczytaną z jednego, określonego kanału — działa jak wejście analogowe Arduino. Notę aplikacyjną układu MCP3008 znajdziesz pod adresem <http://ww1.microchip.com/downloads/en/DeviceDoc/21295d.pdf>. Podano w niej informację, że polecenie konfiguracyjne musi składać się z 4 bitów: pierwszy bit określa tryb pracy (musi on przyjąć wartość 1, aby układ pracował w trybie odczytu danych z jednego złącza), a pozostałe 3 bity określają kanał (liczbę od 0 do 7), z którego dane mają zostać odczytane.

Układ MCP3008 nie został zaprojektowany do jednoczesnego przesyłania bajta danych, a właśnie tego oczekuje biblioteka SPI. Aby moduł MCP3008 rozpoznał opisane wcześniej 4-bitowe polecenie, musimy rozbić je na 2 bajty. Czynność ta jest wykonywana przez poniższy kod:

```
unsigned int configWord = 0b11000 | channel;
byte configByteA = (configWord >> 1);
byte configByteB = ((configWord & 1) << 7);
```

Pierwszy bajt komunikatu konfiguracyjnego zawiera dwie jedynki. Pierwsza z nich może nie być konieczna, druga określa tryb pracy modułu (odczyt danych z jednego wejścia). Dwa pozostałe bity znajdujące się w tym bajcie są dwoma najbardziej znaczącymi bitami liczby określającej numer kanału. Ostatni bit tej liczby jest przesyłany jako najbardziej znaczący bit drugiego bajta wiadomości konfiguracyjnej układu MCP3008.

Kolejna linia kodu podaje niski sygnał na linię wybranego układu podrzędnego (SS). Sygnał ten aktywuje komunikację z modulem przetwornika.

```
digitalWrite(chipSelectPin, LOW);
```

Następnie wysyłany jest pierwszy bit komunikatu konfiguracyjnego:

```
SPI.transfer(configByteA);
byte readingH = SPI.transfer(configByteB);
byte readingL = SPI.transfer(0);
digitalWrite(chipSelectPin, HIGH);
```

Przetwornik nie będzie wysyłał danych do momentu wysłania drugiego bajta komunikatu konfiguracyjnego. Dane wysyłane przez przetwornik A/C (10 bitów) są dzielone na 2 bajty, a więc aby odebrać pozostałe dane, wywoływane jest polecenie „przesyłające” bajt wypełniony samymi zerami.

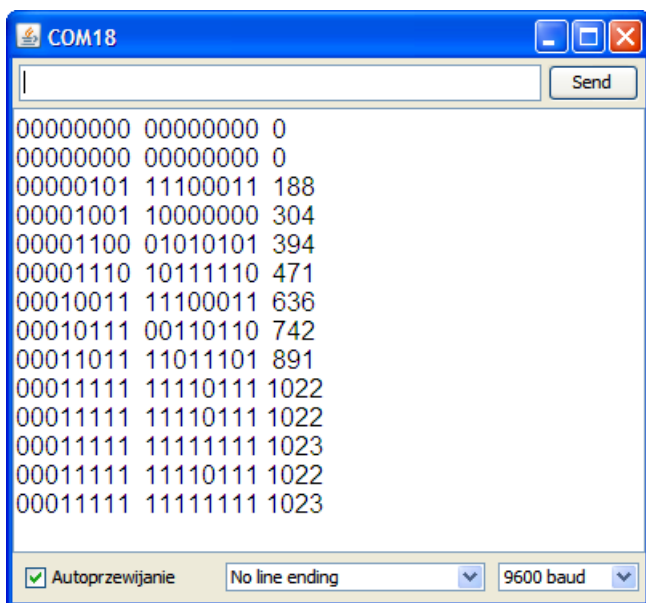
Na linię wyboru układu podrzędnego podawany jest teraz wysoki sygnał — zakończyliśmy komunikację z przetwornikiem.

Na koniec, za pomocą poniższego kodu, obliczana jest 10-bitowa wartość odczytana z analogowego wejścia przetwornika:

```
int reading = ((readingH & 0b00011111) << 5)
              + ((readingL & 0b11111000) >> 3);
```

W każdym z przesłanych 2 bajtów znajduje się po 5 bitów interesującego nas 10-bitowego ciągu danych. Interesujące nas dane znajdują się w pięciu najmniej znaczących bitach pierwszego bajta. Wszystkie bity (poza wspomnianymi pięcioma) są maskowane, a następnie przesuwane o pięć pozycji w górę w 16-bitowej zmiennej typu `int`. Pozostałe potrzebne nam bity znajdują się w pięciu najbardziej znaczących bitach drugiego bajta. Bity znajdujące się w tym bajcie muszą zostać odpowiednio zamaskowane, a następnie przesunięte o trzy pozycje. Dopiero tak przetworzoną wartość można dodać do 16-bitowej zmiennej `int`.

Aby sprawdzić działanie programu, otwórz okno monitora portu szeregowego. Powinny wyświetlać się w nim jakieś dane. Jeżeli obrócisz oś potencjometru zgodnie z ruchem wskazówek zegara, to w otwartym oknie powinny wyświetlić się dane podobne do tych, widocznych na rysunku 9.7. Pierwsze dwie liczby binarne są 2 bajtami przesłanymi przez układ MCP3008, a ostatnia liczba zapisana w systemie binarnym jest wartością odczytaną ze złącza (będzie ona zawierała się w zakresie od 0 do 1023).



Rysunek 9.7. Dane binarne wyświetlane w oknie monitora portu szeregowego

Podsumowanie

Praca z urządzeniami podłączonymi do interfejsu SPI wcale nie jest łatwa bez dedykowanej biblioteki. Czasami w takich sytuacjach zmuszenie urządzenia zewnętrznego do współpracy z Arduino może wymagać konfigurowania interfejsu metodą prób i błędów. Jak zwykle, podczas usuwania błędów w szkicu staraj się zbierać dane wysyłane przez urządzenie zewnętrzne, a następnie je analizować. Z czasem zorientujesz się, co jest nie tak, i poprawisz kod programu w taki sposób, aby działał prawidłowo.

Kolejny rozdział dotyczy tematyki związanej z ostatnim interfejsem obsługiwany przez Arduino — komunikacją szeregową za pomocą układu UART. Interfejs ten nie jest magistralą. Za jego pomocą można połączyć ze sobą tylko dwa urządzenia. Jest to jednakże wygodny, często stosowany mechanizm przesyłania danych.

Rozdział 10.

Szeregowa transmisja danych za pośrednictwem układu UART

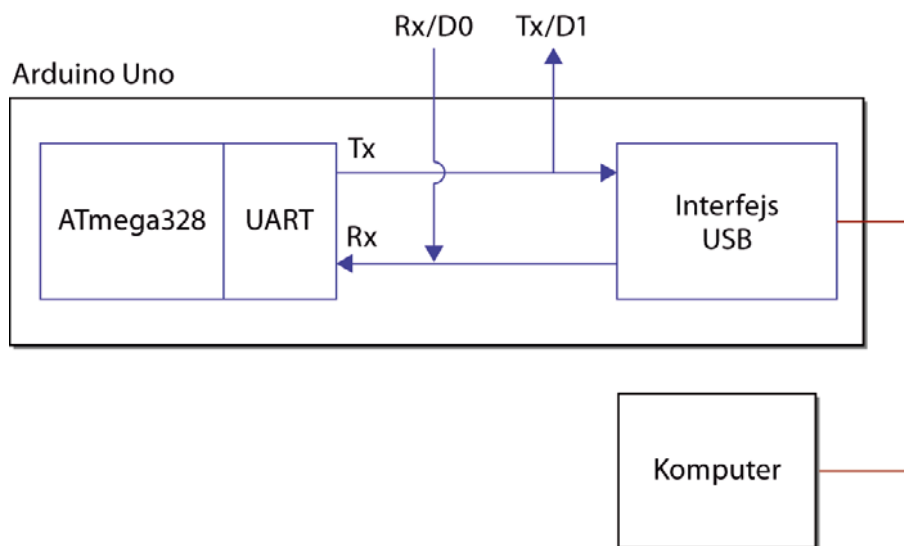
Powinieneś już posiadać pewien zasób wiadomości dotyczących interfejsu szeregowego. Korzystasz z niego, programując płytkę Arduino, za jego pomocą komunikujesz się z monitorem portu szeregowego (przesyłasz dane pomiędzy Arduino a komputerem). Komunikacja ta może być realizowana poprzez kontroler USB albo bezpośrednio poprzez interfejs szeregowy. Bezpośrednią transmisję danych można określić mianem **szeregowej transmisji danych za pośrednictwem układu TTL** lub po prostu **szeregowej transmisji danych**. Układ TTL to tranzystorowo-tranzystorowy układ logiczny. Jest to obecnie przestarzała technologia produkcji układów logicznych zasilanych prądem o napięciu 5 V.

Szeregowa transmisja danych wykonywana za pośrednictwem tego typu układów nie ma funkcjonalności magistrali — może przebiegać tylko pomiędzy dwoma urządzeniami. Zwykle jest to Arduino i urządzenie peryferyjne.

Urządzenia peryferyjne korzystające z szeregowej transmisji danych TTL, niewyposażone w magistralę I2C lub SPI, są zwykle większymi, zaprojektowanymi w przeszłości modułami. Do tej kategorii można również zaliczyć urządzenia, które miały z założenia komunikować się z komputerem poprzez port szeregowy. Są to na przykład moduły GPS, multimetry posiadające funkcję zapisu danych, czytniki kodów kreskowych i czujniki RFID.

Sprzęt służący do szeregowej transmisji danych

Na rysunku 10.1 zaprezentowano układy pozwalające na szeregową transmisję danych przez płytkę Arduino Uno.



Rysunek 10.1. Sprzęt pozwalający na szeregową transmisję danych przez płytke Arduino Uno

Procesor ATmega328, znajdujący się na płytce Arduino Uno, posiada złącza **Rx** (odbiór danych) i **Tx** (transmisja danych). Złącza te pełnią również funkcję cyfrowych portów wejścia-wyjścia (pinów D0 i D1). Jeżeli jednak podłączysz do nich zewnętrzne urządzenia elektroniczne, to nie będziesz w stanie zaprogramować Arduino.

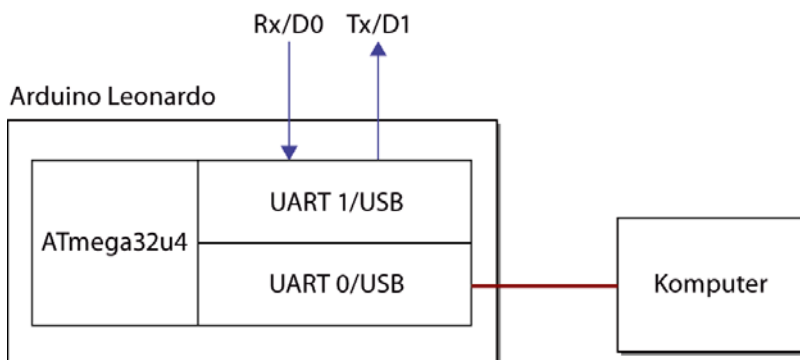
Piny Rx i Tx połączone są ze sprzętowym układem sterującym szeregową transmisją danych — układem UART (ang. *Universal Asynchronous Receiver and Transmitter*), który znajduje się wewnątrz chipa ATmega328. Ten element mikrokontrolera odpowiada za wysyłanie i odbieranie bajtów danych.

Na płytce Arduino Uno znajduje się dodatkowy układ scalony, będący konwerterem pomiędzy interfejsem szeregowym a interfejsem USB. Sygnał przesyłany przez interfejs USB charakteryzuje się innymi właściwościami elektrycznymi. Ponadto magistrala USB obsługiwana jest za pomocą o wiele bardziej skomplikowanego protokołu, a więc wspomniany układ wykonuje wiele czynności, które w efekcie pozwalają na komunikację pomiędzy portem szeregowym procesora ATmega328 a komputerem.

Na płytce Arduino Leonardo nie zamontowano oddzielnego układu scalonego umożliwiającego komunikację poprzez interfejs USB. W chipie ATmega, znajdującym się na tej płytce, zaimplementowano dwa moduły UART oraz interfejs USB (zobacz rysunek 10.2).

Jeden z układów UART obsługuje port USB, a drugi jest podłączony do pinów Rx i Tx (D0 i D1). Dzięki takiemu rozwiązaniu do wspomnianych pinów mogą zostać podłączone urządzenia peryferyjne, a płytka Arduino będzie w stanie równocześnie przysyłać dane do monitora portu szeregowego.

W tabeli 10.1 porównano liczbę portów szeregowych znajdujących się na poszczególnych płytkach Arduino. Zwróć uwagę na to, że płytka Due jest jedynym Arduino, którego port szeregowy pracuje pod napięciem 3,3 V, a nie 5 V.



Rysunek 10.2. Sprzęt pozwalający na szeregową transmisję danych przez płytkę Arduino Leonardo

Tabela 10.1. Szeregowe interfejsy UART znajdujące się na poszczególnych płytkach Arduino

Płytką	Liczba portów szeregowych	Szczegóły
Uno	1	Złącze Rx połączone jest z pinem D0, a Tx — z D1. Z portów tych korzysta również interfejs USB.
Leonardo	2	Jeden port służy do komunikacji z interfejsem USB. Złącza drugiego portu (Rx i Tx) połączono z pinami D0 i D1.
Mega 2560	4	Piny D0 i D1 służą do komunikacji za pośrednictwem interfejsu USB. Trzy pozostałe porty obsługiwane są przez następujące piny: port numer 1 przez piny 19 (Rx) i 18 (Tx), port numer 2 przez piny 17 (Rx) i 16 (Tx), a port numer 3 przez piny 15 (Rx) i 14 (Tx).
Due	4	Służący do obsługi interfejsu USB port szeregowy, któremu przypisano numer 0, korzysta z pinów D0 (Rx) i D1 (Tx). Port numer 1 korzysta z pinów numer 19 (Rx) i 18 (Tx), port numer 2 — z pinów numer 17 (Rx) i 16 (Tx), a port numer 3 — z pinów numer 15 (Rx) i 14 (Tx).

Przewody służące do szeregowej transmisji danych za pomocą układu TTL mogą mieć maksymalną długość od metra do kilku metrów (w zależności od urządzenia peryferyjnego). Krótsze przewody stosowane są w przypadku szybszej transmisji danych. Do komunikacji z urządzeniami znajdującymi się w większej odległości stworzono standard RS232. Jeszcze w poprzedniej dekadzie większość komputerów była wyposażona w porty szeregowe RS232. W tym standardzie przesyłano sygnał o wyższym napięciu niż w przypadku sygnału generowanego przez układ TTL. Dzięki temu urządzenia, między którymi przesyłano dane, mogły znajdować się w większej odległości od siebie.

Protokół obsługujący szeregową transmisję danych

Protokół obsługujący szeregową transmisję danych oraz większość związanej z nim terminologii wywodzą się jeszcze z początków rozwoju sieci komputerowych. Urządzenia nadające i odbierające dane muszą działać z tą samą szybkością. Szybkość transmisji danych wyraża się w **bodach**. Musi ona zostać zdefiniowana dla obu urządzeń jeszcze przed rozpoczęciem transmisji danych. Wartość ta określa liczbę zmian stanu sygnału na sekundę. Wspomniana wartość byłaby równa liczbie bitów danych przesyłanych na sekundę, ale w rzeczywistości między urządzeniami muszą być również przesyłane znaczniki sygnalizujące początek i koniec przesyłanego bajta danych, a także bity parzystości. Dzieliąc wartość wyrażoną w bodach przez 10, dowiesz się, ile bajtów na sekundę możesz w przybliżeniu przesłać.

Istnieją pewne standardowe prędkości przesyłu danych wyrażone w bodach. Być może zauważyłeś je na rozwijanej liście znajdującej się w oknie monitora portu szeregowego. Oprogramowanie Arduino obsługuje przesył danych z szybkością: 300, 1200, 4800, 9600, 14 400, 19 200, 28 800, 38 400, 57 600 i 115 200 bodów.

Arduino najczęściej przesyła dane z domyślną szybkością 9600 bodów. Nie wynika to z żadnych praktycznych powodów. Arduino może równie dobrze przysyłać dane z szybkością 115 200 bodów. Szybkość taka jest stosowana w praktyce w projektach, w których wymagana jest błyskawiczna transmisja danych. Urządzenia peryferyjne, takie jak moduły Bluetooth i GPS, często przesyłają dane z szybkością 2400 bodów.

Innym parametrem opisującym szeregową transmisję danych jest ciąg znaków takich jak 8N1. Ciąg ten może być zastosowany do inicjacji transmisji 8 bitów na pakiet, bez sprawdzania parzystości, z 1 bitem sygnalizującym koniec. Dane mogą być transmitowane również w innych trybach, ale większość urządzeń, z którymi będziesz miał do czynienia, będzie przysyłała dane w trybie 8N1.

Polecenia służące do obsługi szeregowej transmisji danych

Funkcje obsługujące szeregową transmisję danych nie są zdefiniowane w żadnej bibliotece, a więc nie musisz jej dołączać do szkicu za pomocą polecenia `include`.

Komunikację za pośrednictwem portu szeregowego rozpoczyna się poleceniem `Serial.begin`. Jako jego parametr należy podać szybkość transmisji danych wyrażoną w bodach:

```
Serial.begin(9600)
```

Tego typu wiersz znajduje się zwykle tylko w jednym miejscu szkicu, w funkcji `setup`.

Jeżeli korzystasz z płytki wyposażonej w więcej portów szeregowych niż jeden, ale chcesz używać domyślnego portu (portu numer 0), wystarczy zastosować polecenie `Serial.begin`. Jeżeli chcesz przysyłać dane przez jeden z pozostałych portów, to jego numer należy podać po słowie `Serial`. Na przykład w celu rozpoczęcia transmisji danych przez port numer 3, znajdujący się na płycie Arduino Due, należy skorzystać z następującego polecenia:

```
Serial3.begin(9600);
```

Po tym, gdy funkcja `Serial.begin` zostanie wywołana, układ UART będzie oczekiwać na przesył danych i w przypadku transmisji będzie przechowywał je w buforze, a więc nawet gdy procesor będzie zajęty wykonywaniem jakichś innych operacji, przesłane dane nie zostaną utracone, o ile nie dojdzie do przepełnienia bufora.

To, czy układ UART odebrał dane, można sprawdzić w funkcji `loop` za pomocą funkcji `Serial.available`. Zwraca ona liczbę bajtów gotowych do odczytu. Funkcja ta zwraca zero, gdy w buforze nie znajdują się żadne dane. W języku C zero symbolizuje „fałsz”. Dostępność danych jest często sprawdzana za pomocą następującego kodu:

```
void loop()
{
    if (Serial.available())
    {
        // odczytaj i przetwórz kolejny bajt
    }
}
```

Funkcja `read` nie przyjmuje żadnych argumentów — po prostu odczytuje kolejny dostępny bajt danych.

Funkcja `readBytes` wczytuje dostępne bajty do bufora szkicu (wcześniej były one ulokowane w buforze układu UART). Wspomniana funkcja przyjmuje dwa argumenty: bufor, w którym mają zostać zapisane dane (tu powinien znaleźć się odnośnik do tablicy elementów typu `byte`), oraz wartość określającą maksymalną liczbę odczytanych bajtów. Drugi z argumentów jest szczególnie przydatny podczas pracy nad projektem, w którym Arduino musi odbierać łańcuchy o zmiennej długości. Ogólnie lepiej unikać takiej sytuacji. Warto, aby komunikaty wysyłane do Arduino miały stałą długość i były jak najprostsze.

Funkcje `parseInt` i `parseFloat` mogą okazać się bardzo przydatne — pozwalają one na odczyt łańcuchów przesłanych do Arduino jako zmienne liczbowe typu `int` i `float`.

```
void loop()
{
    if (Serial.available())
    {
        int x = parseInt();
    }
}
```

Obie funkcje odczytują znaki do końca łańcucha lub do natrafienia na znak niebędący numerem. Odczytany ciąg znaków jest następnie przekształcany na wartość liczbową.

Zanim skorzystasz z funkcji takich jak `parseInt` i `parseFloat`, upewnij się, że rozumiesz powód ich zastosowania. Spotkałem się z kodem, w którym autor konwertował zmienną typu `int` na tablicę znaków wysyłaną następnie do drugiej płytki Arduino — płytka ta konwertowała wspomnianą tablicę z powrotem na zmienną typu `int`. Tego typu praktyka jest zła z wielu powodów:

- Jest to coś zbędnego. Port szeregowy pozwala na przesyłanie danych binarnych. Zmienną typu `int` wystarczy podzielić przed przesłaniem na 2 bajty, a następnie na podstawie odebranych 2 bajtów odtworzyć zmienną typu `int`.

- Konwersja pomiędzy liczbami i łańcuchami przebiega wolno niezależnie od kierunku operacji.
- Zamiast 2-bajtowej zmiennej typu `int` przez port szeregowy przesyłanych jest 6 znaków (w tym znak końca łańcucha).

Funkcje te są przydatne, gdy urządzenie, z którym współpracuje Arduino, zostało zaprojektowane tak, że nie można komunikować się z nim bezpośrednio, protokół komunikacyjny zapisuje liczby za pomocą łańcuchów lub pola danych mają zmienną długość. W pozostałych przypadkach, gdy masz pełną kontrolę nad protokołem, nie komplikuj sobie życia i unikaj zbędnej konwersji między typami danych, a także komunikatów o zmiennej długości lub różnych formatach.

Przykładowe szkice, przedstawione w sekcji „Przykłady szeregowej transmisji danych” znajdującej się w dalszej części niniejszego rozdziału, mogą posłużyć jako szablony do tworzenia kodu odpowiedzialnego za komunikację w Twoim szkicu.

Port szeregowy ma wiele funkcji. Najbardziej przydatne zostały opisane w tym rozdziale. Więcej informacji na temat tego medium służącego do przesyłu danych zamieszczono w dokumentacji znajdującej się na stronie <http://arduino.cc/en/Reference/Serial>.

Biblioteka SoftwareSerial

Czasami, zwłaszcza podczas pracy z płytką Arduino Uno, może brakować Ci kolejnych portów szeregowych. Biblioteka SoftwareSerial umożliwia prowadzenie szeregowej transmisji danych za pomocą dowolnej pary złączy, ale ma ona pewne ograniczenia:

- W danej chwili możesz odbierać dane przesyłane tylko do jednego portu obsługiwanego przez bibliotekę SoftwareSerial.
- Jeżeli w szkicu korzystasz z przerwań zegarowych lub wywoływanych przez zewnętrzne zdarzenia, to możesz mieć problem z prawidłowym działaniem wspomnianej biblioteki.

W bibliotece zdefiniowano funkcje bliźniacze do tych, obsługujących komunikację przez standardowy port szeregowy. Czasami są one nawet lepiej skonstruowane. Pozwalają na przykład na komunikację z urządzeniami generującymi odwrotne sygnały. Jednym z takich urządzeń jest dalmierz firmy MaxSonar. Dla każdego połączenia możesz utworzyć obiekt SoftwareSerial; praktyka ta jest prostsza od podawania numeru portu po słowie kluczu Serial w przypadku pracy ze standardowym portem szeregowym.

W tabeli 10.2 wymieniono piny płytek Uno i Leonardo, które mogą zostać użyte przez bibliotekę SoftwareSerial. Jeżeli pracujesz z większą płytką, wyposażoną w cztery standardowe porty szeregowy, to najprawdopodobniej nie będziesz musiał korzystać z opisywanej biblioteki. O ile przed numerem pinu nie znajduje się litera A, to numer ten odnosi się do złącza cyfrowego.

Tabela 10.2. Złącza obsługiwane przez bibliotekę *SoftwareSerial*

Płytką	Piny mogące pełnić funkcję szyny Tx	Piny mogące pełnić funkcję szyny Rx
Uno	Wszystkie poza 0 i 1	Wszystkie poza 0 i 1
Leonardo	Wszystkie poza 0 i 1	8, 9, 10, 11, 14 (MISO), 15 (SCK), 16 (MOSI)

Aby utworzyć połączenie za pomocą biblioteki *SoftwareSerial*, zadeklaruj obiekt *SoftwareSerial*, a jako jego parametry podaj piny Rx i Tx. Następnie użyj polecenia *begin*, a jako jego parametr określ szybkość transmisji danych wyrażoną w bodach:

```
#include <SoftwareSerial.h>

SoftwareSerial mySerial(10, 11); // Rx, Tx

void setup()
{
  mySerial.begin(9600);
  mySerial.println("Witaj!");
}
```

Pełną dokumentację biblioteki *SoftwareSerial* znajdziesz na stronie <http://arduino.cc/en/Reference/SoftwareSerial>.

Przykłady szeregowej transmisji danych

W niniejszej sekcji znajdziesz przykłady ilustrujące przesył danych za pomocą układu UART oraz biblioteki *SoftwareSerial*.

Komunikacja pomiędzy komputerem a Arduino za pośrednictwem interfejsu USB

W pierwszym przykładzie będziemy wysyłać polecenia do Arduino za pomocą monitora portu szeregowego. Arduino będzie przysyłać do komputera co sekundę wartość uzyskaną w wyniku pomiaru napięcia sygnału podawanego na złącze A0. Jednocześnie mikrokontroler będzie oczekiwał podania jednoznakowych poleceń: *z* (oznaczającego „zaczynaj”) i *p* (oznaczającego „przestań”). Polecenia te sterują przesyłem odczytanych danych. Na rysunku 10.3 pokazano okno monitora portu szeregowego podczas działania tego szkicu.

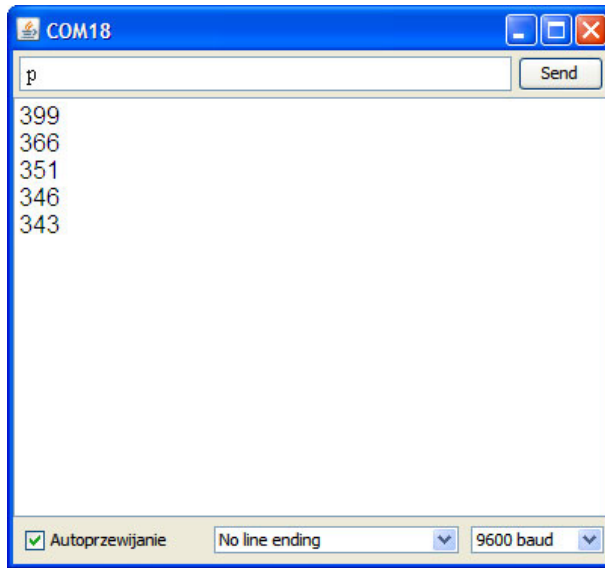
W tym przypadku wartości odczytane przez Arduino będą wyświetlane bezpośrednio w oknie monitora portu szeregowego, a więc mogą być przysyłane jako tekst, a nie dane binarne.

A oto omawiany szkic:

```
// 10_01_komputer_Arduino

const int readingPin = A0;

boolean sendReadings = true;
```



Rysunek 10.3. *Komunikacja pomiędzy Arduino a monitorem portu szeregowego*

```
void setup()
{
  Serial.begin(9600);
}

void loop()
{
  if (Serial.available())
  {
    char ch = Serial.read();
    if (ch == 'z')
    {
      sendReadings = true;
    }
    else if (ch == 'p')
    {
      sendReadings = false;
    }
  }
  if (sendReadings)
  {
    int reading = analogRead(readingPin);
    Serial.println(reading);
    delay(1000);
  }
}
```

Funkcja loop sprawdza, czy nie przesłano danych za pomocą interfejsu szeregowego. Jeżeli takowe dane zostaną napotkane, to zostanie wczytany bajt danych. Będzie on potraktowany jako znak. Znak ten jest następnie porównywany z poleceniami s i g. W zależności od przesłanego polecenia określany jest status zmiennej sendReadings.

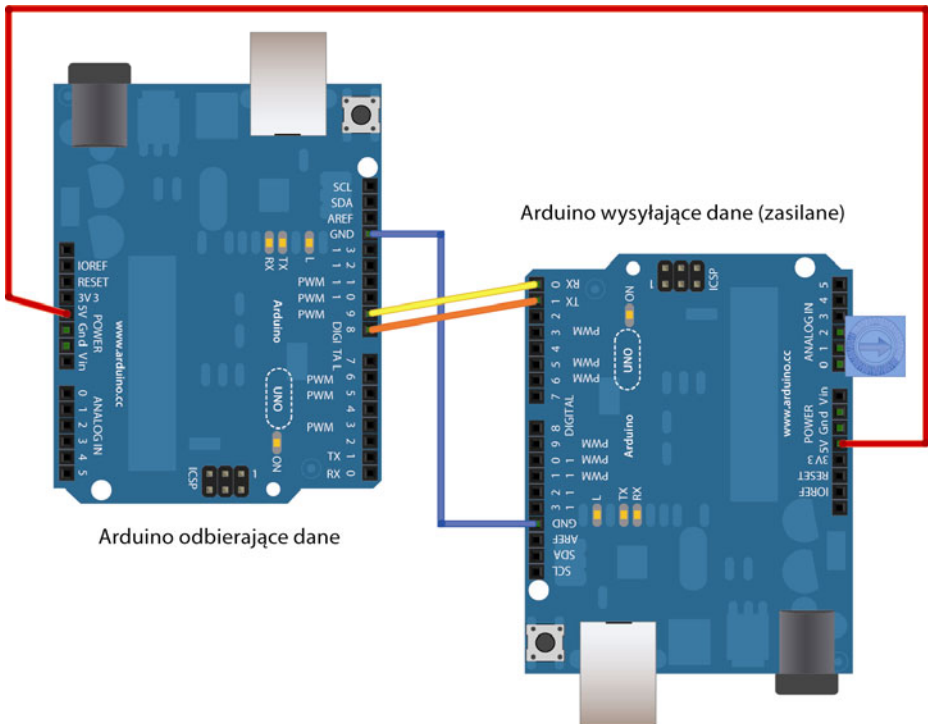
Zmienna `sendReadings` decyduje o tym, czy Arduino powinno odczytywać dane i wysyłać je do monitora portu szeregowego. Jeżeli zmiennej tej przypisany jest znacznik `true`, to mikrokontroler czeka sekundę przed wysłaniem kolejnej odczytanej wartości.

Użycie funkcji `delay` wiąże się z tym, że zmienna `sendReadings` może być zmodyfikowana dopiero w kolejnym obiegu pętli. Nie stanowi to problemu w tym szkicu, ale w innych okolicznościach blokowanie pętli może okazać się problemem. Tego typu problemy zostaną omówione w rozdziale 14.

Komunikacja pomiędzy dwoma płytkami Arduino

W drugim przykładzie zajmiemy się przesyłaniem danych pomiędzy dwoma płytkami Arduino za pośrednictwem połączenia szeregowego. Dane odczytywane przez złącze A1 jednej płytki będą przesyłane do drugiej płytki, która na podstawie przesłanej wartości będzie modyfikowała częstotliwość błysków wbudowanej diody LED, oznaczonej literą *L*.

Na rysunku 10.4 przedstawiono dwie płytki Arduino połączone ze sobą.



Rysunek 10.4. Płytki Arduino komunikujące się ze sobą za pośrednictwem połączenia szeregowego

Pin Tx jednej płytki powinien zostać połączony z pinem Rx drugiej płytki i na odwrót. W tym przykładzie obie płytki Arduino korzystają z biblioteki `SoftwareSerial`. W roli pinu Rx zastosowano pin numer 8, a w roli pinu Tx — pin numer 9.

Arduino wysyłające dane ma zasilac płytke odbierającą dane, a więc należy zewrzeć ze sobą piny masy (oznaczone etykietą *GND*) obu płytek. Połączyć należy również piny zasilania oznaczone etykietami 5V. Pomiędzy piny A0 i A2 Arduino wysyłającego dane należy podłączyć potencjometr dostrojczy (mały rezystor nastawny). Złącza A0 i A2 należy skonfigurować tak, aby działały jako wyjścia. Na złączu A2 należy następnie podać wysoki sygnał. W ten sposób za pomocą potencjometru możesz zmieniać napięcie podawane na pin A1 w zakresie od 0 V do 5 V. To z kolei będzie miało wpływ na częstotliwość błysków diody LED znajdującej się na drugiej płytce Arduino.

Arduino wysyłające dane należy zaprogramować za pomocą następującego szkicu:

```
// 10_02_Arduino_wysylajace_dane

#include "SoftwareSerial.h"

const int readingPin = A1;
const int plusPin = A2;
const int gndPin = A0;

SoftwareSerial sender(8, 9); // Rx, Tx

void setup()
{
    pinMode(gndPin, OUTPUT);
    pinMode(plusPin, OUTPUT);
    digitalWrite(plusPin, HIGH);
    sender.begin(9600);
}

void loop()
{
    int reading = analogRead(readingPin);
    byte h = highByte(reading);
    byte l = lowByte(reading);
    sender.write(h);
    sender.write(l);
    delay(1000);
}
```

Odczytana wartość, mająca formę 16-bitowej zmiennej typu *int*, jest dzielona na 2 bajty (wysoki i niski). Każdy z tych bajtów jest następnie przesyłany poprzez interfejs szeregowy za pomocą polecenia *write*. Funkcje *print* i *println* przeprowadzają konwersję przekazanych do nich elementów na łańcuchy znaków. Polecenie *write* wysyła bajt danych w formie binarnej.

Arduino odbierające dane należy zaprogramować za pomocą następującego szkicu:

```
// 10_03_Arduino_odbierajace_dane

#include "SoftwareSerial.h"

const int ledPin = 13;
int reading = 0;
SoftwareSerial receiver(8, 9); // Rx, Tx
```

```

void setup()
{
    pinMode(ledPin, OUTPUT);
    receiver.begin(9600);
}

void loop()
{
    if (receiver.available() > 1)
    {
        byte h = receiver.read();
        byte l = receiver.read();
        reading = (h << 8) + l;
    }
    flash(reading);
}

void flash(int rate)
{
    // 0 wolno, 1023 bardzo szybko
    int period = (50 + (1023 - rate) / 4);
    digitalWrite(ledPin, HIGH);
    delay(period);
    digitalWrite(ledPin, LOW);
    delay(period);
}

```

Program odbierający dane musi poczekać na przesłanie przynajmniej 2 bajtów danych. Następnie zmienna typu `int` jest budowana w wyniku przesunięcia wysokiego bajta na miejsca 8 najbardziej znaczących bitów tworzonej zmiennej. W dalszej kolejności dodawany jest do niej niski bajt.

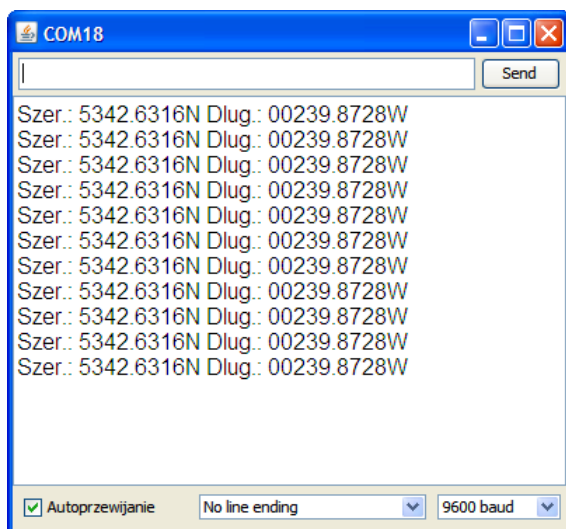
Jeżeli chcesz przysyłać pomiędzy płytkami Arduino jeszcze bardziej złożone struktury danych, to warto, abyś rozważył zastosowanie biblioteki *EasyTransfer*: <http://www.billporter.info/2011/05/30/easytransfer-arduino-library/>.

W zaprezentowanym przykładzie piny Rx i Tx, znajdujące się na dwóch płytkach Arduino, zostały połączone za pomocą przewodów. Równie łatwo można by połączyć je bezprzewodowo. Istnieje wiele bezprzewodowych modułów, które nie wpływają na pracę Arduino — szkice działają tak, jakby płytki były połączone za pomocą przewodów.

Moduł GPS

W ostatnim przykładzie zajmiemy się odbiorem danych przesyłanych szeregowo (za pomocą układu TTL), generowanych przez moduł nawigacji satelitarnej GPS. Arduino będzie formatowało dane określające długość i szerokość geograficzną oraz przysyłało je do monitora portu szeregowego (zobacz rysunek 10.5).

Komunikacja z modułem GPS przebiega tylko w jednym kierunku, a więc wystarczy, że połączysz pin Tx modułu z pinem Rx Arduino. Korzystam z modułu GPS Venus firmy SparkFun (<https://www.sparkfun.com/products/11058>). Jest on wyposażony, podobnie jak większość modułów GPS, w układ TTL odpowiedzialny za szeregową transmisję danych. Moduł wysyła co sekundę serię komunikatów z szybkością 9600 bodów.



Rysunek 10.5. Dane z modułu GPS odbierane przez Arduino

Wysyłane komunikaty są zgodne z protokołem NMEA, opublikowanym przez National Marine Electronics Association. Każdy komunikat jest łańcuchem znaków zakończonym znakiem nowego wiersza. Pola komunikatu są oddzielone od siebie przecinkami. Przykładowy komunikat tego typu wygląda tak:

```
$GPRMC,081019.548,A,5342.6316,N,00239.8728,W,000.0,079.7,110613,,A*76
```

W zaprezentowanym komunikacie znajdują się następujące pola:

- \$GPRMC — typ zdania,
- 081019.548 — bardzo dokładny czas określony w formacie 24-godzinny (8:10:19.548),
- 5342.6316,N — szerokość geograficzna $\times 100$ (53,426316 stopnia szerokości północnej),
- 00239.8728,W — długość geograficzna $\times 100$ (0,2398728 stopnia długości wschodniej),
- 000.0 — prędkość,
- 079.7 — kurs (79,7 stopnia),
- 110613 — data 11 czerwca 2013 r.

Pozostałe przesłane dane nie mają znaczenia w analizowanym przykładzie.

UWAGA

Na stronie <http://aprs.gids.nl/nmea/> znajdziesz opis wszystkich elementów komunikatów generowanych przez odbiorniki GPS zgodne ze standardem NMEA.

To jest szkic obsługujący moduł GPS:

```
// 10_04_GPS

#include <SoftwareSerial.h>

SoftwareSerial gpsSerial(10, 11); // Rx, Tx (złącze Tx nie jest używane)
const int sentenceSize = 80;

char sentence[sentenceSize];

void setup()
{
    Serial.begin(9600);
    gpsSerial.begin(9600);
}

void loop()
{
    static int i = 0;
    if (gpsSerial.available())
    {
        char ch = gpsSerial.read();
        if (ch != '\n' && i < sentenceSize)
        {
            sentence[i] = ch;
            i++;
        }
        else
        {
            sentence[i] = '\0';
            i = 0;
            //Serial.println(sentence);
            displayGPS();
        }
    }
}

void displayGPS()
{
    char field[20];
    getField(field, 0);
    if (strcmp(field, "$GPRMC") == 0)
    {
        Serial.print("Szer.: ");
        getField(field, 3); // liczba
        Serial.print(field);
        getField(field, 4); // północ-południe (N/S)
        Serial.print(field);

        Serial.print(" Dług.: ");
        getField(field, 5); // liczba
        Serial.print(field);
        getField(field, 6); // wschód-zachód (E/W)
        Serial.println(field);
    }
}
```

```

}

void getField(char* buffer, int index)
{
    int sentencePos = 0;
    int fieldPos = 0;
    int commaCount = 0;
    while (sentencePos < sentenceSize)
    {
        if (sentence[sentencePos] == ',')
        {
            commaCount ++;
            sentencePos ++;
        }
        if (commaCount == index)
        {
            buffer[fieldPos] = sentence[sentencePos];
            fieldPos ++;
        }
        sentencePos ++;
    }
    buffer[fieldPos] = '\0';
}

```

Zdania generowane przez moduł GPS są różnej długości, zawsze jednak są krótsze niż 80 znaków. W związku z tym szkic korzysta ze zmiennej pełniącej funkcję bufora — `sentence`. Jest ona wypełniana danymi aż do wczytania znaku nowego wiersza lub wypełnienia bufora.

Po wczytaniu całego zdania umieszczany jest na końcu bufora znak `\0`. Umożliwia to użytkownikowi wyświetlenie niesformatowanych danych.

Pozostała część kodu służy do wyciągnięcia ze zdania poszczególnych pól i sformatowania ich tak, aby mogły zostać wyświetlone w oknie monitora portu szeregowego. Funkcja `getField` wydobywa tekst z pola o określonym indeksie.

Funkcja `displayGPS` ignoruje wszystkie zdania, które są typu innego niż `"$GPRMC"`, a następnie z odebranego ciągu znaków wyciąga dane dotyczące długości i szerokości geograficznej, które zostaną później wyświetlone na ekranie.

Podsumowanie

W tym rozdziale omówiłem kilka technik związanych z obsługą szeregowej transmisji danych między dwoma płytkami Arduino oraz między Arduino a urządzeniami peryferyjnymi i komputerem.

W kolejnym rozdziale omówię dość ciekawą funkcję płytki Arduino Leonardo. Może ona emulować urządzenia peryferyjne wyposażone w interfejs USB, takie jak klawiatura i mysz. Poruszę również inne zagadnienia związane z korzystaniem z portu USB.

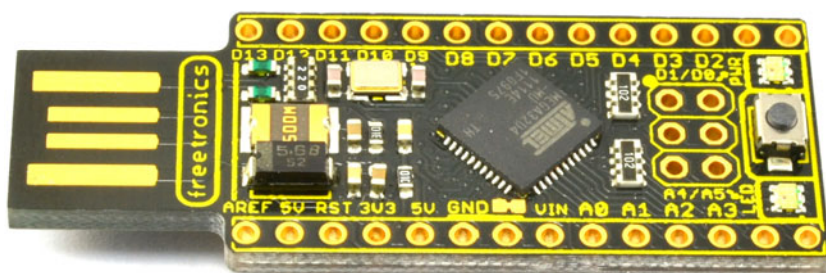
Rozdział 11.

Obsługa interfejsu USB

W tym rozdziale poruszę różne zagadnienia związane z obsługą interfejsu USB przez płytkę Arduino, między innymi emulację klawiatury i myszy przez łytkę Arduino Leonardo. Omówię również odwrotny proces — podłączanie klawiatury i myszy do Arduino.

Emulacja klawiatury i myszy

Oprócz płytki Leonardo również płytki Due i Micro (ich konstrukcja bazuje na płytce Leonardo) mogą emulować klawiaturę lub mysz USB. Taką możliwość oferują również inne płytki kompatybilne z Arduino, takie jak LeoStick produkowana przez firmę Freetronics (zobacz rysunek 11.1).

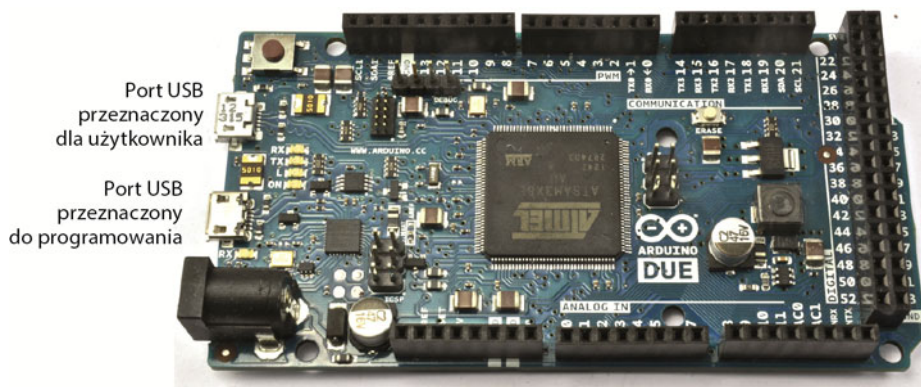


Rysunek 11.1. Płytko LeoStick

Wspomniana funkcja przydaje się zwłaszcza podczas pracy z syntezatorami. Pozwala ona płytce Arduino sterować programami takimi jak Ableton Live. Możesz na przykład stworzyć bazujący na Arduino nowatorski instrument muzyczny, którego pracą sterują przyspieszeniomierze, światłomierze lub pedały znajdujące się w pedalboardzie.

Wspomnianą funkcję Arduino można zastosować nieco mniej ambitnie — w celu robienia żartów. Arduino może sprawić, że mysz czyjegoś komputera zacznie żyć własnym życiem, a klawiatura wypisywać losowo wybrane litery.

Płytką Arduino Due jest wyposażona w dwa porty USB. Emulacja klawiatury i myszy może się odbywać za pomocą **portu USB przeznaczonego dla użytkownika**. Płytką ta jest zwykle programowana za pomocą **portu USB przeznaczonego do programowania** (zobacz rysunek 11.2).



Rysunek 11.2. Porty USB znajdujące się na płytce Arduino Due

Emulacja klawiatury

Funkcje emulujące klawiaturę są dość łatwe w użyciu. To standardowy element języka, z którego korzystamy, a więc nie ma potrzeby dołączania zewnętrznej biblioteki. Aby rozpocząć emulację klawiatury, umieść następujące polecenie w funkcji startup:

```
Keyboard.begin();
```

Tekst „wpisywany” (w dowolnym miejscu, w którym zostanie ustawiony kursor) przez Arduino należy podać jako argument funkcji `print` lub `println`:

```
Keyboard.println("Była to najlepsza z epok.");
```

Jeżeli chcesz korzystać z klawiszy modyfikujących, na przykład podczas emulacji naciśnięcia klawiszy tworzących kombinację `Ctrl+C`, to zastosuj polecenie `press`:

```
Keyboard.press(KEY_LEFT_CTRL);
Keyboard.press('x');
delay(100);
Keyboard.releaseAll();
```

Funkcja `press` przyjmuje w roli argumentu pojedynczy element typu `char`. Poza normalnymi znakami możesz przekazywać jej wiele zdefiniowanych stałych, takich jak `KEY_LEFT_CTRL`. Po tym, gdy uruchomione jest polecenie `press`, Arduino działa tak, jakby zdefiniowany klawisz był wciśnięty — do momentu uruchomienia polecenia `releaseAll`. Pełną listę klawiszy specjalnych znajdziesz pod adresem <http://arduino.cc/en/Reference/KeyboardModifiers>.

UWAGA

Zaprogramowanie Arduino, podczas gdy mikrokontroler emuluje mysz lub klawiaturę, może okazać się niemożliwe, ponieważ podczas próby zaprogramowania układu może on „pisać” jakiś tekst. Problem ten można rozwiązać, naciskając i przytrzymując przycisk Reset aż do wyświetlenia się komunikatu „ładowanie” w pasku statusu środowiska programistycznego Arduino.

Przykład emulacji klawiatury

Poniższy szkic automatycznie (po każdym uruchomieniu Arduino) „wpisuje” tekst określony przez użytkownika (może być to na przykład hasło):

```
// 11_01_klawiatura

char phrase[] = "hasło";

void setup()
{
  Keyboard.begin();
  delay(5000);
  Keyboard.println(phrase);
}

void loop()
{
}
```

Szkic ten byłby lepszy, gdyby proces „pisania” był uruchamiany przez wciśnięcie zewnętrznego przycisku. System operacyjny na komputerze typu Mac wyświetla okno informujące o podłączeniu nowej klawiatury po każdym uruchomieniu Arduino. Tekst nie zostanie „wpisany” przez mikrokontroler dopóty, dopóki okno to nie zostanie zamknięte.

Emulacja myszy

Proces emulacji myszy działa podobnie jak emulacji klawiatury. Tak naprawdę możesz emulować oba urządzenia za pomocą tego samego szkicu.

Emulację myszy należy rozpocząć, stosując polecenie:

```
Mouse.begin();
```

Następnie możesz wykonywać ruchy kursorem za pomocą funkcji `Mouse.move`. Przyjmuje ona trzy parametry: przemieszczanie kursora w płaszczyźnie poziomej i pionowej, a także dane generowane przez pokrętkę *scroll*. Wszystkie te trzy wartości są wyrażone w pikselach. Mogą one przyjmować wartość dodatnią (opisując ruch do góry lub w prawo) albo ujemną (opisując ruch w dół lub w lewo). Cursor myszy przemieszcza się o określoną liczbę pikseli względem swojego początkowego położenia. Emulujemy działanie myszy poruszającej kursorem — nie możemy ustawić kursora w dowolnej pozycji bezwzględnej.

Polecenie `click` emuluje kliknięcie przycisku myszy. Jeżeli nie prześlemy tej funkcji żadnych parametrów, to zostanie wykonane kliknięcie lewym przyciskiem myszy. Za pomocą

dotychczasowych parametrów możemy wygenerować kliknięcie prawym przyciskiem myszy (MOUSE_RIGHT) lub jej środkowym przyciskiem (MOUSE_MIDDLE).

Jeżeli chcesz określić czas naciśnięcia przycisku myszy, to możesz to zrobić za pomocą poleceń `Mouse.press` i `Mouse.release`. Polecenie `Mouse.press` może przyjmować te same parametry, co `Mouse.click`. Polecenia te mogą okazać się przydatne w przypadku tworzenia własnej myszy opartej na Arduino. Wtedy kliknięcie myszy może być wywoływane przez przełącznik podłączony do cyfrowego wejścia mikrokontrolera. W ten sposób z łatwością można wygenerować podwójne lub potrójne kliknięcia.

Przykład emulacji myszy

W poniższym przykładowym szkicu kursor znajdujący się na ekranie komputera będzie poruszany w sposób losowy. Aby zatrzymać działanie szkicu i odzyskać kontrolę nad komputerem, musisz wcisnąć i przytrzymać przycisk *Reset* albo odłączyć Arduino od komputera.

```
// 11_02_mysz

void setup()
{
  Mouse.begin();
}

void loop()
{
  int x = random(61) - 30;
  int y = random(61) - 30;
  Mouse.move(x, y);
  delay(50);
}
```

Programowanie hosta USB

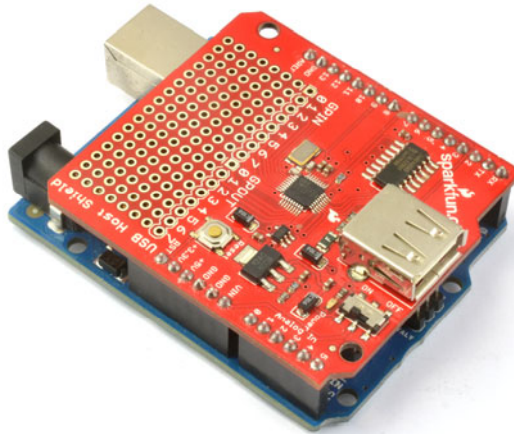
Płytki Leonardo, Due i Micro mogą emulować działanie klawiatury lub myszy. Klawiaturę i mysz można jednak podłączyć tylko do gniazda USB płytki Due lub mniej znanej Arduino Mega ADK. Tylko te płytki pozwolą Ci korzystać z klawiatury i myszy jako urządzeń wejścia. Funkcja ta nosi nazwę **USB Host**. Jest ona obsługiwana bezpośrednio tylko przez płytkę Due, ale istnieją również dodatkowe płytki (tzw. *shields*), rozszerzające o tę funkcję możliwości Arduino Uno lub Leonardo.

Wspomniana funkcja może obsługiwać również klawiatury i myszy bezprzewodowe (o ile nie są to urządzenia korzystające z interfejsu Bluetooth). Wystarczy, że ich klucz sprzętowy zostanie włożony do gniazda USB znajdującego się na płycie pełniącej funkcję hosta USB. W ten sposób możesz dodać do Arduino funkcję zdalnego sterowania.

Host USB może również obsługiwać inne, wyposażone w złącza USB, urządzenia peryferyjne, takie jak kontrolery gier, kamery i kontrolery Bluetooth. Możesz również podłączyć do niego telefon pracujący pod kontrolą systemu Android.

Płytki USB Host i obsługująca ją biblioteka

Płytki USB Host i obsługujące je biblioteki są dostępne od kilku lat. Obecnie mogą obsługiwać wiele urządzeń peryferyjnych. Pierwsza tego typu płytka została opracowana przez firmę Circuits@Home (<http://www.circuitsathome.com/>). W ofercie firm SparkFun, SainSmart i innych znajdują się moduły kompatybilne ze wspomnianą płytką. Na rysunku 11.3 przedstawiono host USB firmy SparkFun, podłączony do płytki Arduino Uno. W chwili, gdy pisać tę książkę, tego typu płytki rozszerzeń nie są kompatybilne z Arduino Leonardo, a tak naprawdę w zasadzie nie są kompatybilne z żadnymi płytkami Arduino, będącymi bardziej „egzotycznymi” od Uno. Sprawdź to przed zakupem.



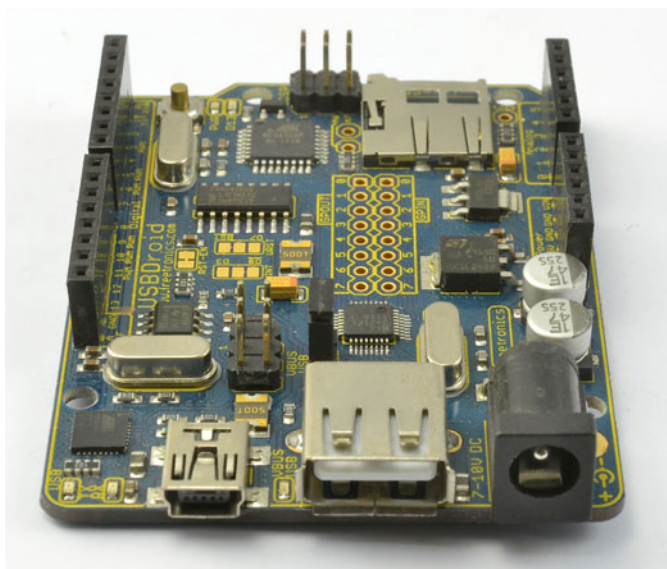
Rysunek 11.3. Płytki USB Host firmy SparkFun

Płytki widoczna na rysunku została ponadto wyposażona w przydatną przestrzeń pozwalającą na dolutowanie dodatkowych komponentów. Alternatywnym rozwiązaniem w zakresie korzystania z płytki rozszerzeń jest zakup urządzenia takiego jak USBDroid firmy Fretronics (zobacz rysunek 11.4). Płytki ta została wyposażona w port mikro-USB służący do programowania płytki, a także w pełnowymiarowe złącze USB, do którego możesz podłączyć klawiaturę lub inne urządzenie peryferyjne.

Do pracy z płytką USBDroid lub nieoficjalną płytką rozszerzeń mogącą pełnić funkcję hosta USB niezbędna jest biblioteka `USB_Host_Shield` firmy Circuits@Home. Jeżeli posiadasz oficjalną płytkę, to możesz korzystać z biblioteki `USB_Host_Shield_2` obsługującej więcej urządzeń.

Programowanie za pomocą wspomnianych bibliotek nie jest czymś łatwym. Biblioteka zapewnia dość niskopoziomowy interfejs obsługujący magistralę USB. W pliku `11_03_host_klawiatura.ino`, który możesz pobrać ze strony <http://www.helion.pl/ksiazki/arpokk.htm>, znajdziesz przykładowy szkic obsługujący klawiaturę podłączoną do hosta USB.

Szkic ten powstał na skutek modyfikacji jednego z przykładowych programów dołączonych do biblioteki `USB_Host_Shield`. Szkic wyświetla wpisywane znaki nie w oknie monitora portu szeregowego (jak oryginalny program), a na wyświetlaczu LCD.



Rysunek 11.4. Płytki USBDroid firmy Freetronics

Zarówno ten szkic, jak i oryginalny przykład mogą posłużyć Ci jako szablon pomocne przy tworzeniu własnego kodu. Oba programy prawidłowo obsługują wszystkie klawisze znajdujące się na klawiaturze. Jeżeli chcesz korzystać tylko z klawiszy kierunkowych albo tylko z klawiatury numerycznej, to możesz znacznie uprościć kod szkicu.

Szkic jest zbyt długi, aby umieszczać go tu w całości, a więc skupię się na jego kluczowych elementach. Przed przystąpieniem do lektury warto załadować szkic do pamięci Arduino.

Importowane są trzy biblioteki:

- `#include <Spi.h>`
- `#include <Max3421e.h>`
- `#include <Usb.h>`

Biblioteka *Spi.h* jest wymagana, ponieważ obsługuje interfejs chipa kontrolera hosta USB. Pracujemy z układem Max3421e, a więc potrzebujemy również obsługującej go biblioteki. Ostatnią importowaną biblioteką jest *Usb.h*, która zawiera funkcje uwalniające użytkownika od konieczności kłopotliwego i złożonego korzystania bezpośrednio z chipa.

Pod poleceniami importującymi biblioteki znajdują się definicje stałych, takich jak:

```
#define BANG          (0x1E)
```

W języku C stałe mogą być definiowane na różne sposoby, równie dobrze moglibyśmy zastosować składnię typu:

```
const int BANG = 0x1E;
```

Następnie tworzone są obiekty MAX3421E i USB, a w funkcji setup wywoływana jest funkcja `powerOn` obiektu `Max`:

```
MAX3421E Max;
USB Usb;
```

W funkcji `loop` wywoływane są funkcje `Task` obiektów `Max` i `Usb`. W ten sposób uruchomiony zostaje interfejs sprawdzający aktywność portu USB.

```
void loop() {
    Max.Task();
    Usb.Task();
    if( Usb.getUsbTaskState() == USB_STATE_CONFIGURING ) { //czeka na stan adresujący
        kbd_init();
        Usb.setUsbTaskState( USB_STATE_RUNNING );
    }
    if( Usb.getUsbTaskState() == USB_STATE_RUNNING ) {
//skanuje klawiaturę
        kbd_poll();
    }
}
```

Po pierwszym uruchomieniu interfejs będzie w trybie `USB_STATE_CONFIGURING`. Połączenie z klawiaturą jest nawiązywane dopiero po wywołaniu funkcji `kbd_init`. Funkcja ta korzysta ze struktury punktu udostępniania danych `ep_record`. W tej strukturze danych umieszczone są fragmenty komunikatu nawiązującego połączenie z klawiaturą:

```
ep_record[ 0 ] = *( Usb.getDevTableEntry( 0,0 ) );
ep_record[ 1 ].MaxPktSize = EP_MAXPKTSIZE;
ep_record[ 1 ].Interval = EP_POLL;
ep_record[ 1 ].sndToggle = bmSNDTOGO;
ep_record[ 1 ].rcvToggle = bmRCVTOGO;
Usb.setDevTableEntry( 1, ep_record );
/* Konfiguracja urządzenia */
rcode = Usb.setConf( KBD_ADDR, 0, 1 );
```

Po tym, gdy inicjalizacja zostanie zakończona, klawiatura będzie działała w głównej pętli w trybie `USB_STATE_RUNNING`. Wtedy wywołana zostanie funkcja `kbd_poll`, która sprawdza, czy na klawiaturze nie wciśnięto przycisku.

Kluczową linią funkcji `kbd_poll` jest:

```
rcode = Usb.inTransfer( KBD_ADDR, KBD_EP, 8, buf );
```

Funkcja ta sprawdza, czy przycisk na klawiaturze nie został wciśnięty podczas wczytywania kodów odbieranych przez interfejs USB. Kody te nie są kodami ASCII. Dopiero funkcja `HidtoA` konwertuje je na kod ASCII. Jest to najbardziej złożona funkcja w szkicu, ale możesz ją z łatwością zaimplementować we własnym szkicu. Więcej informacji na temat kodów generowanych przez klawiaturę USB i mapowania ich do ASCII znajdziesz na stronie <http://www.win.tue.nl/~aeb/linux/kbd/scancodes-1.html>.

Jedną z ciekawszych funkcji protokołu USB HID (ang. *Human Interface Device*) jest umożliwienie sterowania pracą diod LED informujących użytkownika o trybie pracy klawiszy *Scroll Lock* i *Num Lock*. Pracą tych diod steruje funkcja `kbd_poll` odpowiadająca na wciśnięcie klawiszy *Scroll Lock*, *Caps Lock* i *Num Lock*. Możemy również stworzyć oddzielny szkic sterujący pracą wspomnianych diod. Przykład takiego programu znajdziesz w pliku `11_04_host_scroll_lock.ino`.

Główną funkcją tego szkicu jest:

```
void toggleLEDs( void )
{
    if (leds == 0) {
        leds = 0b00000111;
    }
    else {
        leds = 0;
    }
    Usb.setReport( KBD_ADDR, 0, 1, KBD_IF, 0x02, 0, &leds );
}
```

Trzy najmniej znaczące bity znaku są znacznikami określającymi tryb pracy trzech diod LED znajdujących się na klawiaturze.

Host USB płytki Arduino Due

Płytki Arduino Due może pracować tak, jakby wbudowano w nią host USB. W chwili pisanie tej książki funkcja ta jest uznawana przez konstruktorów Arduino za coś eksperymentalnego. Na stronie <http://arduino.cc/en/Reference/USBHost> znajdziesz aktualne informacje dotyczące działania tej technologii i ewentualnych dalszych prac nad nią.

Płytki Due nie ma pełnowymiarowego gniazda USB, a więc nie można do niej podłączyć bezpośrednio klawiatury i myszy. W celu podłączenia urządzeń peryferyjnych do tej płytki musisz kupić kabel Micro USB OTG Host. Przykład takiego przewodu pokazano na rysunku 11.5, na którym widać kontroler bezprzewodowej klawiatury podłączony do Arduino Due. Równie dobrze można korzystać w ten sposób z klawiatury przewodowej.



Rysunek 11.5. Klawiatura podłączona do Arduino Due za pomocą kabla Micro USB OTG Host

Biblioteki płytki Arduino Due są prostsze w użyciu od biblioteki USB Host — zamiast kodu interfejsu HID zwracany jest kod ASCII, przypisany klawiszowi wciśniętemu przez użytkownika. Prezentowany przykład ilustruje obsługę klawiatury przez płytkę Arduino Due. Szkic wyświetla w oknie monitora portu szeregowego informację o tym, który przycisk został wciśnięty.

```
// 11_05_klawiatura_due
#include <KeyboardController.h>
```



```

USBHost usb;
KeyboardController keyboard(usb);

void setup()
{
    Serial.begin(9600);
    Serial.println("Uruchomiono program");
    delay(200);
}

void loop()
{
    usb.Task();
}

// Ta funkcja przechwytuje wciśnięcia klawiszy
void keyPressed()
{
    char key = keyboard.getKey();
    Serial.write(key);
}

```

Każde wciśnięcie klawisza powoduje, że biblioteka `KeyboardController` wywołuje funkcję `keyPressed`. Funkcja `keyRelease` może być użyta do przechwytywania zwolnienia klawisza. Aby określić to, który z klawiszy został wciśnięty, na obiekcie `keyboard` należy wywołać jedną z poniższych funkcji:

- `getModifiers` — zwraca maskę bitów określającą, czy wciśnięto jakiś klawisz modyfikujący (*Shift*, *Ctrl* itd.); na stronie <http://arduino.cc/en/Reference/GetModifiers> znajdziesz przykłady użycia tej funkcji.
- `getKey` — zwraca kod ASCII określający to, który klawisz został wciśnięty.
- `get0emKey` — zwraca kod interfejsu HID określający to, który z klawiszy został wciśnięty.

Obsługa myszy jest równie łatwa, a przy tym podobna do obsługi klawiatury. Przykładowy szkic wyświetla w oknie monitora portu szeregowego litery *L*, *P*, *G* i *D* w zależności od tego, czy mysz została poruszona w lewo, w prawo, w górę czy w dół.

```

// 11_06_mysz_due

#include <MouseController.h>

USBHost usb;
MouseController mouse(usb);

void setup()
{
    Serial.begin(9600);
    Serial.println("Uruchomiono program");
    delay(200);
}

void loop()

```

```

{
    usb.Task();
}

// Ta funkcja przechwytytuje ruchy myszy
void mouseMoved()
{
    int x = mouse.getXChange();
    int y = mouse.getYChange();
    if (x > 50) Serial.print("P");
    if (x < -50) Serial.print("L");
    if (y > 50) Serial.print("D");
    if (y < -50) Serial.print("G");
}

```

Poza funkcją `mouseMoved` do przechwytywania innych zdarzeń generowanych przez mysz możesz stosować następujące funkcje:

- `mouseDragged` — do tego zdarzenia dochodzi, gdy użytkownik przesuwaa mysz, jednocześnie wciskając jej lewy przycisk.
- `mousePressed` — do tego zdarzenia dochodzi, gdy użytkownik wciśnie przycisk myszy. Powinna zostać po nim uruchomiona funkcja `mouse.getButton`, która w roli parametru przyjmuje określenie lewego, prawego lub środkowego przycisku myszy: `LEFT_BUTTON`, `RIGHT_BUTTON` lub `MIDDLE_BUTTON`. Funkcja zwraca wartość logiczną „prawda”, jeżeli dany przycisk jest wciśnięty.
- `mouseReleased` — funkcja ta, stanowiąca uzupełnienie funkcji `mousePressed`, służy do wykrywania zwolnienia przycisku myszy.

Podsumowanie

W tym rozdziale poznałeś zagadnienia związane z łączeniem Arduino z urządzeniami peryferyjnymi za pośrednictwem interfejsu USB.

W kolejnym rozdziale omówię tematykę dotyczącą obsługi przez Arduino sieci przewodowych i bezprzewodowych. Dowiesz się, jak można programować obsługę sieci, a także nauczysz się korzystać z płytek rozszerzeń będących kontrolerami sieci Ethernet oraz Wi-Fi.

Rozdział 12.

Obsługa sieci

Internet zaczął się rozwijać zgodnie z koncepcją **internetu przedmiotów** — to już coś więcej niż tylko zawartość wyświetlana w oknie przeglądarki i serwery sieci Web, to także różnorakie urządzenia korzystające z dostępu do sieci. Drukarki, automatyka domowa, a nawet lodówki stają się nie tylko inteligentne, ale również potrafią korzystać z dostępu do internetu. Arduino może posłużyć do samodzielnej budowy urządzenia korzystającego z internetu. W tym celu można zastosować kartę rozszerzeń obsługującą sieci Ethernet lub Wi-Fi. W tym rozdziale omówię techniki programistyczne związane z tworzeniem szkiców korzystających z połączenia sieciowego.

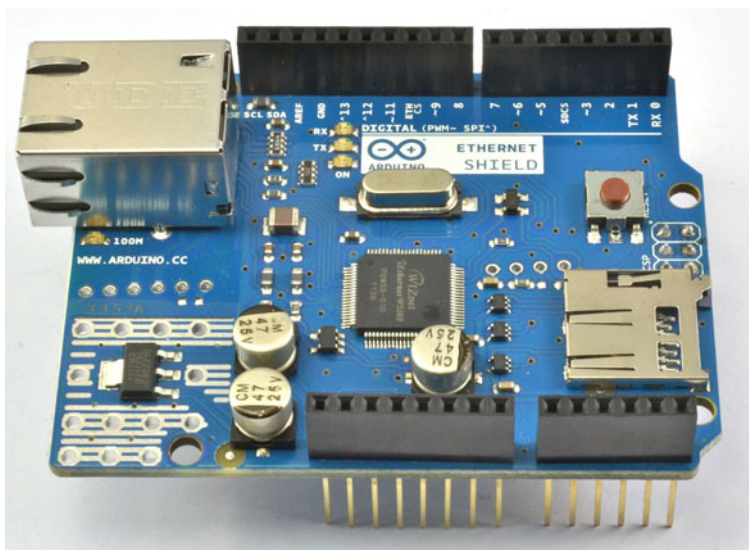
Sprzęt sieciowy

Arduino może zostać podłączone do sieci na wiele sposobów. Możesz w tym celu skorzystać z płytki rozszerzającej, pozwalającej na podłączenie Arduino Uno do sieci Ethernet. Alternatywę stanowi zakup płytki Arduino wyposażonej we wbudowany kontroler sieci Ethernet. Możesz również wydać więcej pieniędzy i kupić płytkę rozszerzeń mającą kontroler sieci Wi-Fi.

Płytki rozszerzeń wyposażona w kontroler sieci Ethernet

Na płytce wyposażonej w kontroler sieci Ethernet (zobacz rysunek 12.1) znajdziesz również gniazdo kart micro-SD. Pozwala ono na zapis danych na kartach pamięci (zajrzyj do umieszczonej w rozdziale 6. sekcji „Zapisywanie danych na kartach SD”).

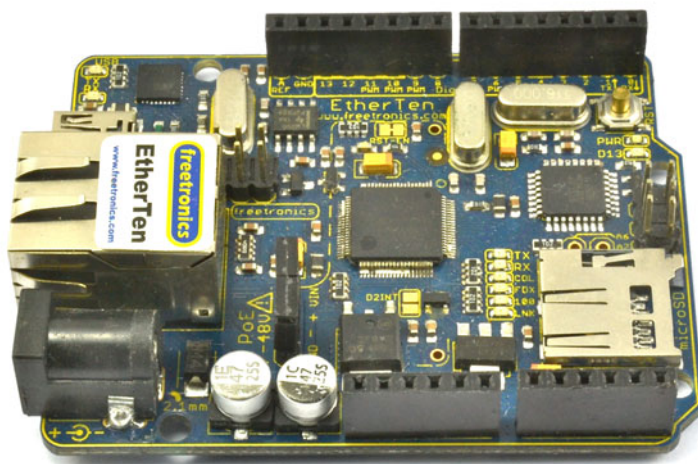
Oficjalne płytki zbudowane są na bazie układu W5100. Możesz jednakże znaleźć o wiele tańsze płytki, korzystające z chipsetu ENC28J60. Tańsze moduły nie są kompatybilne z biblioteką Ethernet. Jeżeli nie dysponujesz dużą ilością czasu i bardzo małym budżetem, to staraj się ich unikać



Rysunek 12.1. Płytkę rozszerzeń wyposażoną w kontroler sieci Ethernet

Arduino Ethernet i Arduino EtherTen

Zamiast korzystać ze specjalnej płytki rozszerzeń, możesz pokusić się o zakup Arduino wyposażonego w kontroler sieci Ethernet. Oficjalna płytkę tego typu to Arduino Ethernet, ale warto jest zainteresować się urządzeniem EtherTen produkowanym przez firmę Freetronics (<http://www.freetronics.com/>). EtherTen (pokazany na rysunku 12.2) jest kompatybilny z Arduino Uno.



Rysunek 12.2. Płytkę EtherTen

Korzystanie z takiego rozwiązania ma sens podczas pracy nad projektami, w których Arduino będzie podłączone do sieci. Płytką Arduino Ethernet może być podłączona do zewnętrznego modułu zasilającego, obsługującego technologię *Power over Ethernet* (PoE). Pozwoli to na zasilanie urządzenia poprzez kabel sieci Ethernet. W ten sposób liczba przewodów biegnących do Arduino może zostać zredukowana do jednego. Płytką EtherTen jest standardowo skonfigurowana tak, aby korzystać z technologii PoE. Więcej informacji na temat zasilania urządzenia EtherTen za pomocą wspomnianej technologii znajdziesz na stronie <http://www.doctormonk.com/2012/01/power-over-ethernet-poe.html>.

Arduino i Wi-Fi

Podłączając Arduino do internetu za pośrednictwem sieci Ethernet, musimy oczywiście korzystać z przewodu. Jeżeli nie chcesz korzystać z sieci przewodowej, to możesz połączyć się z internetem albo siecią bezprzewodową za pomocą specjalnej płytki rozszerzeń wyposażonej w kontroler sieci Wi-Fi (zobacz rysunek 12.3). Takie płytki są dość drogie, ale istnieją tańsze zamienniki innych producentów — np. płytką WiFly firmy SparkFun (<https://www.sparkfun.com/products/9954>).

Biblioteka Ethernet

Biblioteka Ethernet uległa wielu modyfikacjom od wypuszczenia na rynek Arduino 1.0 w 2011 r. Biblioteka ta pozwala płytce Arduino wyposażonej w kontroler sieci Ethernet na pracę w charakterze serwera lub klienta sieci Web (mikrokontroler może wysyłać żądania tak, jak przeglądarka internetowa). Ponadto biblioteka obsługuje protokół DHCP, dzięki czemu płytce Arduino można automatycznie przypisać adres IP.

UWAGA

Wiele przydatnych informacji znajdziesz w oficjalnej dokumentacji biblioteki Ethernet. Możesz ją pobrać ze strony <http://www.arduino.cc/en/reference/ethernet>.

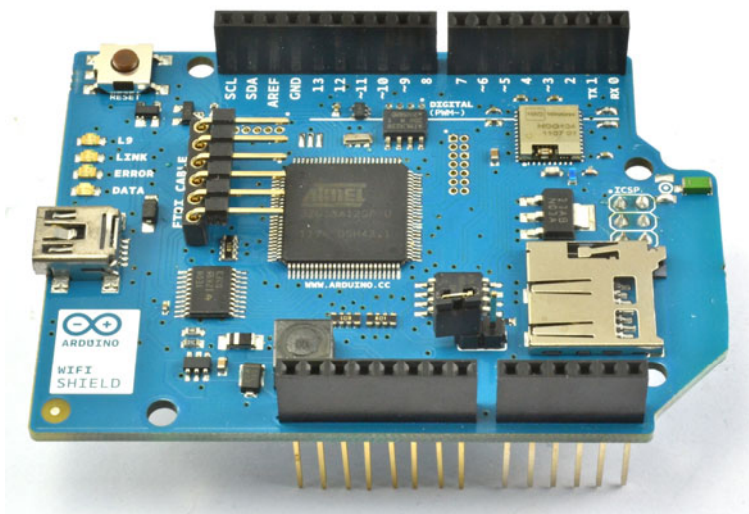
Nawiązywanie połączenia

Zanim jakiegokolwiek dane zostaną przesłane, Arduino musi nawiązać połączenie z siecią. Aby je nawiązać, należy wywołać funkcję `Ethernet.begin()`. Parametry połączenia mogą zostać określone za pomocą następującej składni:

```
Ethernet.begin(mac, ip, dns, brama, maska)
```

Przyjrzyjmy się kolejnym parametrom:

- *mac* — adres MAC karty sieciowej (wyjaśnię to za chwilę);
- *ip* — adres IP przypisany płytce (musisz określić adres z puli obsługiwanej przez Twoją sieć domową);



Rysunek 12.3. Płytkę rozszerzeń wyposażoną w kontroler sieci Wi-Fi

- *dns* — adres IP serwera nazw domen (DNS);
- *brama* — adres IP bramy internetowej (lub twojego domowego routera);
- *maska* — maska podsieci.

O ile nie masz doświadczenia w ręcznej konfiguracji sieci, to składnia ta może wydawać Ci się skomplikowana. Na szczęście wszystkie parametry, poza adresem MAC, są opcjonalne. W 90% przypadków będziesz musiał określać adresy MAC i IP lub sam adres MAC. Wszystkie pozostałe parametry są określane automatycznie.

Adres MAC jest unikalnym wyróżnikiem urządzenia sieciowego. Jest to adres przypisany płytce rozszerzeń Arduino wyposażonej w kontroler sieci Ethernet lub innemu urządzeniu, za pomocą którego Arduino łączy się z internetem.

Żadnym dwóm urządzeniom pracującym w Twojej sieci nie może być przypisany taki sam adres MAC. Ten dziwny kod jest zwykle nadrukowany na tylnej ścianie płytki rozszerzeń zawierającej kontroler sieci Ethernet lub Wi-Fi (zobacz rysunek 12.4). Możesz go również znaleźć na opakowaniu modułu. Jeżeli posiadasz starszą płytkę, której nie przypisano unikalnego adresu MAC, to możesz go wymyślić samodzielnie. Musisz jednak uważać, aby w jednej sieci nie pracowały dwa urządzenia, którym przypisano taki sam adres.

Korzystając z protokołu DHCP, możesz utworzyć połączenie sieciowe, któremu adres IP zostanie przypisany automatycznie. W tym celu zastosuj następujący kod:

```
#include <SPI.h>
#include <Ethernet.h>

byte mac[] = { 0xDE, 0xAD, 0xBE, 0xEF, 0xFE, 0xED };
void setup()
{
  Ethernet.begin(mac);
}
```



Rysunek 12.4. Adres MAC nadrukowany na naklejce płytki kontrolera Wi-Fi

Jeżeli chcesz przypisać płytce Arduino stały adres IP (na przykład w celu przekształcenia mikrokontrolera w serwer sieciowy), możesz to zrobić za pomocą poniższego kodu:

```
#include <SPI.h>
#include <Ethernet.h>
byte mac[] = { 0xDE, 0xAD, 0xBE, 0xEF, 0xFE, 0xED };
byte ip[] = { 10, 0, 1, 200 };

void setup()
{
    Ethernet.begin(mac, ip);
}
```

Pamiętaj o tym, że adres IP musi należeć do puli obsługiwanej przez Twoją sieć domową. Jeżeli nie określisz adresu IP samodzielnie — będziesz korzystał z DHCP — to funkcja `Ethernet.begin` zwróci 1, w sytuacji gdy połączenie zostanie nawiązane i Arduino zostanie przypisany adres IP. W przeciwnym wypadku wspomniana funkcja zwróci 0. Możesz przeprowadzić test — nawiązać połączenie z siecią, a następnie, za pomocą funkcji `localIP`, uzyskać adres IP przypisany Arduino. Przedstawiony szkic wykonuje taki test i wyświetla komunikat dotyczący jego przebiegu w oknie monitora portu szeregowego. Wypróbuj samodzielnie działanie tego szkicu. Pamiętaj o wpisaniu w kodzie szkicu adresu MAC przypisanego Twojemu kontrolerowi sieci Ethernet.

```
// 12_01_dhcp

#include <SPI.h>
#include <Ethernet.h>

byte mac[] = { 0x00, 0xAA, 0xBB, 0xCC, 0xDE, 0x02 };
```

```

void setup()
{
  Serial.begin(9600);
  while (!Serial){}; // zapewnia kompatybilność z płytą Leonardo

  if (Ethernet.begin(mac))
  {
    Serial.println(Ethernet.localIP());
  }
  else
  {
    Serial.println("Nie połączono z siecią");
  }
}

void loop()
{
}

```

Stawianie serwera sieci Web

W dalszej części tego rozdziału zostanie opisany projekt „Sprzętowy serwer sieci Web”. Zobaczysz wówczas szkic z przykładową strukturą serwera sieci Web. W tym fragmencie omówię dostępne funkcje takiego serwera.

W klasie `EthernetServer` umieszczono większość funkcji niezbędnych do obsługi sieci Web. Po utworzeniu połączenia sieciowego należy wykonać jeszcze dwie czynności w celu uruchomienia serwera sieci Web. Najpierw musisz utworzyć nowy obiekt serwera, a następnie określić port, na którym będzie on odbierał polecenia. Poniższą deklarację należy umieścić w szkicu przed funkcją `setup`:

```
EthernetServer server = EthernetServer(80);
```

Strony internetowe są zwykle obsługiwane za pomocą portu 80. Jeżeli uruchomisz serwer sieci Web na porcie 80, to nie będziesz musiał dodawać numeru portu do adresu URL wpisywanego w przeglądarce, która ma się z tym serwerem połączyć.

Aby uruchomić serwer, w funkcji `setup` należy umieścić następujące polecenie:

```
server.begin();
```

Funkcja ta uruchomi serwer, który będzie czekał na dane wysłane przez przeglądarkę próbującą załadować stronę umieszczoną na serwerze. Sprawdzenie, czy przeglądarka nie wysłała żądania, odbywa się za pomocą polecenia `available` umieszczonego w funkcji `loop`. Polecenie to zwraca wartość zerową (jeżeli nie ma żądania wymagającego obsługi) lub obiekt `EthernetClient`. Obiekt ten jest używany również podczas wysyłania żądań przez Arduino do serwerów sieciowych, co może wydawać się mylące. W obu przypadkach `EthernetClient` obsługuje połączenie pomiędzy serwerem sieci Web a przeglądarką.

Po otrzymaniu tego obiektu możesz odczytać żądanie, stosując polecenie `read`. Aby za pomocą tego obiektu przekazać kod HTML, należy użyć funkcji `write`, `print` lub `println`. Po przesłaniu kodu HTML należy przerwać działanie wspomnianego obiektu i zakończyć sesję. Wyjaśnię to w sekcji „Sprzętowy serwer sieci Web” znajdującej się w dalszej części tego rozdziału.

Tworzenie żądań

Arduino potrafi pracować jako serwer sieci Web, ale może również wysyłać żądania HTTP do serwera sieci Web, który może znajdować się wewnątrz Twojej domowej sieci Web lub w dowolnym miejscu internetu.

Przed wysłaniem żądania należy nawiązać połączenie sieciowe tak, jak opisałem to wcześniej w kontekście serwera sieci Web. Zamiast jednak tworzyć obiekt `EthernetServer`, można utworzyć obiekt o nazwie `EthernetClient`:

```
EthernetClient client;
```

Do momentu, w którym chciałbyś wysłać żądanie sieciowe, nie musisz podejmować żadnych działań. Następnie możesz zastosować taki kod:

```
if (client.connect("api.openweathermap.org", 80))
{
    client.println("GET /data/2.5/weather?q=Manchester,uk HTTP/1.0");
    client.println();
    while (client.connected())
    {
        while (client.available())
        {
            Serial.write(client.read());
        }
    }
    client.stop();
}
```

Jeżeli połączenie zostało nawiązane, funkcja `connect` zwraca wartość będącą prawdą logiczną. Dwa polecenia `client.println` są żądaniem określającym witrynę, która ma zostać wysłana przez serwer. Następnie dwie zagnieżdżone pętle `while` wczytują dane dopóty, dopóki klient jest podłączony i dane są dostępne.

Możliwość połączenia dwóch pętli `while` za pomocą polecenia warunkowego `client.available() && client.connected()` może wydawać się czymś kuszącym, ale nie warto tego robić w praktyce, ponieważ na przykład na skutek niskiej prędkości łącza dopływ danych może zostać na chwilę przerwany. Zewnętrzna pętla `while` sprawia, że żądanie nie zostanie przerwane, a dane będą w dalszym ciągu odbierane.

Technika ta „blokuje” Arduino — mikrokontroler nie może wykonywać żadnych innych operacji do momentu zakończenia obsługi żądania. Jeżeli takie rozwiązanie jest nie do zaakceptowania w przypadku Twojego projektu, to w pętli `while` możesz umieścić dodatkowy kod sprawdzający inne warunki.

Przykład szkicu korzystającego z biblioteki Ethernet

Dwa szkice znajdujące się w tej sekcji mają na celu zademonstrowanie praktycznych zastosowań biblioteki `Ethernet`. Te dwa programy wykonują większość czynności, jakich możesz oczekiwać od Arduino podłączonego do sieci.

Sprzętowy serwer sieci Web

Pierwszy przykład ilustruje chyba najpopularniejsze zadanie realizowane przez Arduino podłączone do sieci — pracę w charakterze serwera sieci Web. Użytkownik korzystający z przeglądarki połączonej z Arduino pracującym jako serwer sieci Web może uzyskać dane odczytywane przez analogowe wejścia mikrokontrolera oraz sterować pracą cyfrowych wyjść (zobacz rysunek 12.5).



Rysunek 12.5. Interfejs sprzętowego serwera sieci Web

Za pomocą takiego szkicu można w łatwy sposób łączyć się z Arduino ze smartfona lub tabletu — urządzenia posiadające nawet najprostsza przeglądarkę internetową, która będzie potrafiła wysyłać zapytania do Arduino. Szkic ma 172 linie kodu, a więc nie umieszczam go w książce. Znajdziesz go w pliku `12_02_serwer.ino`. Warto, abyś załadował szkic do pamięci Arduino przed przystąpieniem do lektury jego opisu.

Szkic zaczyna się w sposób standardowy dla programów korzystających z sieci. Po zaimportowaniu bibliotek definiowane są obiekty `EthernetServer` i `EthernetClient`. Zmienne w kolejnej sekcji szkicu mają różne zastosowania:

```

const int numPins = 5;
int pins[] = {3, 4, 5, 6, 7};
int pinState[] = {0, 0, 0, 0, 0};
char line1[100];
char buffer[100];

```

Stała `numPins` definiuje rozmiar tablic `pins` i `pinState`. Tablica `pinState` używana jest do zapisu informacji o tym, jaki sygnał (wysoki czy niski) ma być podawany na poszczególnych wyjściach. Wszystkie piny, których numery zostały umieszczone w tablicy `pins` w funkcji `setup`, są deklarowane jako wyjścia. W funkcji tej nawiązywane jest połączenie sieciowe. Proces ten przebiega tak samo jak w poprzednich przykładach. Na koniec w tablicy znaków `line1` umieszczany jest pierwszy wiersz żądania HTTP, a w tablicy `buffer` umieszczane są jego kolejne wiersze.

Oto kod znajdujący się w funkcji `loop`:

```

void loop()
{
    client = server.available();
    if (client)
    {
        if (client.connected())
        {
            readHeader();
            if (! pageNameIs("/"))
            {
                client.stop();
                return;
            }
            client.println(F("HTTP/1.1 200 OK"));
            client.println(F("Content-Type: text/html"));
            client.println();

            sendBody();
            client.stop();
        }
    }
}

```

Funkcja `loop` sprawdza, czy żadne żądania wysłane przez przeglądarkę nie oczekują na przetworzenie. Jeżeli nawiązano połączenie i wykryto żądanie, to wywoływana jest funkcja `readHeader`. Kod tej funkcji znajduje się pod koniec szkicu. Wczytuje ona zawartość nagłówka żądania do bufora `line1` (pomijając pozostałe linie nagłówka). W ten sposób uzyskasz dostęp do nazwy strony, którą użytkownik chce otworzyć w przeglądarce, a także wszystkich ewentualnych parametrów żądania.

Szkic wysyła dość dużo tekstu przez sieć oraz do okna monitora portu szeregowego, a więc skorzystałem z funkcji `F`, która zapisuje tablice znaków w pamięci flash (zobacz rozdział 6.).

Po tym, gdy nagłówek zostanie odczytany, wywoływana jest funkcja `pageNameIs` (jej kod znajduje się ponownie pod koniec pliku ze szkicem). Funkcja ta sprawdza, czy ładowana jest strona główna (`/`). Żądania dotyczące innych stron są ignorowane. Jest to ważne — wiele przeglądarek automatycznie wysyła do serwera żądanie pobrania ikony witryny sieciowej. Nie chcemy, aby tego typu żądanie było niepotrzebnie przetwarzane przez serwer.

Teraz czas wygenerować odpowiedź mającą formę nagłówka i kodu HTML, który zostanie wyświetlony w oknie przeglądarki. Funkcja `sendHeader` generuje odpowiedź informującą przeglądarkę o tym, że wysłane przez nią żądanie jest poprawne. Poniższa funkcja `sendBody` jest o wiele bardziej skomplikowana:

```
void sendBody()
{
    client.println(F("<html><body>"));
    sendAnalogReadings();
    client.println(F("<h1>Piny wyj.</h1>"));
    client.println(F("<form method='GET'>"));
    setValuesFromParams();
    setPinStates();
    sendHTMLforPins();
    client.println(F("<input type='submit' value='Uaktualnij'>"));
    client.println(F("</form>"));
    client.println(F("</body></html>"));
}
```

Funkcja ta wyświetla podstawowy szablon strony zapisanej w formacie HTML. Kod jest dzielony na mniejsze części przez kilka funkcji pomocniczych. Pierwszą z nich jest `sendAnalogReadings`:

```
void sendAnalogReadings()
{
    client.println(F("<h1>Wej. analogowe</h1>"));
    client.println(F("<table border='1'>"));
    for (int i = 0; i < 5; i++)
    {
        int reading = analogRead(i);
        client.print(F("<tr><td>A")); client.print(i);
        client.print(F("</td><td>")); client.print((float) reading / 205.0);
        client.println(F(" V</td></tr>"));
    }
    client.println("</table>");
}
```

Funkcja ta służy do odczytywania wartości odbieranej przez każde z wejść analogowych. Odczytane napięcia zostaną umieszczone w tabeli wyświetlanej w oknie przeglądarki.

Być może zauważyłeś już wcześniej, że funkcja `sendBody` wywołuje funkcje `setValuesFromParams` i `setPinStates`. Pierwsza z tych funkcji modyfikuje zmienną `pinStates` za pomocą funkcji `valueOfParam`. Zmienna ta, w zależności od parametrów umieszczonych w nagłówku, decyduje o tym, jaki sygnał zostanie podany na piny wyjścia:

```
int valueOfParam(char param)
{
    for (int i = 0; i < strlen(line1); i++)
    {
        if (line1[i] == param && line1[i+1] == '=')
        {
            return (line1[i+2] - '0');
        }
    }
    return 0;
}
```

Funkcja `valueOfParam` obsługuje parametry żądania, które muszą mieć formę pojedynczej cyfry. Jeżeli uruchomisz w przeglądarce stronę wygenerowaną przez omawiany szkic i klikniesz przycisk uaktualniania, to w adresie URL zostaną umieszczone dodatkowe parametry. Adres ten będzie wyglądał następująco:

```
192.168.1.10/?0=1&1=0&2=0&3=0&4=0
```

Parametry są podawane po znaku `?`, mają formę `X=Y` i są oddzielane od siebie znakami `&`. Przed znakiem równości umieszczana jest nazwa parametru (w omawianym przypadku jest to cyfra z zakresu od 0 do 4). Po znaku równości podawana jest wartość przypisana danemu parametrowi (1 podaje wysoki sygnał na wyjściu, a 0 — niski). Dla ułatwienia parametry mogą być tylko jednym znakiem, w tym przypadku jedną cyfrą. Piny są konfigurowane przez funkcję `setPinStates`, która korzysta z danych zapisanych w tablicy `pinStates`.

Wróćmy na chwilę do funkcji `sendBody`. Niezbędne jest wygenerowanie kodu HTML tworzącego rozwijany zestaw list, które posłużą do konfiguracji złączy. Z listy musi być wybrany domyślnie aktualny stan pracy wyjścia. Aby to zrobić, należy dodać wartość `selected` do jednej z opcji znajdujących się na liście. Opcje, do których dodawana jest wspomniana wartość, wybierane są na podstawie tablicy `pinStates`.

Cały kod HTML sterujący pracą złączy wyjściowych jest formularzem. Gdy użytkownik wciśnie przycisk uaktualniania, wtedy przeglądarka wygeneruje żądanie zawierające odpowiednie parametry. Teraz przyjrzymy się kodowi HTML wygenerowanemu przez omawiany szkic:

```
<html><body>
<h1>Wej. Analogowe</h1>
<table border='1'>
<tr><td>A0</td><td>0.58 V</td></tr>
<tr><td>A1</td><td>0.63 V</td></tr>
<tr><td>A2</td><td>0.60 V</td></tr>
<tr><td>A3</td><td>0.65 V</td></tr>
<tr><td>A4</td><td>0.60 V</td></tr>
</table>
<h1>Piny wyj.</h1>

<form method='GET'>
<p>Pin 3<select name='0'>
<option value='0'>Nieaktywny</option>
<option value='1' selected>Aktywny</option>
</select></p>
<p>Pin 4<select name='1'>
<option value='0' selected>Nieaktywny</option>
<option value='1'>Aktywny</option>
</select></p>
<p>Pin 5<select name='2'>
<option value='0' selected>Nieaktywny</option>
<option value='1'>Aktywny</option>
</select></p>
<p>Pin 6<select name='3'>
<option value='0' selected>Nieaktywny</option>
<option value='1'>Aktywny</option>
</select></p>
```

```

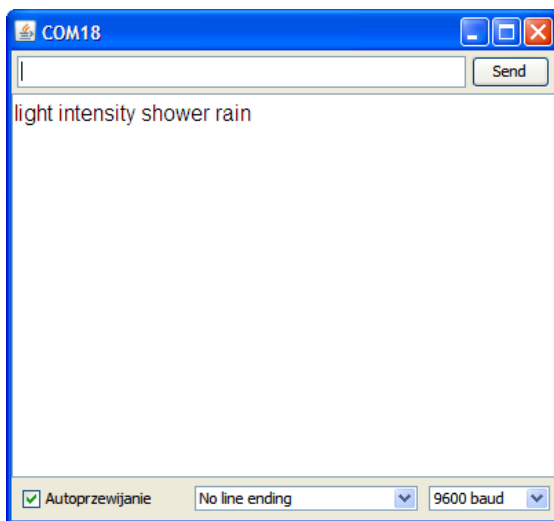
<p>Pin 7<select name='4'>
<option value='0' selected>Nieaktywny</option>
<option value='1'>Aktywny</option>
</select></p>
<input type='submit' value='Uaktualnij' />
</form>
</body></html>

```

Kod ten możesz również podejrzeć samodzielnie — wystarczy w oknie przeglądarki wybrać opcję pokazywania źródła.

Pobieranie danych w formacie JSON

Wysyłanie żądania przez Arduino pokażę Ci na przykładzie serwisu zwracającego krótką informację na temat pogody w lokalizacji interesującej użytkownika. Raport pogodowy (w języku angielskim) zostanie wyświetlony w oknie monitora portu szeregowego (zobacz rysunek 12.6). Szkic po uruchomieniu wysyła jedno żądanie, ale można go zmodyfikować tak, aby żądanie było wysyłane co godzinę, a uzyskane informacje były wyświetlane na ekranie LCD 16×2.



Rysunek 12.6. Informacje o pogodzie uzyskane za pomocą usługi sieciowej

Szkic wysyłający żądania jest dość krótki (zawiera 45 linii kodu). Znajdziesz go w pliku `12_03_żądanie_web.ino`. Jego najbardziej interesującą częścią jest funkcja `hitWebPage`:

```

void hitWebPage()
{
    if (client.connect("api.openweathermap.org", 80))
    {
        client.println("GET /data/2.5/weather?q=Manchester,uk HTTP/1.0");
        client.println();
        while (client.connected())
        {

```

```

        if (client.available())
        {
            client.findUntil("description\\":\\\"", "\\0");
            String description = client.readStringUntil('\\');
            Serial.println(description);
        }
    }
    client.stop();
}
}

```

Szkic na początku musi połączyć się z serwerem, z portem 80. Jeżeli połączenie to zostanie nawiązane, Arduino wyśle do serwera nagłówek:

```
client.println("GET /data/2.5/weather?q=Manchester,uk HTTP/1.0");
```

Dodatkowe polecenie `println` jest wymagane do oznaczenia końca nagłówka żądania i wygenerowania odpowiedzi przez serwer.

Gdy program czeka na połączenie, instrukcja `if` znajdująca się w pętli `while` sprawdza, czy możliwy już jest odczyt danych. Strumień danych jest odczytywany bezpośrednio. Dzięki takiemu rozwiązaniu nie trzeba umieszczać wszystkich danych w pamięci. Przesyłane dane są w formacie JSON:

```

{"coord":{"lon":-2.23743,"lat":53.480949},
"sys":{"country":"GB","sunrise":1371094771,
"sunset":1371155927},"weather":[{"id":520,"main":"Rain",
"description":"light intensity shower rain","icon":"09d"}],
"base":"global stations","main":{"temp":284.87,"pressure":1009,
"humidity":87,"temp_min":283.15,"temp_max":285.93},
"wind":{"speed":5.1,"deg":270},"rain":{"1h":0.83},
"clouds":{"all":40},"dt":1371135000,"id":2643123,
"name":"Manchester","cod":200}

```

Wróćmy do funkcji `hitWebPage`. Potrzebujemy tylko fragmentu tekstu znajdującego się w linii `"description"`. Jest on umieszczony za dwukropkiem, pomiędzy cudzysłowami. Tekst ten zostanie pobrany za pomocą funkcji `findUntil` i `readStringUntil`.

Funkcja `findUntil` ignoruje wszystkie dane wysyłane przez serwer aż do napotkania właściwego łańcucha. Funkcja `readStringUntil` wczytuje kolejne znaki aż do podwójnego cudzysłowu.

Biblioteka WiFi

Jak się zapewne domyślasz, biblioteka obsługująca sieć Wi-Fi działa podobnie do biblioteki obsługującej sieć Ethernet. Jeżeli w swoim szkicu słowa klucze: `Ethernet`, `EthernetServer` i `EthernetClient` zastąpisz, odpowiednio, słowami: `WiFi`, `WiFiServer` i `WiFiClient`, to reszta szkicu prawdopodobnie nie będzie wymagała modyfikacji.

Nawiązywanie połączenia

Największa różnica między bibliotekami obsługującymi sieci Wi-Fi i Ethernet wynika ze sposobu nawiązywania połączenia.

Najpierw należy zaimportować bibliotekę WiFi:

```
#include <SPI.h>
#include <WiFi.h>
```

Aby nawiązać połączenie, skorzystaj z polecenia `WiFi.begin`. Jako jego argumenty podaj nazwę Twojej sieci bezprzewodowej i hasło:

```
WiFi.begin("NAZWA-SIECI", "hasło");
```

W sekcji „Przykładowy szkic korzystający z sieci Wi-Fi” przedstawiono także inne różnice w sposobie obsługi sieci — o tych różnicach również powinieneś wiedzieć.

Funkcje zdefiniowane w bibliotece WiFi

W bibliotece WiFi zdefiniowano różne funkcje przydatne podczas pracy z siecią bezprzewodową. Wymieniono je w tabeli 12.1.

Tabela 12.1. Funkcje zdefiniowane w bibliotece WiFi

Nazwa funkcji	Opis
<code>WiFi.config</code>	Pozwala na przypisanie kontrolerowi sieci bezprzewodowej stałego adresu IP, serwera DNS i bramy.
<code>WiFi.SSID</code>	Zwraca łańcuch zawierający SSID (nazwę sieci bezprzewodowej).
<code>WiFi.BSSID</code>	Zwraca tablicę elementów typu <code>byte</code> , w której znajduje się adres MAC routera, z którym nawiązano połączenie.
<code>WiFi.RSSI</code>	Zwraca zmienną typu <code>long</code> określającą moc sygnału.
<code>WiFi.encryptionType</code>	Zwraca kod liczbowy określający rodzaj szyfrowania.
<code>WiFi.scanNetworks</code>	Zwraca liczbę wykrytych sieci. Funkcja ta nie podaje żadnych innych informacji o wykrytych sieciach.
<code>WiFi.macAddress</code>	Umieszcza adres MAC karty sieci bezprzewodowej w sześciobajtowej tablicy przekazanej tej funkcji jako jej argument.

Pełną, anglojęzyczną dokumentację biblioteki WiFi znajdziesz na stronie <http://arduino.cc/en/Reference/WiFi>.

Przykładowy szkic korzystający z sieci Wi-Fi

Kod szkicu umieściłem w pliku `12_04_serwer_wifi.ino`. Jest to zmodyfikowany szkic `12_02_serwer.ino` — program został przystosowany do pracy z siecią Wi-Fi. Nie będę opisywał całego szkicu, omówię tylko miejsca, w których musiałem coś zmienić.

W celu połączenia się z punktem dostępu do sieci bezprzewodowej musisz podać jej nazwę i hasło:


```
char ssid[] = "nazwa mojej sieci"; // SSID (nazwa) Twojej sieci
char pass[] = "hasło";           // hasło do Twojej sieci
```

Zmienić należy również nazwy klas serwera i klienta — z `EthernetServer` i `EthernetClient` na `WiFiServer` i `WiFiClient`:

```
WiFiServer server(80);
WiFiClient client;
```

Definiując serwer, nadal musisz określić numer portu (80).

Kolejną różnicą między pracą z siecią przewodową a bezprzewodową jest sposób nawiązywania połączenia. W tym szkicu połączenie jest nawiązywane za pomocą polecenia:

```
WiFi.begin(ssid, pass);
```

Pozostała część kodu jest niemalże identyczna jak kod szkicu korzystającego z sieci Ethernet. W funkcji `loop` przed zatrzymaniem klienta umieszczono polecenie `delay(1)`, które pozwala na odczyt danych przez klienta przed zakończeniem komunikacji. Polecenie to nie było potrzebne podczas pracy z siecią Ethernet. W szkicu korzystającym z sieci Wi-Fi rzadziej wywołuję polecenie `client.print` — korzystam z dłuższych łańcuchów. Zabieg taki ma na celu przyspieszenie komunikacji — płytka rozszerzeń wyposażona w kontroler sieci Wi-Fi jest mało efektywna w wysyłaniu krótkich łańcuchów. Pamiętaj o tym, że łańcuch obsługiwany przez funkcje `client.print` i `client.println` nie może przekroczyć 90 bajtów. Dłuższe łańcuchy nie zostaną wysłane.

Szkic korzystający z sieci Wi-Fi działa wolniej od swojego odpowiednika obsługującego sieć Ethernet. Jego ładowanie może trwać nawet 45 sekund. Oprogramowanie układowe płytki rozszerzeń obsługującej sieć Wi-Fi może być aktualizowane. Jeżeli w przyszłości zespół Arduino zwiększy wydajność tej płytki, to warto będzie uaktualnić jej oprogramowanie układowe. Więcej informacji na ten temat znajdziesz na stronie internetowej płytki: <http://arduino.cc/en/Main/ArduinoWiFiShield>.

Podsumowanie

W tym rozdziale omówiłem różne zagadnienia związane z łączeniem Arduino z sieciami Ethernet i Wi-Fi za pomocą specjalnych płytek rozszerzeń. Pokazałem Ci, że Arduino może działać jako serwer oraz jako klient sieci Web.

W kolejnym rozdziale omówię zagadnienia związane z cyfrowym przetwarzaniem sygnałów przez Arduino.

Rozdział 13.

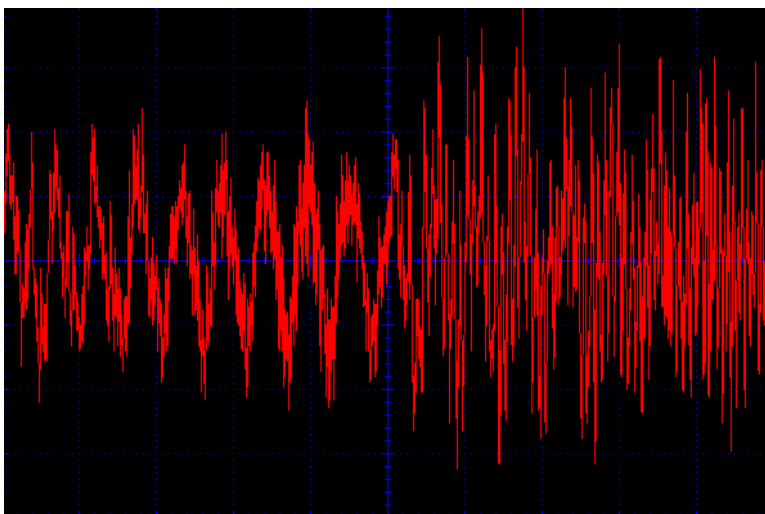
Cyfrowe przetwarzanie sygnałów

Arduino potrafi wykonywać dość proste operacje cyfrowego przetwarzania sygnałów. W tym rozdziale przedstawię różne techniki przetwarzania sygnałów — począwszy od programowego przetwarzania sygnału podawanego na wejście analogowe, a skończywszy na obliczaniu względnej wartości przesunięcia fal sygnału o różnych częstotliwościach za pomocą transformacji Fouriera.

Wprowadzenie do cyfrowego przetwarzania sygnałów

Odczytując wielkość mierzoną przez czujnik, tak naprawdę przeprowadzasz pomiar sygnału. Sygnały są często przedstawiane graficznie jako linie (zwykle faliste) biegnące na wykresie od strony lewej do prawej. W ten sposób na ekranie oscyloskopu przedstawia się sygnały elektryczne. Oś y jest **amplitudą** sygnału (jego siłą), a oś x jest osią czasu. Na rysunku 13.1 przedstawiono wizualizowany przez oscyloskop sygnał fali akustycznej trwającej zaledwie 0,25 s.

Jak widać, w sygnale można odnaleźć powtarzające się elementy. Częstotliwość, z jaką powtarzają się pewne składowe sygnału, nazywamy **częstotliwością sygnału**. Wielkość tę wyraża się w hercach (w skrócie Hz). Sygnał o częstotliwości 1 Hz powtarza się co sekundę. Sygnał o częstotliwości 10 Hz jest powtarzany 10 razy w ciągu 1s. Sygnał widoczny w lewej części rysunku 13.1 powtarza się co 0,6 kratki. Każda kratka to 25 ms (tak skonfigurowano oscyloskop analizujący ten sygnał), a więc częstotliwość sygnału to $1/(0,6 \cdot 0,025) = 67$ Hz. Gdybyś przeskalował ten wykres i rozłożył go na płaszczyźnie uwzględniającej o wiele mniejszą jednostkę czasu, to zauważyłbyś inne częstotliwości składowe dźwięku znajdujące się w tym sygnale. O ile sygnał nie jest idealną sinusoidą (którą pokazano w dalszej części tego rozdziału, na rysunku 13.5), to zawsze będzie zawierał kilka częstotliwości składowych.



Rysunek 13.1. Fala akustyczna (muzyka)

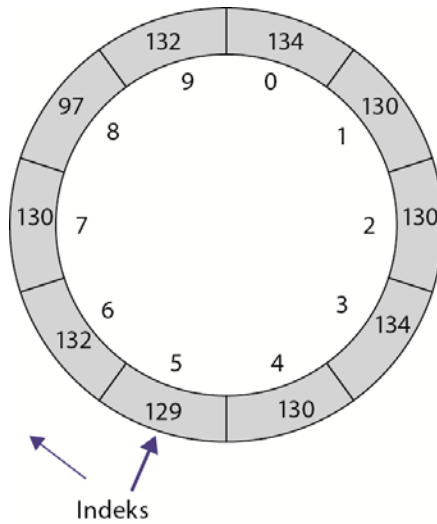
Sygnał przedstawiony na rysunku 13.1 może zostać również zarejestrowany przez jedno z wejść analogowych Arduino. Proces ten nazywa się **cyfryzacją** — polega na konwersji sygnału analogowego na cyfrowy. Podczas tego procesu próbkowanie musi odbywać się z częstotliwością na tyle wysoką, aby sygnał był wiernie odzwierciedlony.

Cyfrowe przetwarzanie sygnału polega na jego cyfryzacji za pomocą przetwornika analogowo-cyfrowego (A/C), poddaniu go obróbce, a następnie wygenerowaniu analogowego sygnału wyjściowego za pomocą przetwornika cyfrowo-analogowego (C/A). Większość nowoczesnego sprzętu audio, odtwarzacze MP3 i telefony komórkowe przetwarzają sygnał cyfrowo — proces ten pozwala na regulację wysokich i niskich częstotliwości w odtwarzanej muzyce. Technologię cyfrowego przetwarzania sygnału można również zastosować do czegoś prostszego — na przykład usunięcia zakłóceń z sygnału generowanego przez czujnik. W ten sposób uzyskamy bardziej dokładny wynik prowadzonego pomiaru.

Ogólnie rzecz biorąc, płytki Arduino nie są idealnymi urządzeniami do cyfrowego przetwarzania sygnału. Układy te nie są w stanie przechwytywać sygnałów analogowych z wysoką częstotliwością, a ich wyjścia są wyposażone w układy PWM, z czym również wiąże się pewne ograniczenia. Wyjątek stanowi płytka Arduino Due, która posiada wiele przetworników A/C, szybki procesor i dwa prawdziwe przetworniki C/A. Urządzenie to jest w stanie dokonać cyfryzacji stereofonicznego sygnału audio, a następnie go przetworzyć.

Uśrednianie odczytów

Bardzo często podczas odczytywania danych z czujników lepsze rezultaty pomiarów można uzyskać wtedy, gdy wykonuje się kilka pomiarów, a następnie wylicza średnią wartość mierzonej wielkości. W tym celu można zastosować na przykład bufor cykliczny (zobacz rysunek 13.2).



Rysunek 13.2. *Bufor cykliczny*

Podczas prowadzenia pomiarów z użyciem bufora cyklicznego każda odczytana wartość jest umieszczana w buforze w pozycji wskazywanej przez indeks. Po zapisaniu danych pod ostatnim indeksem bufora są one zapisywane ponownie od indeksu o numerze 0. W ten sposób stare dane są nadpisywane przez kolejne. Dzięki takiemu rozwiązaniu w buforze o rozmiarze n znajduje się n ostatnio odczytanych wartości.

W prezentowanym szkicu zastosowano bufor cykliczny:

```
// 13_01_uśrednianie

const int samplePin = A1;

const int bufferSize = 10;
int buffer[bufferSize];
int index;

void setup()
{
  Serial.begin(9600);
}

void loop()
{
  int reading = analogRead(samplePin);
  addReading(reading);
  Serial.println(average());
  delay(1000);
}

void addReading(int reading)
{
  buffer[index] = reading;
  index++;
}
```

```

    if (index >= bufferSize) index = 0;
}

int average()
{
    long sum = 0;
    for (int i = 0; i < bufferSize; i++)
    {
        sum += buffer[i];
    }
    return (int)(sum / bufferSize);
}

```

Korzystając z takiego rozwiązania, warto pamiętać o tym, że do momentu zapelnienia wszystkich indeksów bufora wartości średnie generowane przez program będą nieprawidłowe. W praktyce nie stanowi to problemu, ponieważ możemy zacząć obliczać wartości średnie po wypełnieniu bufora.

Zwróć uwagę na to, że funkcja obliczająca średnie (average) zapisuje sumę odczytów za pomocą zmiennej typu long. Stosowanie zmiennej typu long staje się koniecznością, jeżeli bufor jest na tyle duży, że suma zgromadzonych w nim wartości przekracza maksymalną wartość zmiennej typu int, czyli około 32 000. Liczba zwrócona przez funkcję może być zmienną typu int, ponieważ przyjmie ona wartość znajdującą się w przedziale wartości odczytywanych liczb.

Wstęp do filtrowania

W sekcji „Wprowadzenie do cyfrowego przetwarzania sygnałów” napisałem, że sygnały zwykle składają się z kilku różnych częstotliwości składowych. Czasami warto zignorować część tych częstotliwości. Można to osiągnąć przez filtrowanie sygnału.

Arduino podczas filtrowania sygnałów najczęściej pracuje w charakterze **filtru dolnoprzepustowego**. Wyobraź sobie, że dysponujesz światłomierzem i chcesz określić średni poziom jasności, a także zmiany tego poziomu w kolejnych minutach pracy mikrokontrolera. W ten sposób można na przykład wykryć zmierzch i włączyć światło. W takim projekcie warto byłoby wyeliminować wysokie częstotliwości wygenerowane przez czujnik wskutek na przykład przesunięcia dłoni obok czujnika lub oświetlenia go światłem sztucznym migoczącym ze stałą częstotliwością (50 Hz w Europie). Jeżeli interesuje Cię tylko taka część sygnału, która zmienia się dość wolno, to przyda Ci się filtr dolnoprzepustowy.

W przeciwnym wypadku, gdybyś chciał, aby mikrokontroler reagował na sygnały o wysokich częstotliwościach, ignorując sygnały zmieniające się powoli, to przydałby Ci się **filtr górno-przepustowy**.

Wróćmy do problemu zakłóceń liniowych. Na przykład założmy, że potrzebujesz częstotliwości znajdujących się powyżej i poniżej zakłóceń o częstotliwości 50 Hz. W takim przypadku nie można po prostu wyciąć z pasma niskich częstotliwości. Właściwym rozwiązaniem będzie zastosowanie filtru pasmowozaporowego, który wytnie z sygnału pasmo 50 Hz, a raczej wszystkie częstotliwości w zakresie od 49 Hz do 51 Hz.

Prosty filtr dolnoprzepustowy

Jeżeli chcesz tylko wygładzić sygnał, to nie musisz stosować w tym celu bufora. Możesz skorzystać z filtru dolnoprzepustowego — wycięcie z pasma zmian sygnału o wysokiej częstotliwości sprawi, że będziemy mieli do czynienia z sygnałem przedstawiającym ogólną tendencję. Takie filtrowanie przydaje się podczas pracy z czujnikami czułymi na zmiany o wysokiej częstotliwości, takimi jak przyspieszeniomierze. Jeżeli interesuje Cię tylko kąt pochylenia czujnika, to warto zignorować pewne elementy składowe sygnału generowanego przez czujnik.

Istnieje przydatna i prosta w implementacji technika polegająca na uśrednianiu mierzonych wartości. Nowa uśredniana wartość zależy od poprzedniej uśrednionej wartości, a także kolejnej liczby odczytanej przez mikrokontroler na wejściu, do którego podłączony jest czujnik:

$$\text{uśrednionaWartość}_n = (\text{alfa} \cdot \text{uśrednionaWartość}_{n-1}) + ((1 - \text{alfa}) \cdot \text{odczyt}_n)$$

Współczynnik *alfa* jest stałą o wartości znajdującej się w zakresie od 0 do 1. Im wyższą wartość przyjmuje ten współczynnik, tym bardziej wygładzony sygnał otrzymujemy.

Taka operacja może wydawać się czymś skomplikowanym, w rzeczywistości jednak jest dość prosta do zaimplementowania, o czym świadczy poniższy przykładowy szkic:

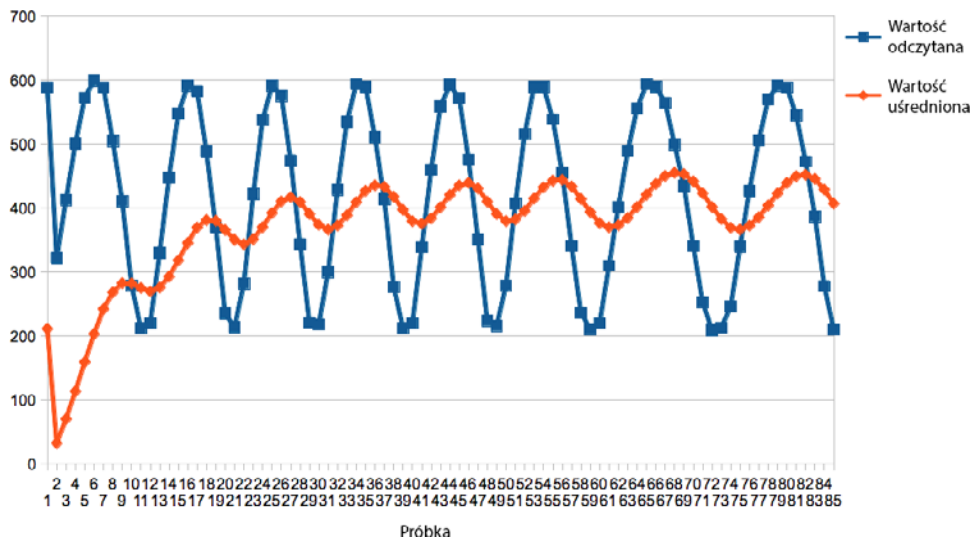
```
// 13_02_proste_wygładzanie
const int samplePin = A1;
const float alfa = 0.9;

void setup()
{
  Serial.begin(9600);
}

void loop()
{
  static float usrednionaWartosc = 0.0;
  float nowyOdczyt = (float)analogRead(samplePin);
  usrednionaWartosc = (alfa * usrednionaWartosc) + ((1 - alfa) * nowyOdczyt);
  Serial.print(nowyOdczyt); Serial.print(",");
  Serial.println(usrednionaWartosc);
  delay(1000);
}
```

Kopiując dane wyświetlane w oknie monitora portu szeregowego do arkusza kalkulacyjnego, możemy sporządzić wykres ilustrujący, jak operacja wygładzania działa w praktyce. Na rysunku 13.3 przedstawiono efekt działania powyższego szkicu. Do złącza A1 płytki Arduino, na której uruchomiono ten szkic, podłączono kawałek drutu tak, aby odbierał jakieś interferencje elektromagnetyczne.

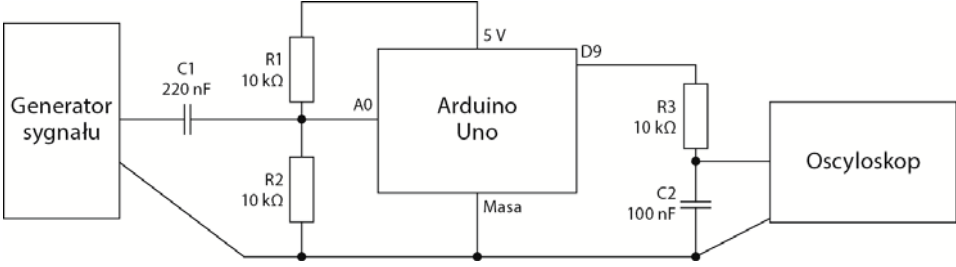
Jak widać, osiągnięcie właściwego zakresu przez wykres wartości uśrednionych zajmuje chwilę. Gdybyś zmienił wartość współczynnika *alfa* na — powiedzmy — 0,95, to wykres byłby jeszcze bardziej wygładzony. Sporządzanie wykresów na podstawie danych generowanych w oknie monitora portu szeregowego jest dobrym sposobem na upewnienie się, czy sygnał jest odpowiednio przetwarzany.



Rysunek 13.3. Wykres wartości uśrednionych

Cyfrowe przetwarzanie sygnałów przez Arduino Uno

Na rysunku 13.4 przedstawiono schemat, na którym sygnał audio jest kierowany do złącza A0. Płytkę Arduino dodatkowo generuje na jednym z wyjść sygnał PWM o częstotliwości 10 kHz. W roli generatora sygnału zastosowałem smartfon wyposażony w odpowiednią aplikację. Wyjście słuchawkowe telefonu połączyłem z wejściem Arduino tak, jak pokazano na rysunku 13.4.



Rysunek 13.4. Obwód pozwalający na przetwarzanie sygnału przez Arduino Uno

UWAGA

Pamiętaj o tym, że stosowanie telefonu w tym obwodzie prawdopodobnie narusza warunki umowy gwarancyjnej i może doprowadzić do jego uszkodzenia.

Przetwornik A/C jest w stanie obsłużyć całą falę sygnału, ponieważ wygenerowany sygnał przechodzi przez kondensator C1 i rezystory R1 i R2 — dzięki temu napięcie sygnału oscyluje wokół 2,5 V. Bez tych dodatkowych komponentów napięcie sygnału przez połowę cyklu byłoby niższe od 0 V.

Częstotliwość nośna sygnału PWM jest usuwana za pomocą dość prostego filtra składającego się z rezystora R3 i kondensatora C2. Niestety częstotliwość ta wynosi 10 kHz i jest zbyt bliska częstotliwości sygnału. W związku z tym usunięcie jej z sygnału jest dość trudne.

Sygnał możesz oglądać na ekranie oscyloskopu, ale równie dobrze do obwodu możesz podłączyć wzmacniacz i usłyszeć falę dźwiękową będącą odwzorowaniem tego sygnału. Musisz jednak upewnić się, że wejście wzmacniacza obsługuje sygnał przemienny.

To jest szkic, w którym biblioteka `TimerOne` została użyta zarówno do wygenerowania sygnału PWM, jak i do próbkowania dźwięku przy częstotliwości 10 kHz:

```
// 13_03_uno_bez_filtru

#include <TimerOne.h>

const int analogInPin = A0;
const int analogOutPin = 9;

void setup()
{
    Timer1.attachInterrupt(sample);
    Timer1.pwm(analogOutPin, 0, 100);
}

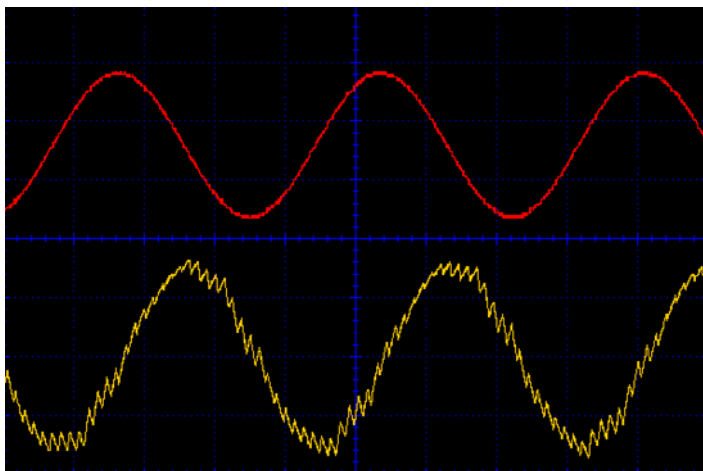
void loop()
{
}

void sample()
{
    int raw = analogRead(analogInPin);
    Timer1.setPwmDuty(analogOutPin, raw);
}
```

Górny ślad widoczny na rysunku 13.5 przedstawia sygnał kierowany na wejście Arduino, a dolny ślad przedstawia sygnał o częstotliwości 1 kHz generowany na wyjściu Arduino. Sygnał jest dość wiernie odwzorowywany, o ile jego częstotliwość nie zacznie zbliżać się do 2 – 3 kHz — wtedy fala staje się trójkątna z powodu zbyt niskiej częstotliwości próbkowania. Jak widać, sygnał wyjściowy jest wciąż poszarpany z powodu nieidealnego odfiltrowania częstotliwości nośnej, ogólny kształt fali sygnału został jednak dość dobrze odwzorowany. Oczywiście taka jakość odwzorowania sygnału wystarcza jedynie do przetwarzania głosu.

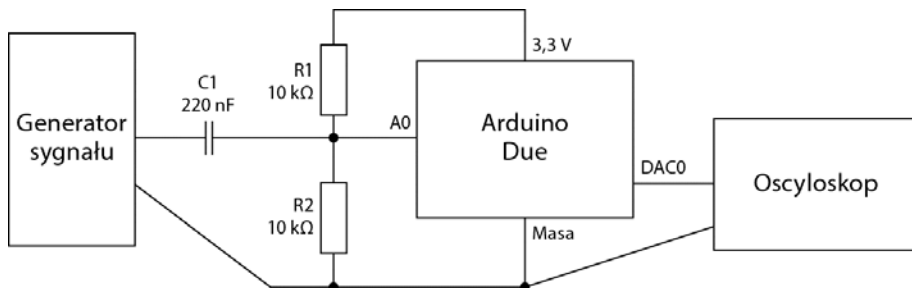
Cyfrowe przetwarzanie sygnałów przez Arduino Due

Przeprowadźmy teraz ten sam eksperyment za pomocą płytki Arduino Due przy znacznie wyższej częstotliwości próbkowania. Przedstawiony wcześniej kod był przeznaczony dla płytki Arduino Uno. Nie można go uruchomić na płytce Arduino Due, ponieważ charakteryzuje się ona inną architekturą i nie obsługuje biblioteki `TimerOne`.



Rysunek 13.5. Reprodukacja sygnału o częstotliwości 1 kHz przez Arduino Uno

Wejścia analogowe Arduino Uno przeznaczone są dla sygnałów o napięciu 3,3 V. W związku z tym górny rezystor R1 podłącz do napięcia 3,3 V, a *nie* 5 V. Arduino Due jest wyposażone w prawdziwe wyjście analogowe, a więc możesz pozbyć się filtra dolnoprzepustowego składającego się z rezystora R3 i kondensatora C2. Oscyloskop można podłączyć bezpośrednio do złącza DAC0. Na rysunku 13.6 przedstawiono schemat połączeń dedykowany płytce Arduino Due.



Rysunek 13.6. Obwód pozwalający na przetwarzanie sygnału przez Arduino Due

W kolejnym szkicu sygnał jest próbkowany z częstotliwością 100 kHz!

// 13_04_due_bez_filtru

```
const long samplePeriod = 10L; // mikrosekundy
```

```
const int analogInPin = A0;
```

```
const int analogOutPin = DAC0;
```

```
void setup()
```

```
{
```

```
    // http://www.djerickson.com/arduino/
```

```
    REG_ADC_MR = (REG_ADC_MR & 0xFFFFFFF) | 0x00020000;
```

```

    analogWriteResolution(8);
    analogReadResolution(8);
}

void loop()
{
    static long lastSampleTime = 0;
    long timeNow = micros();
    if (timeNow > lastSampleTime + samplePeriod)
    {
        int raw = analogRead(analogInPin);
        analogWrite(analogOutPin, raw);
        lastSampleTime = timeNow;
    }
}

```

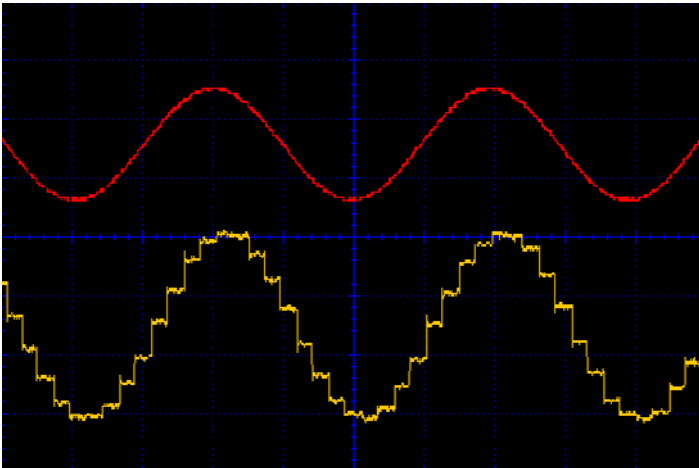
Arduino Due, w przeciwieństwie do innych płytek Arduino, pozwala określić rozdzielczości przetworników A/C i C/A. Dla ułatwienia i utrzymania szybkości działania programu oba przetworniki pracują z rozdzielczością 8 bitów.

Poniższa linia kodu przyspiesza działanie przetwornika A/C, manipulując wartościami rejestru. Więcej informacji na temat tej sztuczki znajdziesz na stronie, której adres podano w komentarzu znajdującym się w szkicu.

```
REG_ADC_MR = (REG_ADC_MR & 0xFFFF0FFF) | 0x00020000;
```

W zaprezentowanym szkicu funkcja `micros` określa częstotliwość próbkowania — kod próbkujący jest uruchamiany po odpowiednim czasie podanym w mikrosekundach.

Na rysunku 13.7 przedstawiono sygnał o częstotliwości 5 kHz reprodukowany przez płytkę Arduino Due. W sygnale wyjściowym widoczne są stopnie — kształt fali jest odzwierciedlany przez 20 próbek (częstotliwość próbkowania sygnału wynosi 100 kHz).



Rysunek 13.7. Reprodukacja sygnału o częstotliwości 5 kHz przez Arduino Due

Generowanie kodu filtrującego

Jeżeli chcesz filtrować sygnał w sposób bardziej zaawansowany, to możesz skorzystać z internetowego generatora kodu. Wygenerowany fragment kodu należy skopiować do szkicu. Wspomniany generator filtrów znajdziesz na stronie <http://www.schwietering.com/jayduino/filtuino/>.

Szczerze mówiąc, jedyną alternatywą dla tego generatora jest samodzielne wykonywanie skomplikowanych obliczeń.

Na rysunku 13.8 przedstawiono interfejs generatora filtrów. W dolnej części okna znajduje się wygenerowany kod. Pokażę Ci, jak zaimplementować go w szkicu Arduino.

The screenshot shows a web browser window with the URL `www.schwietering.com/jayduino/filtuino/index.php?characteristic=bu&passm`. The interface includes several configuration sections:

- character:** A dropdown menu set to "Butterworth", with buttons for "band stop" and "1st order".
- chebyshev ripple:** A text input set to "-3" with the unit "db (<0.0)".
- qfactor (resonator):** A text input set to "1" with the unit "(>0.0)".
- Alpha Low:** A text input set to "0.02267573696".
- Alpha Up:** A text input set to "0.03401360544" with an "or" separator.
- frequency:** A section with a checked "samplerate:" set to "44100" Hz, and "Lower corner" and "Upper corner" both set to "1000" Hz. It also includes "Hz Midi note" fields.
- more:** A section with checkboxes for "matched z-transform (better off, check frequency response)" (unchecked) and "prewarp (better on)" (checked).
- output:** A section with a "float type" dropdown, "Signal bits (including sign):" set to "10 bit", and checkboxes for "1 bit saturation save" (unchecked) and "brutal shift" (unchecked).
- excecution:** Buttons for "Send" and "Resetuj".

At the bottom, the generated C++ code is displayed:

```
//Band stop butterworth filter order=1 alpha1=0.022675736961451 alpha2=0.034013605442177
class filter
{
public:
    filter()
    {
        v[0]=0.0;
        v[1]=0.0;
    }
private:
    float v[3];
public:
    float step(float x) //class II
    {
        v[0] = v[1];
        v[1] = v[2];
        v[2] = (0.659920354492e-1 * x)
            + (-0.0711841160 * v[0])
            + (1.9018448769 * v[1]);
        return
            (v[0] + v[2])
            - 1.969615 * v[1];
    }
};
```

Rysunek 13.8. Generator kodu filtrów Arduino

Masz przed sobą wiele opcji umożliwiających wygenerowanie różnorodnych filtrów. Na rysunku 13.8 wygenerowano przykładowy kod filtra pasmozaporowego, mającego za zadanie zmniejszyć amplitudę składowych sygnału o częstotliwości od 1 kHz do 1,5 kHz. W górnej części okna wybrano ustawienia filtra: *Butterworth*, *band stop* (pasmozaporowy) i *1st order*. Określenie *Butterworth* odnosi się do charakterystyki filtrów pracujących w obwodach urządzeń analogowych (http://pl.wikipedia.org/wiki/Filtr_Butterwortha). Jest to dość uniwersalna charakterystyka i warto korzystać z niej jako parametru domyślnego.

W generatorze wybrałem opcję *1st order*. Skorzystanie z parametru o wyższej wartości zwiększy liczbę wcześniejszych próbek, które będą musiały być przechowywane, a także zwiększy stromość wycięcia niechcianych częstotliwości. W omawianym przykładzie opcja *1st order* jest wystarczająca. Im większa wartość tego parametru, tym więcej obliczeń musi wykonać procesor, co może zmusić Cię do zmniejszenia częstotliwości próbkowania, ponieważ mikrokontroler będzie zbyt mocno obciążony.

W dalszej kolejności należy podać dolną i górną częstotliwość graniczną filtra (*Lower corner* i *Upper corner*). Częstotliwość tę można wyrazić w hercach lub podać jako nutę MIDO.

W sekcji *more* znajdują się kolejne opcje. Na etykietach umieszczono informacje o tym, jak powinno się skonfigurować te opcje w celu uzyskania najlepszych rezultatów. W sekcji *output* możesz określić typ danych stosowany podczas wykonywania operacji filtrowania. Skorzystałem z typu *float*. Aby wygenerować kod, należy kliknąć przycisk *Send*.

W celu sprawdzenia działania wygenerowanego filtra możemy zmodyfikować szkic *13_04_due_bez_filtru.ino*. Zmodyfikowany szkic znajdziesz w pliku *13_05_due_filtr_pasmozaporowy.ino*.

Przed wszystkim należy skopiować wygenerowany kod do podstawowego szkicu „bez filtra”. Należy go umieścić bezpośrednio za definicjami stałych. Dobrą praktyką jest umieszczanie obok kodu adresu URL, który posłużył do jego wygenerowania. Adres ten należy oznaczyć jako komentarz. Pozwoli to na późniejszy szybki powrót do wybranych ustawień generatora filtra i ich modyfikację. W wygenerowanym kodzie cały kod filtra jest zamknięty w jednej klasie. Do zagadnień związanych z klasami wrócę w rozdziale 15. Na razie możesz traktować tę klasę jako komponent filtrujący sygnał.

Pod skopiowanym kodem dodaj następujący wiersz:

```
filter f;
```

Teraz musisz zmodyfikować funkcję *loop* tak, aby Arduino — zamiast podawać na wyjściu nieprzetworzony sygnał wejściowy — podawało sygnał poddany obróbce:

```
void loop()
{
    static long lastSampleTime = 0;
    long timeNow = micros();
    if (timeNow > lastSampleTime + samplePeriod)
    {
        int raw = analogRead(analogInPin);

        float filtered = f.step(raw);
```

```

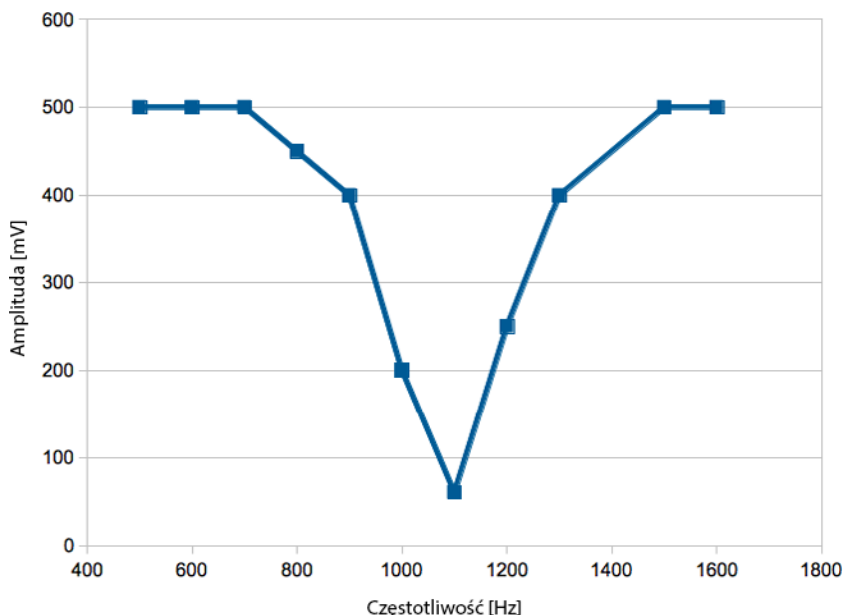
    analogWrite(analogOutPin, (int)filtered);
    lastSampleTime = timeNow;
  }
}

```

Przepuszczenie sygnału przez filtr jest bardzo proste, wystarczy dane odczytane bezpośrednio z wejścia analogowego przekazać jako argument do funkcji `f.step`. Funkcja ta zwróci wartość przetworzoną przez algorytm filtrujący. Można przeprowadzić konwersję zwróconej zmiennej na typ `int` przed przekazaniem jej do przetwornika C/A.

Kod filtrujący, znajdujący się w funkcji `f.step`, przechowuje trzy wcześniejsze wartości oraz trzy nowe wartości. Liczby te są ze sobą mieszane, a następnie przeskalowywane za pomocą pewnych współczynników — w ten sposób tworzona jest zwracana wartość. Czy matematyka nie jest cudowna?

Na rysunku 13.9 przedstawiono efekt działania wspomnianego filtru. Wartości, które posłużyły do wykonania tego wykresu, zostały uzyskane w wyniku pomiaru amplitud wyjściowych powstałych na skutek podawania przez generator na wejście Arduino sygnałów o różnej częstotliwości.



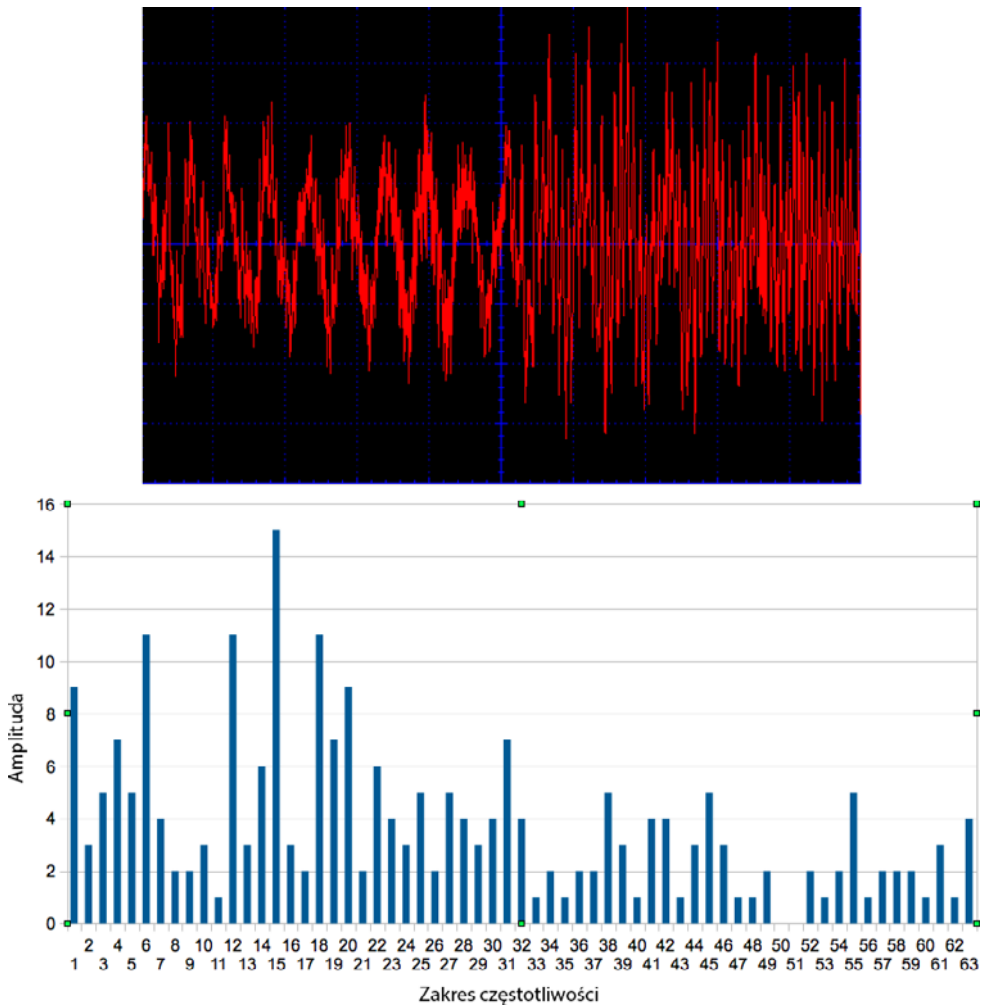
Rysunek 13.9. Charakterystyka częstotliwościowa filtra pasmozaporowego generowanego przez Arduino

Transformacja Fouriera

Transformacja Fouriera umożliwia przeprowadzenie analizy częstotliwościowego składu sygnału. We wstępie do niniejszego rozdziału napisałem, że sygnały mogą składać się z wielu fal

składowych o różnych częstotliwościach. Prawdopodobnie spotkałeś się z analizatorami spektrum wbudowanymi w sprzęt muzyczny lub z narzędziami przedstawiającymi odtwarzany dźwięk w formie graficznej, znajdującymi się w programach przeznaczonych do odtwarzania plików MP3. Spektrum odtwarzanego dźwięku jest przedstawiane w formie wykresów słupkowych. Pionowe słupki określają względną siłę fal składowych o różnych częstotliwościach. W lewej części tych wykresów znajdują się słupki odpowiedzialne za tony niskie (basy), a w prawej części — za tony wysokie (soprany).

Na rysunku 13.10 przedstawiono ten sam sygnał analizowany w **domenie czasu** (falista linia w górnej części rysunku) i w **domenie częstotliwości** (dolny wykres słupkowy pokazujący siłę sygnału w zakresie określonych przedziałów częstotliwości).



Rysunek 13.10. Sygnał analizowany w domenie czasu i domenie częstotliwości

Algorytm obliczający domenę częstotliwości na podstawie domeny czasu nazywany jest **szybką transformacją Fouriera** (ang. *Fast Fourier Transform* — FFT). Algorytm ten przetwarza naprawdę duże liczby i o ile nie kochasz matematyki, to nie musisz wnikać w szczegóły jego działania.

Na szczęście mądrzy ludzie często lubią dzielić się stworzonym przez siebie kodem. W internecie możesz znaleźć kody będące implementacją szybkiej transformacji Fouriera. Przykładowy kod, z którego korzystałem, nie jest biblioteką. Ma on formę pliku nagłówkowego języka C i pliku implementacyjnego C++ (o rozszerzeniach, odpowiednio, *.h* i *.cpp*). Wystarczy, że umieścisz te dwa pliki w folderze ze szkicem. Pliki te dołączyłem do archiwum zawierającego wszystkie szkice, a więc nie musisz samodzielnie pobierać ich z internetu. Kod ten pojawił się w jednym z postów na oficjalnym forum Arduino: <http://forum.arduino.cc/index.php/topic,38153.0.html>. Dwa wspomniane pliki oraz przykłady implementacji algorytmu znajdziesz również na dwóch innych stronach internetowych:

<https://code.google.com/p/arduino-integer-fft/>,

<https://github.com/slytown/arduino-spectrum-analyzer/>.

Kolejne dwa przykłady, które przedstawię, ilustrują użycie tego kodu przez Arduino Uno w celu próbkowania sygnału audio.

Analizator spektrum

W tym przykładzie zmienimy płytkę Arduino Uno w tekstowy analizator spektrum. Kod szkicu znajduje się w pliku *13_06_analizator_spektrum.ino*. Szkic ten jest dość długi, a więc umieścę tutaj tylko fragmenty kodu. Otwórz szkic w środowisku programistycznym Arduino i śledź jego kolejne części podczas lektury tekstu w niniejszej sekcji.

Algorytm szybkiej transformacji Fouriera korzysta z dwóch tablic elementów typu `char`. Korzystamy z typu `char`, a nie `byte`, ponieważ w przypadku języka C, stosowanego do programowania Arduino, zmienna typu `byte` nie może przyjmować wartości ujemnych, a konwertowany sygnał oscyluje wokół wartości 0, tak że może przyjmować wartość ujemną. Wskutek działania algorytmu szybkiej transformacji Fouriera tablica `data` zostanie wypełniona wartościami określającymi siłę kolejnych częstotliwości składowych (zaczynając od częstotliwości dźwięków niskich, a kończąc na wysokich). Zakres częstotliwości zależy od szybkości próbkowania. Omawiany szkic pozwala płytce Uno pracować jak najszybciej, nie kładąc nacisku na dokładność wykonywanych operacji. Umożliwia to uzyskanie maksymalnej częstotliwości 15 kHz. W związku z tym, że dysponujemy 63 równymi zakresami pomiarowymi, różnica częstotliwości między kolejnymi analizowanymi zakresami będzie wynosić około 240 Hz.

Aby konwersja sygnału analogowego przebiegała jak najszybciej, a częstotliwość próbkowania przyjmowała rozsądne wartości, musisz zastosować sztuczkę opisaną w rozdziale 4. Pozwoli ona na przyspieszenie pracy przetwornika A/C. Sztuczka ta sprowadza się do umieszczenia dwóch linii kodu w funkcji `setup`:

```
ADCSRA &= ~PS_128; // usuwa przelicznik wstępny 128
ADCSRA |= PS_16;   // dodaje przelicznik wstępny 16 (1MHz)
```


W głównej funkcji `loop` programu nie umieszczono zbyt wiele kodu:

```
void loop()
{
    sampleWindowFull();
    fix_fft(data, im, 7, 0);
    updateData();

    showSpectrum();
}
```

Funkcja `sampleWindowFull` próbkuje okienko składające się z 128 próbek danych. Omówię to za chwilę. Następnie uruchamiany jest algorytm szybkiej transformacji Fouriera. Parametr 7 jest logarytmem binarnym określającym liczbę próbek. Wartość ta nie ulega zmianie. Parametr 0 jest znacznikiem odwracającym, który zawsze będzie przyjmował wartość logiczną 0 (fałsz). Po wykonaniu obliczeń przez algorytm szybkiej transformacji Fouriera umieszczane są nowe wartości w tablicach. Na koniec wywoływana jest funkcja `showSpectrum`, która wyświetla dane dotyczące częstotliwości.

Funkcja `sampleWindowFull` wczytuje 128 wartości z wejścia analogowego i zakłada, że napięcie sygnału oscyluje wokół wartości 2,5 V, a więc w wyniku odjęcia od odczytanej wartości 512 sygnał będzie mógł przyjąć wartości dodatnie i ujemne. Następnie wartość ta jest skalowana przez stałą `GAIN`. Ma to na celu wzmocnienie słabych sygnałów. Odczytana wartość 10-bitowa jest następnie konwertowana na wartość 8-bitową i umieszczana w tablicy elementów typu `char`. Podczas konwersji stosuje się operację dzielenia przez 4. Tablica `im` jest wypełniana zerami. Jest ona przeznaczona dla urojonych części przetwarzanych liczb. Do operacji tej dochodzi wewnątrz algorytmu. Jeżeli chcesz uzyskać więcej informacji na jego temat, to zajrzyj na stronę http://pl.wikipedia.org/wiki/Szybka_transformacja_Fouriera.

```
void sampleWindowFull()
{
    for (int i = 0; i < 128; i++)
    {
        int val = (analogRead(analogPin) - 512) * GAIN;
        data[i] = val / 4;
        im[i] = 0;
    }
}
```

Amplituda każdej częstotliwości składowej jest obliczana przez funkcję `updateData`. Siła sygnału jest przeciwprostokątną trójkąta prostokątnego, którego dwa pozostałe boki to rzeczywista i urojona część wartości sygnału. Stosujemy w praktyce twierdzenie Pitagorasa!

```
void updateData()
{
    for (int i = 0; i < 64; i++)
    {
        data[i] = sqrt(data[i] * data[i] + im[i] * im[i]);
    }
}
```

Dane są wyświetlane w oknie monitora portu szeregowego. Cały zbiór danych jest umieszczany w jednej linii, a wartości są oddzielane za pomocą przecinków. Pierwsza wartość jest ignorowana, ponieważ zawiera stałą składową sygnału, która zwykle nie interesuje użytkownika.

Dane umieszczone w tablicy `data` mogą zostać użyte do określenia wysokości słupków wyświetlanych na ekranie ciekłokrystalicznym. Aby podłączyć odtwarzacz MP3 (lub dowolne inne źródło sygnału audio), skorzystaj ze schematu przedstawionego wcześniej na rysunku 13.4 — napięcie sygnału powinno oscylować wokół wartości 2,5 V.

Pomiar częstotliwości

W kolejnym przykładzie Arduino Uno będzie mierzyć przybliżoną częstotliwość sygnału, a następnie wyświetlać wynik pomiaru w oknie monitora portu szeregowego. Omawiany szkic znajdziesz w pliku `13_07_częstotliwość.ino`. Większość kodu w tym szkicu jest taka sama jak w poprzednim przykładzie. Główna różnica między programami polega na tym, że teraz — po tym, gdy dane są obliczone i umieszczone w tablicy `data` — indeks najwyższej wartości znajdującej się w tej tablicy jest używany do określenia częstotliwości sygnału. Kod znajdujący się w funkcji `loop` wyświetla tę wartość w oknie monitora portu szeregowego.

```
int findF()
{
    int maxValue = 0;
    int maxIndex = 0;
    for (int i = 1; i < 64; i++)
    {
        int p = data[i];
        if (p > maxValue)
        {
            maxValue = p;
            maxIndex = i;
        }
    }
    int f = maxIndex * 240;
    return f;
}
```

Podsumowanie

Cyfrowe przetwarzanie sygnałów jest bardzo rozległym tematem. Istnieje wiele książek dotyczących wyłącznie tego zagadnienia. Mając to na uwadze, wspomniałem tylko o technikach, które uznałem za najbardziej przydatne w kontekście Arduino.

W kolejnym rozdziale zajmę się problemami związanymi z potrzebą jednoczesnego wykonywania przez Arduino dwóch i większej liczby operacji. Zagadnienie to dotyczy osób mających doświadczenie w programowaniu większych urządzeń, gdzie równoczesne wykonywanie wielu wątków jest normą.

Rozdział 14.

Praca z użyciem tylko jednego procesu

Wielu programistów posiadających doświadczenie w pracy z większymi systemami narzeka, że Arduino nie obsługuje wielowątkowości i współbieżności. W tym rozdziale postaram się pokazać Ci, co tak naprawdę można wycisnąć z jednowątkowego modelu systemu wbudowanego.

Zmiana skali

Arduino przyciąga uwagę wielu entuzjastów programowania. Często są to osoby, które, tak jak ja, spędziły lata na pracy w wieloosobowych zespołach nad złożonymi projektami programistycznymi. Dla takich osób możliwość napisania kilku linijek kodu oraz zobaczenie efektów pracy tego kodu w rzeczywistości jest istnym antidotum na pracę nad złożonymi aplikacjami.

Niestety tacy programiści często próbują podczas pracy z Arduino stosować techniki, do których zostali przyzwyczajeni w swojej pracy. Przenosząc się ze świata dużych projektów programistycznych do miniaturowego świata Arduino, musimy przyzwyczać się do pisania prostych programów. Stworzenie złożonego programu bez użycia techniki Test Driven Development czy innych metod zwinnych jest niemalże szaleństwem. Utworzenie szkicu Arduino obsługującego jakiś skomplikowany projekt wymaga napisania zaledwie 200 linii kodu. Szkic ten może zostać napisany przez jednego programistę mającego w głowie zarys programu — bez użycia dodatkowych, typowych narzędzi programistycznych.

A więc możesz zapomnieć o sprawdzaniu wersji, wzorcach projektowych, testach jednostkowych i refaktoryzacji. Wystarczy opanować obsługę prostego środowiska programistycznego Arduino.

Dlaczego wątki są zbędne

Jeżeli pamiętasz programowanie komputerów w języku BASIC, zapewne jest dla Ciebie czymś normalnym to, że „komputer potrafi wykonywać tylko jedną operację”. Gdyby gra napisana w języku BASIC wymagała jednoczesnego przesuwania kilku sprite’ów, to piszący ją programista musiałby stworzyć kod przesuwający po trochu kolejne elementy znajdujące się na ekranie tak, aby gracz odniósł wrażenie, że są one poruszane jednocześnie.

Zamiast stosować kilka wątków, z których każdy byłby odpowiedzialny za obsługę jednego sprite’a, programista musi stworzyć jeden wątek obsługujący kolejno, fragmentami, każdy sprite bez „blokowania” procesora przez dłuższy czas z powodu obsługi tylko jednego z kilku elementów. Taka technika sprawdzi się również w kontekście Arduino.

Komputery (poza urządzeniami wielordzeniowymi) zwykle nie potrafią wykonywać jednocześnie kilku operacji. System operacyjny przełącza procesor między wykonywaniem wielu uruchomionych procesów. W przypadku Arduino, gdzie nie ma aż takiej potrzeby jednoczesnego wykonywania kilku operacji, możesz samodzielnie stworzyć kod przełączający wykonywane procesy, ponieważ Arduino nie posiada systemu operacyjnego.

Funkcje setup i loop

Każdy szkic Arduino musi składać się z dwóch funkcji: setup i loop. Nie jest to przypadek. Funkcja loop jest powtarzana w nieskończoność, właśnie dlatego nie powinieneś pozwalać na blokowanie jej działania. Kod umieszczony w tej funkcji powinien być wykonywany bez żadnych przeszkód.

Najpierw wykrywaj, a dopiero później reaguj

W większości projektów Arduino steruje pracą jakiegoś elementu. Funkcje loop szkiców bardzo często:

- Sprawdzają, czy użytkownik nie wcisnął przycisku lub czy sygnał generowany przez czujnik przekroczył określony próg.
- Reagują w odpowiedni sposób.

Prostym przykładem takiego szkicu jest program, w którym użytkownik, wciskając przycisk, włącza i wyłącza miganie diody LED.

Poniżej znajduje się właśnie taki szkic. Jak za chwilę zobaczysz, ograniczenia wynikające z faktu, że procesor musi czekać podczas migania diody LED, są czasami nie do zaakceptowania.

```
// 14_01_blyskanie_1
```

```
const int ledPin = 13;
const int switchPin = 5;
const int period = 1000;

boolean flashing = false;
```

```

void setup()
{
    pinMode(ledPin, OUTPUT);
    pinMode(switchPin, INPUT_PULLUP);
}

void loop()
{
    if (digitalRead(switchPin) == LOW)
    {
        flashing = ! flashing;
    }
    if (flashing)
    {
        digitalWrite(ledPin, HIGH);
        delay(period);
        digitalWrite(ledPin, LOW);
        delay(period);
    }
}

```

Szkic ten nie jest w pełni poprawny — program sprawdza wciśnięcie przycisku przez użytkownika tylko po skończeniu operacji błyskania diodą LED. Gdyby przycisk został wciśnięty podczas wykonywania przez program tej operacji, wciśnięcie to nie zostałoby zarejestrowane przez mikrokontroler. Być może nie wpływa to na działanie omawianego szkicu, ale w sytuacjach, gdy niezbędne jest zarejestrowanie każdego wciśnięcia przycisku, nie można umieszczać poleceń typu `delay` w funkcji `loop`. W przedstawionym przykładzie w rzeczywistości Arduino przez większość czasu wykonuje operację błyskania diodą LED, a wciśnięcie przycisku może zostać zarejestrowane tylko w bardzo małym przedziale czasu.

Problem ten rozwiązano w szkicu przedstawionym w kolejnej sekcji.

Pauza, która nie blokuje mikrokontrolera

Przedstawiony wcześniej szkic można zmodyfikować tak, aby nie korzystać z funkcji `delay`:

```

// 14_02_blyskanie_2

const int ledPin = 13;
const int switchPin = 5;
const int period = 1000;

boolean flashing = false;
long lastChangeTime = 0;
int ledState = LOW;

void setup()
{
    pinMode(ledPin, OUTPUT);
    pinMode(switchPin, INPUT_PULLUP);
}

void loop()
{

```

```

if (digitalRead(switchPin) == LOW)
{
    flashing = ! flashing;
    // i wyłączy diodę LED
    if (! flashing)
    {
        digitalWrite(ledPin, LOW);
    }
}
long now = millis();
if (flashing && now > lastChangeTime + period)
{
    ledState = ! ledState;
    digitalWrite(ledPin, ledState);
    lastChangeTime = now;
}
}

```

Do szkicu dodałem dwie nowe zmienne: `lastChangeTime` i `ledState`. Zmienna `lastChangeTime` jest używana do zapisywania informacji o tym, kiedy ostatnio dioda LED była wyłączona. W zmiennej `ledState` zapisywana jest z kolei informacja o tym, czy dioda LED jest obecnie włączona czy wyłączona. Z informacji tej program korzysta wtedy, gdy zachodzi konieczność włączenia lub wyłączenia diody LED.

W funkcji `loop` nie umieszczono żadnych poleceń `delay`. Pierwsza część tej funkcji sprawdza, czy przycisk został wciśnięty. Jeżeli użytkownik nacisnął przycisk, to program zaczyna migać diodą LED. Umieszczenie dodatkowego polecenia `if` (powtórzonego poniżej) jest dość sprytnym sposobem na pewne wyłączenie diody LED w sytuacji, gdyby użytkownik ponownie wciśnął przycisk. Bez tego polecenia dioda LED mogłaby pozostać zapalona po zakończeniu procesu migania.

```

if (! flashing)
{
    digitalWrite(ledPin, LOW);
}

```

W drugiej części funkcji `loop` wartość zwrócona przez funkcję `millis()` jest porównywana z sumą wartości `lastChangeTime` i `period`. Dzięki temu kod znajdujący się wewnątrz instrukcji `if` będzie uruchamiany tylko wtedy, gdy minie więcej czasu (milisekund), niż określa wartość zmiennej `period`.

Następnie zmieniana jest wartość zmiennej `ledState`, a na cyfrowe wyjście podawany jest właściwy sygnał. Wartość zmiennej `now` jest kopiowana do zmiennej `lastChangeTime`. Dzięki temu szkic wykona kolejne operacje dopiero po upływie odpowiedniej ilości czasu, określonej przez wartość zmiennej `period`.

Biblioteka Timer

Technikę przedstawioną w sekcji „Pauza, która nie blokuje mikrokontrolera” można zaimplementować, korzystając z biblioteki pozwalającej na planowanie powtarzania zdarzeń za pomocą funkcji `millis`. Wbrew swojej nazwie biblioteka ta nie wymaga używania sprzętowych układów zegarowych i będzie działała prawidłowo na większości płytek Arduino.

Wspomnianą bibliotekę możesz pobrać ze strony <http://playground.arduino.cc/Code/Timer>.

Prezentowany przykład uproszczono za pomocą biblioteki Timer. Jeżeli pobrałeś tę bibliotekę w wersji oznaczonej numerem 2, to podczas kompilacji szkicu zostanie wyświetlony błąd. Nowsza wersja biblioteki nie jest kompatybilna ze starszą. Dopóki problem ten nie zostanie rozwiązany, w celu kompilacji przedstawionego szkicu pobierz następującą wersję biblioteki: <http://www.simonmonk.org/wp-content/uploads/2013/12/Timer.zip>.

```
// 14_03_blyskanie_3
#include <Timer.h>

const int ledPin = 13;
const int switchPin = 5;
const int period = 1000;

boolean flashing = false;
int ledState = LOW;
Timer t;

void setup()
{
    pinMode(ledPin, OUTPUT);
    pinMode(switchPin, INPUT_PULLUP);
    t.every(period, flashIfRequired);
}

void loop()
{
    if (digitalRead(switchPin) == LOW)
    {
        flashing = ! flashing;
        if (! flashing)
        {
            digitalWrite(ledPin, LOW);
        }
    }
    t.update();
}

void flashIfRequired()
{
    if (flashing)
    {
        ledState = ! ledState;
        digitalWrite(ledPin, ledState);
    }
}
```

Korzystając z biblioteki, należy zdefiniować zegar (w szkicu nazwałem go *t*), a w funkcji *setup* określić funkcję, która będzie okresowo wywoływana za pomocą podanej składni:

```
t.every(period, flashIfRequired);
```

Następnie wystarczy umieścić podaną linię kodu w funkcji *loop*:

```
t.update();
```

Podczas każdego wywołania funkcji `update` funkcja `millis` sprawdza, kiedy poszczególne, zaplanowane czynności mają zostać wykonane. Jeżeli zachodzi konieczność wykonania jakiejś operacji, to wywoływana jest odpowiednia funkcja (w tym przypadku jest to `flashIfRequired`).

W bibliotece `Timer` zdefiniowano również wiele innych przydatnych funkcji. Więcej informacji na temat tej biblioteki znajdziesz na stronie internetowej, której adres podano na początku niniejszej sekcji.

Podsumowanie

Podczas lektury tego rozdziału dowiedziałeś się, jak — bez użycia wielowątkowości — sprawić, aby Arduino działało tak, jakby wykonywało jednocześnie wiele operacji. Wystarczy, że zaplanujesz swój szkic, pamiętając o ograniczeniach, jakimi charakteryzuje się Twój ulubiony mikrokontroler.

W ostatnim rozdziale tej książki dowiesz się, jak możesz udostępnić swój kod innym użytkownikom Arduino. Poznasz zagadnienia związane z tworzeniem i publikowaniem bibliotek Arduino.

Rozdział 15.

Tworzenie bibliotek

Prędzej czy później stworzysz coś na tyle praktycznego, że inni użytkownicy również mogliby skorzystać z napisanego przez Ciebie szkicu. Warto umieścić taki kod w internecie po wcześniejszym przekształceniu go na bibliotekę. W tym rozdziale dowiesz się, jak zrobić to w praktyce.

Kiedy należy tworzyć biblioteki?

Biblioteki mogą tworzyć nie tylko osoby zajmujące się rozwojem Arduino. Mogą one być pisane przez każdego użytkownika tej platformy. Jeżeli ktoś stworzył coś przydatnego, to środowisko użytkowników będzie wdzięczne za udostępnienie takiego kodu. Bibliotek się nie sprzedaje — byłoby to wbrew zasadom panującym w środowisku użytkowników Arduino. Biblioteki powinny być udostępniane jako otwarty kod, który może pomóc innym entuzjastom Arduino.

Chyba najbardziej przydatnymi bibliotekami są te, które pozwalają na współpracę Arduino z zewnętrznym osprzętem. Upraszczają one obsługę danego urządzenia, zwalniając użytkownika z obowiązku korzystania ze złożonego protokołu komunikacyjnego. Nie ma powodu, dla którego więcej użytkowników niż jeden musiałoby wykonywać ciężką pracę polegającą na zmuszeniu danego sprzętu do współpracy z Arduino. Internet umożliwia dzielenie się bibliotekami przez użytkowników.

Wskazówka

Interfejs programistyczny jest zbiorem funkcji, które użytkownik biblioteki będzie stosował w pisaniu przez siebie szkicu. Podczas pracy nad takim interfejsem zawsze musisz odpowiedzieć sobie na pytanie: „Czego tak naprawdę oczekuje użytkownik?”. Niskopoziomowa implementacja kodu powinna być jak najbardziej ukryta przez użytkownika. Zagadnienie to opiszę szczegółowo w sekcji „Przykładowa biblioteka TEA5767 Radio”.

Stosowanie klas i metod

Pisząc szkic, możesz odnieść wrażenie, że stosujesz zasady rządzące językiem C, tak naprawdę jednak korzystasz z zasad C++. Szkice Arduino pisane są w C++ — obiektowym rozszerzeniu języka C. W rozszerzeniu tym występuje pojęcie **klasy** obiektów. Klasy grupują dane związane z obiektem i funkcje używane do przetwarzania danych. Funkcje te wyglądają jak zwyczajne funkcje, ale gdy są związane z jakąś klasą, wtedy można je określić mianem **metod**. Metody mogą być zadeklarowane jako elementy publiczne (mogą być używane wszędzie) lub prywatne (dostęp do nich mogą uzyskać tylko inne metody należące do tej samej klasy).

Informuję Cię o obiektowym rozszerzeniu języka C, ponieważ w przypadku pracy z Arduino bardzo rzadko korzysta się z klas. Klasy pozwalają na obudowanie kodu i stworzenie z niego modułu. O podziale na metody publiczne i prywatne warto pamiętać podczas projektowania interfejsów programistycznych. Osoba pisząca szkic będzie komunikowała się z biblioteką za pomocą metod publicznych, a metody prywatne będą wykorzystywane w operacjach wykonywanych wewnątrz biblioteki.

Przeanalizuj przykład przedstawiony w kolejnej sekcji — dzięki niemu dowiesz się, jak korzystać z klas.

Przykładowa biblioteka TEA5767 Radio

W tym miejscu stworzymy bibliotekę na podstawie kodu, który spotkałeś już w rozdziale 7. Szkic ten służył do obsługi modułu tunera radiowego TEA5767. Program ten jest zbyt prosty, aby przekształcać go na bibliotekę, jest jednak na tyle prosty, że umożliwia stworzenie dobrego przykładu.

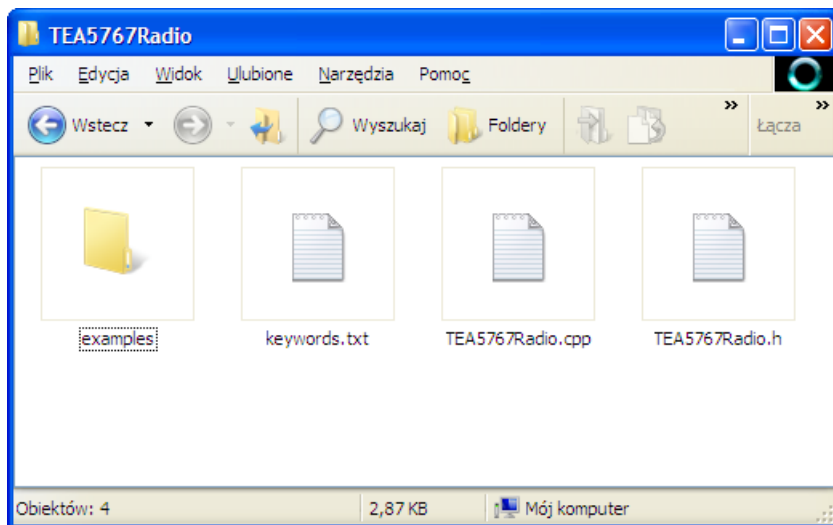
Aby utworzyć bibliotekę należy:

1. określić interfejs programistyczny,
2. napisać plik nagłówkowy,
3. napisać plik implementacji,
4. napisać plik ze słowami kluczowymi,
5. stworzyć kilka przykładów.

Biblioteka ma postać folderu, którego nazwa powinna być taka sama jak nazwa klasy biblioteki. W analizowanym przykładzie klasę biblioteki nazwałem `TEA5767Radio`. W folderze biblioteki należy umieścić dwa pliki: `TEA5767Radio.h` i `TEA5767Radio.cpp`.

Dodatkowo w folderze tym może znajdować się plik `keywords.txt` (zawierający słowa kluczowe) oraz podfolder `examples` (zawierający przykładowe szkice korzystające z biblioteki). Zawartość folderu biblioteki przedstawiono na rysunku 15.1.

Prawdopodobnie najłatwiej pracuje się nad biblioteką bezpośrednio w folderze, w którym znajdują się dodatkowe biblioteki instalowane przez użytkownika, obsługiwane przez środowisko programistyczne Arduino. Środowisko programistyczne wykrywa instalację kolejnej biblioteki podczas następnego uruchomienia, ale od tej pory wszystkie modyfikacje plików biblioteki będą automatycznie uwzględniane w czasie kolejnych kompilacji szkicu.



Rysunek 15.1. Struktura folderu przykładowej biblioteki

Tworzona przez nas biblioteka będzie oparta na kodzie zawartym w pliku `07_01_I2C_TEA5767.ino`. Gotową bibliotekę możesz pobrać ze strony <http://playground.arduino.cc/Main/TEA5767Radio>.

Określ interfejs programistyczny

Pracę nad biblioteką należy zacząć od zdefiniowania interfejsu, z którego będą korzystać użytkownicy.

Jeżeli korzystałeś wcześniej z bibliotek, to zapewne zauważyłeś, że mogą być obsługiwane przez użytkowników na dwa sposoby. Biblioteki, takie jak na przykład `Narcoleptic`, są najprostsze w obsłudze. Wystarczy dodać taką bibliotekę do szkicu, a następnie uzyskiwać dostęp do zdefiniowanych w niej metod, poprzedzając ich nazwy nazwą biblioteki, co ilustruje poniższy przykład:

```
#include <Narcoleptic.h>
// gdzieś w dalszej części szkicu
Narcoleptic.delay(500);
```

Tak samo można używać biblioteki pozwalającej na korzystanie z portu szeregowego. W ten sposób używa się bibliotek obsługujących po jednym urządzeniu. Jeżeli jednak istnieje możliwość obsługi kilku urządzeń przez tę samą bibliotekę, to należy z niej korzystać w nieco inny sposób. Właśnie takie założenie należy przyjąć przed napisaniem naszej biblioteki — do jednej płytki Arduino możemy podłączyć kilka modułów radioodbiornika.

W takim przypadku należy obsługiwać bibliotekę w sposób podobny do tego, w jaki obsługiwaliśmy bibliotekę `SoftwareSerial`. Za pomocą tej biblioteki mogliśmy utworzyć kilka programowych portów szeregowych, a więc musieliśmy tworzyć egzemplarze `SoftwareSerial`, używając następującej składni:

```
#include <SoftwareSerial.h>
SoftwareSerial mySerial(10, 11); // Rx, Tx
```

Gdybyś później chciał skorzystać z jednego z portów szeregowych (na przykład portu wykorzystującego złącza o numerach 10 i 11), to musisz nadać mu nazwę (ja wybrałem nazwę `mySerial`). Dopiero później możesz przysyłać przez niego dane następująco:

```
mySerial.begin(9600);
mySerial.println("Witaj");
```

Nie myśląc o tym, jak napiszesz kod, określ sposób, w jaki chcesz korzystać z kodu w szkicu.

Po zaimportowaniu biblioteki chcesz utworzyć nowe „radio”, nazwać je i określić adres magistrali I2C, pod jakim ma ono działać. Wszystko to jest naprawdę proste, możesz wybrać jedną z dwóch opcji: skorzystać z domyślnego portu (0x60) lub samodzielnie określić numer portu:

```
#include <TEA5767Radio>
TEA5767Radio radio = TEA5767Radio();
// lub TEA5767Radio radio = TEA5767Radio(0x60);
```

Masz do czynienia z radiem FM, a więc musisz określić częstotliwość pracy odbiornika, na przykład za pomocą takiego polecenia:

```
radio.setFrequency(93.0);
```

Wartość podana w nawiasie jest częstotliwością wyrażoną w megahercach. Osoba pisząca szkic chciałaby określić tę częstotliwość w ten sposób, a nie za pomocą zmiennej całkowitoliczbowej bez znaku, przesyłanej do modułu. W bibliotece chcesz umieścić cały skomplikowany kod obsługujący komunikację z radioodbiornikiem.

W tym przypadku musimy zapewnić użytkownikowi tylko takie funkcje. Teraz czas zacząć pisać kod.

Utwórz plik nagłówkowy

Kod biblioteki jest podzielony na kilka plików — zwykle są to dwa pliki. Jeden z nich to **plik nagłówkowy** — jego rozszerzenie to `.h`. To właśnie do tego pliku odnosimy się, umieszczając w szkicu polecenie `#include`. Plik nagłówkowy nie zawiera zasadniczego kodu programu. W pliku tym zdefiniowano nazwy klas i metod znajdujących się w bibliotece. Oto zawartość pliku nagłówkowego omawianej biblioteki:

```
#include <Wire.h>

#ifndef TEA5767Radio_h
#define TEA5767Radio_h

class TEA5767Radio
{
private:
    int _address;
public:
    TEA5767Radio();
    TEA5767Radio(int address);
```

```
void setFrequency(float frequency);
};

#endif
```

Polecenie `#ifndef` zapobiega kilkukrotnemu zaimportowaniu biblioteki. Jest to standardowa praktyka stosowana w plikach nagłówkowych.

W dalszej części pliku nagłówkowego znajduje się definicja klasy. W sekcji z elementami prywatnymi umieszczona jest tylko zmienna o nazwie `_address`. Zmienna ta zawiera adres obsługiwanego urządzenia podłączonego do magistrali I2C.

W sekcji z elementami publicznymi umieszczono dwie funkcje tworzące obiekt radio — jedna z tych funkcji pozwala na określenie adresu urządzenia, a druga korzysta z adresu domyślnego. W tej samej sekcji umieszczono również funkcję `setFrequency`.

Utwórz plik implementacji

Kod implementujący funkcje zdefiniowane w pliku nagłówkowym znajduje się w pliku *TEA5767Radio.cpp*:

```
#include <Arduino.h>
#include <TEA5767Radio.h>

TEA5767Radio::TEA5767Radio(int address)
{
    _address = address;
}

TEA5767Radio::TEA5767Radio()
{
    _address = 0x60;
}

void TEA5767Radio::setFrequency(float frequency)
{
    unsigned int frequencyB = 4 * (frequency * 1000000 + 225000) / 32768;
    byte frequencyH = frequencyB >> 8;
    byte frequencyL = frequencyB & 0xFF;
    Wire.beginTransmission(_address);
    Wire.write(frequencyH);
    Wire.write(frequencyL);
    Wire.write(0xB0);
    Wire.write(0x10);
    Wire.write(0x00);
    Wire.endTransmission();
    delay(100);
}
```

Obie metody odpowiedzialne za utworzenie nowego elementu radio przypisują zmiennej `_address` domyślny adres magistrali I2C (0x60) lub adres przekazany tym metodom jako parametr. Metoda `setFrequency` nie została praktycznie zmodyfikowana w porównaniu

z funkcją znajdującą się w szkicu `07_01_I2C_TEA5767.ino`. Jedyną zmianą jest to, że w poniższej linii kodu nawiązującej połączenie z magistralą I2C zastosowano wartość zmiennej `_address`.

```
Wire.beginTransmission(_address);
```

Utwórz plik ze słowami kluczowymi

Folder biblioteki powinien również zawierać plik o nazwie *keywords.txt*. Tworzenie tego pliku nie jest jednak obowiązkowe, ponieważ biblioteka będzie działała prawidłowo bez niego. Plik ten umożliwia środowisku programistycznemu Arduino oznaczanie kolorem fragmentów kodu będących słowami kluczowymi w bibliotece. Nasza przykładowa biblioteka ma tylko dwa słowa klucze: nazwę biblioteki (TEA5767Radio) i nazwę funkcji `setFrequency`. Plik ze słowami kluczowymi może zawierać komentarze. Należy je umieszczać w liniach rozpoczynających się znakiem `#`. W przypadku naszej biblioteki plik ten ma następującą zawartość:

```
#####
# Syntax Coloring Map for TEA5767Radio
#####
#####
# Datatypes (KEYWORD1)
#####
TEA5767Radio KEYWORD1
#####
# Methods and Functions (KEYWORD2)
#####
setFrequency KEYWORD2
```

Słowa klucze powinny być określone jako `KEYWORD1` lub `KEYWORD2` pomimo tego, że środowisko programistyczne Arduino w wersji 1.4 oznacza słowa klucze obu poziomów kolorem pomarańczowym.

Utwórz folder z przykładami

Jeżeli w folderze biblioteki utworzysz folder o nazwie *examples*, to wszystkie szkice umieszczone w tym folderze będą automatycznie rejestrowane podczas uruchamiania środowiska programistycznego Arduino, aby można było uzyskać dostęp do przykładowych szkiców za pomocą menu *Przykłady*. Użytkownik znajdzie dołączone przez Ciebie szkice w podmenu noszącym nazwę taką jak biblioteka. Przykłady dołączone do bibliotek są zwyczajnymi szkicami, które zostały zapisane w odpowiednim folderze. Do biblioteki obsługującej tuner radiowy można dołączyć następujący przykład:

```
#include <Wire.h>
#include <TEA5767Radio.h>

TEA5767Radio radio = TEA5767Radio();

void setup()
{
  Wire.begin();
  radio.setFrequency(93.0); // podaj żądaną częstotliwość
```

```

}

void loop()
{

```

Testowanie biblioteki

Aby sprawdzić działanie biblioteki, możesz po prostu uruchomić jeden z przykładowych szkiców, które korzystają z napisanej przez Ciebie biblioteki. O ile nie masz szczęścia lub nie zachowałeś dużej ostrożności podczas pracy nad biblioteką, to pierwszy program nie zostanie poprawnie skompilowany. Przeczytaj komunikat o błędzie wyświetlony u dołu okna środowiska programistycznego Arduino.

Publikacja biblioteki

Po napisaniu biblioteki czas udostępnić ją społeczności Arduino. Prawdopodobnie najlepszym sposobem, aby inni użytkownicy znaleźli Twoją bibliotekę, jest dodanie jej do listy dostępnej pod adresem <http://playground.arduino.cc/Main/LibraryList>. Na tej stronie możesz również umieścić archiwum z biblioteką. Niektórzy programiści wolą udostępniać pliki za pomocą serwisów takich jak GitHub, Google Code lub innych platform oferujących hosting plików, a następnie na stronie z listą bibliotek umieszczać odnośnik do pliku.

Jeżeli chcesz umieścić swoją bibliotekę w serwisie Arduino Playground, to musisz wykonać poniższe czynności:

1. Sprawdź bibliotekę i upewnij się, że działa poprawnie.
2. Folder biblioteki spakuj do archiwum w formacie ZIP. Plikowi nadaj nazwę identyczną jak nazwa klasy biblioteki. Archiwum powinno mieć rozszerzenie *.zip*.
3. Załóż konto użytkownika serwisu <http://www.arduino.cc/>.
4. Umieść informację o swojej bibliotece na stronie <http://playground.arduino.cc/Main/LibraryList>. Informacja ta powinna zawierać opis biblioteki i wyjaśnić sposób jej użytkowania. Opis taki najłatwiej jest utworzyć, kopiując fragmenty noty dotyczącej jakiejś innej biblioteki. Stwórz odnośnik do noty, który zostanie umieszczony na liście (na przykład `[[TEA5767Radio]]`). Właśnie pod taką nazwą Twoja biblioteka będzie dodana do listy (obok nazwy zostanie umieszczony znak zapytania). Kliknij utworzony odnośnik. W ten sposób utworzysz nową stronę i otworzysz edytor. Skopiuj treść noty dotyczącej innej biblioteki, a następnie odpowiednio ją zmodyfikuj. Możesz skorzystać na przykład z noty dotyczącej biblioteki *TEA5767Radio*.
5. Aby załadować archiwum z biblioteką, w nocie dotyczącej swojej biblioteki musisz umieścić znacznik typu `Attach:TEA5767Radio.zip`. Po zapisaniu strony kliknij na odnośnik, który później będzie pozwalać na pobieranie pliku, po czym wybierz plik, który zostanie załadowany na serwer.

Podsumowanie

Warto jest tworzyć biblioteki. Zanim zaczniesz tworzyć samodzielnie jakąś bibliotekę, sprawdź, czy ktoś inny nie stworzył jej wcześniej.

Niestety, tworząc książki tego typu, nie mogę zaspokoić oczekiwań wszystkich czytelników. Mam nadzieję, że pomogłem Ci rozwiązać pewne problemy związane z bardziej zaawansowanymi aspektami programowania Arduino.

Możesz mnie śledzić za pomocą serwisu Twitter — @simonmonk2. Więcej informacji na temat tej książki, a także innych, napisanych przeze mnie, znajdziesz na stronie <http://www.helion.pl/ksiazki/arpokk.htm>.

Dodatek A

Podzespoły

Książka ta dotyczy głównie programowania. W treści odnosiłem się tylko do kilku komponentów. W tym dodatku wymieniono je, a także podano przykładowych dystrybutorów, którzy mają je w swojej ofercie.

Płytki Arduino

Arduino jest na tyle popularne, że nie ma problemu z zakupem najczęściej stosowanych płytek, takich jak Uno i Leonardo. Rzadziej spotykane płytki znajdziesz w ofercie firm Adafruit i SparkFun (USA) lub SparkFun (Wielka Brytania). W Polsce płytki te możesz znaleźć w ofercie firm takich jak Kamami, Botland i Nettigo. Ich adresy internetowe podałem w sekcji „Dystrybutorzy”, która znajduje się na końcu tego dodatku.

Płytki rozszerzeń

Firmy takie jak Kamami, Botland i Nettigo mają w swojej ofercie wiele oficjalnych płytek rozszerzeń Arduino, a także ich tańszych odpowiedników wytwarzanych przez inne firmy, na przykład SparkFun.

Płytki rozszerzeń, o których pisałem w tej książce, wymieniono w tabeli A.1. W nawiasach, obok nazwy dystrybutora, podano kody niektórych produktów.

Moduły

W tabeli A.2 wymieniono komponenty i moduły, do których odnosiłem się w tej książce. W nawiasach, obok nazwy dystrybutora, podano kody niektórych produktów.

Tabela A.1. *Płytki rozszerzeń*

Płytki rozszerzeń	Rozdział	Dystrybutor
USB host Shield	11.	Nettigo (usb-host-shield) Botland (SPF-02214)
Ethernet Shield	12.	płytki dostępne u większości dystrybutorów
WiFi Shield	12.	płytki dostępne u większości dystrybutorów

Tabela A.2. *Moduły*

Płytki rozszerzeń	Rozdział	Dystrybutor
moduł radia FM TEA5767	7.	Allegro
kolorowy panel diodowy	7.	Adafruit (902)
moduł zegara czasu rzeczywistego DS1307	7.	Nettigo (mmsmart-rtc)
czujnik temperatury DS18B20	8.	Nettigo (ds18b20)
ośmiokanałowy przetwornik A/C MCP3008	9.	Botland (UCC-02358)
moduł Venus GPS	10.	Nettigo (venus-gps)

Dystrybutorzy

W tabeli A.3 wymieniałem dystrybutorów komponentów elektronicznych oraz rzeczy związanych z Arduino.

Tabela A.3. *Dystrybutorzy*

Dystrybutor	Adres	Uwagi
Adafruit	http://www.adafruit.com/	Produkty firmy Adafruit znajdują się również w ofercie innych dystrybutorów.
SparkFun	http://www.sparkfun.com/	Produkty firmy SparkFun znajdują się również w ofercie innych dystrybutorów.
Seeed Studio	http://www.seeedstudio.com/	Rzadko spotykane i tanie moduły i klony płytek Arduino.
Mouser Electronics	http://pl.mouser.com/	Bogata oferta komponentów elektronicznych.
Digi-Key	http://www.digikey.pl/	Bogata oferta różnych podzespołów elektronicznych.
Farnell	http://pl.farnell.com/	Brytyjska firma operująca w Polsce. Duży asortyment komponentów.

Tabela A.3. *Dystrybutorzy (ciąg dalszy)*

Dystrybutor	Adres	Uwagi
Nettigo	<i>http://nettigo.pl/</i>	Duży asortyment podzespołów elektronicznych, a także modułów i płytek Arduino.
Kamami	<i>http://kamami.pl/</i>	Moduły i płytki Arduino.
Botland	<i>http://botland.com.pl/</i>	Duży wybór artykułów przeznaczonych dla robotyków.

Skorowidz

A

adres
 IP, 185
 MAC, 185–187
akumulatory, 91
algorytm FFT, 212, 213
alokacja pamięci
 dynamiczna, 109
 statyczna, 109
amplituda sygnału, 199
analizator spektrum, 212
anatomia Arduino, 47
Arduino, 15
 Due, 25, 205
 Ethernet, 184
 EtherTen, 184
 Leonardo, 24
 LilyPad, 26
 Mini, 26
 Uno, 204
AT91SAM3X8E, 51
ATmega2560, 50
ATmega328, 48
ATmega32u4, 50
AVR Studio, 56

B

bajt, 82, 146
biblioteka, 42, 221
 Arduino StringObject, 114

avr/pgmspace.h, 121
avr/eeprom.h, 118
DallasTemperature, 142
Ethernet, 183, 185, 189
KeyboardController, 181
MemoryFree, 108
Narcoleptic, 95
OneWire, 139, 141
RTCLib, 134
SoftwareSerial, 164, 165
SPI, 151
Spi.h, 178
StringObject, 114
TEA5767 Radio, 222
Timer, 218
TimerOne, 71, 205
Usb.h, 178
USB_Host_Shield, 177
WiFi, 195
Wire, 126
Wiring, 51
bit, 82, 145
Blink, 18
bod, 162
brama, 186
budzenie, 97
bufor cykliczny, 201

C

charakterystyka częstotliwościowa filtru, 210
cyfrowe przetwarzanie sygnałów, 199, 204, 205

częstotliwość
 sygnału, 199
 taktowania, 92
 zegara, 93
 czujnik DS18B20, 143

D

dioda LED, 217, 218
 długość łańcucha, 113
 DNS, 186
 domena
 czasu, 211
 częstotliwości, 211
 dyrektywa PROGMEM, 120
 dyrektywy przedkompilacyjne, 52
 dystrybutorzy, 230
 działanie magistrali I2C, 128

E

EEPROM, 50, 115–120
 ograniczenia, 120
 zapis zmiennej, 117
 elementy szkicu, 109
 emulacja
 klawiatury, 173–175
 myszy, 175, 176
 Ethernet, 183

F

fala
 akustyczna, 200
 kwadratowa, 70
 FFT, Fast Fourier Transform, 212
 filtr
 dolnoprzepustowy, 202
 górnoprzepustowy, 202
 filtrowanie, 202
 Flash, 120, 121
 format JSON, 194, 195
 formatowanie łańcuchów, 112
 fototranzystor, 100
 funkcja, 28, 32
 attachInterrupt, 66
 bcd2bin, 135
 delay, 217
 digitalWrite, 84
 eeprom_write_block, 119
 Ethernet.begin(), 185

findUntil, 195
 hitWebPage, 194
 initializeCode, 117
 kbd_init, 179
 kbd_poll, 179
 keyRelease, 181
 loop, 28, 31, 166, 209
 malloc, 111
 mouseMoved, 182
 onReceive, 132
 pgm_read_word, 121
 pinMode, 84
 println, 168
 readBytes, 163
 readSecretCodeFromEEPROM, 117
 readStringUntil, 195
 saveSecretCodeToEEPROM, 118
 sendHeader, 192
 Serial.available, 163
 Serial.begin, 163
 setPinStates, 192
 setup, 28, 99, 216
 setValuesFromParams, 192
 strlen, 113
 valueOfParam, 193
 funkcje
 biblioteki StringObject, 115
 biblioteki WiFi, 196
 konfigurujące interfejs SPI, 153
 zarządzające zasilaniem, 95

G

generowanie
 kodu, 80
 kodu filtrującego, 208
 GPS, 169, 172

I

inicjalizowanie biblioteki OneWire, 139
 instalacja
 programu rozruchowego, 58–61
 środowiska programistycznego, 18
 instrukcja if, 30
 interfejs
 1-Wire, 137
 Bluetooth, 176
 generatora filtrów, 208
 programistyczny, 223
 SPI, 145, 153

sprzętowego serwera, 190
 USB, 173
 interfejsy UART, 161

J

jednoczesny odczyt danych, 85
 język programowania, 27

K

karta SD, 121
 klasa, 222
 klasa EthernetServer, 188
 kompensacja równoległa, 87
 komponenty, 17
 komunikacja, 166
 pomiędzy płytkami, 167
 z komputerem, 165
 z modułem GPS, 169
 kontroler sieci
 Ethernet, 183
 sieci Wi-Fi, 186

Ł

ładowanie szkicu, 55
 łańcuch, 37, 107, 111
 łączenie łańcuchów, 114

M

macierz, 37
 magistrala I2C, 123
 inicjacja, 127
 odbieranie danych, 127
 protokół, 126
 Radio FM TEA5767, 128
 SPI, 150
 warstwa sprzętowa, 125
 wysyłanie danych, 127
 zegar DS1307, 134
 malloc, memory allocate, 111
 maska podsieci, 186
 maskowanie bitów, 146
 metoda, 222
 modulacja szerokości impulsu, 40
 moduł, 229
 DS18B20, 137
 GPS, 169, 172

modyfikacja szkicu Blink, 27
 monitor portu szeregowego, 35, 36, 80

N

natężenie prądu, 90
 nawiązywanie połączenia, 185, 195
 notacja szesnastkowa, 146

O

obliczanie, 78
 obsługa
 chipa DS18B20, 141
 interfejsu USB, 173
 sieci, 183
 transmisji szeregowej, 162
 ograniczanie pobieranego prądu, 99
 operacje arytmetyczne, 77
 optymalizacja kodu, 80
 otwarty kolektor, 125

P

pamięć Arduino, 103
 EEPROM, 50, 115–120
 flash, 49, 107, 120
 RAM, 49, 104
 pasożytnicze zasilanie, 137
 pętla, 31
 piny przerwań, 66
 plik
 implementacji, 225
 nagłówkowy, 224
 ze słowami kluczowymi, 226
 płytka
 Arduino Due, 25, 180, 205, 229
 Arduino Uno, 48, 60, 204
 duża, 24
 Ethernet, 184
 EtherTen, 27, 184
 Leonardo, 24
 Leostick, 27
 LilyPad USB, 26
 mała, 25
 Mini, 26
 symbol równości, 30
 Uno, 23
 USB Host, 177
 USBDroid, 178

płytką

- wejścia analogowe, 22
- WiFly, 185
- z diodami LED, 133
- zasilanie, 21
- złącza cyfrowe, 22
- złącza zasilania, 22

płytki rozszerzeń, 183, 229

pobieranie danych, 194

pobór prądu, 89, 93

podzespoły, 229

polecenia Arduino, 44–46

polecenie

- ifndef, 225
- client.println, 189
- delay(), 28
- for, 32, 54
- malloc, 111
- print, 112
- sprint, 112
- while(true), 54
- WiFi.begin, 196
- write, 168

połączenie szeregowo, 167

pomiar

- częstotliwości, 214
- napięcia, 41
- natężenia światła, 100, 101
- wolnej pamięci, 108

porównanie płytek, 76

port

- szeregowy, 35
- USB, 174

porty układu ATmega328, 82

prąd, 91

procedury obsługi przerwań, 65, 67

proces, 215

procesor

- AT91SAM3X8E, 51
- ATmega2560, 50
- ATmega328, 48, 49
- ATmega32u4, 50

procesory AVR, 48

program

- AVR Studio, 57
- rozruchowy, 56, 58

programowanie hosta USB, 176

projekty, 16

protokół

- 1-Wire, 138
- DHCP, 185

NMEA, 170

SPI, 151

przeglądanie, 78

przerwania sprzętowe, 63

przesuwanie bitów, 148

przesyłanie danych, 130

przetwarzanie sygnału, 204

przetwornik

analogowo-cyfrowy, 200

cyfrowo-analogowy, 200

przyspieszanie

Arduino, 75

wejść analogowych, 86

publikacja biblioteki, 227

R

radio FM TEA5767, 128

ramka stosu, 106

rejestr

kierunku danych, 83

wejścia portu, 84

rekurencja, 105

reprodukcja sygnału, 207

rezystor podciągający, 67

S

SCL, Serial Clock Line, 126

SDA, Serial Data Line, 126

serwer sieci Web, 188, 190

sieć Wi-Fi, 196

sinusoida, 79

skanowanie magistrali, 139

sprzęt sieciowy, 183

stała, 109

stawianie serwera, 188

sterta, 111

stos, 106

struktura folderu, 223

struktury danych, 105

sygnał PWM, 72

szeregowo transmisja danych, 159, 165

szkic, 19

Blink, 18, 20

Cardinfo, 122

szybka transformacją Fouriera, 212

szybkie

wejścia cyfrowe, 84

wejścia-wyjścia, 80

szybkość transmisji danych, 162

Ś

środowisko programistyczne, 17

T

tablica, 37

tablica znaków, 37

testowanie

biblioteki, 227

przerwań, 64

wydajności, 76

transformacja Fouriera, 210

tranzystor typu MOSFET, 101

tryby przerwań, 67

TTL, 159

TWI, Two Wire Interface, 123

tworzenie

bibliotek, 221

łańcuchów, 114

zadań, 189

typy danych, 43

char, 111

float, 77

int, 78

long, 77

U

UART, 159

układ

DS18B20, 141, 142

TTL, 159

UART, 159

W5100, 183

USB Host, 176

usypianie, 95

uśrednianie odczytów, 200

W

wątek, 216

wejścia

analogowe, 38, 86

cyfrowe, 33

Wi-Fi, 196

włączanie obsługi przerwań, 69

wybór

płytki, 19

portu szeregowego, 20

wyjścia

analogowe, 40

cyfrowe, 35

wykres wartości uśrednionych, 204

wyłączanie

komponentów, 94

obsługi przerwań, 69

Z

zasilanie

Arduino, 91

układów ATmega, 95

zegar, 70, 219

zegar czasu rzeczywistego, 134

zewnętrzne przerwania, 97

złącza

interfejsu SPI, 150

magistrali I2C, 125

złącze

ICSP, 151

Rx, 160, 161

Tx, 160

zmiana skali, 215

zmienna, 29

delayPeriod, 37

messageIndex, 38

zmienne ulotne, 68

znak końca łańcucha, 111

Ż

zadania HTTP, 189, 191

PROGRAM PARTNERSKI

GRUPY WYDAWNICZEJ HELION



1. ZAREJESTRUJ SIĘ
2. PREZENTUJ KSIĄŻKI
3. ZBIERAJ PROWIZJĘ

Zmień swoją stronę WWW
w działający bankomat!

Dowiedz się więcej i dołącz już dzisiaj!

<http://program-partnerski.helion.pl>

GRUPA WYDAWNICZA

 **Helion SA**